

A Complete Guide to Data Preprocessing
Before Training Any Machine Learning Model

Before Training ML Model

Everything You Need to Know and Do
Before Training a Machine Learning Model

From Scratch • Scikit-learn

— — — — — W R I T T E N B Y — — — — —

Md. Shaon Khan

BSc in IIT, JU

2026

স্বত্ব ও অনুমতি

Before Training ML Model

প্রথম সংস্করণ, ২০২৬

© লেখক কর্তৃক সর্বস্বত্ব সংরক্ষিত।

এই বইয়ের কোনো অংশ প্রকাশক বা লেখকের পূর্বানুমতি ছাড়া পুনর্মুদ্রণ, অনুলিপি, বা কোনো ইলেকট্রনিক মাধ্যমে সংরক্ষণ করা যাবে না।

শিক্ষামূলক ও ব্যক্তিগত ব্যবহারের জন্য বইয়ের উদ্ধৃতি দেওয়া যাবে, তবে উৎস উল্লেখ করা বাধ্যতামূলক।

Disclaimer:

এই বইয়ে ব্যবহৃত সমস্ত code educational উদ্দেশ্যে লেখা হয়েছে। Titanic dataset Kaggle-এর publicly available dataset। Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn সবই open-source BSD-licensed library। লেখক কোনো production project-এ এই code ব্যবহারে সৃষ্ট সমস্যার দায়িত্ব নেন না।

ভূমিকা (Preface)

এই বই কার জন্য?

এই বই মূলত সেইসব শিক্ষার্থী ও পেশাদারদের জন্য যারা Python-এর মৌলিক ধারণা জানেন এবং Machine Learning শিখতে চান, কিন্তু বাংলায় ভালো রিসোর্স পাচ্ছেন না। বিশ্ববিদ্যালয়ের শিক্ষার্থী, চাকরিপ্রার্থী, অথবা নিজে নিজে শিখতে চাওয়া যে কেউ এই বই থেকে উপকৃত হবেন।

কী শিখবেন?

এই বইয়ের মূল বিষয় হলো ML model train করার আগের সম্পূর্ণ প্রক্রিয়া — EDA, Missing Value Handling, Outlier Detection, Feature Encoding, Feature Scaling, Feature Engineering, Feature Selection, Multicollinearity, Class Imbalance, এবং sklearn Pipeline। প্রতিটি বিষয় বাংলায় ব্যাখ্যা করা হয়েছে এবং Titanic dataset দিয়ে Python code-সহ হাতে-কলমে দেখানো হয়েছে।

কীভাবে পড়বেন?

বইটি ক্রমানুসারে পড়া সবচেয়ে ভালো — প্রতিটি অধ্যায় আগের অধ্যায়ের উপর নির্মিত। প্রতিটি code block Jupyter Notebook বা Google Colab-এ চালিয়ে দেখুন। শেষ অধ্যায়ে (অধ্যায় ২০) একটি সম্পূর্ণ End-to-End Titanic Project আছে — সেটি শেষ করলে আপনি নিজেই বুঝবেন কতটা শিখেছেন।

প্রয়োজনীয় পূর্বজ্ঞান:

- Python-এর মৌলিক ধারণা (variable, loop, function, list, dict)
- Jupyter Notebook বা Google Colab ব্যবহার জানা
- উচ্চ মাধ্যমিক পর্যায়ের গণিত (পরিসংখ্যান হলে আরও ভালো)

সবাইকে শুভকামনা। আপনার ML যাত্রা সুন্দর হোক।

সূচিপত্র (Table of Contents)

ভূমিকা (Preface)

- অধ্যায় ১: মেশিন লার্নিং ওয়ার্কফ্লো পরিচিতি
অধ্যায় ২: ডেটাসেট বোঝা
অধ্যায় ৩: এক্সপ্লোরেরি ডেটা অ্যানালাইসিস (EDA)
অধ্যায় ৪: ইউনিভারিয়েট অ্যানালাইসিস
অধ্যায় ৫: বাইভারিয়েট ও মাল্টিভারিয়েট অ্যানালাইসিস
অধ্যায় ৬: ট্রেন-টেস্ট স্প্লিট
অধ্যায় ৭: মিসিং ভ্যালু হ্যান্ডলিং
অধ্যায় ৮: আউটলায়ার ডিটেকশন ও হ্যান্ডলিং
অধ্যায় ৯: ফিচার স্কেলিং
অধ্যায় ১০: ক্যাটাগরিক্যাল এনকোডিং
অধ্যায় ১১: ফিচার ক্রিয়েশন ও এক্সট্রাকশন
অধ্যায় ১২: ফিচার সিলেকশন
অধ্যায় ১৩: মাল্টিকোলিনিয়ারিটি
অধ্যায় ১৪: ক্লাস ইমব্যালেন্স
অধ্যায় ১৫: ডেটা ট্রান্সফরমেশন
অধ্যায় ১৬: ColumnTransformer
অধ্যায় ১৭: Pipeline
অধ্যায় ১৮: ক্রস ভ্যালিডেশন
অধ্যায় ১৯: ইন্ডালুয়েশন মেট্রিক্স
অধ্যায় ২০: সম্পূর্ণ End-to-End Titanic Project
অধ্যায় ২১: নতুন ডেটাসেট হাতে পেলে কীভাবে সিদ্ধান্ত নেব

পরিশিষ্ট ক: শব্দকোষ (Glossary)

পরিশিষ্ট খ: তথ্যসূত্র ও আরও পড়ার উৎস

পরিশিষ্ট গ: লেখক পরিচিতি

**Garbage in,
garbage out.**

অধ্যায় ১: মেশিন লার্নিং ওয়ার্কফ্লো পরিচিতি

১.১ এই বই নিয়ে দুটো কথা

ধরুন আপনি রান্না শিখতে চান। শুধু একটা রেসিপি বই পড়লেই কিন্তু ভালো রাঁধুনি হওয়া যায় না। তার আগে জানতে হয় - কাঁচামাল কীভাবে বাছাই করতে হয়, কোনটা পচা কোনটা তাজা, মসলা কীভাবে পরিমাপ করতে হয়, আগুনের আঁচ কেমন হওয়া উচিত। মেশিন লার্নিং মডেল ট্রেন করাও ঠিক এরকম। অনেকেই সরাসরি `model.fit(X, y)` লিখে মডেল ট্রেন করতে শুরু করে দেন, অথচ তার আগে ডেটার সাথে যে বিশাল কাজ করতে হয় - সেটাই একটা ভালো মডেল আর একটা খারাপ মডেলের মধ্যে আসল পার্থক্য তৈরি করে।

এই বইয়ের নাম তাই 'Before Training ML Model' - অর্থাৎ মডেল ট্রেন করার আগে যা যা জানা ও করা দরকার, তার সম্পূর্ণ এবং প্র্যাক্টিক্যাল গাইড। এই বই সম্পূর্ণ বাংলায় লেখা, যাতে কোনো প্রোগ্রামিং বা ML ব্যাকগ্রাউন্ড ছাড়াও যে কেউ শুরু থেকে শিখতে পারেন। তবে ভাষা সহজ হলেও কনসেপ্টগুলো গভীরভাবে, প্র্যাকটিক্যালি এবং প্রফেশনাল মানে আলোচনা করা হয়েছে - যাতে এটি বিশ্ববিদ্যালয়ের শিক্ষার্থী থেকে শুরু করে কর্মরত ডেটা সায়েন্টিস্ট পর্যন্ত সবার জন্যই একটি রেফারেন্স বই হয়ে উঠতে পারে।

“গারবেজ ইন, গারবেজ আউট” - ডেটা সায়েন্সের সবচেয়ে পুরনো এবং সবচেয়ে সত্যি কথাগুলোর একটি। মডেল যত শক্তিশালীই হোক, ইনপুট ডেটা যদি নোংরা, ভুল বা অগোছালো হয়, আউটপুটও তেমনই হবে।

১.২ মেশিন লার্নিং আসলে কী?

সহজ ভাষায় বললে, মেশিন লার্নিং হলো এমন একটি পদ্ধতি যেখানে কম্পিউটারকে স্পষ্ট নিয়ম (rule) লিখে না দিয়ে, বরং অনেক উদাহরণ (data/example) দেখিয়ে শেখানো হয়।

একটা বাস্তব উদাহরণ দিয়ে বোঝা যাক:

ধরুন আপনি একটা ছোট বাচ্চাকে বিড়াল চিনাতে চান। আপনি কি তাকে বলবেন - “যার চারটা পা, লেজ আছে, লোম আছে, ম্যাও ম্যাও ডাকে, কান খাড়া...” - এভাবে সব নিয়ম লিখে দেবেন? বাস্তবে আমরা তা করি না। আমরা বাচ্চাকে অনেকগুলো বিড়ালের ছবি দেখাই - কালো বিড়াল, সাদা বিড়াল, মোটা বিড়াল, রোগা বিড়াল - আর প্রতিবার বলি “এটা বিড়াল”। কিছুদিন পর বাচ্চা নিজে থেকেই নতুন একটা বিড়াল দেখলে চিনে ফেলে, যদিও সেই নির্দিষ্ট বিড়ালটাকে সে আগে কখনো দেখেনি।

মেশিন লার্নিং একই কাজ করে - এক্সপ্লিসিট নিয়ম না লিখে, ডেটা থেকে প্যাটার্ন শিখে নেয়।
ঐতিহ্যগত প্রোগ্রামিং আর মেশিন লার্নিংয়ের মধ্যে পার্থক্যটা নিচের টেবিলে স্পষ্ট হবে:

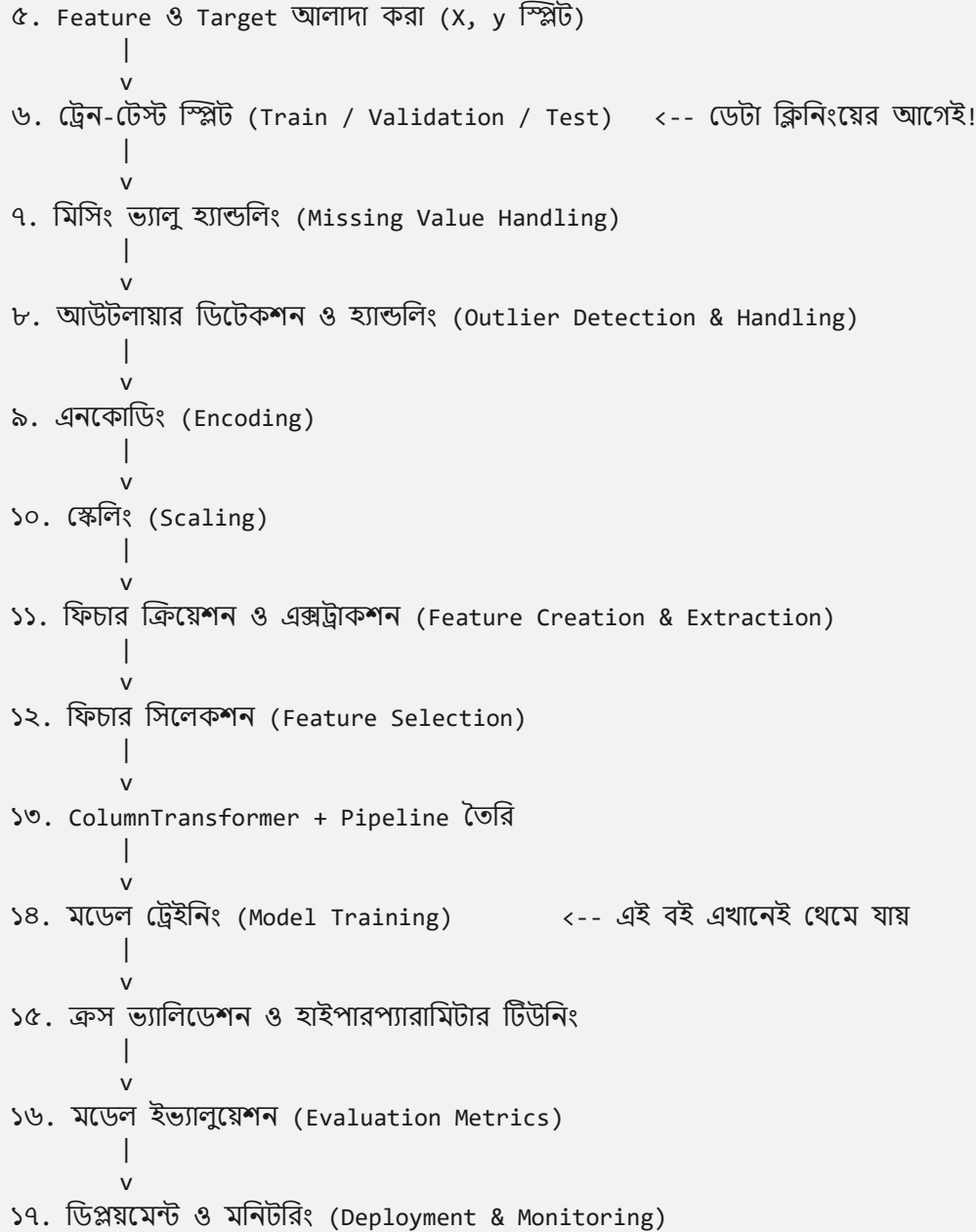
বিষয়	ঐতিহ্যগত প্রোগ্রামিং	মেশিন লার্নিং
ইনপুট	Data + Rules	Data + Output (Answer)
আউটপুট	Answer (ফলাফল)	Rules / Model (নিয়ম শেখে)
যুক্তি	মানুষ নিয়ম লেখে	মডেল নিজে প্যাটার্ন শেখে
উদাহরণ	if age > 18: adult	হাজারো ডেটা দেখে বয়সের প্যাটার্ন শেখা
পরিবর্তনযোগ্যতা	নতুন কেসে কোড পরিবর্তন লাগে	নতুন ডেটা দিয়ে রি-ট্রেন করা যায়

১.৩ সম্পূর্ণ ML পাইপলাইন: কাঁচা ডেটা থেকে ট্রেনড মডেল পর্যন্ত

একটা মেশিন লার্নিং প্রজেক্ট কখনোই “ডেটা নাও, মডেলে ফিট করো, শেষ” - এত সহজ হয় না।
বাস্তব প্রজেক্টে নিচের ধাপগুলো অনুসরণ করতে হয়, এবং এই পুরো বইটি মূলত ধাপ ৪ থেকে ১৩ - অর্থাৎ “training-এর আগের কাজ” নিয়েই লেখা।

একটা বিষয় খেয়াল করুন: Train-Test Split (ধাপ ৬) করা হয় Missing Value ও Outlier হ্যান্ডলিংয়ের (ধাপ ৭-৮) আগেই, কখনোই পরে নয়। কারণ Missing Value impute করা, Outlier ধরা, Encoding বা Scaling - এই প্রতিটা কাজে আমরা ট্রেনিং ডেটা থেকে কিছু না কিছু “শিখি” (যেমন mean, median, mode, min-max)। যদি স্প্লিটের আগে এই শেখাটা ঘটে, তাহলে টেস্ট ডেটার তথ্য ট্রেনিং প্রক্রিয়ায় ফাঁস (leak) হয়ে যায়। তাই নিয়মটা মনে রাখুন: আগে split, তারপর fit।

১. সমস্যা সংজ্ঞায়িত করা (Problem Definition)
 - |
 - v
২. ডেটা সংগ্রহ (Data Collection)
 - |
 - v
৩. এক্সপ্লোরেরি ডেটা অ্যানালিসিস (EDA)
 - |
 - v
৪. অপ্ৰয়োজনীয় কলাম বাদ দেওয়া (Drop Unnecessary Columns)
 - |
 - v



চিত্র ১.১: সম্পূর্ণ মেশিন লার্নিং ওয়ার্কফ্লো - কাঁচা ডেটা থেকে প্রোডাকশন পর্যন্ত। লক্ষ্য করুন, Train-Test Split (ধাপ ৬) ডেটা ক্লিনিংয়ের ধাপগুলোর (৭-১০) আগে ঘটছে।

লক্ষ্য করুন - মডেল ট্রেনিং (ধাপ ১৪) পুরো পাইপলাইনের প্রায় শেষ দিকে। ইন্ডাস্ট্রিতে দেখা যায়, একজন ডেটা সায়েন্টিস্ট তার সময়ের প্রায় ৭০-৮০% ব্যয় করেন ধাপ ২ থেকে ১৩ পর্যন্ত - অর্থাৎ ডেটা বোঝা, পরিষ্কার করা এবং প্রস্তুত করায়। মডেল ট্রেন করা তুলনামূলক সহজ এবং দ্রুত একটা ধাপ - কয়েক লাইনের কোড। আসল চ্যালেঞ্জ থাকে তার আগের অংশে, এবং এই বই সেই অংশটাকেই গভীরভাবে কভার করবে।

১.৪ সুপারভাইজড লার্নিং (Supervised Learning)

সুপারভাইজড লার্নিং হলো মেশিন লার্নিংয়ের সবচেয়ে জনপ্রিয় ও সবচেয়ে বেশি ব্যবহৃত শাখা। এখানে “supervised” শব্দটার মানে হলো - মডেলকে একজন শিক্ষকের মতো সঠিক উত্তর (লেবেল/টার্গেট) সহ ডেটা দেখানো হয়।

analogy:

ভাবুন একজন শিক্ষার্থী অংক পরীক্ষার জন্য পড়ছে, এবং তার কাছে একটা প্রশ্ন ও উত্তরসহ বই আছে (solved examples)। সে প্রতিটা প্রশ্ন সমাধান করে, তারপর বইয়ের দেওয়া সঠিক উত্তরের সাথে মিলিয়ে দেখে কোথায় ভুল হলো। এভাবে বারবার অনুশীলন করে সে শেখে। সুপারভাইজড লার্নিংয়ে মডেলও ঠিক এভাবেই শেখে - প্রতিটা ইনপুট (X) এর সাথে তার সঠিক আউটপুট (y) দেওয়া থাকে, মডেল ভুল করলে তা সংশোধন করে এগোয়।

সুপারভাইজড লার্নিং মূলত দুই ভাগে বিভক্ত:

- **রিগ্রেশন (Regression):** যখন আউটপুট একটা সংখ্যা (continuous value) - যেমন বাড়ির দাম, তাপমাত্রা, বেতন পূর্বাভাস।
- **ক্লাসিফিকেশন (Classification):** যখন আউটপুট একটা শ্রেণি বা ক্যাটাগরি (discrete label) - যেমন স্প্যাম/নট স্প্যাম, টাইটানিকে বেঁচে যাওয়া/না যাওয়া।

১.৫ আনসুপারভাইজড লার্নিং (Unsupervised Learning)

আনসুপারভাইজড লার্নিংয়ে কোনো সঠিক উত্তর বা লেবেল দেওয়া থাকে না। মডেলকে শুধু ডেটা দেওয়া হয়, আর মডেল নিজে থেকেই ডেটার মধ্যে লুকানো প্যাটার্ন, গঠন বা গ্রুপ খুঁজে বের করে।

analogy:

ধরুন আপনাকে একগাদা ফলের ছবি দেওয়া হলো, কিন্তু কোনো নাম লেখা নেই কোনটা কী ফল। আপনি কী করবেন? স্বাভাবিকভাবেই রঙ, আকার, আকৃতি দেখে ছবিগুলোকে কয়েকটা দলে ভাগ করবেন - লাল গোলাকার এক দলে, লম্বা হলুদ আরেক দলে। আপনি ফলের নাম না জেনেও তাদের মধ্যে মিল-অমিল খুঁজে বের করেছেন। এটাই আনসুপারভাইজড লার্নিং - ক্লাস্টারিং (clustering)। এছাড়া ডাইমেনশনালিটি রিডাকশন (যেমন PCA) দিয়ে অনেক ফিচার থেকে গুরুত্বপূর্ণ তথ্য বজায় রেখে ফিচার সংখ্যা কমানো হয়, যা ডেটা ভিজুয়ালাইজেশন ও পরবর্তী প্রসেসিং সহজ করে।

বিষয়	Supervised Learning	Unsupervised Learning
লেবেল/টার্গেট	আছে (y দেওয়া থাকে)	নেই
লক্ষ্য	ভবিষ্যদ্বাণী করা (predict)	প্যাটার্ন/গ্রুপ খুঁজে বের করা
উদাহরণ অ্যালগরিদম	Linear Regression, Logistic Regression, Random Forest	K-Means, Hierarchical Clustering, PCA
মূল্যায়ন	Accuracy, RMSE, F1-score ইত্যাদি দিয়ে সহজে মাপা যায়	মূল্যায়ন তুলনামূলক কঠিন, সরাসরি “সঠিক উত্তর” নেই
বাস্তব উদাহরণ	ইমেইল স্প্যাম শনাক্তকরণ, বাড়ির দাম পূর্বাভাস	কাস্টমার সেগমেন্টেশন, অ্যানোমালি ডিটেকশন

১.৬ রিগ্রেশন বনাম ক্লাসিফিকেশন

যেহেতু এই বইয়ের শেষ প্রজেক্টে (টাইটানিক ডেটাসেট) আমরা ক্লাসিফিকেশন সমস্যা সমাধান করব, তাই রিগ্রেশন ও ক্লাসিফিকেশনের পার্থক্য স্পষ্টভাবে বুঝে নেওয়া জরুরি।

বিষয়	Regression	Classification
আউটপুটের ধরন	Continuous (সংখ্যা)	Discrete (শ্রেণি/ক্যাটাগরি)
উদাহরণ	বাড়ির দাম, তাপমাত্রা, বিক্রয় পরিমাণ	Survived/Not Survived, Spam/Not Spam, রোগ আছে/নেই
মূল্যায়ন মেট্রিক	MAE, MSE, RMSE, R^2	Accuracy, Precision, Recall, F1-score, ROC-AUC
সাধারণ অ্যালগরিদম	Linear Regression, Ridge, Lasso	Logistic Regression, Decision Tree, SVM, KNN
আউটপুট রেঞ্জ	অসীম সংখ্যক মান হতে পারে	নির্দিষ্ট সংখ্যক ক্লাসের মধ্যে একটি

একটা গুরুত্বপূর্ণ বিষয় মনে রাখবেন - কখনো কখনো রিগ্রেশন ও ক্লাসিফিকেশনের সীমারেখা ঝাপসা মনে হতে পারে। যেমন “কাস্টমারের বয়স” যদি ১৮-২৫, ২৬-৩৫ এই রকম গ্রুপে ভাগ করে দেওয়া হয়, তাহলে এটা ক্লাসিফিকেশন সমস্যা হয়ে যায়, যদিও মূলত এটা একটা সংখ্যা (continuous) ছিল। সমস্যাটা কীভাবে সংজ্ঞায়িত করছেন তার ওপর নির্ভর করে এটা রিগ্রেশন বা ক্লাসিফিকেশন - যেকোনো একটা হতে পারে।

১.৭ ওভারফিটিং ও আন্ডারফিটিং

এটি মেশিন লার্নিংয়ের সবচেয়ে গুরুত্বপূর্ণ এবং সবচেয়ে বেশি ইন্টারভিউতে জিজ্ঞাসা করা কনসেপ্টগুলোর একটি।

ধাপ ১: ইনটুইশন ও বাস্তব উদাহরণ

ভাবুন দুজন শিক্ষার্থী পরীক্ষার জন্য পড়ছে।

- **শিক্ষার্থী A (Underfitting):** একেবারেই পড়াশোনা করেনি, শুধু কয়েকটা মূল সূত্র মুখস্থ করেছে। ফলে অনুশীলনী প্রশ্নেও খারাপ করে, পরীক্ষাতেও খারাপ করে। সে আসলে কিছুই ভালোভাবে শেখেনি - এটাই Underfitting, যেখানে মডেল ট্রেনিং ডেটাতেই ভালো পারফর্ম করতে পারে না।
- **শিক্ষার্থী B (Overfitting):** বইয়ের প্রতিটা প্রশ্ন-উত্তর পুরোপুরি মুখস্থ করে ফেলেছে, প্রতিটা শব্দ পর্যন্ত। অনুশীলনীতে (যা সে আগে দেখেছে) সে ১০০% নম্বর পায়। কিন্তু আসল পরীক্ষায় যখন একটু ভিন্নভাবে প্রশ্ন আসে, সে আটকে যায় - কারণ সে বুঝে শেখেনি, মুখস্থ করেছে। এটাই Overfitting, যেখানে মডেল ট্রেনিং ডেটায় চমৎকার করে কিন্তু নতুন (টেস্ট) ডেটায় খারাপ করে।
- **শিক্ষার্থী C (Good Fit):** মূল কনসেপ্ট বুঝে পড়েছে, প্র্যাকটিস করেছে কিন্তু মুখস্থ করেনি। অনুশীলনীতেও ভালো করে, আসল পরীক্ষায় নতুন প্রশ্নেও ভালো করে - কারণ সে প্যাটার্ন/যুক্তি বুঝেছে। এটাই আমাদের লক্ষ্য - একটা সুষম (well-generalized) মডেল।

ধাপ ২: কোডে দেখা - পলিনোমিয়াল রিগ্রেশনের উদাহরণ

নিচের কোডে আমরা $\sin(x)$ ফাংশনের কিছু নয়েজি ডেটা পয়েন্টে তিনটি ভিন্ন ডিগ্রির পলিনোমিয়াল মডেল ফিট করছি - দেখা যাক ফলাফল কেমন হয়:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
X = np.sort(np.random.uniform(0, 1, 25))
y_true = np.sin(2 * np.pi * X)
y = y_true + np.random.normal(0, 0.25, size=X.shape) # নয়েজ যোগ করা হলো

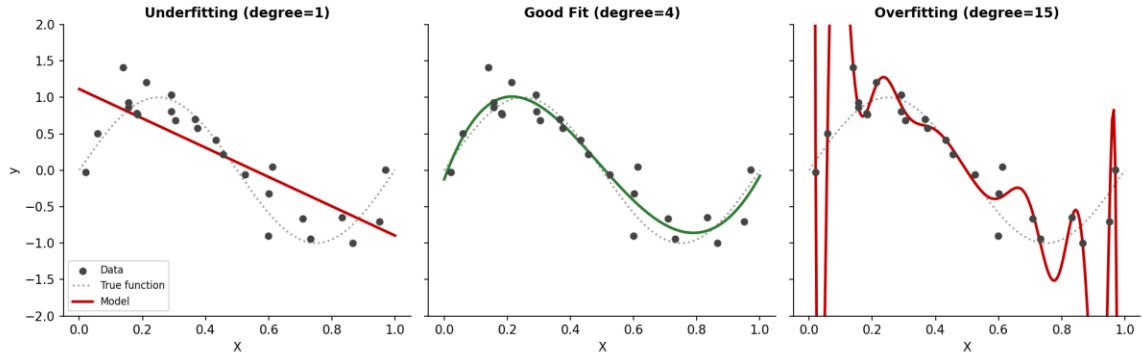
X_plot = np.linspace(0, 1, 200)
degrees = [1, 4, 15] # underfit, good fit, overfit

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
```

```

for ax, deg in zip(axes, degrees):
    coefs = np.polyfit(X, y, deg)          # deg ডিগ্রির পলিনোমিয়াল ফিট
    y_plot = np.polyval(coefs, X_plot)    # প্রেডিকশন
    ax.scatter(X, y)                      # আসল ডেটা পয়েন্ট
    ax.plot(X_plot, y_plot, color='red')  # মডেলের প্রেডিকশন লাইন
    ax.set_title(f'Degree = {deg}')
plt.show()

```



চিত্র ১.২: degree=1 এ আন্ডারফিটিং (সরলরেখা প্যাটার্ন ধরতে পারছে না), degree=4 এ ভালো ফিট (প্যাটার্ন ধরেছে, নয়েজ উপেক্ষা করেছে), degree=15 এ ওভারফিটিং (প্রতিটা নয়েজ পয়েন্ট পর্যন্ত মুখস্থ করে ফেলেছে)

ধাপ ৩: কীভাবে বুঝবেন কোনটা হচ্ছে?

ট্রেনিং এরর ও টেস্ট/ভ্যালিডেশন এরর তুলনা করেই এটা বোঝা যায়:

অবস্থা	Training Error	Test Error	ব্যাখ্যা
Underfitting	বেশি (High)	বেশি (High)	মডেল প্যাটার্নই ধরতে পারেনি
Good Fit	কম (Low)	কম, কাছাকাছি	মডেল ভালোভাবে জেনারেলাইজ করেছে
Overfitting	খুব কম (Very Low)	বেশি (High)	মডেল ডেটা মুখস্থ করেছে, নতুন ডেটায় খারাপ

ধাপ ৪: সমাধানের উপায়

- **Underfitting সমাধানে:** মডেলের জটিলতা বাড়ানো (বেশি ফিচার, উন্নত অ্যালগরিদম), ট্রেনিং সময় বাড়ানো, রেগুলারাইজেশন কমানো।
- **Overfitting সমাধানে:** আরো ডেটা সংগ্রহ করা, ফিচার সংখ্যা কমানো (feature selection), রেগুলারাইজেশন (L1/L2) ব্যবহার করা, ক্রস ভ্যালিডেশন করা, early stopping, ড্রপআউট (নিউরাল নেটওয়ার্কে), মডেলের জটিলতা কমানো।

১.৮ বায়াস ও ভ্যারিয়েন্স (Bias-Variance Tradeoff)

ওভারফিটিং-আন্ডারফিটিংয়ের পেছনের গাণিতিক ব্যাখ্যাটাই হলো বায়াস-ভ্যারিয়েন্স ট্রেডঅফ।

- **বায়াস (Bias):** মডেল বাস্তব সম্পর্ককে কতটা সরলীকৃত (oversimplify) করছে তার পরিমাপ। উচ্চ বায়াস মানে মডেল খুব সরল অনুমান করছে এবং আসল প্যাটার্ন ধরতে পারছে না (Underfitting এর কারণ)।
- **ভ্যারিয়েন্স (Variance):** ট্রেনিং ডেটার ছোট পরিবর্তনে মডেলের প্রেডিকশন কতটা পরিবর্তিত হয় তার পরিমাপ। উচ্চ ভ্যারিয়েন্স মানে মডেল ডেটার নয়েজ পর্যন্ত শিখে ফেলেছে (Overfitting এর কারণ)।

গাণিতিক রূপ (Expected Test Error এর বিভাজন):

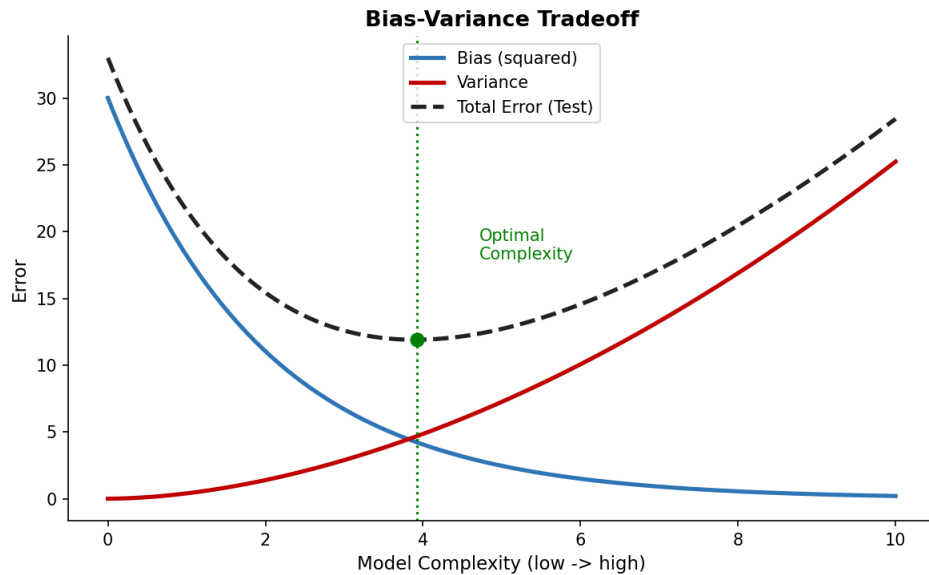
$$\text{Total Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

যেখানে:

$$\text{Bias}^2 = (\text{মডেলের গড় প্রেডিকশন} - \text{আসল মান})^2$$

$$\text{Variance} = \text{মডেল বিভিন্ন ট্রেনিং সেটে কতটা ওঠানামা করে}$$

$$\text{Irreducible Error} = \text{ডেটার মধ্যে থাকা স্বাভাবিক নয়েজ, যা কোনো মডেল দিয়েই দূর করা যায় না}$$



চিত্র ১.৩: মডেলের জটিলতা বাড়ার সাথে সাথে বায়াস কমে কিন্তু ভ্যারিয়েন্স বাড়ে। সবচেয়ে কম টেস্ট এরর পাওয়া যায় একটা “সুইট স্পট”-এ, যেখানে বায়াস ও ভ্যারিয়েন্সের সমষ্টি সর্বনিম্ন।

মডেলের ধরন	Bias	Variance	ফলাফল
খুব সরল মডেল (যেমন: degree=1 লাইন)	High	Low	Underfitting
সুষম জটিলতার মডেল	Moderate	Moderate	Good Fit / Generalization
খুব জটিল মডেল (যেমন: degree=15)	Low	High	Overfitting

এজন্যই একে “tradeoff” বলা হয় - বায়াস কমাতে গেলে সাধারণত ভ্যারিয়েন্স বাড়ে, আর ভ্যারিয়েন্স কমাতে গেলে বায়াস বাড়ে। মেশিন লার্নিং ইঞ্জিনিয়ারের কাজ হলো এই দুইয়ের মধ্যে একটা ভারসাম্য খুঁজে বের করা, যাতে মোট এরর সর্বনিম্ন হয়।

১.৯ ট্রেন, ভ্যালিডেশন এবং টেস্ট ডেটা

মডেলের জেনারেলাইজেশন ক্ষমতা সঠিকভাবে যাচাই করার জন্য পুরো ডেটাসেটকে সাধারণত তিনটি ভাগে ভাগ করা হয়:

- **ট্রেনিং সেট (Training Set):** মডেল এই ডেটা দেখে শেখে (fit করে)। সাধারণত মোট ডেটার ৬০-৮০%।
- **ভ্যালিডেশন সেট (Validation Set):** মডেল ট্রেন করার সময় হাইপারপ্যারামিটার টিউন করতে এবং কোন মডেল ভালো তা তুলনা করতে ব্যবহৃত হয়। মডেল এই ডেটা থেকে সরাসরি শেখে না, কিন্তু পরোক্ষভাবে এর প্রভাব মডেল সিলেকশনে পড়ে। সাধারণত ১০-২০%।
- **টেস্ট সেট (Test Set):** চূড়ান্ত মূল্যায়নের জন্য সংরক্ষিত - এই ডেটা মডেল প্রশিক্ষণের কোনো পর্যায়েই “দেখে না”। এটাই বাস্তব জগতে নতুন, অদেখা ডেটার প্রতিনিধিত্ব করে। সাধারণত ১০-২০%।

কেন আগে স্প্লিট করতে হয়? (Data Leakage থেকে বাঁচতে)

একটা সবচেয়ে গুরুত্বপূর্ণ এবং সবচেয়ে বেশি করা ভুল হলো - প্রথমে পুরো ডেটাসেটে EDA, মিসিং ভ্যালু ইম্পিউটেশন, স্কেলিং ইত্যাদি করে ফেলা, তারপর ট্রেন-টেস্ট স্প্লিট করা। এতে টেস্ট ডেটার তথ্য (যেমন তার mean, std, distribution) অজান্তেই ট্রেনিং প্রক্রিয়ায় “ফাঁস” (leak) হয়ে যায়, ফলে মডেলের পারফরম্যান্স বাস্তবের চেয়ে ভালো দেখায় - কিন্তু প্রোডাকশনে গিয়ে ভেঙে পড়ে। এই বইয়ের পরবর্তী প্রতিটা অধ্যায়েই (Missing Value, Outlier, Encoding, Scaling) এই নিয়মটা বারবার গুরুত্ব দিয়ে আলোচনা করা হবে - সবসময় আগে split, তারপর fit।

```

from sklearn.model_selection import train_test_split

X = df.drop('Survived', axis=1)
y = df['Survived']

# ধাপ ১: প্রথমে train ও temp (validation+test) আলাদা করা
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# ধাপ ২: temp থেকে validation ও test আলাদা করা
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)

print(f'Train: {X_train.shape[0]} samples') # ~70%
print(f'Validation: {X_val.shape[0]} samples') # ~15%
print(f'Test: {X_test.shape[0]} samples') # ~15%

```

লক্ষ্য করুন stratify=y প্যারামিটারটি - এটি নিশ্চিত করে যে ট্রেন, ভ্যালিডেশন ও টেস্ট প্রতিটা সেটে টার্গেট ক্লাসের অনুপাত (যেমন Survived=0 vs Survived=1) প্রায় একই থাকে। ক্লাস ইমব্যালেন্স থাকা ডেটাসেটে (যেমন টাইটানিক, যেখানে বেঁচে যাওয়া আর না যাওয়ার সংখ্যা সমান নয়) এটি অত্যন্ত গুরুত্বপূর্ণ।

সোনালী নিয়ম: fit_transform() শুধু X_train-এ, transform() শুধু X_test-এ

স্প্লিট করার পর Missing Value Imputer, Encoder, Scaler - যেকোনো ট্রান্সফর্মার ব্যবহারের সময় এই নিয়মটা সবসময় মাথায় রাখবেন: ট্রান্সফর্মার শুধু X_train থেকে শেখা (mean, mode, min-max ইত্যাদি), এবং সেই শেখা জ্ঞান X_train ও X_test উভয়ের ওপর প্রয়োগ করা হয় - কখনোই X_test থেকে আলাদা করে শেখা হয় না। এই বইয়ের প্রতিটা পরবর্তী অধ্যায়ে (Missing Value, Outlier, Encoding, Scaling) এই একই প্যাটার্ন বারবার ব্যবহৃত হবে।

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# X_train-এ fit_transform: প্রথমে train থেকে mean/std শেখা (fit),
# তারপর সাথে সাথে সেই train ডেটাকেই transform করা
X_train_scaled = scaler.fit_transform(X_train)

# X_test-এ শুধু transform: train থেকে শেখা mean/std প্রয়োগ করা হচ্ছে,
# নতুন করে fit করা হচ্ছে না (fit_transform নয়!)

```

```
X_test_scaled = scaler.transform(X_test)
```

```
# ❌ ভুল - X_test-এ কখনো fit() বা fit_transform() করবেন না:
```

```
# X_test_scaled = scaler.fit_transform(X_test) # এটি Data Leakage!
```

স্প্লিট	সাধারণ অনুপাত	উদ্দেশ্য
Train	৬০-৮০%	মডেল প্যারামিটার শেখা (fit)
Validation	১০-২০%	হাইপারপ্যারামিটার টিউনিং, মডেল সিলেকশন
Test	১০-২০%	চূড়ান্ত, নিরপেক্ষ মূল্যায়ন (একবারই ব্যবহার করা উচিত)

ছোট ডেটাসেটে (যেমন কয়েকশ' স্যাম্পল) আলাদা ভ্যালিডেশন সেট রাখলে ট্রেনিং ডেটা অনেক কমে যেতে পারে। এক্ষেত্রে ভ্যালিডেশন সেটের পরিবর্তে K-Fold Cross Validation ব্যবহার করা ভালো অভ্যাস (এই বইয়ের Cross Validation অধ্যায়ে বিস্তারিত আলোচনা করা হবে)।

১.১০ সাধারণ ভুল (Common Mistakes)

- স্প্লিট করার আগে পুরো ডেটাসেটে স্কেলিং/ইম্পিউটেশন করে ফেলা (Data Leakage)।
- টেস্ট সেট বারবার দেখে দেখে মডেল টিউন করা - এতে টেস্ট সেটও কার্যত “ট্রেনিং ডেটার অংশ” হয়ে যায়।
- শুধুমাত্র ট্রেনিং accuracy দেখে মডেলকে “ভালো” বলে সিদ্ধান্ত নেওয়া, টেস্ট পারফরম্যান্স যাচাই না করা।
- ইমব্যালান্সড ডেটাসেটে stratify ছাড়া random split করা।
- random_state সেট না করায় প্রতিবার ভিন্ন ভিন্ন রেজাল্ট পাওয়া এবং তুলনা করতে না পারা।
- মডেলকে অতিরিক্ত জটিল করে ফেলা, যখন সহজ মডেলই যথেষ্ট ছিল (Occam's Razor উপেক্ষা করা)।

১.১০.১ Data Leakage-এর বাস্তব পরিণতি: Accuracy কেন মিথ্যা ভালো দেখায়?

Data Leakage-এর সবচেয়ে বিপজ্জনক দিক হলো এটি মডেলকে বাস্তবের চেয়ে অনেক বেশি ভালো দেখাতে পারে। Training-এর সময় যদি Test Data-এর তথ্য (mean, standard deviation,

minimum, maximum, category frequency ইত্যাদি) ভুলবশত ব্যবহার হয়ে যায়, তাহলে Test Set আর প্রকৃতপক্ষে "অদেখা" থাকে না।

ফলাফল:

- Validation/Test Accuracy কৃত্রিমভাবে বেড়ে যায়।
- Cross Validation Result বাস্তবের চেয়ে ভালো দেখায়।
- Production-এ Deploy করার পরে Performance হঠাৎ কমে যায়।
- Model Selection ভুল হতে পারে।

নিচের উদাহরণে আমরা একই Dataset-এ দুটি পদ্ধতি ব্যবহার করব:

ভুল পদ্ধতি (Leakage):

- প্রথমে পুরো Dataset Scale করা
- তারপর Train-Test Split করা

সঠিক পদ্ধতি (No Leakage):

- প্রথমে Train-Test Split করা
- তারপর শুধুমাত্র Train Data থেকে Scaler শেখানো

Code:

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# reproducibility নিশ্চিত করার জন্য
np.random.seed(42)

# classification dataset তৈরি
X, y = make_classification(
    n_samples=5000,
    n_features=20,
    n_informative=10,
    random_state=42
)
```

```

# =====
# WRONG APPROACH (Data Leakage)
# =====

# পুরো dataset থেকে mean বের করা (ভুল)
global_mean = X.mean(axis=0)

# পুরো dataset থেকে std বের করা (ভুল)
global_std = X.std(axis=0)

# পুরো dataset scale করা (ভুল)
X_scaled_wrong = (X - global_mean) / global_std

# scale করার পরে split করা (ভুল)
X_train_w, X_test_w, y_train_w, y_test_w = train_test_split(
    X_scaled_wrong,
    y,
    test_size=0.2,
    random_state=42
)

# model train
model_wrong = LogisticRegression(max_iter=500)

model_wrong.fit(X_train_w, y_train_w)

# prediction
y_pred_wrong = model_wrong.predict(X_test_w)

# accuracy
acc_wrong = accuracy_score(y_test_w, y_pred_wrong)

print("Wrong Accuracy:", acc_wrong)

# =====
# CORRECT APPROACH (No Leakage)
# =====

# প্রথমে split করা
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

```

```

)

# শুধুমাত্র train data থেকে mean শেখা
train_mean = X_train.mean(axis=0)

# শুধুমাত্র train data থেকে std শেখা
train_std = X_train.std(axis=0)

# train scale করা
X_train_scaled = (X_train - train_mean) / train_std

# test scale করা (train statistics ব্যবহার করে)
X_test_scaled = (X_test - train_mean) / train_std

# model train
model_correct = LogisticRegression(max_iter=500)

model_correct.fit(X_train_scaled, y_train)

# prediction
y_pred_correct = model_correct.predict(X_test_scaled)

# accuracy
acc_correct = accuracy_score(y_test, y_pred_correct)

print("Correct Accuracy:", acc_correct)

# leakage এর প্রভাব
print("Artificial Gain:", acc_wrong - acc_correct)

```

scikit-learn Code (Pipeline/ColumnTransformer)

```

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score

# dataset তৈরি
X, y = make_classification(
    n_samples=5000,
    n_features=20,

```

```

        n_informative=10,
        random_state=42
    )

# =====
# WRONG APPROACH
# =====

# পুরো dataset এ scaler fit (ভুল)
scaler = StandardScaler()

X_scaled_wrong = scaler.fit_transform(X)

# পরে split (ভুল)
X_train_w, X_test_w, y_train_w, y_test_w = train_test_split(
    X_scaled_wrong,
    y,
    test_size=0.2,
    random_state=42
)

wrong_model = LogisticRegression(max_iter=500)

wrong_model.fit(X_train_w, y_train_w)

wrong_pred = wrong_model.predict(X_test_w)

wrong_acc = accuracy_score(y_test_w, wrong_pred)

print("Wrong Accuracy:", wrong_acc)

# =====
# CORRECT APPROACH
# =====

# প্রথমে split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# pipeline তৈরি
pipeline = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),
    (
        "model",

```



```

        LogisticRegression(max_iter=500)
    )
])

# scaler শুধুমাত্র train data থেকে শিখবে
pipeline.fit(X_train, y_train)

# prediction
correct_pred = pipeline.predict(X_test)

# accuracy
correct_acc = accuracy_score(y_test, correct_pred)

print("Correct Accuracy:", correct_acc)

print("Artificial Gain:", wrong_acc - correct_acc)

```

কেন Pipeline ব্যবহার করা উচিত?

Pipeline-এর সবচেয়ে বড় সুবিধা হলো:

- StandardScaler শুধুমাত্র Training Data-তে fit হয়।
- Test Data কখনো fit হয় না।
- Cross Validation-এর সময়ও Leakage হয় না।
- Production Code অনেক বেশি নিরাপদ হয়।

১.১১ বেস্ট প্র্যাকটিস (Best Practices)

- যেকোনো ডেটা প্রসেসিং (impute, scale, encode) শুরুর আগেই train-test split করুন।
- সবসময় random_state ফিক্সড রাখুন, যাতে রেজাল্ট রিপ্ৰোডিউসিবল হয়।
- ইমব্যালান্সড ক্লাসিফিকেশন সমস্যায় stratify=y ব্যবহার করুন।
- টেস্ট সেটকে “ভল্ট” হিসেবে ভাবুন - একদম শেষে, একবার মাত্র খুলবেন।
- মডেলের জটিলতা ডেটার পরিমাণ ও সমস্যার জটিলতার সাথে সামঞ্জস্যপূর্ণ রাখুন - সবচেয়ে জটিল মডেলই সবসময় সেরা নয়।
- Train ও Validation/Test এররের মধ্যে ব্যবধান (gap) নিয়মিত পর্যবেক্ষণ করুন - এই ব্যবধানই ওভারফিটিং এর সবচেয়ে নির্ভরযোগ্য সংকেত।

১.১২ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Overfitting এবং Underfitting এর মধ্যে পার্থক্য কী?

উত্তর: Underfitting-এ মডেল ট্রেনিং ও টেস্ট উভয় ডেটাসেটেই খারাপ পারফরম করে কারণ এটি প্যাটার্ন ধরতে পারে না (High Bias)। Overfitting-এ মডেল ট্রেনিং ডেটায় চমৎকার করে কিন্তু টেস্ট ডেটায় খারাপ করে কারণ এটি নয়েজ পর্যন্ত মুখস্থ করে ফেলে (High Variance)।

প্রশ্ন: কেন আমরা ভ্যালিডেশন সেট আলাদা রাখি, যখন টেস্ট সেট আছেই?

উত্তর: ভ্যালিডেশন সেট ব্যবহার করা হয় হাইপারপ্যারামিটার টিউনিং ও মডেল সিলেকশনের জন্য, যাতে টেস্ট সেট পুরোপুরি “অদেখা” থেকে যায় এবং চূড়ান্ত মূল্যায়ন নিরপেক্ষ হয়। যদি টিউনিং-এর জন্য টেস্ট সেট ব্যবহার করা হয়, তাহলে টেস্ট সেটের তথ্য পরোক্ষভাবে মডেলে ফাঁস হয়ে যায়।

প্রশ্ন: Data Leakage কী এবং এটা কীভাবে এড়ানো যায়?

উত্তর: Data Leakage ঘটে যখন টেস্ট/ভ্যালিডেশন ডেটার তথ্য, প্রত্যক্ষ বা পরোক্ষভাবে, ট্রেনিং প্রক্রিয়ায় ব্যবহৃত হয়ে যায় - যেমন পুরো ডেটাসেট থেকে mean/std বের করে স্কেলিং করা, তারপর split করা। এটি এড়াতে সবসময় প্রথমে split করতে হবে, এবং যেকোনো ট্রান্সফর্মার শুধু ট্রেনিং সেটে fit করতে হবে, টেস্ট সেটে শুধু transform করতে হবে।

প্রশ্ন: Bias-Variance Tradeoff কীভাবে মডেল সিলেকশনে সাহায্য করে?

উত্তর: এটি বোঝায় কেন সবচেয়ে জটিল মডেলই সবসময় সেরা নয়। মডেলের জটিলতা বাড়ালে বায়াস কমে কিন্তু ভ্যারিয়েন্স বাড়ে। সর্বোত্তম মডেল সেটাই, যেখানে এই দুইয়ের সমষ্টি (অর্থাৎ মোট টেস্ট এরর) সর্বনিম্ন হয় - এটি লার্নিং কার্ভ ও ক্রস-ভ্যালিডেশনের মাধ্যমে অভিজ্ঞতামূলকভাবে খুঁজে বের করা হয়।

প্রশ্ন: Regression ও Classification সমস্যা কীভাবে আলাদা করবেন?

উত্তর: টার্গেট ভ্যারিয়েবলের ধরন দেখে। যদি টার্গেট একটি continuous সংখ্যা হয় (যেমন দাম, তাপমাত্রা), সেটি Regression। যদি টার্গেট একটি নির্দিষ্ট সংখ্যক শ্রেণির মধ্যে একটি হয় (যেমন Yes/No, একাধিক ক্যাটাগরি), সেটি Classification।

১.১৩ অধ্যায় সারসংক্ষেপ

- মেশিন লার্নিং হলো ডেটা থেকে প্যাটার্ন শেখার পদ্ধতি, এক্সপ্লিসিট নিয়ম লেখার বদলে।
- সম্পূর্ণ ML পাইপলাইনে মডেল ট্রেনিং একটা ছোট অংশ মাত্র - বেশিরভাগ সময় যায় ডেটা প্রস্তুতিতে, যা এই বইয়ের মূল বিষয়।
- Supervised Learning-এ লেবেলসহ ডেটা থাকে (Regression ও Classification); Unsupervised Learning-এ লেবেল ছাড়াই প্যাটার্ন খুঁজে বের করা হয়।
- Underfitting = High Bias (খুব সরল মডেল); Overfitting = High Variance (খুব জটিল মডেল)। লক্ষ্য একটা সুষম মডেল।
- ডেটাকে সবসময় কোনো প্রসেসিং করার আগেই Train/Validation/Test এ ভাগ করে নিতে হবে, যাতে Data Leakage না ঘটে।

অধ্যায় ২: ডেটাসেট বোঝা

২.১ ভূমিকা: কেন ডেটাসেট বোঝা গুরুত্বপূর্ণ

অধ্যায় ১-এ আমরা দেখেছি, ML পাইপলাইনের প্রতিটা ধাপ - মিসিং ভ্যালু হ্যান্ডলিং, এনকোডিং, স্কেলিং - সবকিছুর সিদ্ধান্ত নিৰ্ভর করে একটা প্রশ্নের ওপর: “এই কলামটা আসলে কী ধরনের ডেটা?” একটা কলাম যদি সংখ্যা (numerical) হয়, সেখানে mean দিয়ে মিসিং ভ্যালু পূরণ করা যায়; কিন্তু সেটা যদি ক্যাটাগরি (categorical) হয়, mean বের করার কোনো অর্থই হয় না - সেখানে mode ব্যবহার করতে হয়। তাই মডেল ট্রেনিং তো দূরের কথা, কোনো ডেটা ক্লিনিংয়ের আগেও একটা ডেটাসেটের প্রতিটা কলাম গঠনগতভাবে বুঝে নেওয়া অপরিহার্য।

এই অধ্যায়ে আমরা একটা ডেটাসেটকে “ভেঙে” দেখব - রো-কলাম থেকে শুরু করে ফিচারের বিভিন্ন প্রকারভেদ পর্যন্ত।

২.২ রো ও কলাম (Rows and Columns)

একটা ট্যাবুলার ডেটাসেট (যেমন CSV ফাইল বা pandas DataFrame) মূলত একটা গ্রিড বা টেবিল, যা সাজানো থাকে রো এবং কলামে।

- **রো (Row):** একে “স্যাম্পল”, “রেকর্ড” বা “অবজারভেশন”ও বলা হয়। প্রতিটা রো একটা একক ঘটনা বা সত্তা বোঝায় - টাইটানিক ডেটাসেটে প্রতিটা রো মানে একজন যাত্রী।
- **কলাম (Column):** একে “ফিচার”, “ভেরিয়েবল” বা “অ্যাট্রিবিউট”ও বলা হয়। প্রতিটা কলাম একটা নির্দিষ্ট বৈশিষ্ট্য পরিমাপ করে - যেমন বয়স, লিঙ্গ, টিকেটের দাম।

analogy:

একটা স্কুলের ছাত্র-ছাত্রীদের রেজিস্টার খাতার কথা ভাবুন। প্রতিটা সারি (row) একজন ছাত্র-ছাত্রীর তথ্য - নাম, রোল, বয়স, ক্লাস। প্রতিটা কলাম একটা নির্দিষ্ট তথ্যের ধরন - সব ছাত্র-ছাত্রীর নাম এক কলামে, সবার বয়স আরেক কলামে। ডেটাসেটও ঠিক এভাবেই সংগঠিত।

```
import pandas as pd

df = pd.read_csv('titanic_data_updated.csv')

df.shape          # (891, 12) -> ৮৯১টি রো (যাত্রী), ১২টি কলাম (ফিচার)
df.shape[0]       # মোট রো সংখ্যা -> 891
df.shape[1]       # মোট কলাম সংখ্যা -> 12
```

```
df.columns      # সব কলামের নাম দেখায়
df.head(3)      # প্রথম ৩টি রো দেখায়
```

২.৩ ফিচার ও টার্গেট (Features and Target)

সুপারভাইজড লার্নিং সমস্যায় কলামগুলোকে দুই ভাগে ভাগ করা হয়:

- **ফিচার (Features / X):** যেসব কলাম দিয়ে মডেল পূর্বাভাস দেবে - এগুলো ইনপুট। টাইটানিক ডেটাসেটে Pclass, Sex, Age, Fare ইত্যাদি ফিচার।
- **টার্গেট (Target / Label / y):** যে কলামটা মডেল পূর্বাভাস দেওয়ার চেষ্টা করবে - এটা আউটপুট। টাইটানিক ডেটাসেটে Survived (বেঁচে গেছে কি না) হলো টার্গেট।

একটা গুরুত্বপূর্ণ নিয়ম মনে রাখবেন - এই বিভাজনটা ট্রেন-টেস্ট স্প্লিটের আগেই করতে হয় (অধ্যায় ১-এর পাইপলাইনে ধাপ ৫ লক্ষ্য করুন), কারণ স্প্লিট করার সময় X ও y আলাদাভাবে ভাগ হয়, যাতে প্রতিটা রোয়ের ফিচার ও তার সঠিক টার্গেট একসাথে থাকে।

```
# অপয়োজনীয় কলাম বাদ দেওয়া (ধাপ ৪)
df.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)

# Feature ও Target আলাদা করা (ধাপ ৫)
X = df.drop('Survived', axis=1) # Survived ছাড়া বাকি সব কলাম -> ফিচার
y = df['Survived']              # শুধু Survived কলাম -> টার্গেট

print(X.shape) # (891, 8) -> ৪টি ফিচার
print(y.shape) # (891,) -> ১টি টার্গেট ভেরিয়েবল
```

২.৪ নিউমেরিক্যাল বনাম ক্যাটাগরিক্যাল ফিচার

প্রতিটা ফিচারকে মোটাদাগে দুই ভাগে ভাগ করা যায়, যা নির্ভর করে সেটি কী ধরনের মান ধারণ করছে তার ওপর।

- **নিউমেরিক্যাল (Numerical) ফিচার:** যেসব ফিচারের মান সংখ্যা, এবং সেই সংখ্যা দিয়ে গাণিতিক হিসাব (যোগ, গড়, তুলনা) করা অর্থবহ। যেমন Age (বয়স), Fare (ভাড়া)।
- **ক্যাটাগরিক্যাল (Categorical) ফিচার:** যেসব ফিচার একটা নির্দিষ্ট সংখ্যক শ্রেণি বা গ্রুপের মধ্যে একটি প্রতিনিধিত্ব করে, যেখানে সংখ্যা দিয়ে গাণিতিক হিসাব করা অর্থহীন। যেমন Sex (Male/Female), Embarked (S/C/Q)।

একটা সহজ পরীক্ষা: যদি দুটো মানের গড় (average) বের করা অর্থবহ হয়, সেটা নিউমেরিক্যাল। যেমন বয়স ২২ এবং ৩৮-এর গড় ৩০ একটা অর্থবহ সংখ্যা। কিন্তু Pclass-এর মান ১ আর ৩-এর “গড়” ২ - এটা আসলে অর্থহীন, কারণ Pclass একটা ক্যাটাগরি (যদিও সংখ্যা দিয়ে লেখা)।

```
df.dtypes
# PassengerId      int64
# Survived         int64    <- আসলে categorical (0/1), যদিও dtype int64
# Pclass           int64    <- আসলে categorical (1/2/3), যদিও dtype int64
# Sex              object    <- categorical
# Age              float64   <- numerical
# Fare             float64   <- numerical
# Embarked         object    <- categorical

# numerical ও categorical কলাম আলাদা করা
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df.select_dtypes(include=['object']).columns
print('Numerical:', list(num_cols))
print('Categorical:', list(cat_cols))
```

সতর্কতা: pandas-এর dtype (int64, float64, object) সবসময় ফিচারের আসল প্রকৃতি সঠিকভাবে বলে না। Survived ও Pclass সংখ্যা দিয়ে লেখা হলেও এরা প্রকৃতপক্ষে categorical। তাই শুধু dtype-এর ওপর ভরসা না করে, প্রতিটা কলামের প্রকৃত অর্থ বুঝে সিদ্ধান্ত নিতে হবে।

২.৫ কন্টিনিউয়াস বনাম ডিসক্রিট ভেরিয়েবল

নিউমেরিক্যাল ফিচারকেও আরো দুই ভাগে ভাগ করা যায়:

- **কন্টিনিউয়াস (Continuous):** দুটি মানের মধ্যে অসীম সংখ্যক মধ্যবর্তী মান থাকতে পারে। যেমন Age (২২.৫ বছর হতে পারে), Fare (১০৫.৩৭ ডলার হতে পারে), উচ্চতা, ওজন, তাপমাত্রা।
- **ডিসক্রিট (Discrete):** শুধু নির্দিষ্ট, গণনাযোগ্য (countable) মান নিতে পারে, সাধারণত পূর্ণসংখ্যা। যেমন SibSp (ভাইবোন/স্বামী-স্ত্রীর সংখ্যা: 0, 1, 2...), পরিবারের সদস্য সংখ্যা, একটা বাড়ির রুম সংখ্যা।

analogy:

একটা গ্লাসে পানি ঢালার কথা ভাবুন - পানি ১০০ মিলিলিটার, ১০০.৫ মিলিলিটার, ১০০.৫৭ মিলিলিটার - যেকোনো মান হতে পারে (continuous)। কিন্তু গ্লাসের সংখ্যা গুনতে গেলে আপনি ১টা, ২টা, ৩টা গ্লাস বলবেন - কখনো ২.৫টা গ্লাস বলবেন না (discrete)।

২.৬ নমিনাল বনাম অর্ডিনাল ভেরিয়েবল

একইভাবে, ক্যাটাগরিক্যাল ফিচারকেও দুই ভাগে ভাগ করা যায় - এবং এই পার্থক্যটা পরবর্তী অধ্যায়ে এনকোডিং পদ্ধতি বাছাইয়ের জন্য অত্যন্ত গুরুত্বপূর্ণ হবে।

- **নমিনাল (Nominal):** ক্যাটাগরিগুলোর মধ্যে কোনো স্বাভাবিক ক্রম (order) নেই। যেমন Sex (Male, Female), Embarked (S, C, Q), রঙ (লাল, নীল, সবুজ)। “Male” কে “Female” এর চেয়ে “বড়” বা “ছোট” বলা যায় না।
- **অর্ডিনাল (Ordinal):** ক্যাটাগরিগুলোর মধ্যে একটা স্পষ্ট, স্বাভাবিক ক্রম আছে। যেমন Education (SSC < HSC < Bachelor < Masters), Pclass (1st class > 2nd class > 3rd class মর্যাদার দিক থেকে), Performance (Poor < Average < Good < Excellent)।

কেন এই পার্থক্যটা গুরুত্বপূর্ণ? কারণ অর্ডিনাল ডেটার ক্রমটা সংখ্যায় রূপান্তর করার সময় (Ordinal Encoding দিয়ে) সংরক্ষণ করা উচিত - SSC=0, HSC=1, Bachelor=2, Masters=3, যাতে ক্রমটা বজায় থাকে। কিন্তু নমিনাল ডেটাকে এভাবে সংখ্যায় রূপান্তর করলে (যেমন Red=0, Blue=1, Green=2) মডেল ভুলভাবে ধরে নিতে পারে যে Green, Red-এর চেয়ে “বড়” - যেটা ভুল। নমিনাল ডেটার জন্য তাই One-Hot Encoding ব্যবহার করা হয় (অধ্যায় ১০-এ বিস্তারিত)।

ভেরিয়েবলের প্রকারভেদ:

```

Feature
|
+-- Numerical (সংখ্যা, গাণিতিক হিসাব অর্থবহ)
|   |
|   +-- Continuous (Age, Fare, উচ্চতা)
|   +-- Discrete (SibSp, Parch, পরিবার সদস্য সংখ্যা)
|
+-- Categorical (নির্দিষ্ট সংখ্যক শ্রেণি)
    |
    +-- Nominal (Sex, Embarked, রঙ) - কোনো ক্রম নেই
    +-- Ordinal (Pclass, Education, Performance) - স্বাভাবিক ক্রম আছে
  
```

চিত্র ২.১: ফিচার ভেরিয়েবলের সম্পূর্ণ শ্রেণিবিন্যাস

টাইটানিক কলাম	প্রকার	ব্যাখ্যা
Age	Numerical - Continuous	যেকোনো দশমিক মান হতে পারে (২৯.৫ বছর)
Fare	Numerical - Continuous	যেকোনো দশমিক মান হতে পারে
SibSp	Numerical - Discrete	শুধু পূর্ণসংখ্যা (0, 1, 2...)

টাইটানিক কলাম	প্রকার	ব্যাখ্যা
Sex	Categorical - Nominal	Male/Female-এর মধ্যে কোনো ক্রম নেই
Embarked	Categorical - Nominal	S/C/Q বন্দরের মধ্যে কোনো স্বাভাবিক ক্রম নেই
Pclass	Categorical - Ordinal	1st > 2nd > 3rd - একটা স্পষ্ট ক্রম আছে
Survived	Categorical - Nominal (Binary)	0/1, দুটি ক্লাসের মধ্যে কোনো ক্রম নেই

২.৭ কোডে ডেটা টাইপ যাচাই করা

একটা নতুন ডেটাসেট পেলে, যেকোনো প্রসেসিং শুরুর আগে নিচের কোডগুলো দিয়ে প্রতিটা কলামের প্রকৃতি যাচাই করে নেওয়া একটা ভালো অভ্যাস:

```
# প্রতিটা কলামের ইউনিক মান সংখ্যা - ক্যাটাগরিক্যাল চেনার একটা সহজ উপায়
df.nunique()

# Survived      2      <- খুব কম ইউনিক মান, সম্ভবত categorical
# Pclass        3      <- খুব কম ইউনিক মান, সম্ভবত categorical (ordinal)
# Sex           2      <- categorical (nominal)
# Age          88      <- অনেক ইউনিক মান, সম্ভবত continuous numerical
# Fare         248     <- অনেক ইউনিক মান, সম্ভবত continuous numerical

# নির্দিষ্ট কলামের ইউনিক ক্যাটাগরিগুলো দেখা
df['Embarked'].unique()      # array(['S', 'C', 'Q', nan], dtype=object)
df['Pclass'].unique()        # array([3, 1, 2])

# প্রতিটা ক্যাটাগরির ফ্রিকোয়েন্সি গণনা
df['Sex'].value_counts()
# male         577
# female       314

# একটা সাধারণ নিয়ম (rule of thumb, পুরোপুরি নির্ভরযোগ্য নয়):
# nunique() < ~১৫-২০ এবং dtype int/object হলে -> probably categorical
for col in df.columns:
    print(f'{col}: {df[col].nunique()} unique values, dtype={df[col].dtype}')
```

২.৮ সাধারণ ভুল (Common Mistakes)

- শুধুমাত্র pandas dtype দেখে সিদ্ধান্ত নেওয়া যে কোনো কলাম numerical, যেমন Pclass বা Survived-কে সরাসরি numerical ধরে নেওয়া - অথচ এরা প্রকৃতপক্ষে categorical।

8. Ordinal ফিচারকে নমিনালের মতো One-Hot Encode করা, ফলে ক্রমের তথ্য হারিয়ে যাওয়া।
9. Nominal ফিচারকে Ordinal Encode করে দেওয়া (যেমন Red=0, Blue=1, Green=2), যা মডেলকে একটা ভুয়া ক্রম শেখায়।
10. Discrete numerical ফিচারকে (যেমন SibSp) categorical মনে করে One-Hot Encode করে ফেলা, যেখানে এটাকে numerical হিসেবেই রাখা ভালো ছিল।
11. ID জাতীয় কলাম (PassengerId) ফিচার হিসেবে রেখে দেওয়া, যা মডেলের জন্য কোনো প্রকৃত তথ্য বহন করে না এবং ওভারফিটিংয়ের ঝুঁকি বাড়ায়।

২.৯ বেস্ট প্র্যাকটিস (Best Practices)

- প্রতিটা কলামের জন্য নিজেকে প্রশ্ন করুন: এর মান দিয়ে কি গড় বের করা অর্থবহ? উত্তর হ্যাঁ হলে numerical, না হলে categorical।
- categorical ফিচারের জন্য আরেকটা প্রশ্ন করুন: এই ক্যাটাগরিগুলোর মধ্যে কি কোনো স্বাভাবিক ক্রম আছে? থাকলে ordinal, না থাকলে nominal।
- df.dtypes, df.nunique(), এবং df['col'].unique() - এই তিনটি কমান্ড দিয়ে শুরুতেই প্রতিটা কলাম পরীক্ষা করুন।
- ID/index জাতীয় কলাম (PassengerId, Name, Ticket Number) সাধারণত মডেলিংয়ের আগেই বাদ দিয়ে দিন, যদি না সেগুলো থেকে অর্থবহ নতুন ফিচার তৈরি করা যায় (যেমন Name থেকে Title বের করা)।
- ডেটা ডিকশনারি (data dictionary) তৈরি করুন - প্রতিটা কলামের নাম, প্রকার (numerical/categorical, continuous/discrete, nominal/ordinal) এবং সংক্ষিপ্ত বর্ণনা লিখে রাখুন। এটা টিমের সবার জন্য একটা রেফারেন্স হিসেবে কাজ করে।

২.১০ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Numerical এবং Categorical ফিচারের মধ্যে পার্থক্য কী, এবং কীভাবে চিনবেন?

উত্তর: Numerical ফিচারের মান সংখ্যা এবং তাদের ওপর গাণিতিক হিসাব (গড়, যোগ) অর্থবহ - যেমন Age, Fare। Categorical ফিচার নির্দিষ্ট সংখ্যক শ্রেণির একটি প্রতিনিধিত্ব করে, যেখানে গাণিতিক হিসাব অর্থহীন - যেমন Sex, Embarked। শুধু dtype না দেখে, কলামের ইউনিক ভ্যালুর

সংখ্যা (nunique()) এবং কলামটার বাস্তব অর্থ বিবেচনা করে সিদ্ধান্ত নিতে হয়, কারণ Pclass-এর মতো কলাম সংখ্যা দিয়ে লেখা হলেও আসলে categorical।

প্রশ্ন: Nominal এবং Ordinal ভেরিয়েবলের মধ্যে পার্থক্য কী, এবং কেন এটা এনকোডিংয়ের জন্য গুরুত্বপূর্ণ?

উত্তর: Nominal ভেরিয়েবলের ক্যাটাগরিগুলোর মধ্যে কোনো স্বাভাবিক ক্রম নেই (যেমন Sex, Embarked), আর Ordinal ভেরিয়েবলে একটা স্পষ্ট ক্রম থাকে (যেমন Education, Pclass)। এটা গুরুত্বপূর্ণ কারণ Ordinal ডেটাকে এমনভাবে এনকোড করতে হয় যাতে ক্রমটা বজায় থাকে (Ordinal Encoding), কিন্তু Nominal ডেটাকে এভাবে এনকোড করলে মডেল ভুলভাবে একটা ক্রম অনুমান করতে পারে, তাই সাধারণত One-Hot Encoding ব্যবহার করা হয়।

প্রশ্ন: Continuous এবং Discrete ভেরিয়েবলের মধ্যে পার্থক্য কী?

উত্তর: Continuous ভেরিয়েবল দুটি মানের মধ্যে অসীম সংখ্যক মধ্যবর্তী মান নিতে পারে (যেমন Age, Fare), যেখানে Discrete ভেরিয়েবল শুধু নির্দিষ্ট, গণনাযোগ্য মান নিতে পারে, সাধারণত পূর্ণসংখ্যা (যেমন SibSp, পরিবারের সদস্য সংখ্যা)।

প্রশ্ন: কেন PassengerId-এর মতো কলাম সাধারণত ফিচার হিসেবে ব্যবহার করা হয় না?

উত্তর: PassengerId শুধু একটা ক্রমিক সনাক্তকারী সংখ্যা (identifier), যার সাথে টার্গেট ভেরিয়েবলের (Survived) কোনো প্রকৃত সম্পর্ক নেই। এটাকে ফিচার হিসেবে রাখলে মডেল এই অর্থহীন সংখ্যার সাথে ভুয়া প্যাটার্ন শিখে ফেলতে পারে (ওভারফিটিং), তাই মডেলিংয়ের আগেই এই ধরনের কলাম বাদ দেওয়া হয়।

২.১১ অধ্যায় সারসংক্ষেপ

- একটা ডেটাসেট রো (স্যাম্পল/রেকর্ড) এবং কলাম (ফিচার/ভেরিয়েবল) দিয়ে গঠিত।
- সুপারভাইজড লার্নিংয়ে কলামগুলো Feature (X, ইনপুট) এবং Target (y, আউটপুট) - এ ভাগ হয়, এবং এই বিভাজন ট্রেন-টেস্ট স্প্লিটের আগেই করতে হয়।

- ফিচার মূলত দুই প্রকার: Numerical (Continuous ও Discrete) এবং Categorical (Nominal ও Ordinal)।
- শুধু pandas dtype-এর ওপর ভরসা না করে প্রতিটা কলামের প্রকৃত অর্থ বুঝে তার প্রকার নির্ধারণ করতে হবে - কারণ Pclass বা Survived-এর মতো সংখ্যায় লেখা কলামও আসলে categorical হতে পারে।
- Nominal বনাম Ordinal-এর পার্থক্য বোঝা পরবর্তী এনকোডিং অধ্যায়ের জন্য অপরিহার্য একটা ভিত্তি।

অধ্যায় ৩: এক্সপ্লোরেরি ডেটা অ্যানালাইসিস (EDA)

৩.১ EDA কী এবং কেন এটি গুরুত্বপূর্ণ

Exploratory Data Analysis (EDA) হলো একটা ডেটাসেটকে মডেলিং শুরু করার আগে “ভালোভাবে চেনার” প্রক্রিয়া। এখানে আমরা সংখ্যা, টেবিল ও গ্রাফের মাধ্যমে ডেটার গঠন, প্যাটার্ন, অসঙ্গতি এবং সম্পর্ক বোঝার চেষ্টা করি - কোনো অনুমান ছাড়াই, শুধু ডেটা যা বলছে তা দেখে।

analogy:

একজন ডাক্তার রোগীকে অস্ত্রোপচার করার আগে প্রথমে পরীক্ষা-নিরীক্ষা করেন - রক্ত পরীক্ষা, এক্স-রে, রক্তচাপ মাপা। এই পরীক্ষাগুলো ছাড়া সরাসরি অস্ত্রোপচারে চলে গেলে বিপর্যয় ঘটতে পারে। EDA ঠিক এই পরীক্ষা-নিরীক্ষার মতো কাজ করে - মডেল ট্রেন করার “অস্ত্রোপচারে” যাওয়ার আগে ডেটার “স্বাস্থ্য পরীক্ষা”।

EDA না করে সরাসরি মডেল ট্রেন করলে যেসব সমস্যা অলঙ্কিত থেকে যেতে পারে:

- কোন কলামে কত মিসিং ভ্যালু আছে তা না জানা, ফলে ভুল ইম্পিউটেশন পদ্ধতি বেছে নেওয়া।
- ডুপ্লিকেট রো থেকে যাওয়া, যা মডেলকে নির্দিষ্ট কিছু প্যাটার্নের দিকে পক্ষপাতদুষ্ট করে তোলে।
- আউটলায়ার বা অস্বাভাবিক মান (যেমন বয়স = -৫ বা ৫০০) অলঙ্কিত থেকে যাওয়া।
- টার্গেট ভেরিয়েবলে ক্লাস ইমব্যালেন্স আছে কি না তা না বোঝা।
- কোন ফিচার numerical, কোনটা categorical তা ভুল বোঝা, যা ভুল এনকোডিং/স্কেলিং সিদ্ধান্তের দিকে নিয়ে যায়।

৩.২ EDA ওয়ার্কফ্লো ওভারভিউ

একটা সুশৃঙ্খল EDA প্রক্রিয়া সাধারণত নিচের ধাপ অনুসরণ করে:

১. অ্যানালিটিক্যাল ভিউ -> shape, columns, head(), tail(), sample()
২. ফিচার স্ক্রিনিং -> isnull().sum(), duplicated().sum(), dtypes
৩. স্ট্যাটিস্টিক্যাল প্রোফাইলিং -> info(), describe()
৪. ইউনিভারিয়েট অ্যানালাইসিস -> একটা একটা কলাম আলাদাভাবে বিশ্লেষণ (পরবর্তী অধ্যায়ে)

৫. বাইভ্যারিয়েট/মাল্টিভ্যারিয়েট -> কলামগুলোর মধ্যে সম্পর্ক বিশ্লেষণ (পরবর্তী অধ্যায়ে)
 ৬. স্বয়ংক্রিয় প্রোফাইলিং -> ydata-profiling দিয়ে সম্পূর্ণ রিপোর্ট

এই অধ্যায়ে আমরা ধাপ ১, ২, ৩ এবং ৬ নিয়ে বিস্তারিত আলোচনা করব। ইউনিভ্যারিয়েট ও বাইভ্যারিয়েট/মাল্টিভ্যারিয়েট অ্যানালাইসিস (countplot, histogram, KDE, scatterplot, correlation heatmap ইত্যাদি) নিয়ে পরবর্তী দুটি অধ্যায়ে গভীরভাবে আলোচনা করা হবে।

৩.৩ ডেটাসেটের অ্যানালিটিক্যাল ভিউ

যেকোনো নতুন ডেটাসেট হাতে পেলে সবার আগে কয়েকটা মৌলিক প্রশ্নের উত্তর জানা দরকার - কত রো-কলাম আছে? কলামগুলোর নাম কী? ডেটা দেখতে কেমন?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('titanic_data_updated.csv')

df.shape
# (891, 12)

print(f'Total potential features: {df.shape[1]}') # 12
print(f'Total sample data: {df.shape[0]}') # 891

df.columns
# Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age',
#        'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'], dtype='object')

df.head(10) # প্রথম ১০টি রো - কলামের গঠন ও প্রথম কিছু মান দেখা
df.tail(5) # শেষ ৫টি রো - ডেটার শেষ অংশে কোনো অসঙ্গতি আছে কি না দেখা
df.sample(10) # র্যান্ডম ১০টি রো - শুধু শুরু-শেষ না দেখে পুরো ডেটাসেট জুড়ে একটা
প্রতিনিধিত্বমূলক ধারণা পাওয়া
```

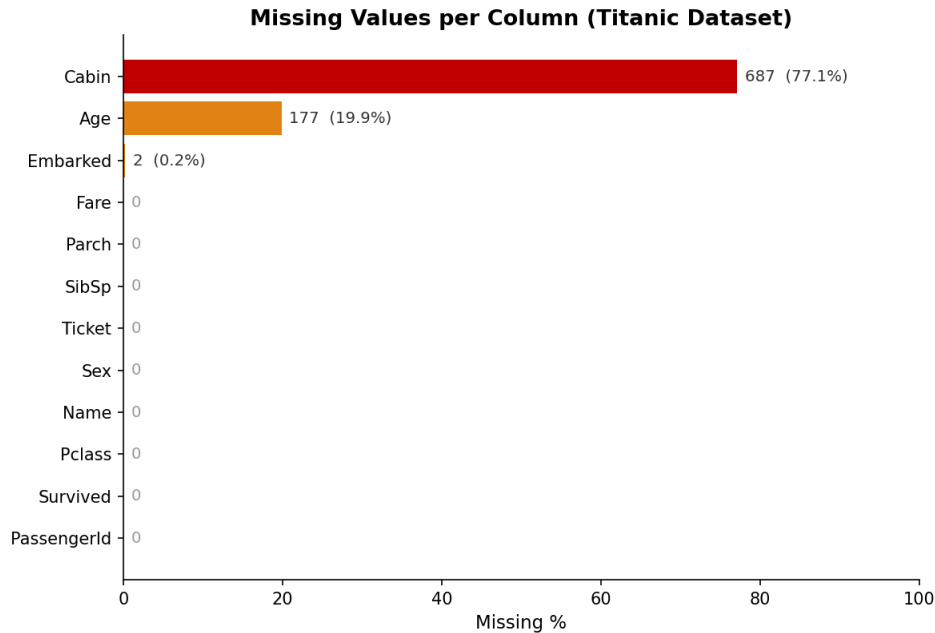
head() বনাম sample(): df.head() সবসময় প্রথম কয়েকটা রো দেখায়, যা অনেক সময় বিভ্রান্তিকর হতে পারে - কারণ ডেটা যদি কোনোভাবে সর্ট করা থাকে (যেমন তারিখ অনুযায়ী, বা ক্লাস অনুযায়ী), প্রথম কয়েকটা রো পুরো ডেটাসেটের প্রতিনিধিত্ব নাও করতে পারে। df.sample() র্যান্ডমভাবে রো বেছে নেয় বলে এটি প্রায়ই বেশি নির্ভরযোগ্য একটা প্রাথমিক ধারণা দেয়।

৩.৪ ফিচার স্ক্রিনিং: মিসিং ভ্যালু শনাক্তকরণ

মিসিং ভ্যালু হলো ডেটাসেটে এমন জায়গা যেখানে কোনো মান রেকর্ড করা হয়নি (NaN/null)। ডেটা ক্লিনিংয়ের প্রথম এবং সবচেয়ে গুরুত্বপূর্ণ ধাপগুলোর একটি হলো প্রতিটা কলামে কত মিসিং ভ্যালু আছে তা পরিমাপ করা।

```
df.isnull().sum()
# PassengerId      0
# Survived         0
# Pclass           0
# Name             0
# Sex              0
# Age             177
# SibSp            0
# Parch            0
# Ticket           0
# Fare             0
# Cabin           687
# Embarked         2

# শতাংশ আকারে দেখা প্রায়ই বেশি অর্থবহ
(df.isnull().sum() / len(df) * 100).round(2)
# Age             19.87
# Cabin           77.10
# Embarked         0.22
```



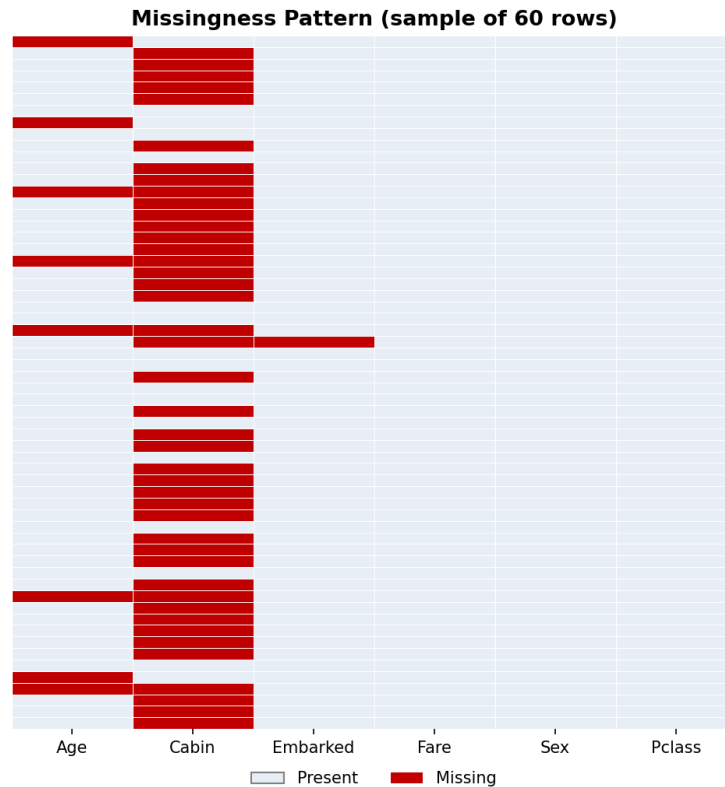
চিত্র ৩.১: কলাম অনুযায়ী মিসিং ভ্যালুর পরিমাণ। Cabin-এ প্রায় ৭৭% মান অনুপস্থিত, Age-এ প্রায় ২০%, Embarked-এ মাত্র ২টি মান অনুপস্থিত।

এই গ্রাফ থেকে গুরুত্বপূর্ণ সিদ্ধান্ত নেওয়া যায়: Cabin কলামে এত বিপুল পরিমাণ (৭৭%) মিসিং ভ্যালু থাকায় এটাকে সরাসরি impute করার বদলে হয়তো বাদ দেওয়া, বা “মিসিং কি না” তা একটা নতুন ইন্ডিকেটর ফিচার বানানো বেশি যুক্তিসঙ্গত (এই বইয়ের Missing Value অধ্যায়ে বিস্তারিত)। Age-এর ২০% মিসিং ভ্যালুর জন্য median/mean ইম্পিউটেশন যথেষ্ট, আর Embarked-এর মাত্র ২টি মিসিং ভ্যালুর জন্য mode ইম্পিউটেশন বা সেই দুটো রো বাদ দেওয়া - দুটোই গ্রহণযোগ্য।

মিসিংনেস প্যাটার্ন ভিজুয়ালাইজেশন

শুধু সংখ্যা না দেখে, মিসিং ভ্যালুগুলো ডেটাসেটের কোথায় কোথায় ছড়িয়ে আছে তা একটা হিটম্যাপের মাধ্যমে ভিজুয়ালি দেখা যায়। এটি বিশেষভাবে সাহায্য করে যখন বুঝতে হয় একাধিক কলামের মিসিং ভ্যালু একসাথে ঘটছে কি না (যা MCAR/MAR/MNAR বিশ্লেষণে গুরুত্বপূর্ণ - এই বইয়ের Missing Value অধ্যায়ে আলোচিত হবে)।

```
plt.figure(figsize=(10, 6))
sns.heatmap(df.isnull(), cbar=False, cmap=['#E8EEF5', '#C00000'], yticklabels=False)
plt.title('Missingness Heatmap')
plt.show()
```



চিত্র ৩.২: মিসিংনেস হিটম্যাপ (৬০টি রোয়ের নমুনা)। লাল অংশ মিসিং ভ্যালু নির্দেশ করে। লক্ষ্য করুন Cabin কলামে লাল অংশ কতটা ঘন, যেখানে Age-এ বিক্ষিপ্তভাবে ছড়িয়ে আছে।

৩.৫ ফিচার স্ক্রিনিং: ডুপ্লিকেট ডাটাসেট শনাক্তকরণ

ডুপ্লিকেট রো মানে এমন রো যা ডেটাসেটে একাধিকবার হুবহু একইভাবে উপস্থিত। এগুলো থেকে যাওয়ার বিভিন্ন কারণ থাকতে পারে - ডেটা সংগ্রহের সময় একই এন্ট্রি দুইবার রেকর্ড হওয়া, একাধিক উৎস থেকে ডেটা একত্র করার সময় ওভারল্যাপ হওয়া ইত্যাদি। ডুপ্লিকেট রো রেখে দিলে মডেল সেই নির্দিষ্ট প্যাটার্নকে অতিরিক্ত গুরুত্ব দিয়ে শিখে ফেলতে পারে (একপ্রকার ডেটা লিকেজ এবং পক্ষপাতের কারণ)।

```
df.duplicated().sum()
# 0 (টাইটানিক ডেটাসেটে সাধারণত কোনো ডুপ্লিকেট রো থাকে না)

# ডুপ্লিকেট রোগুলো দেখা (যদি থাকে)
df[df.duplicated()]

# ডুপ্লিকেট রো বাদ দেওয়া
df.drop_duplicates(inplace=True)

# নির্দিষ্ট কলাম অনুযায়ী ডুপ্লিকেট চেক করা (যেমন শুধু Name একই কি না)
df.duplicated(subset=['Name']).sum()
```

সতর্কতা: drop_duplicates() ব্যবহার করার আগে ভেবে দেখুন এটা সত্যিই ডুপ্লিকেট নাকি কাকতালীয়ভাবে একই মান। উদাহরণস্বরূপ, দুজন ভিন্ন যাত্রীর Age, Pclass, Fare হুবহু এক হতে পারে - এটা ডুপ্লিকেট নয়। সাধারণত একটা ইউনিক আইডেন্টিফায়ার কলাম (যেমন PassengerId) থাকলে সেটা দিয়ে প্রকৃত ডুপ্লিকেট যাচাই করা সবচেয়ে নির্ভরযোগ্য।

৩.৬ ডেটা টাইপ পুনঃযাচাই

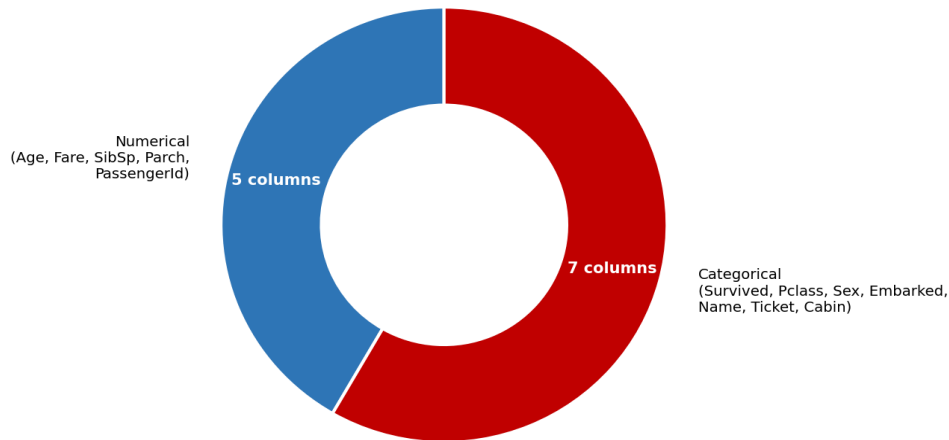
অধ্যায় ২-এ আমরা numerical/categorical এবং nominal/ordinal পার্থক্য নিয়ে বিস্তারিত আলোচনা করেছি। EDA-এর এই ধাপে আমরা সেই জ্ঞান প্রয়োগ করে প্রতিটা কলামকে শ্রেণিবদ্ধ করি, যা পরবর্তী প্রতিটা ধাপের (ইম্পিউটেশন, এনকোডিং, স্কেলিং) ভিত্তি তৈরি করে।

```
df.dtypes
df.info()

num_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = df.select_dtypes(include=['object']).columns.tolist()

print('Numerical columns:', num_cols)
print('Categorical columns:', cat_cols)
```


Numerical vs Categorical Columns (Titanic, 12 total)



চিত্র ৩.৩: ১২টি কলামের মধ্যে ৫টি numerical এবং ৭টি categorical (যেখানে Survived ও Pclass সংখ্যা দিয়ে লেখা হলেও প্রকৃতপক্ষে categorical, অধ্যায় ২-এ আলোচিত)।

৩.৭ স্ট্যাটিস্টিক্যাল প্রোফাইলিং

পরবর্তী ধাপ হলো প্রতিটা কলামের পরিসংখ্যানগত সারসংক্ষেপ দেখা - কেন্দ্রীয় প্রবণতা (mean, median), বিস্তৃতি (std, min, max), এবং কোয়ার্টাইল।

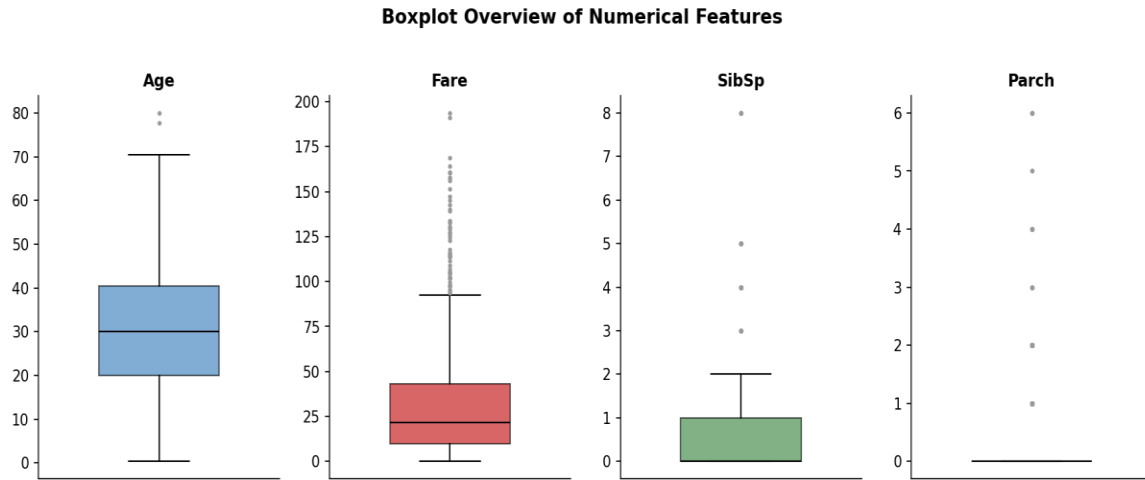
```
df.info()
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 891 entries, 0 to 890
# Data columns (total 12 columns):
#  #   Column        Non-Null Count  Dtype
# ---  ---
#  0   PassengerId    891 non-null    int64
#  1   Survived       891 non-null    int64
#  2   Pclass         891 non-null    int64
#  3   Name           891 non-null    object
#  4   Sex            891 non-null    object
#  5   Age            714 non-null    float64    <- 177টি মিসিং
#  6   SibSp          891 non-null    int64
#  7   Parch          891 non-null    int64
#  8   Ticket         891 non-null    object
#  9   Fare           891 non-null    float64
#  10  Cabin          204 non-null    object      <- 687টি মিসিং
#  11  Embarked       889 non-null    object      <- 2টি মিসিং

df.describe()
#           Age          Fare          SibSp          Parch
# count    714.000      891.000      891.000      891.000
```

```
# mean      29.699      32.204      0.523      0.382
# std       14.526      49.693      1.103      0.806
# min        0.420        0.000        0.000        0.000
# 25%       20.125        7.910        0.000        0.000
# 50%       28.000       14.454        0.000        0.000
# 75%       38.000       31.000        1.000        0.000
# max       80.000      512.329        8.000        6.000
```

```
df.describe(include='object') # categorical কলামের সারসংক্ষেপ
#      Sex  Embarked
# count  891      889
# unique   2        3
# top    male      S
# freq   577      644
```

describe() থেকে গুরুত্বপূর্ণ পর্যবেক্ষণ: Fare-এর mean (৩২.২) ও median/50% (১৪.৪৫)-এর মধ্যে বিশাল ব্যবধান, এবং max (৫১২.৩) অনেক বেশি - এটি একটা স্পষ্ট সংকেত যে Fare ডানদিকে স্কিউড (right-skewed) এবং সম্ভবত আউটলায়ার আছে। নিচের বক্সপ্লট গ্রিড দিয়ে এটা ভিজুয়ালি নিশ্চিত করা যায়:



চিত্র ৩.৪: numerical ফিচারগুলোর বক্সপ্লট ও ভারভিউ। Fare-এ প্রচুর আউটলায়ার (উপরের বিন্দুগুলো) দেখা যাচ্ছে, যেখানে Age তুলনামূলক সুসম বিতরণ দেখাচ্ছে।

আউটলায়ার ও স্কিউনেস নিয়ে গভীর আলোচনা এই বইয়ের Outlier Detection অধ্যায়ে করা হবে - এখানে describe() ও boxplot শুধু একটা প্রাথমিক সংকেত দিচ্ছে যে Fare কলামটির বিশেষ মনোযোগ প্রয়োজন।

৩.৮ Y Data Profiling: স্বয়ংক্রিয় EDA রিপোর্ট

এতক্ষণ আমরা যা ম্যানুয়ালি করলাম - shape, isnull(), duplicated(), describe(), dtype চেক - তার প্রায় সবকিছু এবং আরও অনেক কিছু একসাথে একটা সুন্দর, ইন্টারেক্টিভ HTML রিপোর্টে স্বয়ংক্রিয়ভাবে তৈরি করে দেয় ydata-profiling লাইব্রেরি। এটি বড় ডেটাসেটে দ্রুত একটা সামগ্রিক ধারণা পেতে অত্যন্ত উপযোগী।

```
pip install ydata_profiling

import pandas as pd
from ydata_profiling import ProfileReport

df = pd.read_csv('titanic_data_updated.csv')

profile = ProfileReport(df, title='Titanic Profile Report')
profile.to_file('my_report.html') # রিপোর্ট HTML ফাইল হিসেবে সংরক্ষিত হবে

# Jupyter Notebook-এর ভেতরেই রিপোর্ট দেখতে চাইলে:
profile.to_notebook_iframe()
```

এই রিপোর্টে যা যা স্বয়ংক্রিয়ভাবে পাওয়া যায়:

- **Overview:** মোট রো-কলাম সংখ্যা, মিসিং সেলের শতাংশ, ডুপ্লিকেট রোয়ের সংখ্যা, মেমরি ব্যবহার।
- **Variables:** প্রতিটা কলামের জন্য আলাদা বিস্তারিত প্রোফাইল - distribution, missing %, unique values, সবচেয়ে কমন মান, histogram।
- **Interactions:** যেকোনো দুটি কলামের মধ্যে স্ক্যাটারপ্লট ইন্টারঅ্যাকটিভভাবে দেখা।
- **Correlations:** Pearson, Spearman, Kendall - একাধিক কোরিলেশন ম্যাট্রিক্স স্বয়ংক্রিয়ভাবে তৈরি।
- **Missing values:** মিসিং ভ্যালুর ম্যাট্রিক্স, বার চার্ট, এবং ডেনড্রোগ্রাম (কোন কলামগুলোর মিসিংনেস একসাথে ঘটে তা দেখানোর জন্য)।
- **Alerts:** স্বয়ংক্রিয়ভাবে সতর্কবার্তা - যেমন “Cabin has 77.1% missing values”, “Fare is highly skewed”, “PassengerId is uniformly distributed (likely an ID)”।

বিষয়	ম্যানুয়াল EDA	YData Profiling
সময়	অনেক বেশি (প্রতিটা কোড নিজে লিখতে হয়)	কয়েক সেকেন্ড-মিনিটে সম্পূর্ণ রিপোর্ট

বিষয়	ম্যানুয়াল EDA	YData Profiling
নিয়ন্ত্রণ	সম্পূর্ণ নিয়ন্ত্রণ - নিজের প্রয়োজন অনুযায়ী বিশ্লেষণ	সীমিত - প্রি-ডিফাইন্ড বিশ্লেষণ
গভীরতা	নির্দিষ্ট প্রশ্নের গভীর উত্তর দেওয়া যায়	ব্যাপক কিন্তু সাধারণীকৃত (generic) ওভারভিউ
বড় ডেটাসেট	সময়সাপেক্ষ	বড় ডেটাসেটে স্লো হতে পারে, sample নিয়ে চালানো ভালো
উপযুক্ত ব্যবহার	চূড়ান্ত, নির্দিষ্ট সিদ্ধান্তের জন্য	প্রাথমিক, দ্রুত ওভারভিউয়ের জন্য

সর্বোত্তম অভ্যাস হলো দুটোকেই একসাথে ব্যবহার করা: প্রথমে ydata-profiling চালিয়ে একটা দ্রুত, ব্যাপক ওভারভিউ নিন, তারপর সেই রিপোর্টে যেসব সমস্যা বা প্রশ্ন চিহ্নিত হলো (যেমন কোন কলামে বেশি মিসিং, কোনটায় আউটলায়ার), সেগুলোর জন্য ম্যানুয়ালি গভীরভাবে বিশ্লেষণ করুন।

৩.৯ ইউনিভ্যারিয়েট অ্যানালাইসিস: ক্যাটাগরিক্যাল ফিচার

ইউনিভ্যারিয়েট (Univariate) অ্যানালাইসিস মানে একটা সময়ে একটা মাত্র কলাম আলাদাভাবে বিশ্লেষণ করা - অন্য কোনো কলামের সাথে সম্পর্ক না দেখে। ক্যাটাগরিক্যাল ফিচারের ক্ষেত্রে আমরা মূলত জানতে চাই - প্রতিটা ক্যাটাগরিতে কতগুলো রো আছে এবং তাদের অনুপাত কেমন।

Countplot: প্রতিটা ক্যাটাগরির ফ্রিকোয়েন্সি

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.countplot(data=df, x=df['Survived'])
plt.title('Countplot of Survived')
plt.show()

df['Survived'].value_counts()
# 0    557
# 1    334

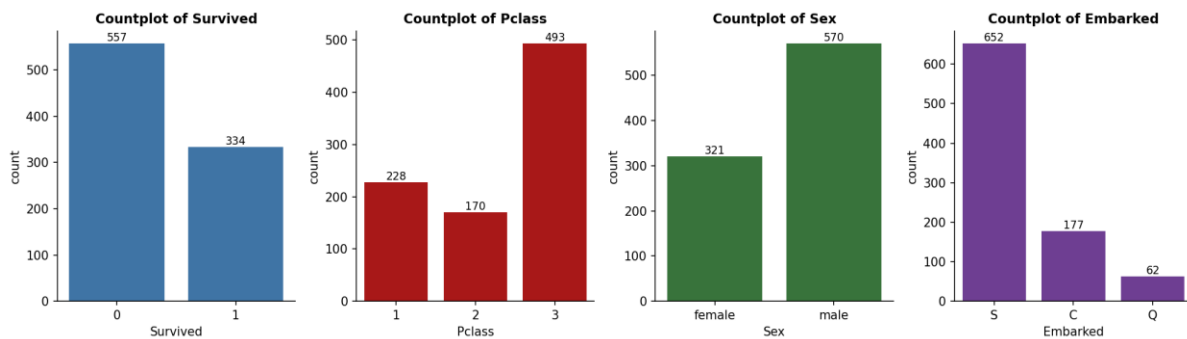
sns.countplot(data=df, x=df['Pclass'])
plt.title('Countplot of Pclass')
plt.show()
```

```
(df['Pclass'].value_counts() / len(df)) * 100 # শতকরা হিসেবে

sns.countplot(data=df, x=df['Sex'])
plt.show()

sns.countplot(data=df, x=df['Embarked'])
plt.show()

(df['Embarked'].value_counts() / len(df)) * 100
```



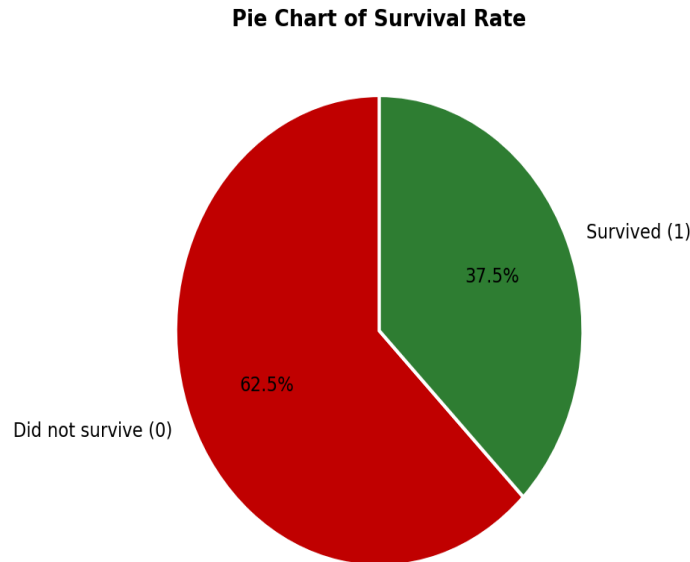
চিত্র ৩.৫: চারটি ক্যাটাগরিক্যাল কলামের countplot / Pclass-এ ৩য় শ্রেণির যাত্রী সবচেয়ে বেশি, Sex-এ পুরুষ যাত্রী বেশি, Embarked-এ Southampton (S) বন্দর থেকে সবচেয়ে বেশি যাত্রী উঠেছেন।

Pie Chart: অনুপাত ভিজুয়ালাইজেশন

Countplot সংখ্যা দেখায়, কিন্তু কখনো কখনো শতকরা অনুপাত (proportion) দেখানোর জন্য পাই চার্ট বেশি স্বজ্ঞাত (intuitive)। তবে মনে রাখবেন, ৩টির বেশি ক্যাটাগরি থাকলে পাই চার্ট পড়া কঠিন হয়ে যায় - সেক্ষেত্রে countplot বা bar chart বেশি উপযোগী।

```
survived_count = df['Survived'].value_counts()
survived_label = df['Survived'].unique()

plt.pie(survived_count, labels=survived_label, autopct='%1.1f%%')
plt.title('Pie chart of survival rate')
plt.show()
```



চিত্র ৩.৬: পাই চার্ট থেকে স্পষ্ট দেখা যাচ্ছে মাত্র ৩৭.৫% যাত্রী বেঁচে গিয়েছিলেন - অর্থাৎ টার্গেট ভেরিয়েবলে সামান্য ক্লাস ইমব্যালেন্স আছে, যা মডেল ইন্ট্রানুয়েশনের সময় (শুধু accuracy না দেখে precision/recall/F1 ব্যবহার) মাথায় রাখতে হবে।

৩.১০ ইউনিভ্যারিয়েট অ্যানালাইসিস: নিউমেরিক্যাল ফিচার

নিউমেরিক্যাল ফিচারের ক্ষেত্রে আমরা জানতে চাই মানগুলো কীভাবে বিতরণ (distribute) হয়ে আছে - কেন্দ্রবিন্দু কোথায়, কতটা ছড়ানো, এবং বিতরণের আকৃতি প্রতিসম (symmetric) নাকি স্কিউড (skewed)। এর জন্য তিনটি প্রধান টুল ব্যবহার করা হয়: Histogram, KDE Plot এবং Boxplot।

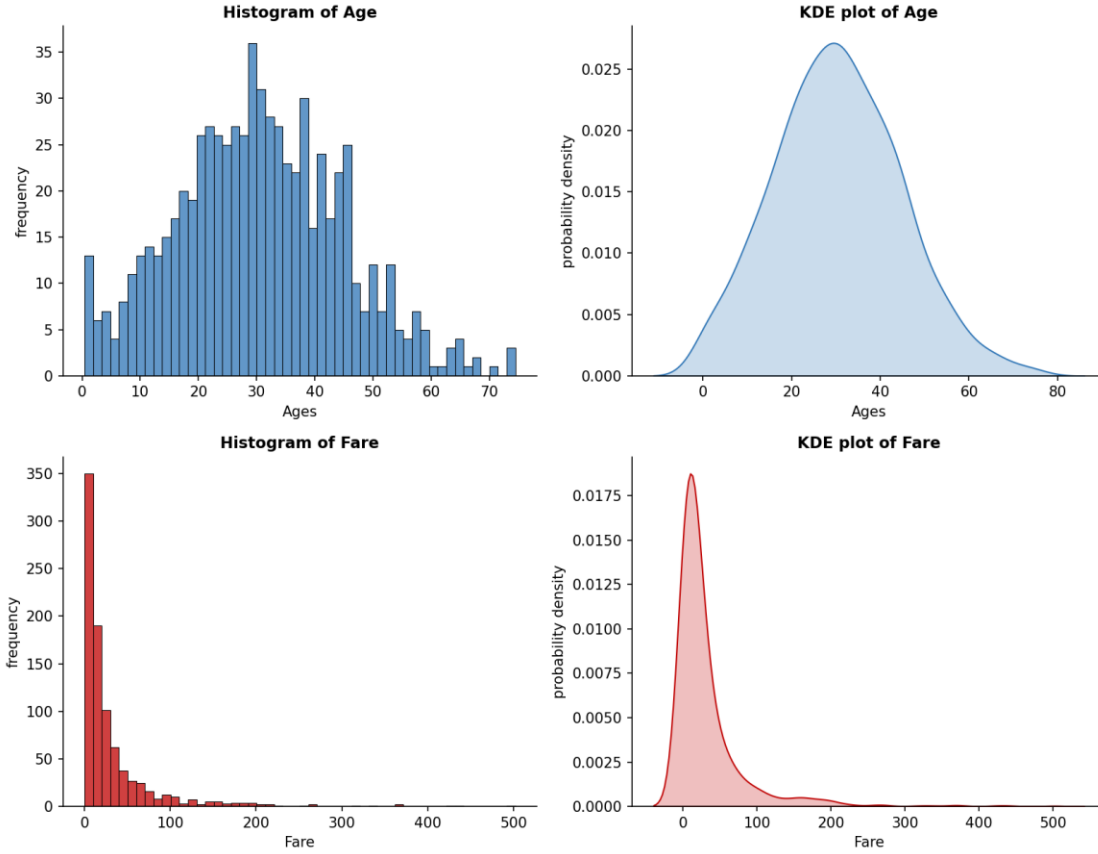
```
sns.histplot(df['Age'], bins=50)
plt.title('Histogram of Age')
plt.xlabel('Ages'); plt.ylabel('frequency')
plt.show()

sns.kdeplot(df['Age'])
plt.title('KDE plot of Age')
plt.xlabel('Ages'); plt.ylabel('probability distribution')
plt.show()

sns.histplot(df['Fare'], bins=50)
plt.title('Histogram of Fare')
plt.xlabel('Fare'); plt.ylabel('frequency')
plt.show()

sns.kdeplot(df['Fare'])
```

```
plt.title('KDE plot of Fare')
plt.xlabel('Fare'); plt.ylabel('probability distribution')
plt.show()
```



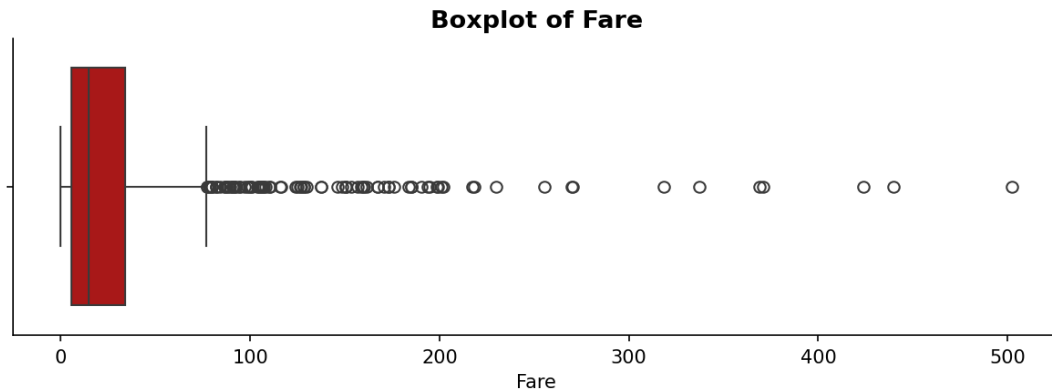
চিত্র ৩.৭: Age মোটামুটি ঘণ্টা-আকৃতির (bell-shaped), normal distribution-এর কাছাকাছি। Fare প্রবলভাবে ডান-দিকে ঝুঁকিড (right-skewed) - বেশিরভাগ যাত্রীর ভাড়া কম, কিন্তু কিছু যাত্রীর ভাড়া অনেক বেশি, যা একটা লম্বা লেজ (tail) তৈরি করেছে।

Normal বনাম Skewed Distribution

বৈশিষ্ট্য	Normal Distribution	Skewed Distribution
আকৃতি	প্রতিসম, ঘণ্টা-আকৃতির (bell-shaped)	অপ্রতিসম, একদিকে লম্বা লেজ
Mean বনাম Median	প্রায় সমান	ভিন্ন (right-skewed হলে mean > median)
উদাহরণ	Age (অনেকটা কাছাকাছি)	Fare, আয়, বাড়ির দাম
প্রভাব	অনেক ML অ্যালগরিদমে সরাসরি ব্যবহারযোগ্য	Log/Power transformation প্রয়োজন হতে পারে

Boxplot: একনজরে কেন্দ্রীয় প্রবণতা, বিস্তৃতি ও আউটলায়ার

```
sns.boxplot(x=df['Fare'])
plt.title('Boxplot of Fare')
plt.show()
```



চিত্র ৩.৮: Fare-এর বক্সপ্লটে ডানদিকে প্রচুর বিন্দু (আউটলায়ার) দেখা যাচ্ছে - এগুলো সেইসব যাত্রী যারা গড়ের তুলনায় অনেক বেশি ভাড়া দিয়েছিলেন (সম্ভবত প্রথম শ্রেণির বিলাসবহুল কেবিন)।

৩.১১ বাইভ্যারিয়েট ও মাল্টিভ্যারিয়েট অ্যানালাইসিস

বাইভ্যারিয়েট (Bivariate) অ্যানালাইসিস মানে দুটো কলামের মধ্যে সম্পর্ক দেখা, আর মাল্টিভ্যারিয়েট (Multivariate) মানে তিন বা ততোধিক কলামের মধ্যে সম্পর্ক একসাথে দেখা। এটি বিশেষভাবে গুরুত্বপূর্ণ কারণ এখান থেকেই বোঝা যায় কোন ফিচারগুলো টার্গেট ভেরিয়েবল পূর্বাভাসে সবচেয়ে বেশি সাহায্য করতে পারে।

Countplot with Hue: দুটো ক্যাটাগরিক্যাল ফিচারের সম্পর্ক

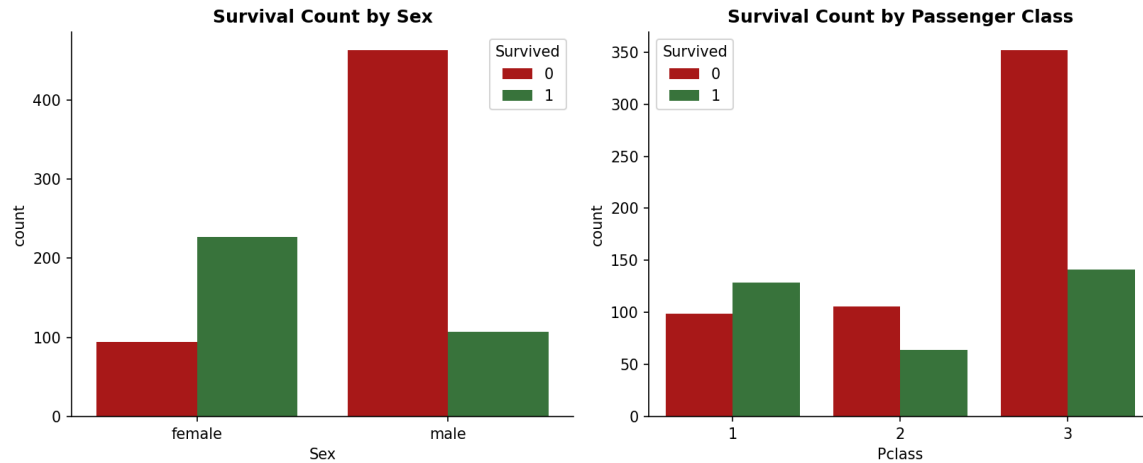
```
sns.countplot(x=df['Sex'], hue=df['Survived'])
plt.title('Survival Count by Sex')
plt.show()

gender_split = df.groupby('Sex')['Survived'].value_counts(normalize=True)
print(gender_split)
# Sex      Survived
# female  1          0.706
#         0          0.294
# male    0          0.811
#         1          0.189

sns.countplot(x=df['Pclass'], hue=df['Survived'])
plt.title('Survival Count by Passenger Class')
plt.show()
```



```
pclass_split = df.groupby('Pclass')['Survived'].value_counts(normalize=True)
print(pclass_split)
```



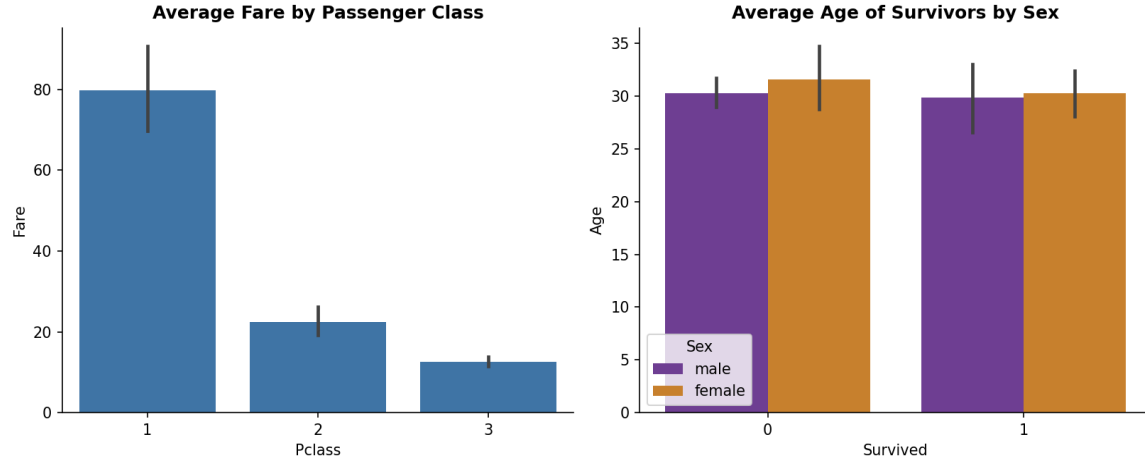
চিত্র ৩.৯: নারী যাত্রীদের মধ্যে বেঁচে যাওয়ার হার পুরুষদের চেয়ে অনেক বেশি (“Women and children first” নীতি)। একইভাবে ১ম শ্রেণির যাত্রীদের বেঁচে যাওয়ার হার ৩য় শ্রেণির চেয়ে বেশি - Sex ও Pclass দুটোই Survived-এর শক্তিশালী predictor।

Barplot: ক্যাটাগরিক্যাল বনাম নিউমেরিক্যাল

Barplot ব্যবহার করা হয় যখন একটা ক্যাটাগরিক্যাল ফিচারের প্রতিটা গ্রুপে একটা নিউমেরিক্যাল ফিচারের গড় (বা অন্য কোনো সারসংক্ষেপ পরিসংখ্যান) তুলনা করতে হয়।

```
sns.barplot(x=df['Pclass'], y=df['Fare'])
plt.title('Average Fare by Passenger Class')
plt.show()

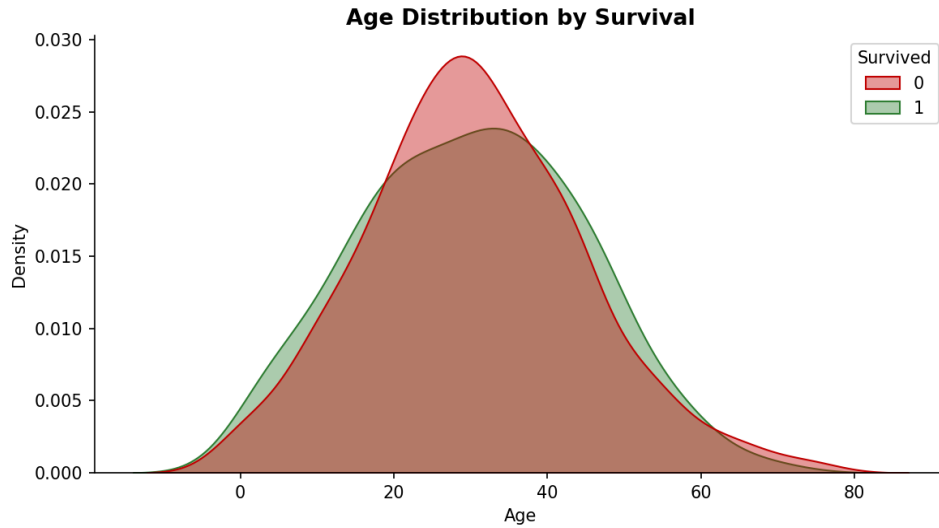
sns.barplot(x=df['Survived'], y=df['Age'], hue=df['Sex'])
plt.title('Average Age of Survivors by Sex')
plt.show()
```



চিত্র ৩.১০: ১ম শ্রেণির গড় ভাড়া ৩য় শ্রেণির প্রায় ৬ গুণ বেশি- Pclass ও Fare পরস্পর সম্পর্কিত (এই বিষয়টা Multicollinearity অধ্যায়ে গুরুত্বপূর্ণ হবে)। ডানদিকের গ্রাফে কালো রেখাগুলো (error bar) প্রতিটা গড়ের আত্মবিশ্বাস ব্যবধান (confidence interval) দেখাচ্ছে।

KDE Plot with Hue: একাধিক গ্রুপের ডিস্ট্রিবিউশন তুলনা

```
sns.kdeplot(data=df, x='Age', hue='Survived', fill=True, common_norm=False)
plt.title('Age Distribution by Survival')
plt.show()
```



চিত্র ৩.১১: বেঁচে যাওয়া (সবুজ) ও না-যাওয়া (লাল) যাত্রীদের বয়স-বিতরণ একে অপরের ওপর ওভারল্যাপ করে দেখানো হয়েছে। লক্ষ্য করুন ছোট বয়সে (২০ বছরের নিচে) সবুজ রেখা লালের চেয়ে কিছুটা উঁচু- শিশুদের বেঁচে যাওয়ার প্রবণতা কিছুটা বেশি ছিল।

Scatterplot: দুটো নিউমেরিক্যাল ফিচারের সম্পর্ক

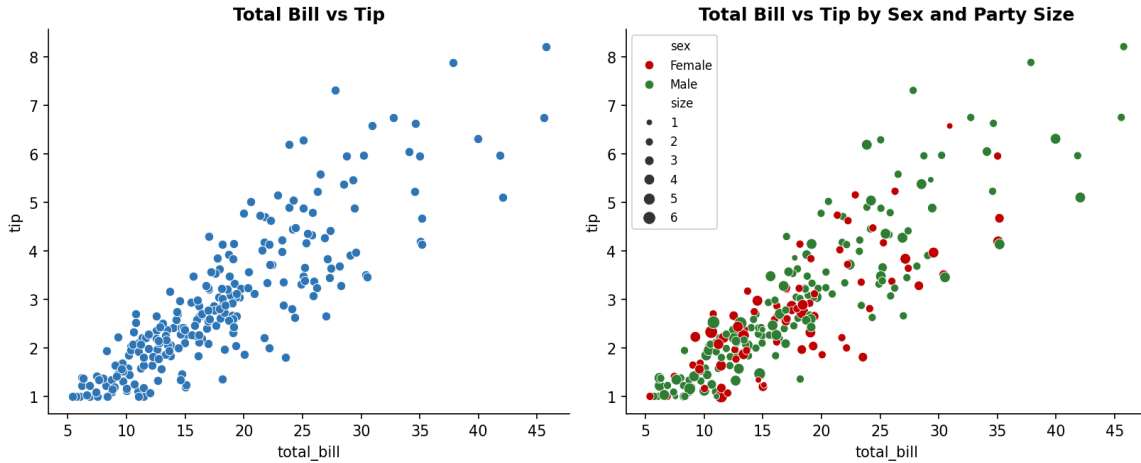
Scatterplot ব্যবহার করা হয় দুটো নিউমেরিক্যাল ফিচারের মধ্যে সম্পর্ক (linear/non-linear, positive/negative, শক্তিশালী/দুর্বল) ভিজুয়ালি দেখার জন্য। নিচের উদাহরণে আমরা সিবোর্নের

বিল্ট-ইন tips ডেটাসেট ব্যবহার করছি, যেখানে total_bill (মোট বিল) ও tip (বকশিশ)-এর সম্পর্ক দেখানো হচ্ছে।

```
import seaborn as sns
tips = sns.load_dataset('tips')

sns.scatterplot(data=tips, x='total_bill', y='tip')
plt.title('Total Bill vs Tip')
plt.show()

sns.scatterplot(data=tips, x='total_bill', y='tip', hue='sex', size='size')
plt.title('Total Bill vs Tip by Sex and Party Size')
plt.show()
```



চিত্র ৩.১২: total_bill ও tip-এর মধ্যে স্পষ্ট পজিটিভ (ধনাত্মক) লিনিয়ার সম্পর্ক - বিল যত বেশি, বকশিশও সাধারণত তত বেশি। ডানদিকের গ্রাফে hue ও size যোগ করে আমরা একসাথে চারটি ভেরিয়েবল (total_bill, tip, sex, party size) একই গ্রাফে দেখাতে পেরেছি - এটাই মান্টিভ্যারিয়েট ভিজুয়ালাইজেশনের শক্তি।

মনে রাখবেন: Scatterplot-এ একটা সম্পর্ক দেখা মানেই কার্যকারণ (causation) প্রমাণিত হয় না - “Correlation is not Causation”। total_bill বেশি হলে tip বেশি হওয়ার পেছনে সরাসরি কার্যকারণ থাকলেও, অনেক ক্ষেত্রে দুটো ভেরিয়েবলের মধ্যে সম্পর্ক একটা তৃতীয়, লুকানো ভেরিয়েবলের (confounding variable) কারণেও হতে পারে।

৩.১২ EDA চেকলিস্ট

মডেলিংয়ের আগে নিচের চেকলিস্টটা অনুসরণ করলে কোনো গুরুত্বপূর্ণ বিষয় বাদ পড়ার সম্ভাবনা কমে যায়:

- [] df.shape দিয়ে রো-কলাম সংখ্যা যাচাই করা হয়েছে
- [] df.head(), df.tail(), df.sample() দিয়ে ডেটার চেহারা দেখা হয়েছে

- [] `df.isnull().sum()` দিয়ে প্রতিটা কলামের মিসিং ভ্যালু গণনা করা হয়েছে
- [] `df.duplicated().sum()` দিয়ে ডুপ্লিকেট রো চেক করা হয়েছে
- [] `df.dtypes` এবং `df.info()` দিয়ে প্রতিটা কলামের ডেটা টাইপ যাচাই করা হয়েছে
- [] `df.describe()` এবং `df.describe(include='object')` দিয়ে পরিসংখ্যানগত সারসংক্ষেপ দেখা হয়েছে
- [] টার্গেট ভেরিয়েবলের ক্লাস ডিস্ট্রিবিউশন (ইমব্যালেন্স আছে কি না) যাচাই করা হয়েছে
- [] প্রতিটা numerical কলামে সম্ভাব্য আউটলায়ার আছে কি না বক্সপ্লট দিয়ে প্রাথমিকভাবে দেখা হয়েছে
- [] `ydata-profiling` দিয়ে একটা সম্পূর্ণ স্বয়ংক্রিয় রিপোর্ট তৈরি করে ক্রস-চেক করা হয়েছে

৩.১৩ সাধারণ ভুল (Common Mistakes)

12. শুধু `df.head()` দেখে পুরো ডেটাসেট সম্পর্কে ধারণা করা, যেখানে `df.sample()` বা পুরো `describe()` না দেখা।
13. `isnull().sum()` দিয়ে শুধু সংখ্যা দেখে কলাম বাদ দেওয়ার সিদ্ধান্ত নেওয়া, কিন্তু কেন মিসিং হচ্ছে (MCAR/MAR/MNAR) তা বিশ্লেষণ না করা।
14. ডুপ্লিকেট রো অন্ধভাবে `drop_duplicates()` দিয়ে বাদ দেওয়া, প্রকৃতপক্ষে সেগুলো ডুপ্লিকেট কি না যাচাই না করে।
15. `describe()` এর mean/std দেখেই সিদ্ধান্ত নেওয়া, কোয়ার্টাইল ও min-max উপেক্ষা করা - যা স্কিউনেস ও আউটলায়ারের গুরুত্বপূর্ণ সংকেত দেয়।
16. `ydata-profiling`-এর রিপোর্টকেই EDA-এর শেষ ধাপ মনে করা, যেখানে ডোমেইন-নির্দিষ্ট প্রশ্নের জন্য নিজস্ব ম্যানুয়াল বিশ্লেষণও প্রয়োজন।
17. EDA-কে শুধু একবারের কাজ মনে করা - বাস্তবে ডেটা ক্লিনিং বা ফিচার ইঞ্জিনিয়ারিংয়ের প্রতিটা ধাপের পরে আবার দ্রুত EDA করে দেখে নেওয়া উচিত পরিবর্তনগুলো প্রত্যাশিত ফল দিচ্ছে কি না।

৩.১৪ বেস্ট প্র্যাকটিস (Best Practices)

- EDA শুরু করুন বড় ছবি থেকে (shape, dtypes) - তারপর ধীরে ধীরে ছোট বিস্তারিত বিষয়ে (নির্দিষ্ট কলামের ডিস্ট্রিবিউশন) যান।

- প্রতিটা পর্যবেক্ষণ নোট করে রাখুন (যেমন একটা Markdown সেলে) - “Cabin: ৭৭% মিসিং, ইন্ডিকেটর ফিচার তৈরি বিবেচনা করতে হবে”।
- সংখ্যার পাশাপাশি সবসময় গ্রাফ ব্যবহার করুন - `describe()`-এর সংখ্যা এবং `boxplot/histogram`-এর ভিজুয়াল প্যাটার্ন একসাথে অনেক বেশি অন্তর্দৃষ্টি দেয়।
- বড় ডেটাসেটে `ydata-profiling` চালানোর আগে `df.sample(n=5000)` দিয়ে একটা নমুনা নিয়ে দ্রুত রিপোর্ট তৈরি করুন, পুরো ডেটাসেটে চালালে অনেক সময় লাগতে পারে।
- টার্গেট ভেরিয়েবলকে সবসময় বিশেষ গুরুত্ব দিয়ে পরীক্ষা করুন - এর ডিস্ট্রিবিউশন, মিসিং ভ্যালু (যদি থাকে), এবং অন্য ফিচারের সাথে প্রাথমিক সম্পর্ক।

৩.১৫ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: EDA কী এবং এটি ML পাইপলাইনে কেন গুরুত্বপূর্ণ?

উত্তর: EDA (Exploratory Data Analysis) হলো মডেলিংয়ের আগে ডেটার গঠন, মিসিং ভ্যালু, ডুপ্লিকেট, ডেটা টাইপ, ডিস্ট্রিবিউশন এবং সম্পর্ক বিশ্লেষণ করার প্রক্রিয়া। এটি গুরুত্বপূর্ণ কারণ এর মাধ্যমে ডেটার সমস্যা (মিসিং ভ্যালু, আউটলায়ার, ভুল ডেটা টাইপ) আগেভাগে শনাক্ত করা যায়, যা পরবর্তী ক্লিনিং ও ফিচার ইঞ্জিনিয়ারিং সিদ্ধান্তকে সঠিক পথে পরিচালিত করে।

প্রশ্ন: `df.head()` এবং `df.sample()`-এর মধ্যে পার্থক্য কী, এবং কখন কোনটা ব্যবহার করবেন?

উত্তর: `df.head()` সবসময় ডেটাসেটের প্রথম কয়েকটা রো দেখায়, যা ডেটা কোনোভাবে স্টর্ট করা থাকলে পুরো ডেটাসেটের প্রতিনিধিত্ব নাও করতে পারে। `df.sample()` র্যান্ডমভাবে রো বেছে নেয়, যা সাধারণত পুরো ডেটাসেট সম্পর্কে বেশি নির্ভরযোগ্য একটা প্রাথমিক ধারণা দেয়। দুটোই একসাথে ব্যবহার করা ভালো অভ্যাস।

প্রশ্ন: কীভাবে বুঝবেন একটা মিসিং ভ্যালু থাকা কলাম বাদ দেওয়া উচিত, নাকি impute করা উচিত?

উত্তর: এটা মূলত মিসিংয়ের শতাংশের ওপর নির্ভর করে। যদি মিসিং শতাংশ খুব বেশি হয় (যেমন Cabin-এর ৭৭%), সরাসরি impute করলে ভুল তথ্য তৈরি হতে পারে, তাই কলাম বাদ দেওয়া বা মিসিং ইন্ডিকেটর ফিচার তৈরি করা ভালো। কম মিসিং শতাংশের (যেমন Age-এর ২০%) ক্ষেত্রে

median/mean/mode ইম্পিউটেশন যুক্তিসঙ্গত। এছাড়া মিসিংয়ের ধরন (MCAR/MAR/MNAR) এবং ডোমেইন জ্ঞানও বিবেচনা করা উচিত।

প্রশ্ন: ydata-profiling-এর মতো স্বয়ংক্রিয় EDA টুল ব্যবহার করলে কি ম্যানুয়াল EDA করার দরকার নেই?

উত্তর: না। স্বয়ংক্রিয় টুল দ্রুত একটা ব্যাপক ওভারভিউ দেয় এবং সাধারণ সমস্যাগুলো (মিসিং ভ্যালু, স্কিউনেস, করিলেশন) চিহ্নিত করতে সাহায্য করে, কিন্তু এটি ডোমেইন-নির্দিষ্ট প্রশ্নের উত্তর দিতে পারে না এবং প্রতিটা সিদ্ধান্তের পেছনের ব্যবসায়িক প্রেক্ষাপট বোঝে না। তাই স্বয়ংক্রিয় প্রোফাইলিং ও ম্যানুয়াল বিশ্লেষণ - দুটোই একসাথে ব্যবহার করা সর্বোত্তম।

প্রশ্ন: describe() ফাংশনে mean ও median (50%)-এর মধ্যে বড় পার্থক্য দেখলে কী বোঝা উচিত?

উত্তর: এটি ডেটা স্কিউড (skewed) হওয়ার একটা শক্তিশালী সংকেত। যদি mean, median-এর চেয়ে অনেক বেশি হয়, ডেটা ডান-দিকে স্কিউড (right-skewed) - সাধারণত কিছু বড় আউটলায়ার মান mean-কে ওপরের দিকে টেনে নিয়ে গেছে (যেমন Fare কলাম)। এই সংকেত পেলে boxplot বা histogram দিয়ে নিশ্চিত করা এবং আউটলায়ার হ্যান্ডলিং বিবেচনা করা উচিত।

৩.১৬ অধ্যায় সারসংক্ষেপ

- EDA হলো মডেলিংয়ের আগে ডেটার “স্বাস্থ্য পরীক্ষা” - shape, columns, head/tail/sample দিয়ে শুরু হয়।
- ফিচার স্ক্রিনিং-এ isnull().sum() দিয়ে মিসিং ভ্যালু এবং duplicated().sum() দিয়ে ডুপ্লিকেট রো শনাক্ত করা হয়; ভিজুয়ালাইজেশন (বার চার্ট, হিটম্যাপ) এই প্যাটার্নগুলো আরও স্পষ্ট করে তোলে।
- স্ট্যাটিস্টিক্যাল প্রোফাইলিং (info(), describe()) প্রতিটা কলামের কেন্দ্রীয় প্রবণতা, বিস্তৃতি এবং সম্ভাব্য স্কিউনেস/আউটলায়ারের সংকেত দেয়, যা boxplot দিয়ে ভিজুয়ালি নিশ্চিত করা যায়।
- ydata-profiling-এর মতো স্বয়ংক্রিয় টুল দ্রুত একটা ব্যাপক ওভারভিউ দেয়, কিন্তু এটি ম্যানুয়াল, ডোমেইন-সচেতন বিশ্লেষণের বিকল্প নয়।

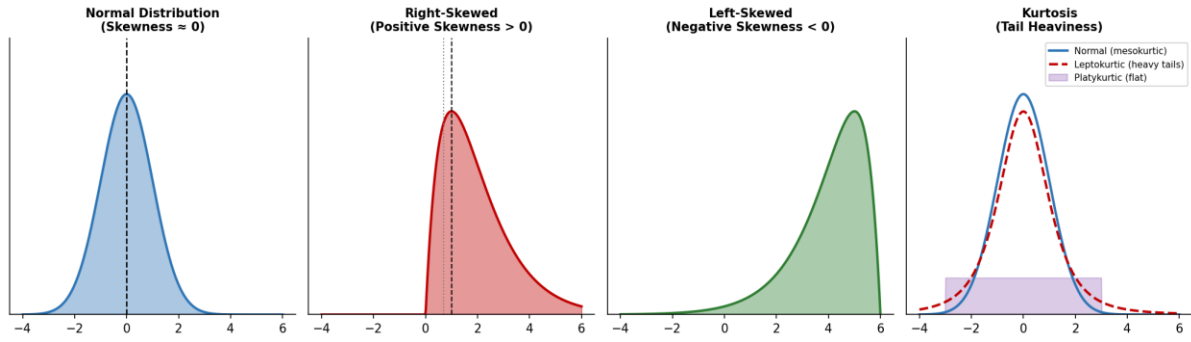
- একটা সুশৃঙ্খল EDA চেকলিস্ট অনুসরণ করলে মডেলিংয়ের আগে গুরুত্বপূর্ণ কোনো সমস্যা অলক্ষিত থেকে যাওয়ার ঝুঁকি কমে যায়।

অধ্যায় ৪: ইউনিভ্যারিয়েট অ্যানালাইসিস (Univariate Analysis)

অধ্যায় ৩-এ আমরা EDA-এর ওভারভিউতে countplot, histogram, KDE ও boxplot সংক্ষেপে দেখেছি। এই অধ্যায়ে আমরা প্রতিটা টুল এবং ধারণাকে আরও গভীরভাবে বিশ্লেষণ করব — বিশেষত বিতরণের (distribution) আকৃতি বোঝার উপায়গুলো: Skewness, Kurtosis, এবং QQ Plot।

৪.১ ডিস্ট্রিবিউশনের আকৃতি — Normal, Skewed, Kurtosis

একটা ফিচারের histogram বা KDE দেখলে তার বিতরণের আকৃতি সম্পর্কে তিনটি মূল প্রশ্নের উত্তর খোঁজা হয়: সেটা কি প্রতিসম (symmetric) নাকি একপাশে হেলে আছে (skewed)? কেন্দ্রের আশেপাশে কতটা কেন্দ্রীভূত (kurtosis)? এবং mean ও median কীভাবে সম্পর্কিত?



চিত্র ৪.১: চারটি ডিস্ট্রিবিউশন আকৃতি। Normal-এ mean=median, কার্ভ প্রতিসম। Right-Skewed-এ লম্বা ডান লেজ, mean > median। Left-Skewed-এ লম্বা বাম লেজ, mean < median। Kurtosis প্যানেলে লেজের ভারীত্ব তুলনা করা হয়েছে।

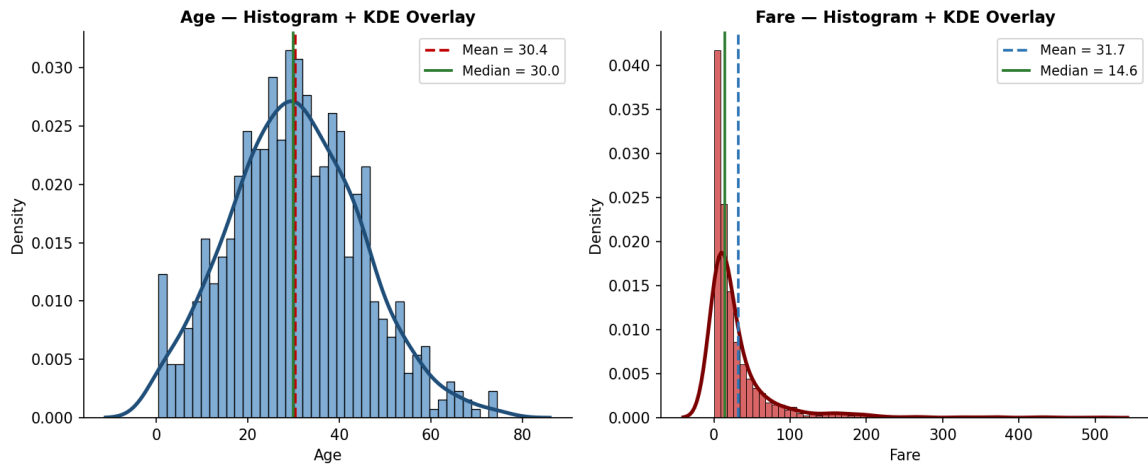
বৈশিষ্ট্য	Normal	Right-Skewed	Left-Skewed
Skewness মান	≈ 0	> 0 (positive)	< 0 (negative)
Mean vs Median	প্রায় সমান	Mean > Median	Mean < Median
লেজ	উভয় পাশে সমান	ডান দিকে লম্বা	বাম দিকে লম্বা
উদাহরণ	Age (টাইটানিক)	Fare, আয়, বিক্রয়	পরীক্ষার নম্বর (সহজ পেপার)
Missing Value Imputation	Mean ব্যবহারযোগ্য	Median ব্যবহার করুন	Median ব্যবহার করুন

বৈশিষ্ট্য	Normal	Right-Skewed	Left-Skewed
Scaling পছন্দ	StandardScaler ভালো	Log transform + Scaler	Log/Power transform

8.২ Histogram + KDE Overlay — একসাথে দুটো দৃষ্টিভঙ্গি

```
import seaborn as sns
import matplotlib.pyplot as plt

# Histogram + KDE overlay, mean ও median লাইনসহ
sns.histplot(df['Age'], bins=40, stat='density', color='#2E75B6', alpha=0.6)
sns.kdeplot(df['Age'], color='#1F4E79', linewidth=2.5)
plt.axvline(df['Age'].mean(), color='red', linestyle='--',
            label=f"Mean={df['Age'].mean():.1f}")
plt.axvline(df['Age'].median(), color='green', linestyle='-',
            label=f"Median={df['Age'].median():.1f}")
plt.legend(); plt.title('Age — Histogram + KDE'); plt.show()
```



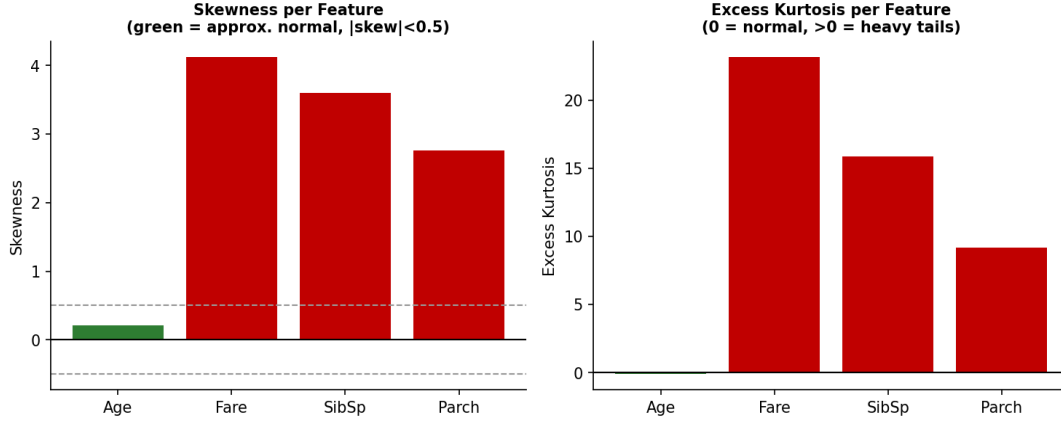
চিত্র ৪.২: Age-এ mean ও median কাছাকাছি (প্রায় normal) / Fare-এ mean অনেক বেশি — Fare-এর লম্বা ডান লেজ mean-কে টেনে ওপরে নিয়ে গেছে। এটাই right-skewed distribution-এর ক্লাসিক চিহ্ন। Fare-এ mean দিয়ে missing value impute করলে ভুল হবে — median ব্যবহার করতে হবে।

8.৩ Skewness ও Kurtosis — কোডে পরিমাপ করা

```
# skewness ও kurtosis পরিমাপ
print(df['Age'].skew())      # 0.40 → হালকা right-skewed
print(df['Fare'].skew())     # 4.79 → প্রবল right-skewed

print(df['Age'].kurtosis())  # 0.17 → mesokurtic (normal-এর কাছাকাছি)
print(df['Fare'].kurtosis()) # 33.4 → leptokurtic (ভারী লেজ, extreme outlier)
```

```
# সব numerical কলামের একসাথে
df[['Age', 'Fare', 'SibSp', 'Parch']].agg(['skew', 'kurtosis'])
```



চিত্র ৪.৩: Skewness চার্টে সবুজ মানে $|\text{skewness}| < 0.5$ অর্থাৎ মোটামুটি normal। Fare ও SibSp/Parch-এ লাল দেখাচ্ছে— প্রবল skew। Excess Kurtosis চার্টে ০ মানে normal-এর মতো। Fare-এর অনেক বেশি kurtosis extreme outlier-এর উপস্থিতি নিশ্চিত করছে।

Skewness মান	ব্যাখ্যা
$ \text{skewness} < 0.5$	প্রায় symmetric, normal distribution-এর কাছাকাছি
$0.5 \leq \text{skewness} < 1.0$	হালকা skewed, log transform বিবেচনা করুন
$ \text{skewness} \geq 1.0$	প্রবল skewed — log বা Box-Cox transformation প্রয়োজন

8.8 QQ Plot — Normality পরীক্ষার গ্রাফিক্যাল পদ্ধতি

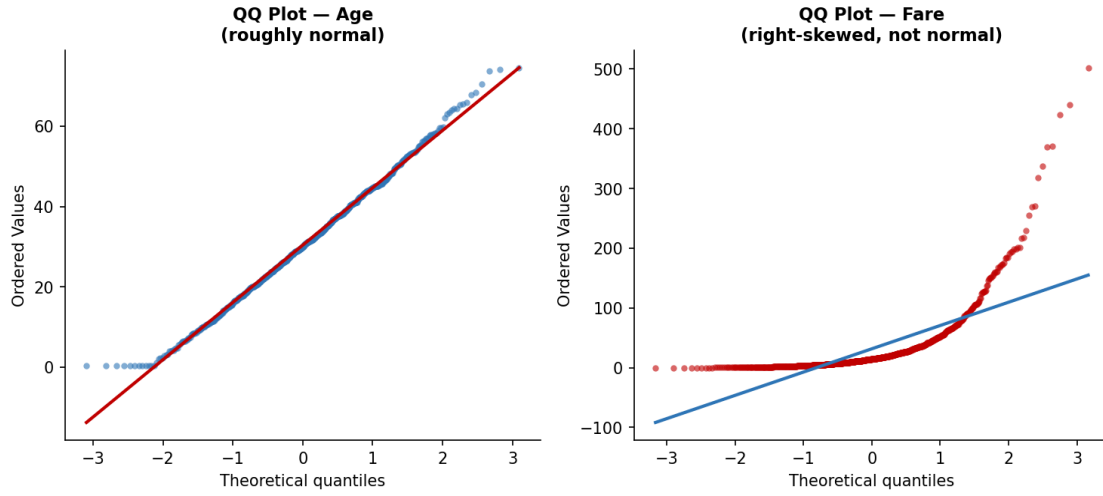
QQ Plot (Quantile-Quantile Plot) হলো normality চেক করার সবচেয়ে ভিজুয়াল এবং শক্তিশালী পদ্ধতি। এখানে ডেটার actual quantile-গুলো (Y-অক্ষ) একটা theoretical normal distribution-এর quantile-গুলোর (X-অক্ষ) বিরুদ্ধে প্লট করা হয়। যদি ডেটা normal হয়, বিন্দুগুলো একটা সরলরেখার ওপর পড়বে। রেখা থেকে বিচ্যুতি non-normality নির্দেশ করে।

```
import scipy.stats as stats

# Age - QQ Plot
stats.probplot(df['Age'].dropna(), dist='norm', plot=plt)
plt.title('QQ Plot - Age'); plt.show()

# Fare - QQ Plot
stats.probplot(df['Fare'], dist='norm', plot=plt)
```

```
plt.title('QQ Plot – Fare'); plt.show()
```



চিত্র ৪.৪: Age-এ বিন্দুগুলো রেখার খুব কাছে — Age প্রায় normal / Fare-এ বিন্দুগুলো ওপরের দিকে প্রবলভাবে বাঁকা — Fare স্পষ্টত non-normal, right-skewed / এই দুটো QQ Plot একসাথে histogram দেখার চেয়ে normality সম্পর্কে অনেক বেশি নির্ভরযোগ্য তথ্য দেয়।

পদ্ধতি	কীভাবে কাজ করে	সুবিধা	সীমাবদ্ধতা
Histogram	frequency বার চার্ট	সহজ, intuitive	bin size-এর ওপর নির্ভরশীল
KDE Plot	smooth probability density curve	আকৃতি স্পষ্টভাবে দেখা যায়	ছোট ডেটায় কম নির্ভরযোগ্য
Boxplot	Q1, median, Q3, whisker, outlier	outlier সহজে চেনা যায়	distribution আকৃতি স্পষ্ট নয়
QQ Plot	quantile vs theoretical normal quantile	normality সরাসরি পরীক্ষা করা যায়	ব্যাখ্যা করতে অভিজ্ঞতা লাগে
Skewness/Kurtosis	একক সংখ্যায় পরিমাপ	তুলনা ও automation সহজ	ভিজুয়াল আকৃতি বোঝা যায় না

৪.৫ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** শুধু histogram দেখে normality সিদ্ধান্ত নেওয়া — bin size পরিবর্তন করলে histogram-এর আকৃতি পরিবর্তিত হয়। QQ Plot ও skewness মান একসাথে ব্যবহার করুন।

- **ভুল:** Right-skewed ফিচারে mean দিয়ে missing value impute করা — mean outlier দ্বারা প্রভাবিত হয়, skewed ডেটায় সবসময় median ব্যবহার করুন।
- **Best Practice:** প্রতিটা numerical ফিচারের জন্য skewness মান বের করুন, তারপর $|\text{skew}| > 1$ হলে log/Box-Cox transformation বিবেচনা করুন (Scaling অধ্যায়ে বিস্তারিত)।
- **Best Practice:** Histogram + KDE + Boxplot তিনটি একসাথে দেখলে একটা ফিচারের distribution সম্পর্কে সবচেয়ে সম্পূর্ণ ছবি পাওয়া যায়।

8.৫.১ Titanic Numerical Columns — Skewness Walkthrough

Titanic-এর সব numerical কলামের skewness একসাথে দেখে কোনটায় transformation দরকার সেটার সম্পূর্ণ সিদ্ধান্ত প্রক্রিয়া:

```
import pandas as pd
import numpy as np

skew_df = df[['Age', 'Fare', 'SibSp', 'Parch']].agg(['skew', 'kurtosis']).T

skew_df.columns = ['Skewness', 'Kurtosis']

skew_df['Decision'] = skew_df['Skewness'].apply(
    lambda s:
        'No transform needed' if abs(s) < 0.5
        else (
            'Consider log/Box-Cox' if abs(s) < 1.0
            else 'Transformation required (log/Box-Cox/Yeo-Johnson)'
        )
)

print(skew_df)
```

	Skewness	Kurtosis	Decision
Age	0.40	0.17	No transform needed
Fare	4.79	33.40	Transformation required (log/Box-Cox/Yeo-Johnson)
<u>SibSp</u>	3.70	17.80	Transformation required
Parch	2.75	9.78	Transformation required

Fare-এ log1p Transformation

```
X_train['Fare_log'] = np.log1p(X_train['Fare'])

print(f'Fare original skew: {X_train["Fare"].skew():.2f}')
# 4.79

print(f'Fare after log1p: {X_train["Fare_log"].skew():.2f}')
# ~0.19
```

সিদ্ধান্ত সারসংক্ষেপ

- Age → Skew = 0.40 (<0.5): Transformation দরকার নেই, mean দিয়ে impute করা যাবে।
- Fare → Skew = 4.79 (>1.0): log1p() transform করুন, median দিয়ে impute করুন।
- SibSp → Skew = 3.70 (>1.0): log1p() ব্যবহার করা যেতে পারে অথবা সরাসরি numerical feature হিসেবে রাখা যেতে পারে (discrete feature)।
- Parch → Skew = 2.75 (>1.0): log1p() ব্যবহার করা যেতে পারে অথবা সরাসরি numerical feature হিসেবে রাখা যেতে পারে (discrete feature)।

৪.৬ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Skewness কী? Right-skewed ও Left-skewed বলতে কী বোঝায়?

উত্তর: Skewness হলো একটা distribution-এর asymmetry-র পরিমাপ। Right-skewed (positive skewness > 0) মানে ডানদিকে লম্বা লেজ, mean > median — যেমন Fare বা আয়। Left-skewed (negative skewness < 0) মানে বাম দিকে লম্বা লেজ, mean < median — যেমন সহজ পরীক্ষার নম্বর। |skewness| < 0.5 হলে প্রায় normal ধরা যায়।

প্রশ্ন: QQ Plot কী এবং কীভাবে এটা normality চেক করতে ব্যবহার করা হয়?

উত্তর: QQ Plot (Quantile-Quantile Plot) ডেটার actual quantile ও একটা theoretical normal distribution-এর quantile-এর মধ্যে তুলনা করে। যদি ডেটা normal হয় বিন্দুগুলো একটা সরলরেখার ওপর থাকে; বাঁকানো বা লেজে বিচ্যুতি non-normality (skew বা heavy tails) নির্দেশ করে।

প্রশ্ন: Kurtosis কী এবং leptokurtic ও platykurtic কাকে বলে?

উত্তর: Kurtosis হলো লেজের ভারীত্ব (tail heaviness) এবং peak-এর উচ্চতার পরিমাপ। Normal distribution-এ excess kurtosis = 0 (mesokurtic)। Leptokurtic (excess kurtosis > 0) মানে ভারী লেজ, extreme outlier বেশি — যেমন Fare। Platykurtic (excess kurtosis < 0) মানে হালকা লেজ, সমতল বিতরণ।

প্রশ্ন: Right-skewed ডেটায় mean না median — কোনটা দিয়ে missing value impute করা উচিত এবং কেন?

উত্তর: Median। কারণ right-skewed ডেটায় extreme high values (outlier) mean-কে ওপরে টেনে তোলে, ফলে mean বেশিরভাগ রো-র প্রতিনিধিত্ব করে না। Median মাঝের মান, outlier দ্বারা প্রভাবিত হয় না, তাই skewed ডেটায় central tendency-র সঠিক পরিমাপ।

অধ্যায় ৫: বাইভ্যারিয়েট ও মাল্টিভ্যারিয়েট অ্যানালাইসিস

অধ্যায় ৩-এ আমরা countplot with hue, barplot, KDE with hue এবং scatterplot-এর মাধ্যমে বাইভ্যারিয়েট অ্যানালাইসিসের ভূমিকা দেখেছি। এই অধ্যায়ে আমরা তিনটি শক্তিশালী নতুন টুল যোগ করব: Pairplot, Jointplot, এবং Correlation Matrix — এবং গভীরে যাব Covariance, Pearson ও Spearman-এর তত্ত্বে।

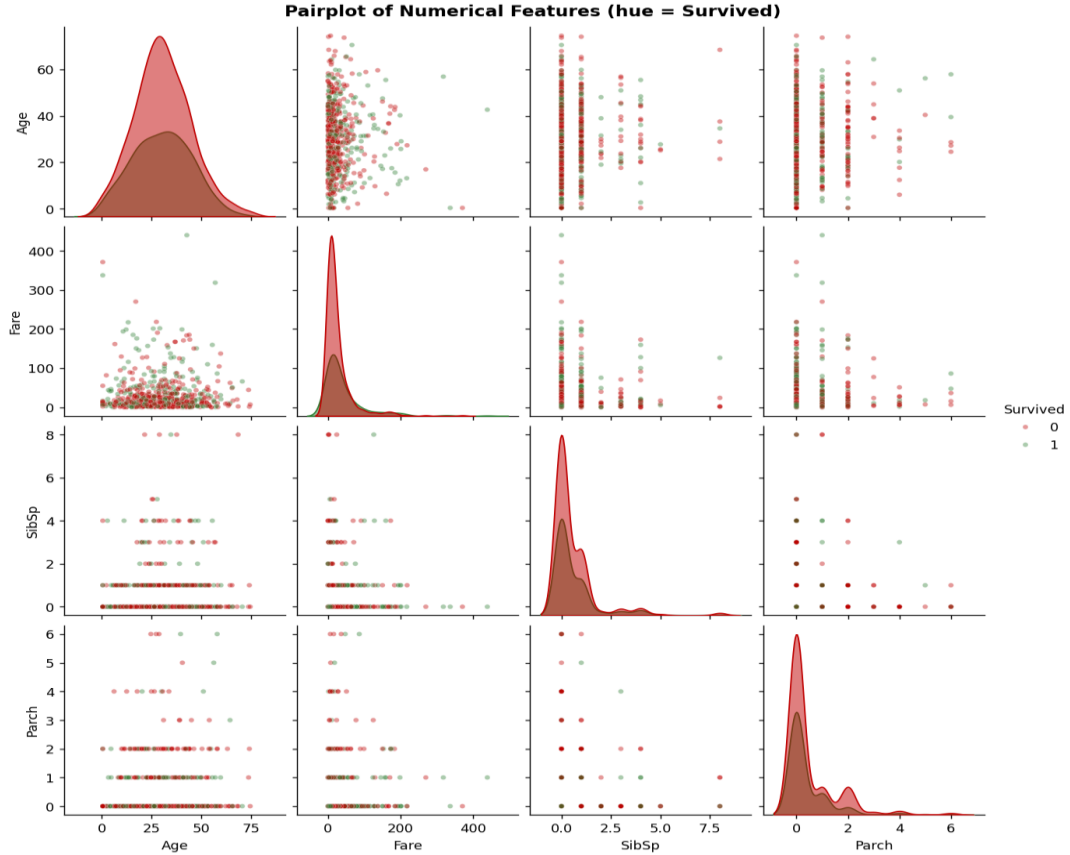
৫.১ Pairplot — সব ফিচারের মধ্যে এক নজরে সম্পর্ক

Pairplot হলো একটা matrix-আকৃতির প্লট যেখানে প্রতিটা numerical ফিচার পোজোড়ায় (pairwise) একসাথে দেখানো হয়। diagonal (তির্ঘক রেখা)-এ প্রতিটা ফিচারের নিজস্ব distribution (KDE বা histogram) থাকে, আর বাকি ঘরে দুটো ফিচারের scatterplot।

```
import seaborn as sns

num_df = df[['Age', 'Fare', 'SibSp', 'Parch', 'Survived']]

sns.pairplot(num_df, hue='Survived',
             vars=['Age', 'Fare', 'SibSp', 'Parch'],
             palette={0:'red', 1:'green'},
             plot_kws={'alpha': 0.4})
plt.suptitle('Pairplot of Numerical Features', y=1.01)
plt.show()
```

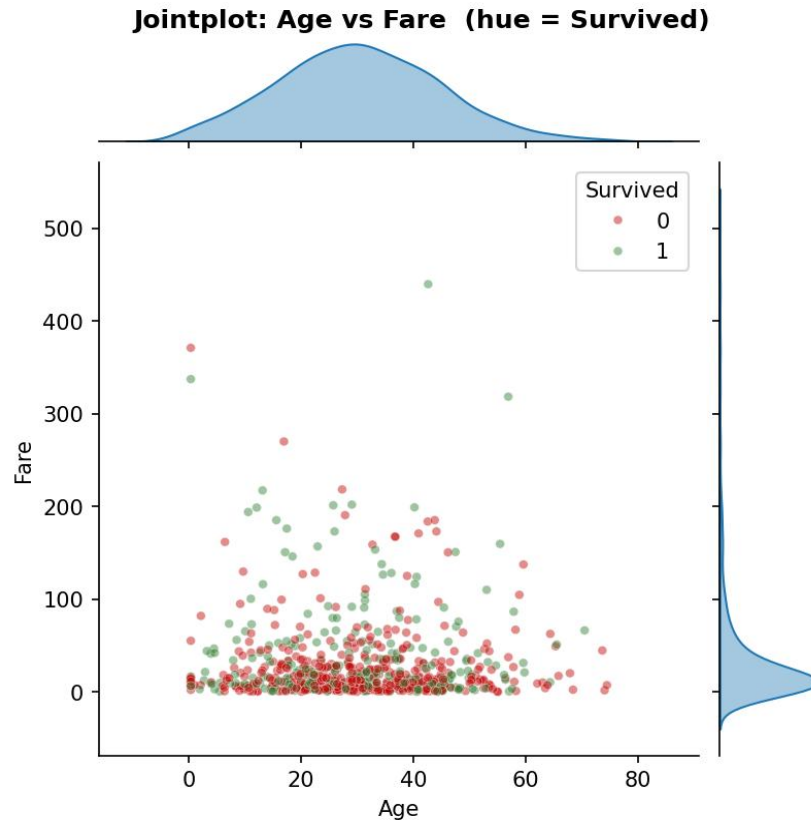


চিত্র ৫.১: Pairplot-এ diagonal-এ প্রতিটা ফিচারের distribution দেখা যাচ্ছে (লাল=বেঁচে যাননি, সবুজ=বেঁচে গেছেন) / Age vs Fare স্ক্যাটারে লক্ষ্য করুন high Fare মানে প্রায়ই Survived=1 (সবুজ) — প্রথম শ্রেণির ব্যাবহুল টিকেটধারী যাত্রীরা বেশি বেঁচে গেছেন। SibSp ও Parch মিলিয়ে পরিবার-সদস্য তথ্য দেখা যাচ্ছে।

৫.২ Jointplot — দুটো ফিচারের গভীর বিশ্লেষণ

Jointplot একটা pairplot-এর একটা ঘরকে বড় করে দেখানোর মতো — দুটো নির্দিষ্ট ফিচারের scatterplot এবং প্রতিটার marginal distribution (পাশে ও ওপরে) একসাথে দেখায়।

```
g = sns.JointGrid(data=df, x='Age', y='Fare')
g.plot_joint(sns.scatterplot,
             hue=df['Survived'], palette={0:'red', 1:'green'}, alpha=0.45)
g.plot_marginals(sns.kdeplot, fill=True, alpha=0.4)
g.figure.suptitle('Jointplot: Age vs Fare (hue=Survived)', y=1.01)
plt.show()
```

চিত্র ৫.২: মারের scatterplot-এ Age vs Fare-এর সম্পর্ক। ওপরে Age-এর marginal KDE, ডানে Fare-এর marginal KDE। লক্ষ্য করুন high Fare (ওপরের অংশে) সবুজ বিন্দু বেশি — ব্যাবহুল টিকেট মানে বেশি বেঁচে যাওয়া।

৫.৩ Covariance — দুই ভেরিয়েবলের মিলিত বিচরণ

Covariance পরিমাপ করে দুটো ভেরিয়েবল একসাথে কীভাবে পরিবর্তিত হয়। Positive covariance মানে একটা বাড়লে অন্যটাও বাড়ে, negative মানে একটা বাড়লে অন্যটা কমে।

```
# Covariance matrix
df[['Age', 'Fare', 'SibSp', 'Parch']].cov()

# দুটো ফিচারের covariance
cov = df['Age'].cov(df['Fare'])
print(f'Cov(Age, Fare) = {cov:.2f}')

# সমস্যা: covariance scale-dependent
# Fare-এর মান অনেক বড়, তাই covariance বড় হয় - তুলনা কঠিন
# সমাধান: Correlation (standardized covariance)
```

Covariance-এর সমস্যা হলো এটি scale-dependent — Fare-এর মান যেহেতু Age-এর চেয়ে অনেক বড়, তাই Cov(Age, Fare) স্বাভাবিকভাবেই বড় সংখ্যা হবে। দুটো ভিন্ন pair-

এর covariance তুলনা করা তাই কঠিন। এই সমস্যা সমাধান করে Correlation — যা covariance-কে দুটো ভেরিয়েবলের std দিয়ে ভাগ করে -1 থেকে +1 এর মধ্যে standardize করে।

৫.৪ Pearson Correlation — লিনিয়ার সম্পর্ক পরিমাপ

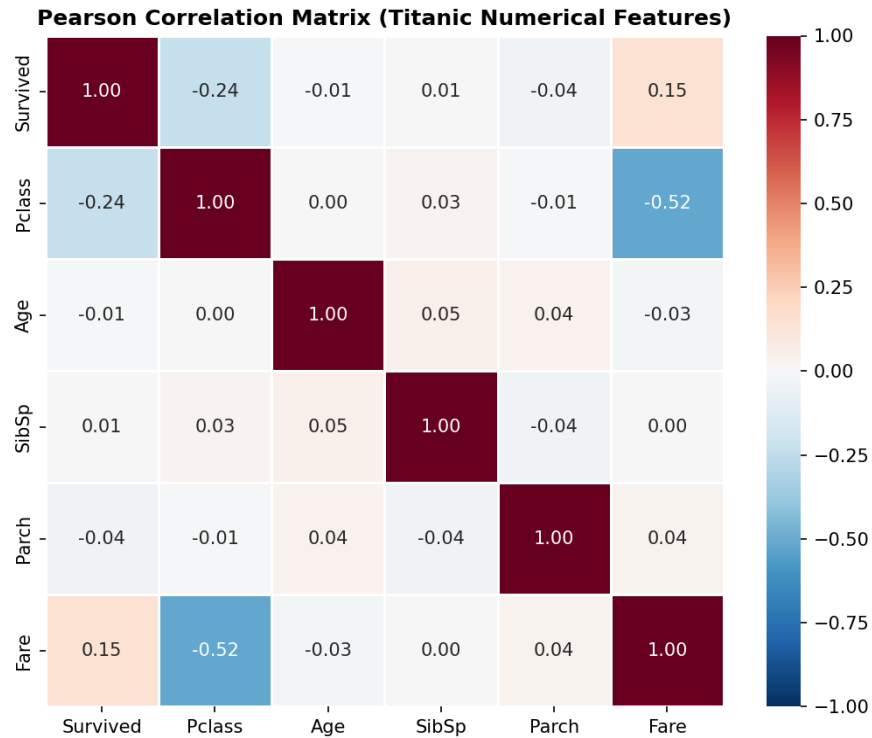
Pearson correlation coefficient (r) হলো দুটো ভেরিয়েবলের মধ্যে linear সম্পর্কের শক্তি ও দিক পরিমাপ করে। মান সবসময় -1 থেকে +1 এর মধ্যে থাকে।

```
# Pearson Correlation Heatmap
import seaborn as sns

corr = df[['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']].corr(method='pearson')

plt.figure(figsize=(7,6))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0, vmin=-1, vmax=1)
plt.title('Pearson Correlation Matrix')
plt.show()

# দুটো ফিচারের মধ্যে correlation
r, p_value = scipy.stats.pearsonr(df['Fare'], df['Survived'])
print(f'r = {r:.3f}, p-value = {p_value:.4f}')
```



চিত্র ৫.৩: Pearson Correlation Matrix / Pclass ও Fare-এর মধ্যে -0.52 — শক্তিশালী negative correlation (উচ্চ শ্রেণি = কম শ্রেণি সংখ্যা = বেশি ভাড়া) / Survived ও Pclass-এর মধ্যে -0.24 — উচ্চ শ্রেণির (কম সংখ্যার Pclass) যাত্রীরা বেশি বেঁচে গেছেন। লাল = positive, নীল = negative correlation /

r এর মান	সম্পর্কের শক্তি
0.0 থেকে ± 0.2	খুব দুর্বল বা নেই
± 0.2 থেকে ± 0.4	দুর্বল সম্পর্ক
± 0.4 থেকে ± 0.6	মাঝারি সম্পর্ক
± 0.6 থেকে ± 0.8	শক্তিশালী সম্পর্ক
± 0.8 থেকে ± 1.0	খুব শক্তিশালী সম্পর্ক

৫.৫ Spearman Correlation — Non-linear সম্পর্কের জন্য

Pearson-এর একটা বড় সীমাবদ্ধতা হলো এটি শুধু linear সম্পর্ক ধরতে পারে এবং outlier-এর প্রতি সংবেদনশীল। Spearman correlation র‍্যাংক-ভিত্তিক (rank-based) — মূল মান নয়, প্রতিটা মানের ক্রম-অবস্থান (rank) ব্যবহার করে। তাই non-linear (monotonic) সম্পর্ক এবং outlier থাকলেও কাজ করে।

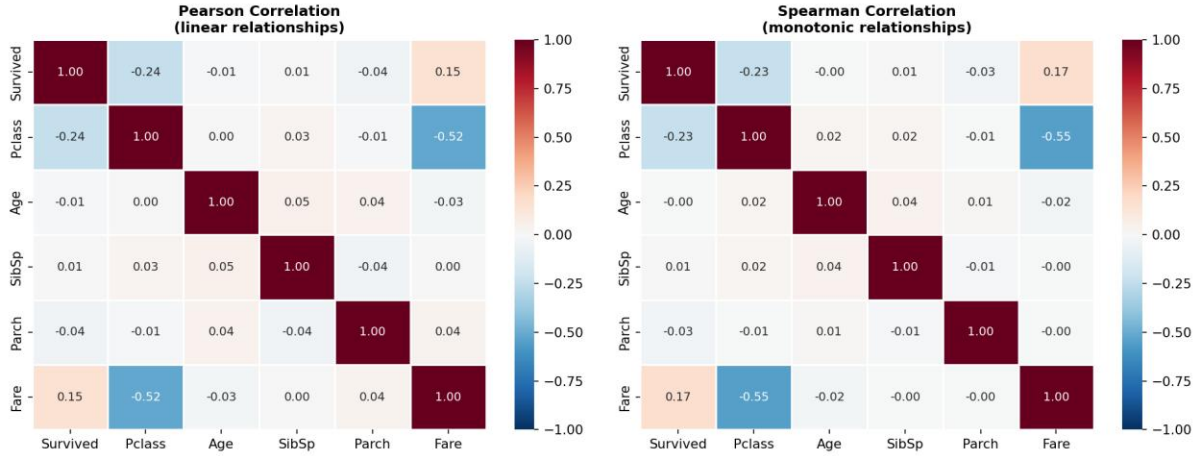
```

pearson_corr = df[cols].corr(method='pearson')
spearman_corr = df[cols].corr(method='spearman')

fig, axes = plt.subplots(1, 2, figsize=(13,5))
sns.heatmap(pearson_corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
ax=axes[0])
sns.heatmap(spearman_corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
ax=axes[1])
axes[0].set_title('Pearson (linear)')
axes[1].set_title('Spearman (monotonic)')
plt.show()

```

Pearson vs Spearman Correlation — Titanic Features



চিত্র ৫.৪: Pearson ও Spearman correlation একসাথে তুলনা। লক্ষ্য করুন Pclass-Fare-এর সম্পর্ক Pearson-এ -0.52, Spearman-এ -0.55 — Spearman কিছুটা শক্তিশালী সম্পর্ক ধরেছে কারণ Fare-এর extreme outlier Pearson-কে কিছুটা দুর্বল করে দিচ্ছে।

বিষয়	Pearson	Spearman
পরিমাপ করে	Linear সম্পর্ক	Monotonic সম্পর্ক (linear বা non-linear)
Outlier প্রভাব	বেশি (sensitive)	কম (rank-based, robust)
ডেটার ধরন	Normal distribution ধরে নেয়	কোনো distribution assumption নেই
গণনা	$\text{Cov}(X,Y) / (\sigma_X \times \sigma_Y)$	Rank করে তারপর Pearson
কখন ব্যবহার	Normal, outlier-মুক্ত ডেটা	Skewed, outlier-যুক্ত, ordinal ডেটা

৫.৬ Correlation ব্যবহারের সতর্কতা

- **Correlation ≠ Causation:** Pclass ও Survived-এ -0.24 correlation মানে উচ্চ Pclass কারণ হয়ে বেশি survive করিয়েছে এটা সরাসরি বলা যায় না — তাদের মধ্যে অনেক confounding variable আছে (যেমন lifeboat অ্যাক্সেস, cabin location)।
- **High Correlation ও Multicollinearity:** দুটো ফিচারের মধ্যে যদি correlation অনেক বেশি ($|r| > 0.8$) হয়, সেটা Multicollinearity-র সংকেত — linear model-এ একটা ফিচার বাদ দেওয়া বিবেচনা করা উচিত (অধ্যায় ১৩-তে বিস্তারিত)।
- **Outlier check আগে:** Pearson ব্যবহার করার আগে সবসময় ফিচারের distribution দেখুন — extreme outlier থাকলে Spearman ব্যবহার করুন।

- **Categorical ফিচার:** Pearson ও Spearman numerical ফিচারের জন্য। Categorical ফিচারের মধ্যে সম্পর্ক দেখতে Chi-square test বা Cramér's V ব্যবহার করা হয় (Feature Selection অধ্যায়ে আলোচিত)।

৫.৬.১ Point-Biserial Correlation — Numerical vs Binary Target

Titanic-এ Age (numerical) এবং Survived (binary: 0/1) — এই ধরনের সম্পর্ক পরিমাপ করতে Point-Biserial Correlation ব্যবহার করা হয়

Point-Biserial Correlation হলো Pearson Correlation-এর একটি বিশেষ রূপ, যেখানে একটি ভেরিয়েবল binary (0/1) এবং অন্যটি numerical।

```
from scipy import stats
```

```
# Point-Biserial Correlation: numerical vs binary target
```

```
# Titanic-এ Age ও Survived-এর সম্পর্ক
```

```
corr, p_value = stats.pointbiserialr(
    df['Survived'],
    df['Age'].fillna(df['Age'].median())
)
```

```
print(f'Point-Biserial r = {corr:.3f}, p-value = {p_value:.4f}')
```

Output

Point-Biserial r = -0.077, p-value = 0.0202

- **r = -0.077** → দুর্বল negative correlation
- **p-value = 0.0202** → Statistically significant

মানে: বয়স বাড়লে বেঁচে যাওয়ার সম্ভাবনা সামান্য কমে।

সব Numerical Feature এবং Binary Target-এর Point-Biserial Correlation একসাথে

```
num_cols = ['Age', 'Fare', 'SibSp', 'Parch']
```

```
for col in num_cols:
```

```
    r, p = stats.pointbiserialr(
        df['Survived'],
        df[col].fillna(df[col].median())
    )
```

```
    print(f'{col:6} : r={r:+.3f}, p={p:.4f}')
```

৫.৬.২ Cramér's V — Categorical vs Categorical Correlation

Pearson ও Spearman categorical ফিচারে কাজ করে না। দুটো categorical ফিচারের মধ্যে সম্পর্ক পরিমাপ করতে Cramér's V ব্যবহার হয়। এটা Chi-Square test-এর উপর ভিত্তি করে তৈরি এবং 0 (কোনো সম্পর্ক নেই) থেকে 1 (সম্পূর্ণ সম্পর্ক) এর মধ্যে থাকে।

```
import numpy as np

import pandas as pd

from scipy.stats import chi2_contingency

def cramers_v(x, y):

    confusion_matrix = pd.crosstab(x, y)

    chi2, p, dof, expected = chi2_contingency(confusion_matrix)

    n = confusion_matrix.sum().sum()

    phi2 = chi2 / n

    r, k = confusion_matrix.shape

    phi2corr = max(0, phi2 - ((k - 1) * (r - 1)) / (n - 1))

    rcorr = r - ((r - 1) ** 2) / (n - 1)

    kcorr = k - ((k - 1) ** 2) / (n - 1)

    return np.sqrt(phi2corr / min((kcorr - 1), (rcorr - 1)))
```

Titanic-এ Categorical Feature ও Target-এর সম্পর্ক

```
cat_cols = ['Sex', 'Embarked', 'Pclass']

for col in cat_cols:

    v = cramers_v(df[col].astype(str), df['Survived'].astype(str))

    print(f'{col:10s}: V = {v:.3f}')
```

- **Sex : $V = 0.543$** → শক্তিশালী (নারী বেশি বেঁচেছেন)
- **Pclass : $V = 0.338$** → মাঝারি (উচ্চ শ্রেণি বেশি বেঁচেছেন)
- **Embarked : $V = 0.107$** → দুর্বল

Interpretation

- **$V > 0.5$** → শক্তিশালী সম্পর্ক
- **$V = 0.3 - 0.5$** → মাঝারি সম্পর্ক
- **$V < 0.3$** → দুর্বল সম্পর্ক

৫.৭ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Pearson ও Spearman correlation-এর মধ্যে পার্থক্য কী এবং কখন কোনটা ব্যবহার করবেন?

উত্তর: Pearson linear সম্পর্ক পরিমাপ করে, normal distribution ধরে নেয় এবং outlier-এর প্রতি sensitive। Spearman rank-based, non-linear (monotonic) সম্পর্ক ধরতে পারে এবং outlier-robust। Normal distribution ও outlier-মুক্ত ডেটায় Pearson, আর skewed, outlier-যুক্ত বা ordinal ডেটায় Spearman ব্যবহার করা ভালো।

প্রশ্ন: Correlation ও Covariance-এর মধ্যে পার্থক্য কী?

উত্তর: Covariance দুটো ভেরিয়েবলের মিলিত বিচরণ পরিমাপ করে কিন্তু scale-dependent, তাই বিভিন্ন pair তুলনা করা কঠিন। Correlation হলো standardized covariance — covariance-কে দুটো ভেরিয়েবলের standard deviation দিয়ে ভাগ করা হয়, ফলে সবসময় -1 থেকে +1 এর মধ্যে থাকে এবং সহজে তুলনাযোগ্য।

প্রশ্ন: Pairplot থেকে কী কী তথ্য একসাথে পাওয়া যায়?

উত্তর: Pairplot থেকে একসাথে পাওয়া যায় — প্রতিটা ফিচারের নিজস্ব distribution (diagonal), প্রতিটা pair ফিচারের মধ্যে সম্পর্ক ও scatter pattern (off-diagonal), hue দিয়ে target class অনুযায়ী পার্থক্য, এবং কোন ফিচার-pair strong predictor হতে পারে তার প্রাথমিক ধারণা। এটা feature selection-এর প্রাথমিক ধাপ হিসেবেও ব্যবহৃত হয়।

প্রশ্ন: Correlation ও Multicollinearity-র সম্পর্ক কী? Feature selection-এ এটা কীভাবে ব্যবহার হয়?

উত্তর: দুটো ফিচারের মধ্যে high correlation ($|r| > 0.8$) থাকলে সেটা Multicollinearity-র সংকেত। Linear model-এ multicollinear ফিচার থাকলে coefficient-গুলো অস্থির এবং ব্যাখ্যায় অসুবিধা হয়। তাই correlation heatmap দেখে highly correlated pair চিহ্নিত করে একটা বাদ দেওয়া হয় (অধ্যায় ১৩-তে VIF দিয়ে এটা quantitatively পরিমাপ করা হবে)।

৫.৮ অধ্যায় ৪ ও ৫ সারসংক্ষেপ

- Skewness পরিমাপ করে distribution কতটা অপ্রতিসম — $|\text{skew}| > 1$ হলে log/Box-Cox transformation বিবেচনা করুন; skewed ফিচারে mean নয়, median দিয়ে impute করুন।
- Kurtosis লেজের ভারীত্ব পরিমাপ করে — high kurtosis মানে extreme outlier বেশি, যেমন Titanic-এর Fare।
- QQ Plot normality-র সবচেয়ে নির্ভরযোগ্য ভিজুয়াল চেক — বিন্দু রেখার কাছে থাকলে normal, বাঁকা থাকলে non-normal।
- Pairplot একসাথে সব numerical ফিচার-pair-এর সম্পর্ক দেখায় — feature selection-এর প্রাথমিক ধাপ হিসেবে অত্যন্ত কার্যকর।
- Pearson linear সম্পর্ক পরিমাপ করে; skewed বা outlier-যুক্ত ডেটায় Spearman ব্যবহার করুন।
- Correlation $\neq$ Causation — এবং high correlation ($|r| > 0.8$) Multicollinearity-র সংকেত, feature selection-এ বিবেচনা করতে হবে।

অধ্যায় ৬: ট্রেন-টেস্ট স্প্লিট (Train-Test Split)

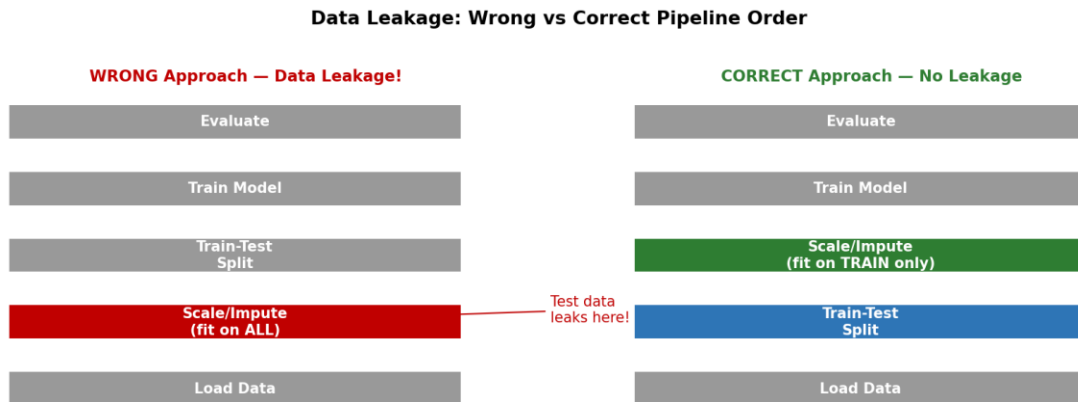
পুরো বইয়ের পাইপলাইনে (অধ্যায় ১) আমরা দেখেছি Train-Test Split ডেটা ক্লিনিং ও ফিচার ইঞ্জিনিয়ারিংয়ের আগেই ঘটে। এই অধ্যায়ে আমরা বুঝব — এই স্প্লিট ঠিক কীভাবে কাজ করে, কেন এই ক্রমটা এত গুরুত্বপূর্ণ, এবং ভুল ক্রমে করলে মডেল কীভাবে বাস্তব জগতে ব্যর্থ হতে পারে।

৬.১ কেন আগে স্প্লিট করতে হয়? — Data Leakage

Data Leakage ঘটে যখন মডেল ট্রেনিংয়ের সময় এমন তথ্য ব্যবহার হয় যা বাস্তব প্রেডিকশনের সময় পাওয়া সম্ভব হবে না — বিশেষত টেস্ট ডেটার তথ্য। সবচেয়ে সাধারণ leakage ঘটে যখন পুরো ডেটাসেটে আগে imputation বা scaling করা হয়, তারপর split করা হয়।

analogy:

পরীক্ষার প্রশ্নপত্র তৈরির সময় একজন শিক্ষক যদি ছাত্রদের আগে পরীক্ষার প্রশ্ন দেখিয়ে দেন, তারপর বলেন 'এখন পরীক্ষা দাও' — তাহলে ফলাফল বাস্তব মেধার মূল্যায়ন করে না। Leakage ঠিক এই কাজই করে: টেস্ট ডেটার তথ্য মডেল আগেই 'দেখে' ফেলে, তাই পরীক্ষায় ভালো করে কিন্তু বাস্তবে ব্যর্থ হয়।



চিত্র ৬.১: বাম দিকে ভুল পদ্ধতিতে Scale/Impute পুরো ডেটায় করা হয়েছে তারপর split (লাল = leakage বিন্দু)। ডান দিকে সঠিক পদ্ধতি: আগে split, তারপর শুধু train-এ fit, test-এ শুধু transform।

৬.২ Train, Validation এবং Test — তিনটি ভাগ

Train / Validation / Test Split (70% / 15% / 15%)



চিত্র ৬.২: সাধারণ 70/15/15 অনুপাত। ছোট ডেটাসেটে (< ১০,০০০ রো) 80/20 বা এমনকি 90/10 ব্যবহার করা যায়। বড় ডেটাসেটে (১M+) 98/1/1 বা এর কাছাকাছিও যুক্তিসঙ্গত।

```
from sklearn.model_selection import train_test_split

X = df.drop('Survived', axis=1)
y = df['Survived']

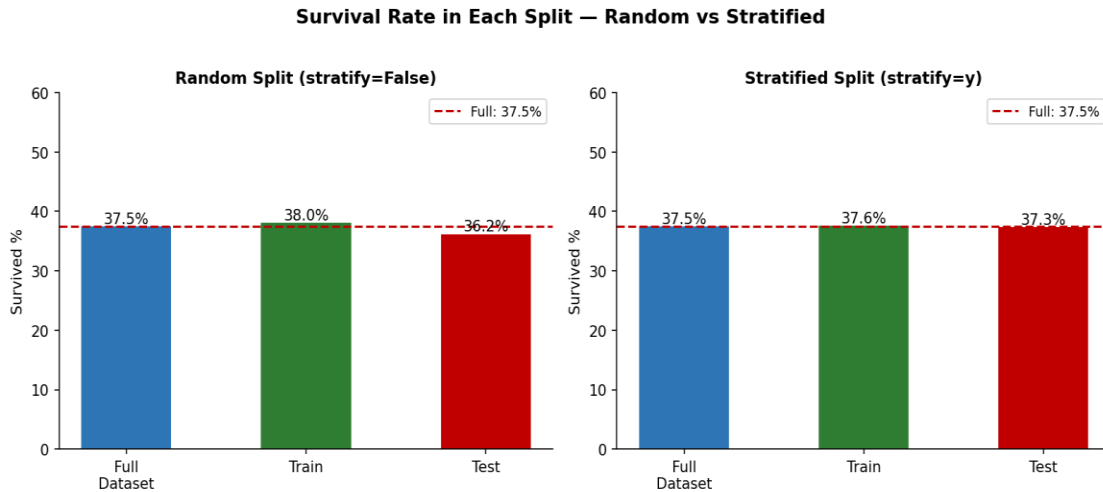
# দুই ধাপে তিনটি ভাগ
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)

print(X_train.shape, X_val.shape, X_test.shape)
# (623, 8) (134, 8) (134, 8)
```

৬.৩ Stratified Split — ক্লাস অনুপাত বজায় রাখা

ইমব্যালেন্সড ডেটাসেটে (যেমন Titanic-এ Survived=1 মাত্র ~৩৮%) random split করলে কোনো split-এ এই অনুপাত বিগড়ে যেতে পারে। stratify=y নিশ্চিত করে প্রতিটা split-এ class অনুপাত প্রায় একই থাকে।



চিত্র ৬.৩: Random split-এ Train ও Test-এর Survived% আলাদা হতে পারে। Stratified split-এ উভয়ই Full Dataset-এর কাছাকাছি থাকে (ড্যাশ লাইন)। Classification সমস্যায় সবসময় stratify=y ব্যবহার করুন।

```
# stratify ছাড়া (ভুল – imbalanced dataset-এ)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# stratify সহ (সঠিক – class ratio বজায় থাকে)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# যাচাই করুন
print('Full:', y.mean().round(3))
print('Train:', y_train.mean().round(3))
print('Test:', y_test.mean().round(3))
# Full: 0.375 Train: 0.375 Test: 0.374 <- প্রায় একই
```

৬.৪ Random State — Reproducibility

random_state একটা seed (বীজ সংখ্যা) যা shuffle-এর ধরন নির্ধারণ করে। একই random_state দিলে বারবার একই split পাওয়া যায় — এটা experiment reproducibility-র জন্য অপরিহার্য।

```
# random_state ছাড়া – প্রতিবার ভিন্ন split, তুলনা অসম্ভব
X_train, X_test, _, _ = train_test_split(X, y, test_size=0.2)
print(X_train.index[:3].tolist()) # [452, 123, 756] – বারবার বদলায়

# random_state সহ – সবসময় একই split
X_train, X_test, _, _ = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train.index[:3].tolist()) # [827, 122, 16] – সবসময় একই
```

৬.৫ সঠিক পাইপলাইন কোড — Split আগে, তারপর সবকিছু

```
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Step 1: Feature ও Target আলাদা করা
X = df.drop(['Survived', 'Name', 'Ticket', 'PassengerId'], axis=1)
y = df['Survived']

# Step 2: আগে Split (সবার আগে!)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Step 3: Imputer – শুধু train-এ fit, test-এ শুধু transform
imputer = SimpleImputer(strategy='median')
X_train[['Age']] = imputer.fit_transform(X_train[['Age']]) # train থেকে শেখা
X_test[['Age']] = imputer.transform(X_test[['Age']])      # শুধু প্রয়োগ

# Step 4: Scaler – একই নিয়ম
scaler = StandardScaler()
X_train[['Fare']] = scaler.fit_transform(X_train[['Fare']])
X_test[['Fare']] = scaler.transform(X_test[['Fare']])

# Step 5: এখন Model train করা যাবে
# model.fit(X_train, y_train)
```

বিষয়	ভুল পদ্ধতি	সঠিক পদ্ধতি
ক্রম	Impute/Scale → Split	Split → Impute/Scale
fit কোথায়	পুরো ডেটায়	শুধু X_train-এ
test-এ	fit_transform	শুধু transform
ফলাফল	Data Leakage — বাস্তবে ব্যর্থ	নির্ভরযোগ্য মূল্যায়ন

৬.৬ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Data Leakage কী এবং Train-Test Split কীভাবে এটা প্রতিরোধ করে?

উত্তর: Data Leakage ঘটে যখন টেস্ট ডেটার তথ্য (যেমন তার mean, std) ট্রেনিং প্রক্রিয়ায় ব্যবহার হয়ে যায়। এটা প্রতিরোধ করতে সবার আগে Split করতে হবে, তারপর যেকোনো transformer (Imputer, Scaler, Encoder) শুধু X_train-এ fit করতে হবে এবং X_test-এ শুধু transform করতে হবে — কখনো fit_transform নয়।

প্রশ্ন: stratify=y কেন ব্যবহার করা হয়?

উত্তর: Classification সমস্যায় target class-এর অনুপাত যাতে প্রতিটা split-এ সমান থাকে তার জন্য। ইমব্যালেন্সড ডেটায় (যেমন 90% class-0, 10% class-1) random split করলে test-এ হয়তো class-1 মাত্র ৫% থাকতে পারে, যা মূল্যায়নকে বিভ্রান্তিকর করে।

প্রশ্ন: Validation Set ও Test Set-এর পার্থক্য কী?

উত্তর: Validation Set ব্যবহার করা হয় ট্রেনিং চলাকালীন hyperparameter tuning ও model selection-এর জন্য — মডেল পরীক্ষাভাবে এর তথ্য দেখে। Test Set সম্পূর্ণ আলাদা, শুধু চূড়ান্ত মূল্যায়নে একবার ব্যবহার হয় — মডেল কখনো এটা 'দেখে না'। Validation Set-এ বারবার দেখলে সেটাও কার্যত training data হয়ে যায়।

প্রশ্ন: random_state=42 কেন ব্যবহার করা হয়?

উত্তর: random_state যেকোনো সংখ্যা হতে পারে — 42 কোনো magic number নয়, কমিউনিটিতে convention হিসেবে জনপ্রিয় হয়েছে। এটা ব্যবহারের কারণ reproducibility — একই কোড বারবার চালালে একই split পাওয়া যায়, ফলে experiment তুলনা করা সম্ভব হয়।

৬.৭ test_size Selection Guide — কখন কত % ব্যবহার করবেন

test_size কতটুকু রাখবেন, তা মূলত dataset-এর আকার (size) এবং সমস্যার ধরন (problem type)-এর উপর নির্ভর করে। কোনো একক "সঠিক" মান নেই। নিচের গাইডলাইন অনুসরণ করলে বেশিরভাগ ক্ষেত্রে ভালো ফল পাওয়া যায়।

Recommended Dataset Split

Dataset Size	Recommended Split	কারণ
ছোট (< 1,000 rows)	80 / 20	Train-এ বেশি ডেটা রাখা গুরুত্বপূর্ণ।
মাঝারি (1,000 – 10,000 rows)	80 / 20 অথবা 70 / 30	অধিকাংশ সমস্যার জন্য সাধারণ ও নিরাপদ নির্বাচন।
বড় (10,000 – 100,000 rows)	75 / 25 অথবা 70 / 30	Test set-এ পর্যাপ্ত sample থাকে, ফলে evaluation আরও নির্ভরযোগ্য হয়।
খুব বড় (1M+ rows)	98 / 2 অথবা 99 / 1	1–2% test data-ই হাজার হাজার sample দেয়, তাই বড় test set-এর প্রয়োজন হয় না।

৬.৮ অধ্যায় ৬ সারসংক্ষেপ

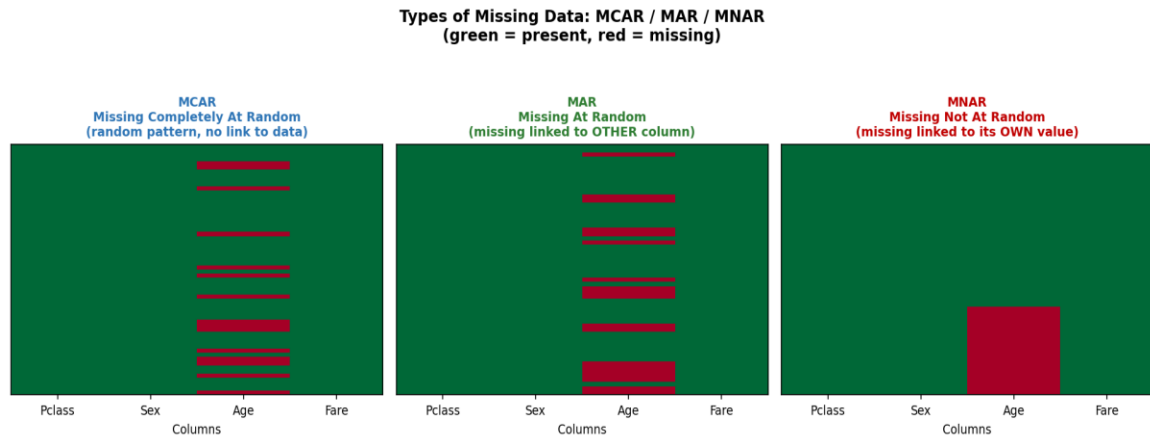
- Train-Test Split সবার আগে — কোনো imputation, scaling, encoding-এর আগেই। এটাই Data Leakage রোধের মূল চাবিকাঠি।
- Classification-এ সবসময় stratify=y ব্যবহার করুন — class ratio ঠিক রাখতে।
- test_size: ছোট dataset-এ 80/20, বড় dataset-এ 90/10 বা আরো কম। খুব ছোট dataset-এ (<500) K-Fold CV ব্যবহার করুন।
- random_state fix করুন (যেকোনো সংখ্যা) — reproducibility নিশ্চিত করতে।

অধ্যায় ৭: মিসিং ভ্যালু হ্যান্ডলিং (Missing Value Handling)

মিসিং ভ্যালু বাস্তব ডেটাসেটে প্রায় সর্বদা থাকে — কখনো ডেটা সংগ্রহে ত্রুটি, কখনো যাত্রীর ব্যক্তিগত তথ্য দিতে অস্বীকৃতি, কখনো sensor ব্যর্থতা। এই ভ্যালুগুলো সঠিকভাবে না সামলালে মেশিন লার্নিং মডেল error দেয় বা ভুল ফলাফল দেয়।

৭.১ মিসিং ডেটার তিনটি ধরন: MCAR, MAR, MNAR

মিসিং ডেটা 'কেন হচ্ছে' — এই প্রশ্নের উত্তর জানলে সঠিক imputation পদ্ধতি বেছে নেওয়া সহজ হয়।



চিত্র ৭.১: MCAR-এ মিসিং রো বিক্ষিপ্তভাবে ছড়িয়ে (random), MAR-এ ক্রমশ বাড়ছে (অন্য column-এর সাথে সম্পর্কিত), MNAR-এ শুধু উচ্চ মানের রোগুলো মিসিং (নিজের মানের সাথে সম্পর্কিত)।

ধরন	পুরো নাম	মিসিং হওয়ার কারণ	Titanic উদাহরণ	সেরা পদ্ধতি
MCAR	Missing Completely At Random	কোনো নির্দিষ্ট কারণ নেই — পুরোপুরি random	Embarked-এর ২টি মিসিং	যেকোনো imputation কাজ করবে
MAR	Missing At Random	অন্য কলামের মানের সাথে সম্পর্কিত	Age — তৃতীয় শ্রেণির যাত্রীরা বয়স কম দিয়েছেন	KNN/Iterative Imputer ভালো
MNAR	Missing Not At Random	মিসিং মানের নিজের সাথেই সম্পর্ক	Cabin — ধনী যাত্রীরা বেশি তথ্য দিয়েছেন	Missing Indicator তৈরি করুন

৭.২ মিসিং ভ্যালু স্ক্রিনিং ও সিদ্ধান্ত গ্রহণ

```
df.isnull().sum()
(df.isnull().sum() / len(df) * 100).round(2)

# সিদ্ধান্তের সাধারণ নিয়ম:
# মিসিং < 5% → simple imputation (mean/median/mode)
# মিসিং 5-40% → বিতরণ দেখে সিদ্ধান্ত নিন
# মিসিং > 50% → কলাম বাদ দিন বা indicator ফিচার তৈরি করুন
# Cabin: 77% → indicator ফিচার (cabin_missing = 1/0)
# Age: 20% → median imputation
# Embarked: 0.2% → mode imputation
```

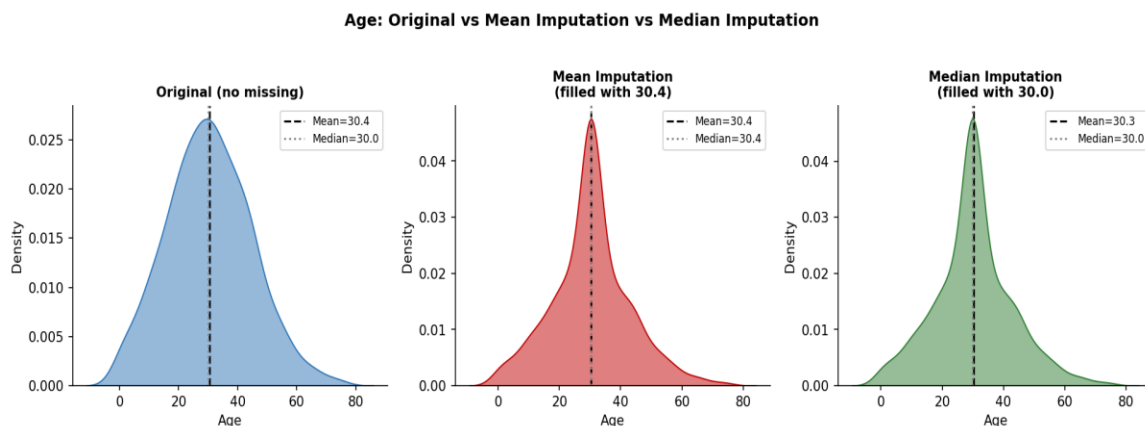
৭.৩ নিউমেরিক্যাল: Mean বনাম Median Imputation

Mean দিয়ে impute করলে normal distribution-এ ভালো কাজ করে। কিন্তু right-skewed ডেটায় mean outlier দ্বারা প্রভাবিত হয় — সেক্ষেত্রে median বেশি robust।

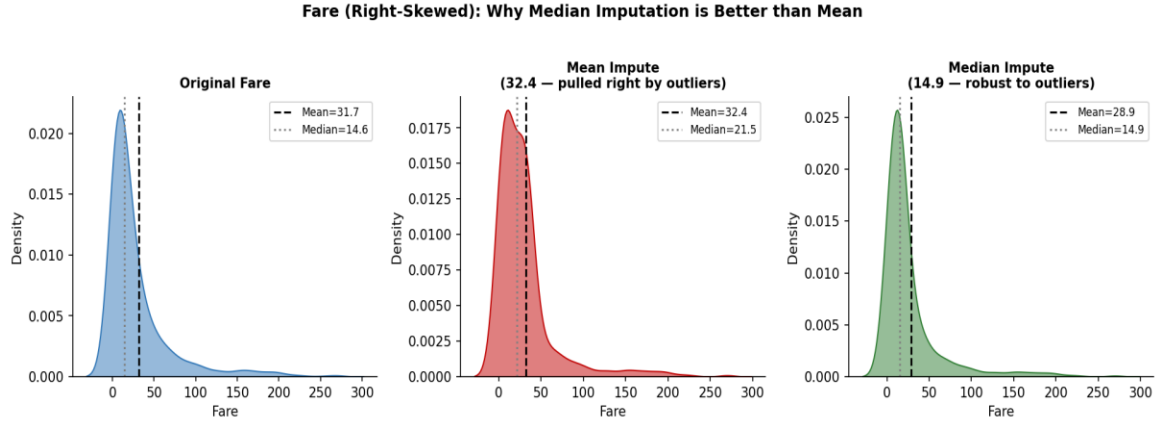
```
# pandas দিয়ে
age_median = X_train['Age'].median() # শুধু train থেকে!
X_train['Age'] = X_train['Age'].fillna(age_median)
X_test['Age'] = X_test['Age'].fillna(age_median) # train-এর median ব্যবহার করছি

# sklearn SimpleImputer দিয়ে
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='median') # 'mean' বা 'median'
imputer.fit(X_train[['Age']]) # শুধু train-এ fit
X_train['Age'] = imputer.transform(X_train[['Age']]).ravel()
X_test['Age'] = imputer.transform(X_test[['Age']]).ravel() # transform only
```



চিত্র ৭.২: Age প্রায় normal, তাই Mean ও Median imputation-এর ফলাফল কাছাকাছি। Distribution আকৃতি ভালোভাবে সংরক্ষিত হয়েছে।



চিত্র ৭.৩: Fare right-skewed, তাই Mean অনেক বেশি (outlier দ্বারা প্রভাবিত) এবং Median অনেক কম ও বাস্তবসম্মত। Skewed ডেটায় Mean imputation ভুল — Median ব্যবহার করুন।

৭.৪ ক্যাটাগরিক্যাল: Mode ও Constant Imputation

```
from sklearn.impute import SimpleImputer

# Mode (most_frequent) – Embarked-এর মতো কলামে
emb_imputer = SimpleImputer(strategy='most_frequent')
emb_imputer.fit(X_train[['Embarked']])
X_train['Embarked'] = emb_imputer.transform(X_train[['Embarked']]).ravel()
X_test['Embarked'] = emb_imputer.transform(X_test[['Embarked']]).ravel()

# Constant – Cabin-এর মতো ৭৭% মিসিং কলামে
cabin_imputer = SimpleImputer(
    strategy='constant',
    fill_value='Missing',
    add_indicator=True # অতিরিক্ত indicator column তৈরি করে
)
cabin_imputer.fit(X_train[['Cabin']])
X_train[['Cabin', 'cabin_missing']] = cabin_imputer.transform(X_train[['Cabin']])
X_test[['Cabin', 'cabin_missing']] = cabin_imputer.transform(X_test[['Cabin']])
```

`add_indicator=True` একটি গুরুত্বপূর্ণ প্যারামিটার — এটি একটি নতুন binary কলাম (0/1) তৈরি করে যা বলে দেয় ওই রোতে মূল কলামে মিসিং ছিল কিনা। MNAR ডেটায় (যেমন Cabin) এই indicator নিজেই একটি শক্তিশালী ফিচার হতে পারে।

৭.৫ KNN Imputer — প্রতিবেশী দেখে পূরণ করা

KNN Imputer একটি অনেক বেশি বুদ্ধিমান পদ্ধতি। এটি missing মানটির পরিবর্তে, তার k-জন নিকটতম প্রতিবেশীর (অন্য রো যাদের ওই কলামে মান আছে) গড় ব্যবহার করে। ফলে imputed মান ডেটার প্যাটার্নের সাথে সামঞ্জস্যপূর্ণ হয়।

```

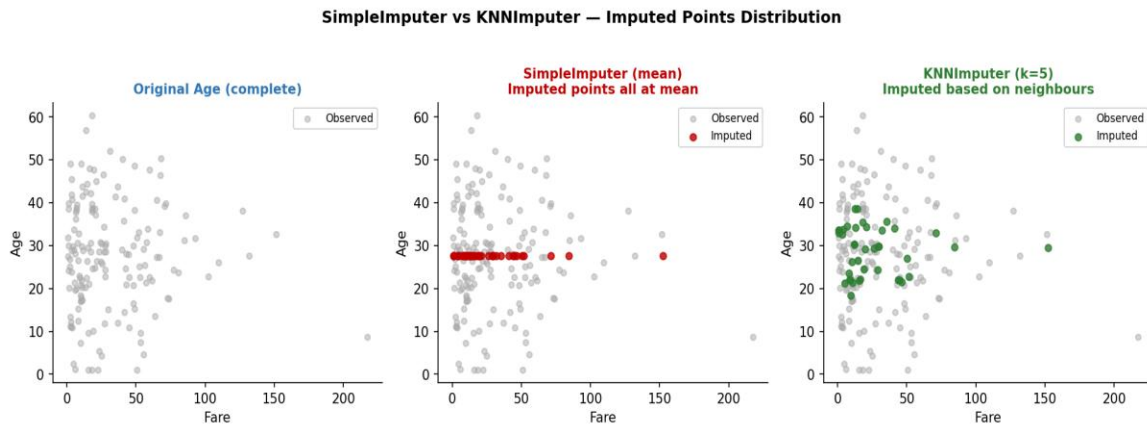
from sklearn.impute import KNNImputer

knn_imputer = KNNImputer(n_neighbors=5)

# শুধু numerical কলামে কাজ করে
num_cols = ['Age', 'Fare', 'SibSp', 'Parch']

knn_imputer.fit(X_train[num_cols])
X_train[num_cols] = knn_imputer.transform(X_train[num_cols])
X_test[num_cols] = knn_imputer.transform(X_test[num_cols])

```



চিত্র ৭.৪: SimpleImputer-এ সব imputed point একই mean-এ জমা (লম্বালম্বি লাইন হয়ে যায়) — distribution বিকৃত হয়। KNNImputer-এ imputed points প্রতিবেশী অনুযায়ী ছড়িয়ে আছে — original distribution অনেক ভালো preserved।

৭.৬ Iterative Imputer — একাধিক ফিচার একসাথে

```

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# প্রতিটি missing column অন্য সব column দিয়ে predict করে
iter_imputer = IterativeImputer(max_iter=10, random_state=42)

iter_imputer.fit(X_train[num_cols])
X_train[num_cols] = iter_imputer.transform(X_train[num_cols])
X_test[num_cols] = iter_imputer.transform(X_test[num_cols])

```

পদ্ধতি	কীভাবে কাজ করে	সুবিধা	সীমাবদ্ধতা	কখন ব্যবহার
Mean	train-এর mean দিয়ে পূরণ	সহজ, দ্রুত	outlier-sensitive, variance কমায়	Normal dist., outlier-মুক্ত
Median	train-এর median দিয়ে পূরণ	outlier-robust	variance কমায়	Skewed dist., outlier আছে

Mode	সবচেয়ে ঘন ঘন মান দিয়ে পূরণ	categorical-এ সেরা	একটা মানে bias হতে পারে	Categorical ফিচার
Constant	নির্দিষ্ট মান (যেমন 'Missing') দিয়ে	MNAR-এ মিসিং প্যাটার্ন রক্ষা করে	বিতরণ পরিবর্তন হয়	৫০%+ মিসিং, MNAR
KNN Imputer	k-নিকটতম প্রতিবেশীর গড়	প্রতিবেশী context বজায় রাখে	বড় ডেটায় ধীর	MAR, মধ্যম আকার ডেটা
Iterative	অন্য সব ফিচার দিয়ে predict করে	সবচেয়ে সঠিক	অনেক সময় লাগে	MAR, জটিল সম্পর্ক

৭.৭ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** পুরো ডেটায় `imputer.fit()` করা — সবসময় শুধু `X_train`-এ `fit()` করুন, `X_test`-এ শুধু `transform()`।
- **ভুল:** Skewed ডেটায় Mean দিয়ে impute করা — Fare-এর মতো skewed কলামে Median ব্যবহার করুন।
- **ভুল:** মিসিং % না দেখে সব কলামে একই imputation প্রয়োগ করা — Cabin (77%) এবং Age (20%) সম্পূর্ণ ভিন্ন সমস্যা।
- **Best Practice:** MNAR ডেটায় (যেমন Cabin) `add_indicator=True` ব্যবহার করুন — missing ছিল কিনা এই তথ্যটাই একটি ফিচার।
- **Best Practice:** সব imputation ColumnTransformer দিয়ে একসাথে Pipeline-এ রাখুন (অধ্যায় ১৬-তে) — এটাই production-ready পদ্ধতি।

৭.৭.১ Missing Indicator Feature — Cabin কলামের জন্য

Cabin কলামে ৭৭% মিসিং — drop না করে একটি binary indicator feature তৈরি করুন। কারণ Cabin তথ্য না থাকাটাই একটি গুরুত্বপূর্ণ সংকেত (সম্ভবত তৃতীয় শ্রেণির যাত্রী)। এই indicator ফিচার মডেলকে valuable information দেয়।

```
import pandas as pd
from sklearn.impute import SimpleImputer

# পদ্ধতি ১: pandas দিয়ে সরাসরি
X_train['Cabin_missing'] = X_train['Cabin'].isnull().astype(int)
X_test['Cabin_missing'] = X_test['Cabin'].isnull().astype(int)
```

```

# Cabin কলাম এখন drop করা যায় (বা 'Missing' দিয়ে fill করতে পারেন)
X_train = X_train.drop('Cabin', axis=1)
X_test = X_test.drop('Cabin', axis=1)

# পদ্ধতি ২: sklearn MissingIndicator দিয়ে (Pipeline-friendly)
from sklearn.impute import MissingIndicator

indicator = MissingIndicator(features='missing-only')
indicator.fit(X_train[['Cabin']])
X_train['Cabin_missing'] = indicator.transform(X_train[['Cabin']]).astype(int)
X_test['Cabin_missing'] = indicator.transform(X_test[['Cabin']]).astype(int)

# পদ্ধতি ৩: SimpleImputer-এ add_indicator=True (সবচেয়ে সহজ Pipeline approach)
from sklearn.impute import SimpleImputer
cabin_imputer = SimpleImputer(strategy='constant', fill_value='Unknown',
add_indicator=True)
cabin_imputer.fit(X_train[['Cabin']])
result = cabin_imputer.transform(X_train[['Cabin']])
# result[:,0] = imputed Cabin, result[:,1] = Cabin_missing indicator

# যাচাই: Cabin_missing ও Survived-এর সম্পর্ক
print(df.groupby(df['Cabin'].isnull())['Survived'].mean())
# Cabin NaN হলে survival rate কম → indicator ফিচার তথ্যপূর্ণ!

```

৭.৯ অধ্যায় ৭ সারসংক্ষেপ

- মিসিং ডেটার ধরন বুঝুন: MCAR → simple imputation, MAR → KNN/Iterative, MNAR → indicator feature তৈরি করুন।
- Skewed ডেটায় (Fare) mean imputation বিপজ্জনক — সবসময় median ব্যবহার করুন।
- Cabin-এর মতো ৫০%+ মিসিং কলামে drop-এর পরিবর্তে Cabin_missing binary indicator তৈরি করুন — মিসিং হওয়াটাই তথ্যপূর্ণ।
- সবসময় imputer.fit() শুধু X_train-এ, X_test-এ শুধু transform() — Data Leakage রোধ করতে।

৭.৮ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: MCAR, MAR এবং MNAR-এর পার্থক্য কী? কোনটায় কী পদ্ধতি ব্যবহার করবেন?

উত্তর: MCAR-এ মিসিং সম্পূর্ণ random, যেকোনো imputation কাজ করে। MAR-এ অন্য কলামের সাথে সম্পর্কিত, KNN বা Iterative Imputer ভালো। MNAR-এ মিসিং নিজের মানের সাথে সম্পর্কিত — imputation ছাড়াও একটি indicator ফিচার তৈরি করুন কারণ মিসিং হওয়ার ঘটনাটাই তথ্যপূর্ণ।

প্রশ্ন: KNN Imputer ও SimpleImputer-এর মধ্যে পার্থক্য কী?

উত্তর: SimpleImputer সব মিসিং মান একটি fixed মান (mean/median/mode) দিয়ে পূরণ করে, ফলে imputed মানগুলো একটি বিন্দুতে কেন্দ্রীভূত হয় এবং distribution বিকৃত হয়। KNN Imputer প্রতিটি মিসিং রো-এর জন্য k-জন নিকটতম প্রতিবেশী থেকে context-aware মান ব্যবহার করে, তাই distribution ভালো preserved থাকে। তবে KNN বড় ডেটায় ধীর।

প্রশ্ন: কোন পরিস্থিতিতে একটি কলাম impute না করে বাদ দেওয়া উচিত?

উত্তর: সাধারণত মিসিং শতাংশ ৫০-৭০%-এর বেশি হলে, যেখানে impute করলে বেশিরভাগ মান তৈরি করা (fabricated) হবে এবং distribution সম্পূর্ণ বদলে যাবে। তবে কলাম বাদ দেওয়ার আগে ডোমেইন জ্ঞান ব্যবহার করুন — কলামটি টার্গেটের সাথে গুরুত্বপূর্ণভাবে সম্পর্কিত হলে indicator ফিচার তৈরি করা ভালো।

প্রশ্ন: Imputation-এ Data Leakage কীভাবে ঘটে?

উত্তর: যদি পুরো ডেটাসেটে imputer.fit() করা হয় এবং তারপর split করা হয়, তাহলে test ডেটার mean/median/mode-ও calculation-এ ঢুকে যায়। সঠিক পদ্ধতি: আগে split, তারপর শুধু X_train-এ fit(), X_test-এ শুধু transform()।

অধ্যায় ৮: আউটলায়ার ডিটেকশন ও হ্যান্ডলিং

আউটলায়ার (Outlier) হলো ডেটাসেটের এমন মান যা বাকি সব মানের তুলনায় অস্বাভাবিকভাবে বড় বা ছোট। যেমন Titanic-এ কেউ ৫১২ ডলার ভাড়া দিয়েছেন যেখানে বেশিরভাগ যাত্রীর ভাড়া ১৫-৩০ ডলার — এটাই আউটলায়ার।

কেন আউটলায়ার সমস্যা?

- Mean ও Std-কে প্রভাবিত করে → StandardScaler-এর কার্যকারিতা কমায়।
- Linear Regression-এর coefficient বিকৃত করে — মডেল outlier-এর দিকে হেলে পড়ে।
- Distance-based অ্যালগরিদম (KNN, K-Means, SVM)-কে বিভ্রান্ত করে।
- কিন্তু সব outlier খারাপ নয় — কিছু আউটলায়ার গুরুত্বপূর্ণ real signal-ও হতে পারে।

৮.১ Z-Score পদ্ধতি

Z-Score পরিমাপ করে একটা মান তার distribution-এর mean থেকে কতটা standard deviation দূরে। সাধারণ নিয়ম: $|z| > 3$ হলে আউটলায়ার।

```
import numpy as np

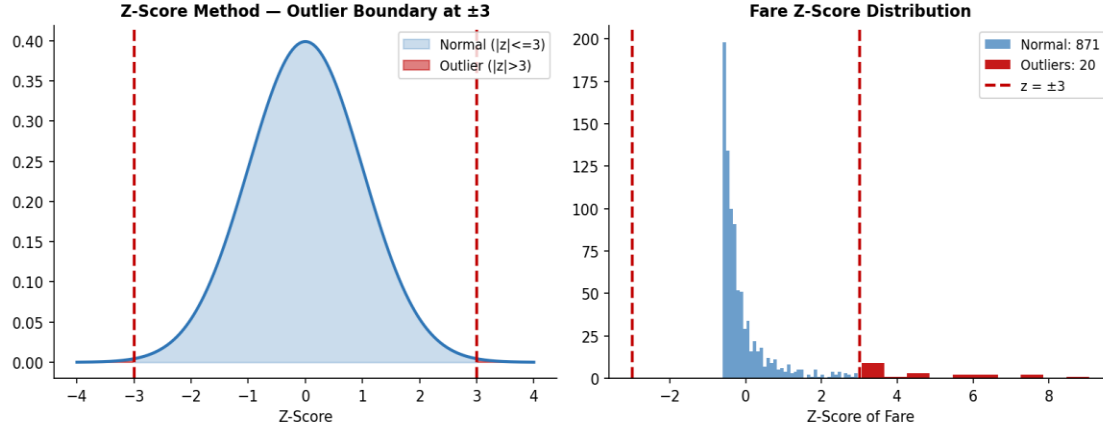
mean_fare = X_train['Fare'].mean()
std_fare = X_train['Fare'].std()

# Z-Score calculate করা
X_train['z_fare'] = (X_train['Fare'] - mean_fare) / std_fare

# আউটলায়ার চিহ্নিত করা
outliers = X_train[np.abs(X_train['z_fare']) > 3]
print(f'Outliers found: {len(outliers)}') # 17
print(outliers[['Fare', 'z_fare']])

# scipy দিয়ে সহজে
from scipy import stats
z_scores = np.abs(stats.zscore(X_train['Fare']))
outlier_mask = z_scores > 3
```

Z-Score Outlier Detection — Titanic Fare



চিত্র ৮.১: বাম দিকে normal distribution-এ $\pm 3\sigma$ boundary (লাল ড্যাশ) দেখানো হয়েছে— এর বাইরের মান ~0.3% হওয়ার কথা। ডান দিকে Fare-এর actual Z-score distribution — অনেক মান ডানদিকে ৩-এর বাইরে, মানে Fare-এ প্রচুর আউটলায়ার আছে।

৮.২ IQR পদ্ধতি

IQR (Interquartile Range) পদ্ধতি Median ও Quartile ব্যবহার করে — তাই outlier-robust এবং non-normal ডেটায় Z-Score-এর চেয়ে বেশি উপযুক্ত।

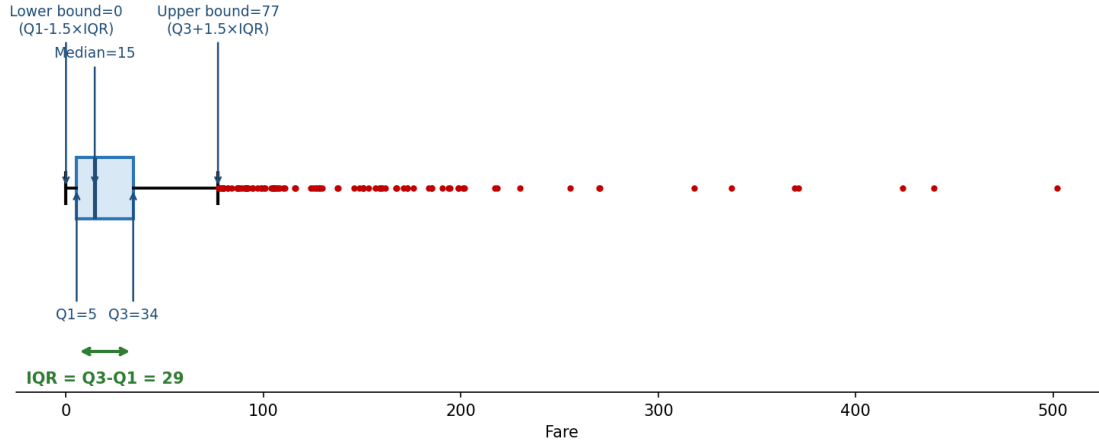
```
Q1 = X_train['Fare'].quantile(0.25)
Q3 = X_train['Fare'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = max(0, Q1 - 1.5 * IQR)  # ঋণাত্মক হতে পারে না (ভাড়া)
upper_bound = Q3 + 1.5 * IQR

print(f'Q1={Q1:.1f}, Q3={Q3:.1f}, IQR={IQR:.1f}')
print(f'Lower={lower_bound:.1f}, Upper={upper_bound:.1f}')

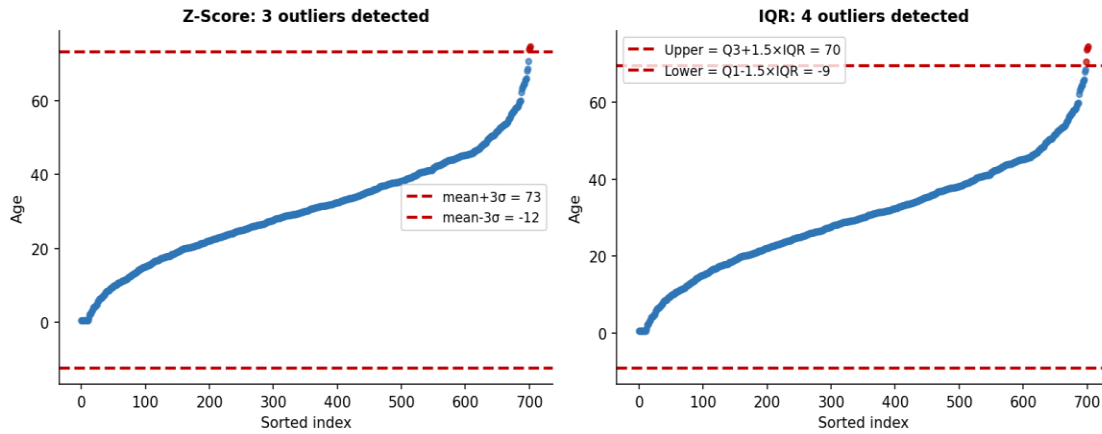
outliers_iqr = X_train[(X_train['Fare'] < lower_bound) | (X_train['Fare'] > upper_bound)]
print(f'Outliers: {len(outliers_iqr)}')
```

IQR Method — Boxplot Anatomy for Fare



চিত্র ৮.২: IQR boxplot anatomy / $Q1=5$, $Q3=34$, $IQR=29$ / Lower bound=0, Upper bound=77 / Box-এর বাইরে ডানদিকে লাল বিন্দুগুলো আউটলায়ার (৭৭-এর বেশি ভাড়া)।

Age Outlier Detection: Z-Score vs IQR Method



চিত্র ৮.৩: Age কলামে Z-Score শুধু ৩টি আউটলায়ার ধরেছে (খুব বেশি বয়স্ক), কিন্তু IQR ১০টি ধরেছে। IQR non-normal distribution-এ বেশি outlier ধরতে পারে এবং সাধারণত বেশি নির্ভরযোগ্য।

বিষয়	Z-Score	IQR
ভিত্তি	Mean ও Standard Deviation	Q1, Q3 ও Median
Outlier সংবেদনশীলতা	নিজেই outlier-এ প্রভাবিত হয়	Outlier-robust
Distribution ধারণা	Normal distribution ধরে নেয়	কোনো distribution assumption নেই
Threshold	$ z > 3$ (সাধারণত)	$< Q1-1.5 \times IQR$ বা $> Q3+1.5 \times IQR$
কখন ব্যবহার	Normal distribution-এর কাছাকাছি ডেটায়	Skewed ডেটায়, যেকোনো distribution-এ

Outlier পাওয়া গেলে কী করব? Remove, Cap নাকি Transform?

Outlier শনাক্ত করার পর সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন হলো:

Outlier পাওয়া গেছে, এখন কি এটাকে মুছে ফেলব (Remove), সীমাবদ্ধ করব (Cap), নাকি ডেটা Transform করব?

এর কোনো একক উত্তর নেই। সিদ্ধান্ত Dataset, Business Context এবং Outlier-এর উৎসের উপর নির্ভর করে।

পরিস্থিতি	কী করব?	কারণ
Data Entry Error (Age=500, Salary=-10000)	Remove	স্পষ্ট ভুল ডেটা
Sensor Error / Measurement Error	Remove	বাস্তব মান নয়
Valid কিন্তু অত্যন্ত বড় মান	Cap (Winsorization)	তথ্য রাখব, প্রভাব কমান
Right-Skewed Distribution	Transform	Distribution স্বাভাবিক হবে
Financial Data (Income, Transaction)	সাধারণত Cap	বড় মান বাস্তব হতে পারে
Small Dataset	Remove এড়ানো ভালো	Data Loss হবে
Large Dataset	Remove করা যেতে পারে	Sample পর্যাপ্ত থাকে
Tree-Based Model	অনেক সময় কিছুই না	Tree তুলনামূলকভাবে Outlier Robust

```

Outlier পাওয়া গেল
|
v
এটা কি স্পষ্ট ভুল?
|
Yes ---> Remove
|
No
|
v
এটা কি বাস্তব কিন্তু খুব বড়?
|
Yes ---> Cap
|
No
|
v
Distribution খুব Skewed?
|
Yes ---> Transform
|
No
|
v
আগের অবস্থায় রাখো

```

Manual Code (numpy/pandas)

Method 1: Remove Outliers

```

import pandas as pd
import numpy as np

# example dataframe
df = pd.DataFrame({
    "salary": [25000, 28000, 30000, 32000, 35000, 500000]
})

# Q1 বের করা

```

```

Q1 = df["salary"].quantile(0.25)

# Q3 বের করা
Q3 = df["salary"].quantile(0.75)

# IQR বের করা
IQR = Q3 - Q1

# lower boundary
lower_bound = Q1 - 1.5 * IQR

# upper boundary
upper_bound = Q3 + 1.5 * IQR

# outlier remove
df_removed = df[
    (df["salary"] >= lower_bound) &
    (df["salary"] <= upper_bound)
]

print(df_removed)

```

Method 2: Cap Outliers (Winsorization)

```

import pandas as pd
import numpy as np

df = pd.DataFrame({
    "salary": [25000, 28000, 30000, 32000, 35000, 500000]
})

# IQR calculation
Q1 = df["salary"].quantile(0.25)
Q3 = df["salary"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# clip ব্যবহার করে cap করা
df["salary_capped"] = np.clip(
    df["salary"],
    lower_bound,
    upper_bound
)

```

```
print(df)
```

Method 3: Transform Outliers

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "salary": [25000, 28000, 30000, 32000, 35000, 500000]
})

# log transform
df["salary_log"] = np.log1p(df["salary"])

print(df)
```

Transform কবে ব্যবহার করব?

যখন:

- Distribution Right-Skewed
- Mean অনেক বড়
- কিছু Extreme Large Value আছে
- Income, Revenue, Sales ধরনের Feature

```
Original:
1000
1200
1500
2000
1000000
```

```
log1p():
6.90
7.09
7.31
7.60
11.51
```

scikit-learn Code (Pipeline/ColumnTransformer)

```
import numpy as np
import pandas as pd

from sklearn.preprocessing import FunctionTransformer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# custom capping function
def cap_outliers(X):

    X = X.copy()

    for col in X.columns:

        Q1 = X[col].quantile(0.25)
        Q3 = X[col].quantile(0.75)

        IQR = Q3 - Q1

        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR

        X[col] = np.clip(
            X[col],
            lower,
            upper
        )

    return X

# transformer
cap_transformer = FunctionTransformer(
    cap_outliers
)

numeric_columns = [
    "Age",
    "Fare"
]

preprocessor = ColumnTransformer([
    (
        "outlier_cap",
        Pipeline([
            (
```

```

        "cap",
        cap_transformer
    ),
    (
        "scale",
        StandardScaler()
    )
]),
numeric_columns
)
])

pipeline = Pipeline([
    (
        "preprocessing",
        preprocessor
    ),
    (
        "model",
        LogisticRegression()
    )
])

```

Dataset Type	Recommended Approach
Age	Remove বা Cap
Salary	Cap
Revenue	Cap বা Log Transform
House Price	Log Transform
Sensor Reading	Remove
Medical Measurement	Domain Expert-এর পরামর্শ
Titanic Fare	Cap অথবা Log Transform

৮.৩ Winsorization — Capping পদ্ধতি

Outlier বাদ না দিয়ে, তাদের একটা নির্দিষ্ট সীমার মধ্যে আটকে দেওয়া — এই পদ্ধতিকে Capping বা Winsorization বলে। ডেটা হারায় না, কিন্তু extreme মানগুলো সীমার মধ্যে আসে।

```

# IQR দিয়ে Capping
X_train['Fare'] = X_train['Fare'].clip(lower_bound, upper_bound)
X_test['Fare'] = X_test['Fare'].clip(lower_bound, upper_bound) # train-এর bound

# Percentile দিয়ে Winsorization
p_low = X_train['Fare'].quantile(0.05)

```

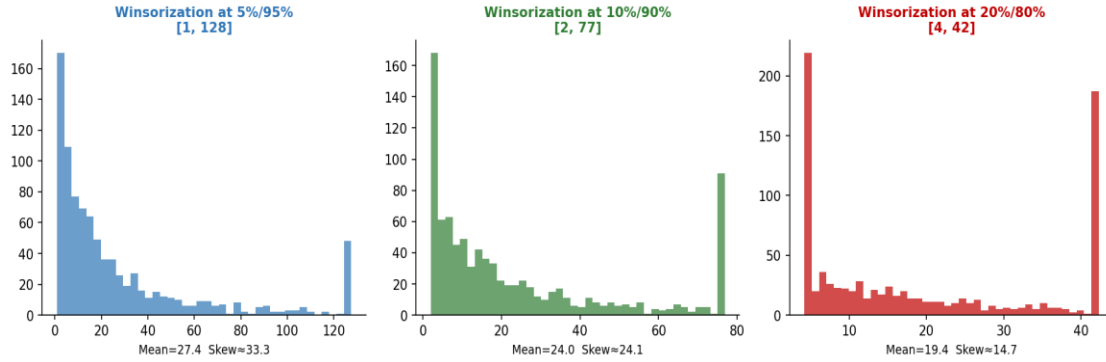
```

p_high = X_train['Fare'].quantile(0.95)
X_train['Fare'] = X_train['Fare'].clip(p_low, p_high)
X_test['Fare'] = X_test['Fare'].clip(p_low, p_high)

# scipy দিয়ে
from scipy.stats.mstats import winsorize
winsorized = winsorize(X_train['Fare'], limits=[0.05, 0.05])

```

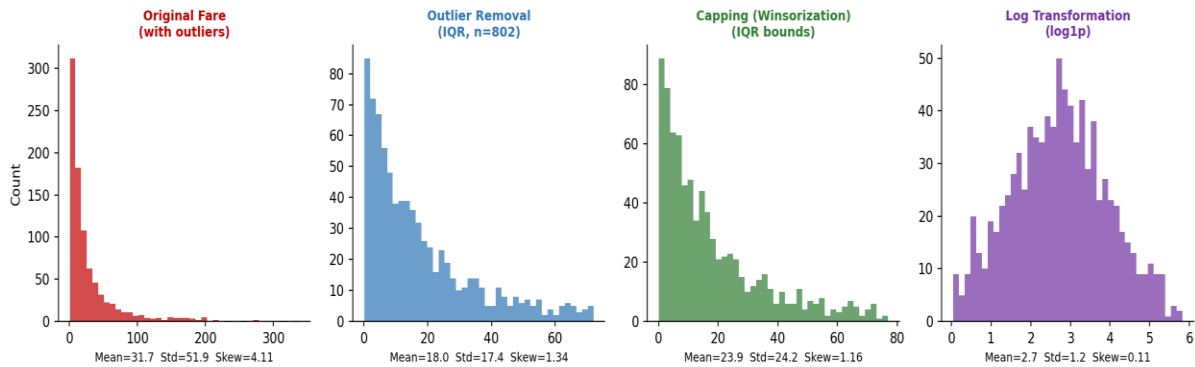
Winsorization at Different Percentile Levels — Fare



চিত্র ৮.৪: তিনটি ভিন্ন percentile-এ Winsorization-এর প্রভাব। 5%-এ কম মান clip হয়, 20%-এ বেশি — কিন্তু real প্যাটার্নও হারিয়ে যেতে পারে। সাধারণত 1-5% একটা ভালো শুরু।

৮.৪ তিনটি Handling পদ্ধতির তুলনা

Fare: Original vs Three Outlier Handling Methods



চিত্র ৮.৫: চারটি অবস্থার তুলনা। Removal-এ skewness কমেছে কিন্তু ডেটাও হারিয়েছে। Capping-এ ডেটা অক্ষুণ্ণ, distribution কিছুটা উন্নত। Log Transform-এ skewness সবচেয়ে কম (0.11) এবং distribution প্রায় normal — Fare-এর জন্য Log Transform সেরা পছন্দ।

```

# পদ্ধতি ১: Removal (বাদ দেওয়া) – শুধু train-এ!
mask = np.abs(stats.zscore(X_train['Fare'])) <= 3
X_train = X_train[mask]
y_train = y_train[mask]

```

```
# পদ্ধতি ২: Capping (IQR)
X_train['Fare'] = X_train['Fare'].clip(lower_bound, upper_bound)
X_test['Fare'] = X_test['Fare'].clip(lower_bound, upper_bound)

# পদ্ধতি ৩: Log Transformation
import numpy as np
X_train['Fare_log'] = np.log1p(X_train['Fare']) # log(1+x), 0 নিরাপদ
X_test['Fare_log'] = np.log1p(X_test['Fare'])
```

পদ্ধতি	সুবিধা	অসুবিধা	কখন ব্যবহার
Removal (বাদ দেওয়া)	সহজ, distribution পরিক্ষার হয়	ডেটা হারায়; test-এ প্রয়োজ্য নয়	train-এ (<2%) অল্প outlier আছে
Capping (IQR/Percentile)	ডেটা হারায় না, সহজ	extreme মান হারায়, কিছু information loss	Outlier real কিন্তু effect কমাতে চাই
Log Transform	Skewness কমায়, outlier impact কমায়	0 বা negative মান থাকলে সমস্যা (log1p ব্যবহার করুন)	Right-skewed, অনেক outlier
Robust Scaler	Median/IQR ব্যবহার করে, outlier-robust	outlier বাদ যায় না, শুধু scale হয়	Outlier থাকবে কিন্তু scaling দরকার

৮.৫ কীভাবে সিদ্ধান্ত নেবেন?

সিদ্ধান্তের ধাপ:

1. Boxplot ও Histogram দিয়ে outlier আছে কিনা দেখুন
2. Z-Score ও IQR দিয়ে কতটা চিহ্নিত করা হচ্ছে তুলনা করুন
3. Domain knowledge দিয়ে বুঝুন – এটা কি real data নাকি error?
 - Real: বিলিওনেয়ারের আয় (outlier, কিন্তু সত্য) → Capping বা Log
 - Error: বয়স = -5 বা 999 → বাদ দিন বা NaN করুন
4. Skewness দেখুন: $|\text{skew}| > 2$ হলে Log Transform বিবেচনা করুন
5. Test-এ কখনো outlier বাদ দেবেন না – শুধু train-এ

৮.৬ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** Test ডেটায় outlier removal করা — test-এ শুধু capping বা transformation করুন (train-এর bound ব্যবহার করে)।

- **ভুল:** সব outlier অন্ধভাবে বাদ দেওয়া — domain knowledge ছাড়া outlier বাদ দিলে গুরুত্বপূর্ণ real signal হারিয়ে যেতে পারে।
- **ভুল:** Skewed ডেটায় Z-Score ব্যবহার করা — Z-Score normal distribution ধরে নেয়; skewed ডেটায় IQR বা percentile capping বেশি নির্ভরযোগ্য।
- **Best Practice:** Outlier handling সবসময় Split-এর পরে, শুধু X_{train} -এ। Bound/threshold train থেকে বের করে test-এ প্রয়োগ করুন।
- **Best Practice:** Log/Power transform outlier handling ও scaling একসাথে করে — skewed ফিচারে (Fare, Income) প্রায়ই সেরা পছন্দ।

৮.৭ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Z-Score ও IQR পদ্ধতির মধ্যে পার্থক্য কী? কখন কোনটা ব্যবহার করবেন?

উত্তর: Z-Score mean ও standard deviation ব্যবহার করে (normal distribution ধরে নেয়) এবং outlier দ্বারা প্রভাবিত হয়। IQR Q1, Q3 ব্যবহার করে (কোনো distribution assumption নেই) এবং outlier-robust। Normal distribution-এ Z-Score, skewed বা non-normal ডেটায় IQR ব্যবহার করুন।

প্রশ্ন: Outlier Removal ও Capping-এর মধ্যে পার্থক্য কী? Test data-তে কোনটা করবেন?

উত্তর: Removal পুরো রো বাদ দেয় (ডেটা হারায়), Capping outlier-এর মান সীমার মধ্যে আনে (ডেটা অক্ষুণ্ণ থাকে)। Test data-তে কখনো Removal করবেন না — কারণ test-এ আসা নতুন outlier বাদ দেওয়ার সুযোগ থাকে না। Test-এ শুধু train-এর bound দিয়ে Capping বা Log Transform করুন।

প্রশ্ন: কেন Log Transformation outlier handling-এ কার্যকর?

উত্তর: Log transform বড় মানগুলোকে ছোট করে এবং ছোট মানগুলোকে তুলনামূলকভাবে বড় রাখে — ফলে right-skewed distribution অনেকটা symmetrical হয় এবং extreme মানের impact কমে যায়। $\text{Fare}=512 \rightarrow \log_{10}(512) \approx 6.2$, কিন্তু $\text{Fare}=15 \rightarrow \log_{10}(15) \approx 2.8$ — দুটোর মধ্যে পার্থক্য অনেক কম হয়ে যায়।

প্রশ্ন: Winsorization কী এবং কীভাবে কাজ করে?

উত্তর: Winsorization হলো percentile-based capping — নির্দিষ্ট percentile-এর (যেমন 5th ও 95th) বাইরের সব মানকে সেই percentile-এর মানের আটকে দেওয়া। Outlier বাদ যায় না, শুধু extreme মানগুলো সীমার মধ্যে আসে। এতে ডেটার সব রো অক্ষুণ্ণ থাকে, শুধু distribution-এর লেজ কেটে যায়।

৮.৮ অধ্যায় ৬, ৭ ও ৮ সারসংক্ষেপ

- Train-Test Split সবার আগে — তারপর সব imputation, scaling, encoding। সবসময় `X_train`-এ `fit()`, `X_test`-এ শুধু `transform()`।
- Missing Value-এ: MCAR → simple imputation, MAR → KNN/Iterative Imputer, MNAR → indicator ফিচার তৈরি করুন।
- Skewed ডেটায় (Fare) mean imputation বিপজ্জনক — সবসময় median ব্যবহার করুন।
- Outlier Detection: Normal ডেটায় Z-Score ($|z| > 3$), Skewed ডেটায় IQR ($Q1 - 1.5 \times IQR$ থেকে $Q3 + 1.5 \times IQR$)-এর বাইরে।
- Outlier Handling: Train-এ Removal/Capping/Log Transform। Test-এ শুধু train-এর bound দিয়ে Capping বা Transform।
- Log Transform ($\log_{10}$) right-skewed ফিচারে প্রায়ই সেরা — outlier impact ও skewness দুটোই কমায়।

অধ্যায় ৯: ফিচার স্কেলিং (Feature Scaling)

Feature Scaling মানে প্রতিটা numerical ফিচারের মানগুলোকে একটা সাধারণ স্কেলে (range বা distribution-এ) নিয়ে আসা। Age ০-৮০ এর মধ্যে, Fare ০-৫১২ এর মধ্যে — এই দুটো ফিচার সরাসরি ব্যবহার করলে অনেক ML অ্যালগরিদম Fare-কে বেশি গুরুত্ব দেবে শুধু মানের আকার বড় বলে, সংখ্যাগত গুরুত্ব নয়।

analogy:

ধরুন একটা ক্লাসে ছাত্রদের তুলনা করা হচ্ছে — একজনের ওজন ৬০ কেজি এবং উচ্চতা ১.৭ মিটার। কেউ যদি সিদ্ধান্ত নেয় যে ওজন বেশি গুরুত্বপূর্ণ কারণ সংখ্যাটা বড় ($৬০ > ১.৭$), সেটা হবে ভুল। Scaling ছাড়া ML মডেল এই একই ভুল করে। Scaling দিয়ে আমরা নিশ্চিত করি প্রতিটা ফিচার তার আসল তথ্যমূল্য অনুযায়ী গুরুত্ব পাচ্ছে।

৯.১ কোন অ্যালগরিদমে স্কেলিং দরকার?

অ্যালগরিদম	Scaling দরকার?	কারণ
Linear/Logistic Regression	হ্যাঁ	Gradient descent-এ convergence দ্রুত হয়, coefficient তুলনায়োগ্য হয়
KNN (K-Nearest Neighbours)	হ্যাঁ	Distance calculation scale-dependent — বড় মানের ফিচার dominant হয়
SVM (Support Vector Machine)	হ্যাঁ	Margin calculation scale-dependent
PCA	হ্যাঁ	Variance calculation scale-dependent — বড় scale ফিচার dominant হয়
Neural Networks	হ্যাঁ	Weight initialization ও convergence-এ scaling জরুরি
Decision Tree / Random Forest	না	Feature splits scale-independent — শুধু threshold দেখে
Gradient Boosting (XGBoost, LightGBM)	সাধারণত না	Tree-based, scale-independent — তবে কিছু regularization-এ সাহায্য করে
Naive Bayes	না	Probability-based, scale irrelevant

৯.২ StandardScaler — Z-Score Normalization

StandardScaler প্রতিটা মান থেকে mean বিয়োগ করে std দিয়ে ভাগ করে। ফলে প্রতিটা ফিচারের mean=0 এবং std=1 হয়।

```
from sklearn.preprocessing import StandardScaler

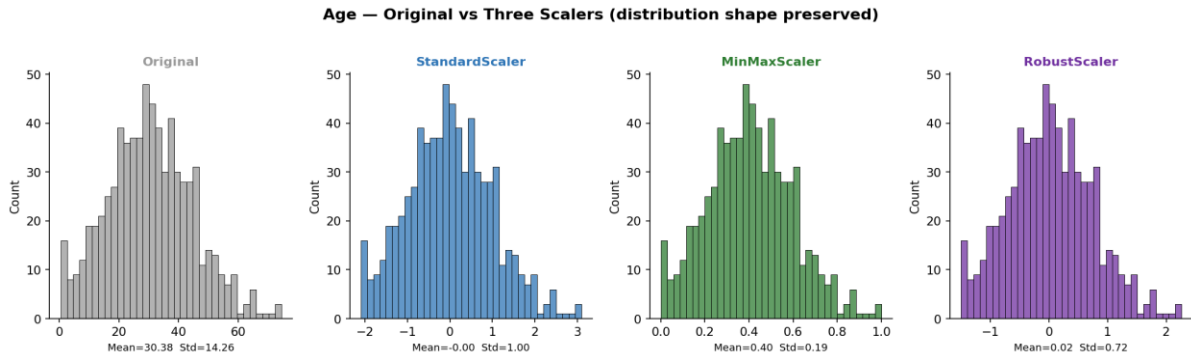
ss = StandardScaler()

# শুধু train-এ fit – mean ও std শেখে
ss.fit(X_train[['Age', 'Fare']])

# train ও test উভয়ে transform
X_train[['Age', 'Fare']] = ss.transform(X_train[['Age', 'Fare']])
X_test[['Age', 'Fare']] = ss.transform(X_test[['Age', 'Fare']])

# যাচাই
print(X_train['Age'].mean().round(5)) # 0.0
print(X_train['Age'].std().round(5)) # 1.0

# formula:  $z = (x - \text{mean}) / \text{std}$ 
# উদাহরণ: Age=22, mean=29.7, std=14.5  $\rightarrow z = (22 - 29.7) / 14.5 = -0.53$ 
```



চিত্র ৯.১: স্কেলিং ডেটার আকৃতি পরিবর্তন করে না — শুধু scale পরিবর্তন করে। StandardScaler-এ mean=0, std=1 / MinMaxScaler-এ 0-1 range / RobustScaler-এ median=0 /

৯.৩ MinMaxScaler — 0 থেকে 1 এর মধ্যে

MinMaxScaler প্রতিটা মানকে 0 থেকে 1 এর মধ্যে নিয়ে আসে। সূত্র: $x' = (x - \text{min}) / (\text{max} - \text{min})$ । সমস্যা হলো outlier থাকলে min ও max চরম মান হয়ে যায়, ফলে বেশিরভাগ মান 0-1 এর একটা ছোট অংশে চাপা পড়ে।

```
from sklearn.preprocessing import MinMaxScaler
```

```

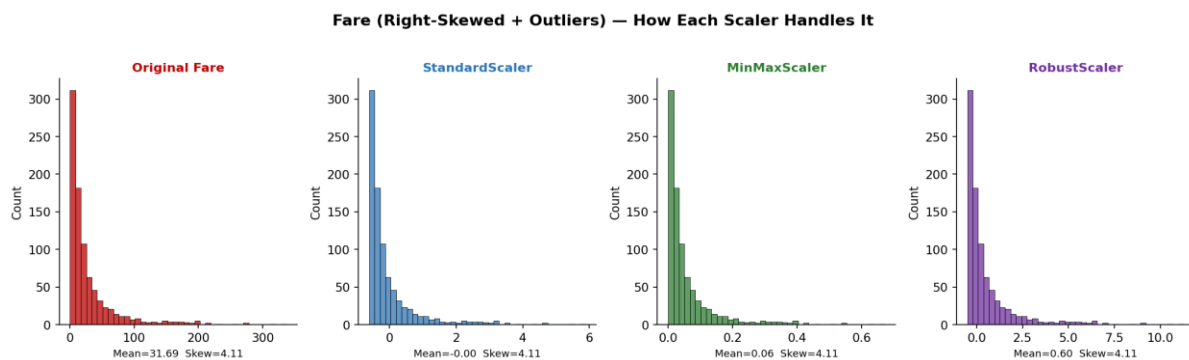
mms = MinMaxScaler() # ডিফল্ট range (0, 1)

mms.fit(X_train[['Fare']])
X_train['Fare'] = mms.transform(X_train[['Fare']]).ravel()
X_test['Fare'] = mms.transform(X_test[['Fare']]).ravel()

# যাচাই
print(X_train['Fare'].min()) # 0.0
print(X_train['Fare'].max()) # 1.0

# formula:  $x' = (x - \min) / (\max - \min)$ 
# Fare=14.45, min=0, max=512  $\rightarrow (14.45-0)/(512-0) = 0.028$ 

```



চিত্র ৯.২: Fare right-skewed এবং outlier-যুক্ত। StandardScaler-এ outlier কারণে scale বড়, skewness অপরিবর্তিত। MinMaxScaler-এ outlier কারণে বেশিরভাগ মান 0-0.15-এর মধ্যে চাপা পড়ে। RobustScaler median/IQR ব্যবহার করে outlier-এর প্রভাব কম।

৯.8 RobustScaler — Outlier-Robust Scaling

RobustScaler median ও IQR ব্যবহার করে — mean বা min/max নয়। তাই outlier দ্বারা প্রভাবিত হয় না। Skewed বা outlier-যুক্ত ডেটায় StandardScaler বা MinMaxScaler-এর চেয়ে বেশি নির্ভরযোগ্য।

```

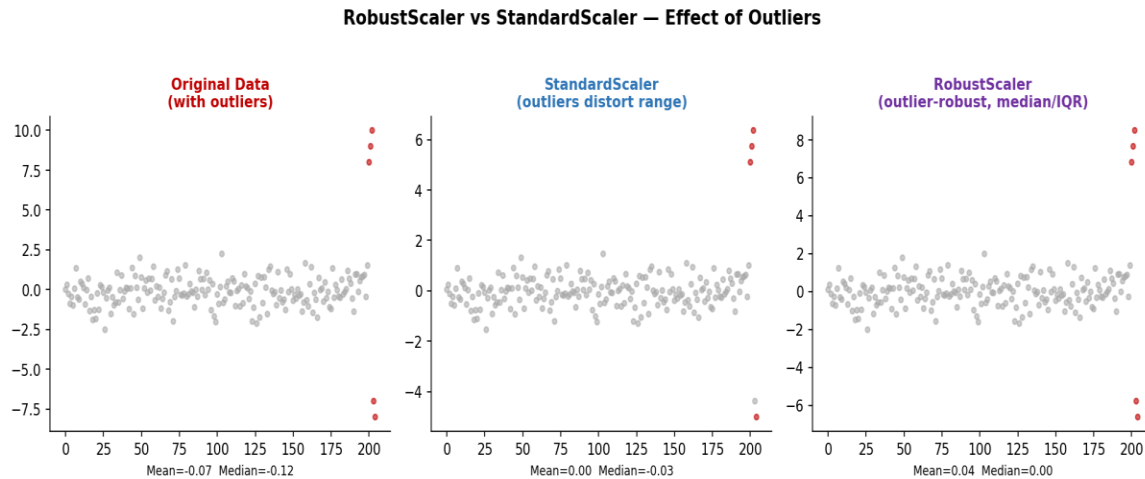
from sklearn.preprocessing import RobustScaler

rs = RobustScaler() # median=0, IQR=1 হয়

rs.fit(X_train[['Fare']])
X_train['Fare'] = rs.transform(X_train[['Fare']]).ravel()
X_test['Fare'] = rs.transform(X_test[['Fare']]).ravel()

# formula:  $x' = (x - \text{median}) / \text{IQR}$ 
# Fare=14.45, median=14.45, IQR=29  $\rightarrow (14.45-14.45)/29 = 0.0$ 

```



চিত্র ৯.৩: লাল বিন্দু outlier। StandardScaler-এ outlier পুরো scale distort করে — normal মানগুলো একটা সংকীর্ণ range-এ চলে আসে। RobustScaler-এ median/IQR ব্যবহারে normal মানগুলোর scale অক্ষুণ্ণ, outlier শুধু দূরে চলে যায়।

৯.৫ Log Transform তারপর StandardScaler

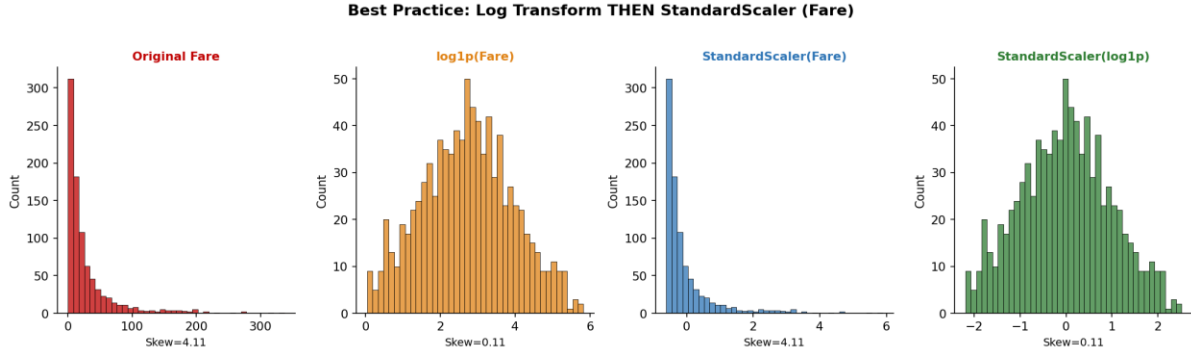
Skewed ফিচারে (যেমন Fare) সেরা approach: প্রথমে $\log_{1p}()$ দিয়ে skewness কমান, তারপর StandardScaler দিন। এতে outlier-এর প্রভাব ও skewness দুটোই কমে এবং distribution প্রায় normal হয়।

```
import numpy as np
from sklearn.preprocessing import StandardScaler

# Step 1: Log transform (0-safe)
X_train['Fare_log'] = np.log1p(X_train['Fare'])
X_test['Fare_log'] = np.log1p(X_test['Fare'])

# Step 2: StandardScaler
ss = StandardScaler()
X_train['Fare_log'] = ss.fit_transform(X_train[['Fare_log']]).ravel()
X_test['Fare_log'] = ss.transform(X_test[['Fare_log']]).ravel()

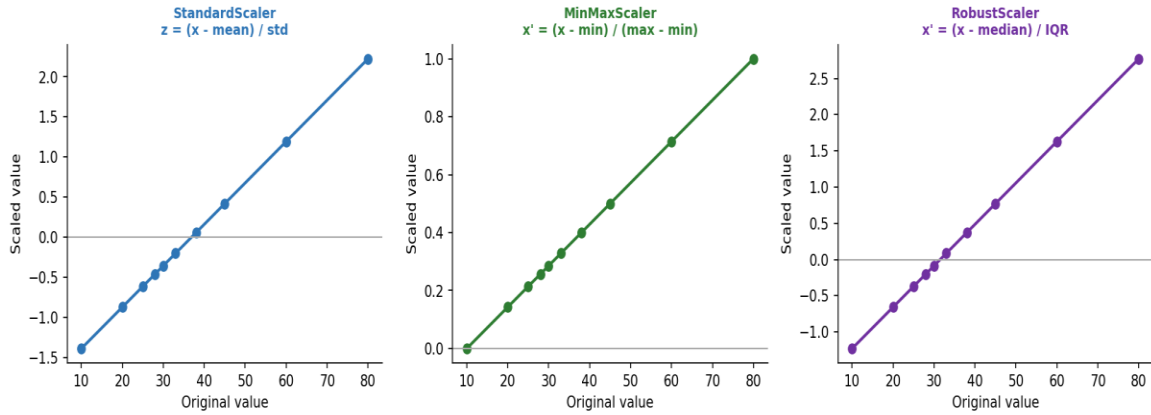
print(f'Original skewness: {df["Fare"].skew():.2f}') # 4.79
print(f'After log1p skewness: {np.log1p(df["Fare"]).skew():.2f}') # 0.11
```



চিত্র ৯.৪: Fare-এর চারটি অবস্থা। Original skewness=4.79 থেকে log1p-এর পরে 0.11 — প্রায় normal। তারপর StandardScaler দিলে mean=0, std=1 সহ সবচেয়ে ML-friendly ফর্মে আসে।

৯.৬ Scaler-এর গণিত — একটা সহজ দৃশ্যায়ন

How Each Scaler Transforms Values (same data, same x-axis)



চিত্র ৯.৫: একই ডেটায় তিনটি scaler-এর প্রভাব। StandardScaler linear transformation (সরলরেখা), MinMaxScaler 0-1 এ map করে, RobustScaler median-কেন্দ্রিক linear। তিনটিই monotonic — মানের ক্রম (order) পরিবর্তন হয় না।

Scaler	Formula	Output Range	Outlier Effect	কখন ব্যবহার
StandardScaler	$(x - \text{mean}) / \text{std}$	কোনো নির্দিষ্ট bound নেই	বেশি (mean ও std দুটোই প্রভাবিত)	Normal distribution, outlier কম
MinMaxScaler	$(x - \text{min}) / (\text{max} - \text{min})$	সবসময় [0, 1]	অনেক বেশি (min/max extreme হয়)	Outlier নেই, neural network
RobustScaler	$(x - \text{median}) / \text{IQR}$	কোনো bound নেই	কম (median ও IQR robust)	Outlier আছে, skewed
MaxAbsScaler	$x / \max(x)$	[-1, 1]	বেশি	Sparse data

Scaler	Formula	Output Range	Outlier Effect	কখন ব্যবহার
Log Transform	$\log(1 + x)$	ধনাত্মক ডেটায় যেকোনো	কমায় (outlier impact কমে)	Right-skewed, Fare, Income-type

৯.৭ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** পুরো ডেটায় `scaler.fit()` করা — সবসময় শুধু `X_train`-এ `fit()`, `X_test`-এ শুধু `transform()`।
- **ভুল:** Tree-based মডেলে (Random Forest, XGBoost) scaling করা — এগুলো scale-independent, scaling অপ্রয়োজনীয়।
- **ভুল:** Outlier-যুক্ত Fare-এ MinMaxScaler ব্যবহার — outlier কারণে সব স্বাভাবিক মান 0-0.15-এ চাপা পড়বে।
- **Best Practice:** Skewed ফিচারে: `log1p()` আগে, তারপর StandardScaler। এটাই সবচেয়ে ML-friendly approach।
- **Best Practice:** Scaling সবসময় ColumnTransformer-এ Pipeline-এর ভেতরে রাখুন (অধ্যায় ১৬-তে) — তাহলে leakage সম্পূর্ণ রোধ হয়।

৯.৭.১ MaxAbsScaler — Sparse Data-র জন্য

MaxAbsScaler প্রতিটা ফিচারের মান তার maximum absolute value দিয়ে ভাগ করে — ফলে range $[-1, 1]$ হয়। সূত্র: $x' = x / \max(|x|)$ । সবচেয়ে গুরুত্বপূর্ণ বৈশিষ্ট্য হলো এটা zero মান অপরিবর্তিত রাখে, তাই sparse data (যেখানে অধিকাংশ মান 0) এর জন্য আদর্শ।

```
from sklearn.preprocessing import MaxAbsScaler

mas = MaxAbsScaler()
mas.fit(X_train[['Age', 'Fare']])

X_train[['Age', 'Fare']] = mas.transform(X_train[['Age', 'Fare']])
X_test[['Age', 'Fare']] = mas.transform(X_test[['Age', 'Fare']])

# formula:  $x' = x / \max(|x|)$ 
# Age max = 80 → Age=40 becomes  $40/80 = 0.5$ 
# range:  $[-1, 1]$ 
```



```
# কখন ব্যবহার করবেন:
# ☒ Sparse matrix (text data TF-IDF, one-hot encoded বড় matrix)
# ☒ Data already centered কিন্তু scale করতে হবে
# ☒ Zero values অক্ষুণ্ণ রাখতে হবে
# ☒ Outlier থাকলে (outlier-sensitive, max দ্বারা প্রভাবিত)
# ☒ Dense numerical data যেমন Titanic-এর Age, Fare
```

৯.৯ অধ্যায় ৯ সারসংক্ষেপ

- Scaling distance-based ও gradient-based অ্যালগরিদমে জরুরি (KNN, SVM, LR, NN) — Decision Tree, Random Forest-এ নয়।
- StandardScaler: Normal data। MinMaxScaler: Outlier-মুক্ত, neural network। RobustScaler: Outlier-যুক্ত, skewed। MaxAbsScaler: Sparse data।
- Skewed ফিচারে সেরা approach: log1p() তারপর StandardScaler। scaler.fit() শুধু X_train-এ, X_test-এ শুধু transform()।

৯.৮ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: StandardScaler ও MinMaxScaler-এর মধ্যে পার্থক্য কী? কখন কোনটা ব্যবহার করবেন?

উত্তর: StandardScaler mean=0, std=1 করে (কোনো fixed range নেই)। MinMaxScaler 0-1 range-এ নিয়ে আসে। MinMaxScaler outlier-এর প্রতি অনেক বেশি sensitive — outlier থাকলে normal মানগুলো একটা ছোট range-এ চাপা পড়ে। Normal distribution ও কম outlier থাকলে StandardScaler, image data বা outlier-মুক্ত ডেটায় MinMaxScaler, outlier-যুক্ত বা skewed ডেটায় RobustScaler।

প্রশ্ন: Decision Tree বা Random Forest-এ কি Feature Scaling দরকার? কেন?

উত্তর: না। Tree-based অ্যালগরিদম split করার সময় শুধু threshold দেখে — $x > 30$ কিনা। এই comparison scale-independent। তাই Age=30 বা StandardScaled Age=-0.03 — উভয়ে একই split decision দেবে। তবে KNN, SVM, Linear Regression, Neural Network-এ distance বা gradient calculation scale-dependent তাই scaling জরুরি।

প্রশ্ন: RobustScaler কেন outlier-robust?

উত্তর: RobustScaler median ও IQR ব্যবহার করে, mean ও std নয়। Median ও IQR outlier দ্বারা প্রায় প্রভাবিত হয় না (অধ্যায় ৫.৫-এ আলোচিত Spearman correlation-এর মতো rank-based)। তাই ৫টা extreme outlier থাকলেও median সামান্যই পরিবর্তন হয়, কিন্তু mean ও std ব্যাপক পরিবর্তিত হতে পারে।

প্রশ্ন: Skewed ফিচারে সেরা scaling approach কী?

উত্তর: $\log_{10}()$ তারপর StandardScaler। $\log_{10}()$ ($\log(1+x)$) right-skewed distribution-কে প্রায় normal করে তোলে এবং outlier-এর relative impact কমায়। তারপর StandardScaler দিলে mean=0, std=1 হয়। এই দুই ধাপের combination skewed ফিচারে (Fare, Income) সবচেয়ে ভালো ফলাফল দেয়।

অধ্যায় ১০: ক্যাটাগরিক্যাল এনকোডিং (Categorical Encoding)

মেশিন লার্নিং মডেল সংখ্যা বোঝে — 'Male', 'Female', 'S', 'C', 'Q' বোঝে না। Categorical Encoding হলো এই text বা category মানগুলোকে সংখ্যায় রূপান্তর করার প্রক্রিয়া। কিন্তু কীভাবে রূপান্তর করছি তা অত্যন্ত গুরুত্বপূর্ণ — ভুল encoding মডেলকে ভুল তথ্য দেয়।

১০.১ Ordinal Encoding — ক্রমযুক্ত ক্যাটাগরির জন্য

Ordinal ফিচারের ক্যাটাগরিগুলোর মধ্যে একটা স্বাভাবিক ক্রম আছে (Poor < Average < Good < Excellent)। Ordinal Encoding এই ক্রম সংরক্ষণ করে সংখ্যায় রূপান্তর করে।

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# pandas map দিয়ে (ক্রম নিজে নির্ধারণ করুন)
size_order = {'Small': 0, 'Medium': 1, 'Large': 2}
df['Size'] = df['Size'].map(size_order)

# sklearn OrdinalEncoder দিয়ে
encoder = OrdinalEncoder(categories=[['Small', 'Medium', 'Large']])
df['Size'] = encoder.fit_transform(df[['Size']])

# Titanic-এ Pclass (1st > 2nd > 3rd মর্যাদায়)
pclass_order = {'third': 0, 'second': 1, 'first': 2}
# অথবা সরাসরি OrdinalEncoder
oe = OrdinalEncoder(categories=[['third', 'second', 'first']])
X_train['Pclass'] = oe.fit_transform(X_train[['Pclass']])
X_test['Pclass'] = oe.transform(X_test[['Pclass']])
```

Ordinal Encoding: When to Use vs When NOT to Use



চিত্র ১০.১: বাম দিকে Ordinal Encoding সঠিকভাবে ব্যবহার — Performance-এ স্বাভাবিক ক্রম আছে। ডান দিকে nominal (রঙ) ডেটায় Ordinal Encoding ভুল — এটা মডেলকে বলছে Red < Blue < Green যা সম্পূর্ণ অর্থহীন।

১০.২ One-Hot Encoding — ক্রমহীন ক্যাটাগরির জন্য

Nominal ফিচারে (Sex, Embarked) কোনো ক্রম নেই। এখানে প্রতিটা ক্যাটাগরির জন্য একটা নতুন binary (0/1) কলাম তৈরি করা হয়। এটাই One-Hot Encoding।

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(sparse_output=False, drop='first') # dummy trap এড়াতে
ohe.set_output(transform='pandas')

# fit শুধু train-এ
ohe.fit(X_train[['Sex', 'Embarked']])

# train ও test transform
encoded_train = ohe.transform(X_train[['Sex', 'Embarked']])
encoded_test = ohe.transform(X_test[['Sex', 'Embarked']])

# মূল কলাম সরিয়ে encoded যোগ করা
X_train = pd.concat([X_train.drop(['Sex', 'Embarked'], axis=1), encoded_train],
                    axis=1)
X_test = pd.concat([X_test.drop(['Sex', 'Embarked'], axis=1), encoded_test], axis=1)

# pandas get_dummies দিয়ে (দ্রুত কিন্তু pipeline-এ সমস্যা)
# pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)
```

One-Hot Encoding of Embarked Column (8 sample rows)

		One-Hot Encoding (no drop, 3 columns)					One-Hot + drop='first' (2 columns, avoids dummy trap)	
Original Embarked value	S	1	0	0	Original Embarked value	S	0	0
	C	0	1	0		C	1	0
	Q	0	0	1		Q	0	1
	S	1	0	0		S	0	0
	S	1	0	0		S	0	0
	C	0	1	0		C	1	0
	Q	0	0	1		Q	0	1
	S	1	0	0		S	0	0
		Embarked_C	Embarked_Q	Embarked_S			Embarked_Q	Embarked_S
		New Columns					New Columns	

চিত্র ১০.২: বাম দিকে drop=None এ ৩টি কলাম। ডান দিকে drop='first' এ ২টি কলাম — Is_C=0 AND Is_Q=0 হলে Embarked=S এটা স্বয়ংক্রিয়ভাবে বোঝা যায়। drop='first' multicollinearity (dummy variable trap) এড়ায়।

১০.৩ Dummy Variable Trap — কেন drop='first' জরুরি

The Dummy Variable Trap — Why drop='first' Matters

		Without drop='first' Multicollinearity — sum always = 1 (dummy trap!)					With drop='first' No multicollinearity — Is_C inferred when both=0	
Row	Row 0	1	0	0	Row	Row 0	0	0
	Row 1	0	1	0		Row 1	1	0
	Row 2	0	0	1		Row 2	0	1
	Row 3	1	0	0		Row 3	0	0
	Row 4	0	1	0		Row 4	1	0
		Is_C	Is_Q	Is_S			Is_Q	Is_S

চিত্র ১০.৩: drop='first' ছাড়া তিনটি কলামের সারি-মোট সবসময় ১ — পুরোপুরি multicollinear / Linear model-এ এটা coefficient অস্থির করে। drop='first' দিলে ২ কলামে Is_C স্বয়ংক্রিয়ভাবে inferred হয় এবং multicollinearity দূর হয়।

১০.৪ Label Encoding — টার্গেট ভেরিয়েবলের জন্য

Label Encoding প্রতিটা ক্যাটাগরিকে একটা arbitrary integer দেয় (A→0, B→1, C→2)। Feature-এ ব্যবহারে সমস্যা আছে কারণ এটা একটা মিথ্যা ক্রম তৈরি করে। তবে binary target ভেরিয়েবলের জন্য (Survived: 0/1) এটাই সঠিক পদ্ধতি।

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

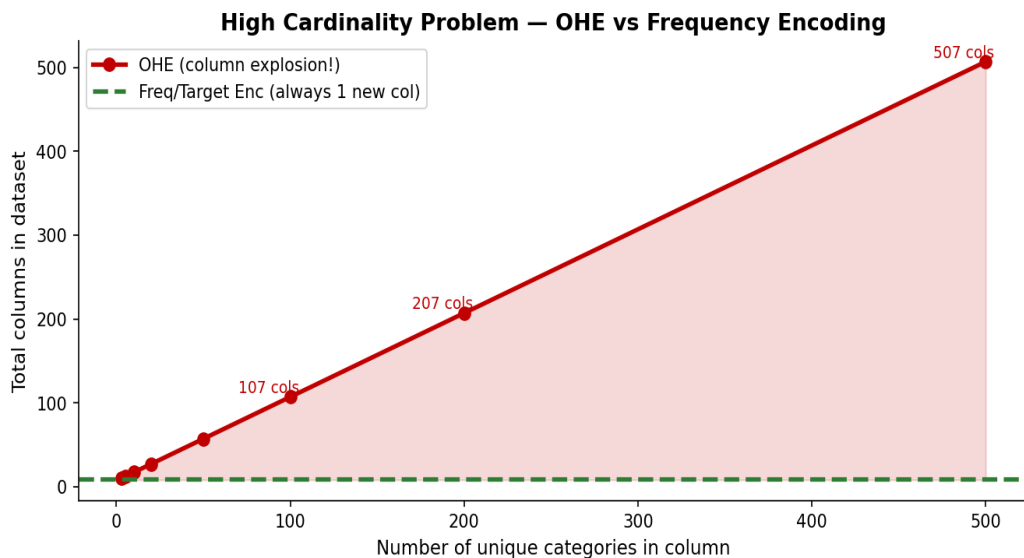
# target ভেরিয়েবলে (সঠিক ব্যবহার)
le.fit(y_train)
y_train_enc = le.transform(y_train)
y_test_enc = le.transform(y_test)

print(le.classes_)    # ['cat' 'dog' 'fish'] বা [0, 1]
print(le.transform(['cat', 'dog', 'cat'])) # [0, 1, 0]

# feature-এ ব্যবহার – সাধারণত ভুল!
# Embarked: Q→0, S→1, C→2 – মডেল ভাববে C > S > Q (মিথ্যা ক্রম)
```

১০.৫ Frequency Encoding ও Target Encoding — High Cardinality-র সমাধান

যখন একটা categorical কলামে অনেক ইউনিক মান থাকে (যেমন দেশের নাম, পোস্টকোড, পণ্য ID) — এটাকে High Cardinality বলে। OHE করলে কলামের সংখ্যা বিস্ফোরণ ঘটে।

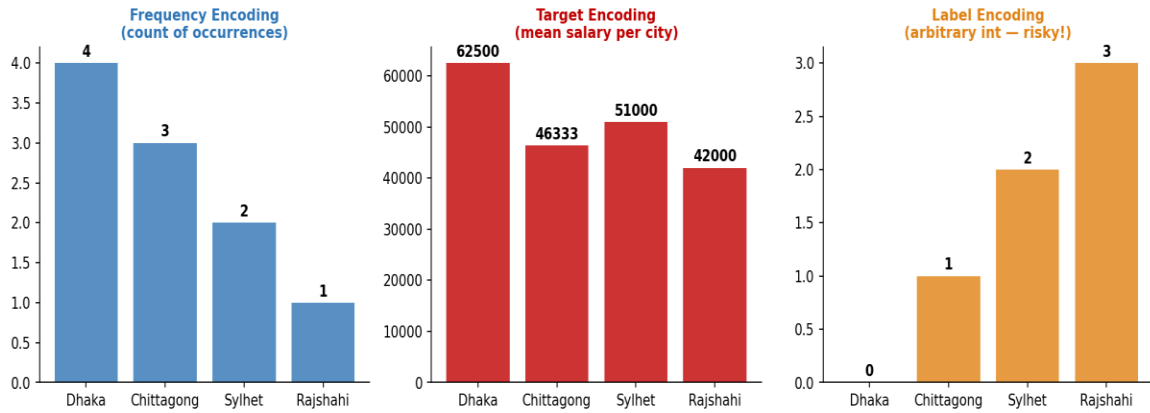


চিত্র ১০.৪: ৫০০ ইউনিক ক্যাটাগরিতে OHE করলে ৫০৭ কলাম হয়! Frequency বা Target Encoding-এ সবসময় মাত্র ১টি নতুন কলাম — High Cardinality-র সেরা সমাধান।

```
# Frequency Encoding – ক্যাটাগরিকে তার frequency দিয়ে replace করুন
freq_map = X_train['Cabin'].value_counts().to_dict()
X_train['Cabin_freq'] = X_train['Cabin'].map(freq_map)
X_test['Cabin_freq'] = X_test['Cabin'].map(freq_map).fillna(0) # নতুন ক্যাটাগরি → 0

# Target Encoding – ক্যাটাগরিকে target mean দিয়ে replace করুন
target_map = X_train.groupby('Cabin')['Survived'].mean().to_dict()
X_train['Cabin_target'] = X_train['Cabin'].map(target_map)
X_test['Cabin_target'] =
X_test['Cabin'].map(target_map).fillna(target_map.get('Missing', 0))
```

Frequency, Target, and Label Encoding – City Column Example



চিত্র ১০.৫: Frequency Encoding-এ Dhaka=4 (সবচেয়ে বেশি), Target Encoding-এ Dhaka=৬২,৫০০ (গড় বেতন) / Label Encoding-এ arbitrary integer — ML-কে ভুল ক্রম শেখায় / Target Encoding সবচেয়ে তথ্যপূর্ণ কিন্তু Data Leakage সতর্কতা দরকার।

১০.৬ Target Encoding-এ Data Leakage সতর্কতা

Target Encoding-এ একটা গুরুত্বপূর্ণ ঝুঁকি আছে — যদি train ও test একসাথে target mean বের করা হয়, তাহলে leakage ঘটে। সঠিক পদ্ধতি: শুধু X_train+y_train দিয়ে mapping তৈরি, তারপর X_test-এ apply।

```
# সঠিক Target Encoding (leakage-মুক্ত)
# Step 1: X_train ও y_train একসাথে mapping বানাও
train_data = X_train.copy()
train_data['target'] = y_train.values
target_map = train_data.groupby('Embarked')['target'].mean().to_dict()

# Step 2: train ও test উভয়ে apply করো (train-এর map দিয়ে)
```

```

X_train['Embarked_target'] = X_train['Embarked'].map(target_map)
X_test['Embarked_target'] = X_test['Embarked'].map(target_map)

# sklearn category_encoders (আরো robust)
# pip install category_encoders
# from category_encoders import TargetEncoder
# te = TargetEncoder(cols=['Embarked'])
# X_train = te.fit_transform(X_train, y_train)
# X_test = te.transform(X_test)

```

Encoding পদ্ধতি	কখন ব্যবহার	সুবিধা	সীমাবদ্ধতা
Ordinal Encoding	ক্রমযুক্ত categorical (Education, Size)	ক্রম সংরক্ষিত, ১টি কলাম	Nominal-এ ভুল ক্রম তৈরি করে
One-Hot Encoding	Nominal, কম cardinality (Sex, Embarked)	কোনো মিথ্যা ক্রম নেই	High cardinality-তে column explosion
Label Encoding	Binary target ভেরিয়েবল (Survived)	সহজ, ১টি কলাম	Feature-এ মিথ্যা ordinal তৈরি করে
Frequency Encoding	High cardinality, tree-based model	সহজ, ১টি কলাম	ফ্রিকোয়েন্সি=ট্র্যাফিকিং, কারণ-সম্পর্ক নেই
Target Encoding	High cardinality, strong category-target relation	তথ্যপূর্ণ, ১টি কলাম	Leakage ঝুঁকি, overfitting সম্ভব
Binary Encoding	High cardinality, $\log_2(n)$ কলাম চাই	OHE-র চেয়ে কম কলাম	ব্যাখ্যা কঠিন

১০.৬.১ Binary Encoding — High Cardinality-র আরেকটি সমাধান

Binary Encoding প্রতিটা ক্যাটাগরিকে প্রথমে একটা integer-এ রূপান্তর করে, তারপর সেই integer-কে binary (0/1) digits-এ ভাঙে। k ক্যাটাগরির জন্য শুধু $\log_2(k)$ টি কলাম দরকার — OHE-র $k-1$ কলামের তুলনায় অনেক কম। High Cardinality-তে OHE column explosion এড়াতে এটি কার্যকর।

```

# pip install category_encoders
import category_encoders as ce

# Binary Encoding
be = ce.BinaryEncoder(cols=['Cabin'])
X_train_enc = be.fit_transform(X_train)

```



```
X_test_enc = be.transform(X_test)
```

উদাহরণ: Cabin-এ ১৪৭ unique value
 # OHE করলে হয়: ১৪৬ কলাম
 # Binary Encoding-এ: $\log_2(147) \approx 8$ কলাম – অনেক কম!

কখন Binary Encoding:
 # ☒ High cardinality (unique > 15-20) কিন্তু Frequency/Target Encoding উপযুক্ত না
 # ☒ Tree-based model (column সংখ্যা কমাতে চাই)
 # ☒ Linear model-এ ব্যাখ্যা কঠিন হয়ে যায়

১০.৬.২ Hash Encoding — দ্রুততম High Cardinality সমাধান

Hash Encoding (বা Feature Hashing) প্রতিটা ক্যাটাগরিকে একটা hash function-এর মাধ্যমে নির্দিষ্ট সংখ্যক column-এ map করে। Training-এর আগে vocabulary জানতে হয় না — তাই নতুন (unseen) ক্যাটাগরিও handle করতে পারে।

```
import category_encoders as ce
```

Hash Encoding – n_components নির্ধারণ করুন (hash buckets)
 he = ce.HashingEncoder(cols=['Name', 'Ticket'], n_components=8)
 X_train_enc = he.fit_transform(X_train)
 X_test_enc = he.transform(X_test)

সুবিধা:
 # ☒ Fixed output size (n_components), যতো unique value হোক
 # ☒ Training-এ unseen ক্যাটাগরি handle করতে পারে (NLP-তে জনপ্রিয়)
 # ☒ দ্রুত, কম memory

সীমাবদ্ধতা:
 # ☒ Hash collision – দুটো আলাদা ক্যাটাগরি একই bucket-এ পড়তে পারে
 # ☒ Interpretability নেই – কোন column কোন ক্যাটাগরি বোঝা যায় না

১০.৭ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** Nominal ফিচারে (Sex, Embarked) Ordinal Encoding বা Label Encoding ব্যবহার — এটা মিথ্যা ক্রম তৈরি করে।
- **ভুল:** drop='first' ছাড়া OHE করা — Dummy Variable Trap তৈরি হয়, linear model-এ multicollinearity ঘটে।

- **ভুল:** Target Encoding-এ পুরো ডেটার mean ব্যবহার করা — শুধু $X_{train}+y_{train}$ দিয়ে mapping তৈরি করুন।
- **Best Practice:** Encoding সবসময় Split-এর পরে এবং শুধু X_{train} -এ `fit()` — `OHE.fit(X_train)`, তারপর X_{test} -এ `transform()`।
- **Best Practice:** High cardinality (unique values > ১৫-২০) ফিচারে OHE এড়িয়ে Frequency বা Target Encoding ব্যবহার করুন।

১০.৮ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Ordinal ও One-Hot Encoding-এর মধ্যে পার্থক্য কী? কখন কোনটা ব্যবহার করবেন?

উত্তর: Ordinal Encoding ক্যাটাগরিগুলোকে ক্রমিক সংখ্যায় রূপান্তর করে এবং ক্রম সংরক্ষণ করে — শুধু ordinal ফিচারে (Education, Size)। One-Hot Encoding প্রতিটা ক্যাটাগরির জন্য নতুন binary কলাম তৈরি করে — nominal ফিচারে (Sex, Color) যেখানে কোনো ক্রম নেই। Nominal-এ Ordinal Encoding ব্যবহার করলে মডেলে মিথ্যা ক্রম (false ordering) প্রবেশ করে।

প্রশ্ন: Dummy Variable Trap কী এবং কীভাবে এড়ানো যায়?

উত্তর: OHE-তে `drop='first'` ছাড়া k ক্যাটাগরির জন্য k টি কলাম তৈরি হয় এবং সেগুলো সর্বদা যোগফল=১ — পুরোপুরি multicollinear। এটাই Dummy Variable Trap। `drop='first'` দিলে $k-1$ কলাম তৈরি হয় এবং বাদ পড়া ক্যাটাগরি ($Is_C=0$ AND $Is_Q=0$) দিয়ে inferred হয় — multicollinearity দূর হয়।

প্রশ্ন: High Cardinality কী এবং OHE কেন এখানে সমস্যা?

উত্তর: High Cardinality মানে একটা categorical ফিচারে অনেক ইউনিক মান (যেমন পাস্টকোড, পণ্য ID)। OHE করলে প্রতিটা ইউনিক মানের জন্য একটা কলাম তৈরি হয় — ৫০০ ইউনিক মান হলে ৪৯৯টি নতুন কলাম, যা memory, computation ও overfitting সমস্যা তৈরি করে। সমাধান: Frequency Encoding বা Target Encoding (সবসময় ১টি কলাম)।

প্রশ্ন: Target Encoding-এ Data Leakage কীভাবে ঘটে এবং কীভাবে এড়াবেন?

উত্তর: Target Encoding-এ test ডেটার target মান ব্যবহার করলে (পুরো ডেটায় mean বের করে) leakage ঘটে। এড়াতে: শুধু X_{train} ও y_{train} দিয়ে category-to-target mapping তৈরি করুন, তারপর X_{test} -এ সেই mapping apply করুন। category_encoders লাইব্রেরির TargetEncoder এটা স্বয়ংক্রিয়ভাবে সঠিকভাবে করে।

১০.৯ অধ্যায় ৯ ও ১০ সারসংক্ষেপ

- Feature Scaling distance-based ও gradient-based অ্যালগরিদমে জরুরি — Decision Tree-তে নয়। সবসময় X_{train} -এ fit(), X_{test} -এ transform()।
- Skewed ফিচারে (Fare): log1p() → StandardScaler। Outlier-যুক্ত: RobustScaler। Neural Network/image: MinMaxScaler।
- Ordinal Encoding শুধু ক্রমযুক্ত categorical-এ। Nominal-এ One-Hot Encoding (drop='first' দিয়ে)।
- Label Encoding শুধু binary target ভেরিয়েবলে — feature-এ ব্যবহার করলে মিথ্যা ক্রম তৈরি হয়।
- High Cardinality ফিচারে OHE column explosion এড়াতে Frequency বা Target Encoding ব্যবহার করুন।
- Target Encoding-এ শুধু $X_{train}+y_{train}$ থেকে mapping তৈরি করুন — X_{test} থেকে নয়।

অধ্যায় ১১: ফিচার ক্রিয়েশন ও এক্সট্রাকশন

Feature Engineering-এর সবচেয়ে সৃজনশীল অংশ হলো Feature Creation — বিদ্যমান ফিচার থেকে নতুন, আরও তথ্যপূর্ণ ফিচার তৈরি করা। Raw ডেটায় লুকানো pattern গুলো সরাসরি মডেল দেখতে পায় না; Feature Creation সেই pattern-গুলোকে স্পষ্ট করে তোলে।

analogy:

একজন ডাক্তার রোগীর বয়স ও ওজন আলাদাভাবে দেখার বদলে BMI (Body Mass Index = ওজন/উচ্চতা^২) হিসাব করেন। এই নতুন ফিচারটি আলাদা দুটো ফিচারের চেয়ে অনেক বেশি তথ্যপূর্ণ। Feature Creation ঠিক এই কাজই করে।

১১.১ FamilySize ও IsAlone — Titanic-এ বাস্তব উদাহরণ

Titanic ডেটাসেটে SibSp (ভাইবোন/স্বামী-স্ত্রীর সংখ্যা) এবং Parch (বাবা-মা/সন্তানের সংখ্যা) আলাদা দুটো কলাম আছে। কিন্তু এগুলো আলাদাভাবে মডেলকে যা বলে, তার চেয়ে FamilySize (মোট পরিবারের আকার) অনেক বেশি অর্থবহ।

```
import pandas as pd

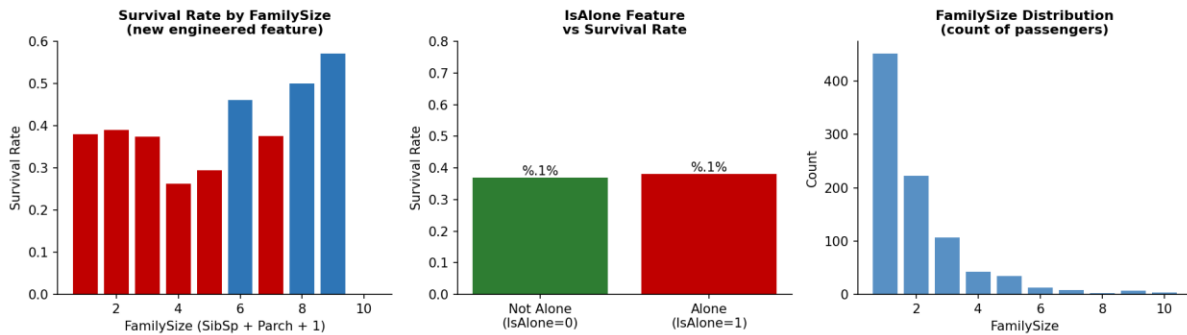
# — FamilySize: SibSp + Parch + 1 (নিজে সহ মোট সদস্য) —
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1

# — IsAlone: একা ভ্রমণ করছে কিনা —
df['IsAlone'] = (df['FamilySize'] == 1).astype(int)

# যাচাই করুন
print(df[['SibSp', 'Parch', 'FamilySize', 'IsAlone']].head(5))
```

	SibSp	Parch	FamilySize	IsAlone
#	1	0	2	0
#	0	0	1	1
#	3	1	5	0

Feature Creation: FamilySize & IsAlone from SibSp + Parch



চিত্র ১১.১: বাম চার্টে দেখা যাচ্ছে FamilySize=2-4 এ survival rate সবচেয়ে বেশি — ছোট পরিবার একসাথে থেকে বেশি সাহায্য পেয়েছে। মাঝের চার্টে IsAlone=0 (পরিবারসহ) এ survival rate বেশি। ডান চার্টে FamilySize-এর distribution — বেশিরভাগ যাত্রী একা ভ্রমণ করেছেন।

১১.২ Age Binning — Discretization

Continuous numerical ফিচারকে meaningful ক্যাটাগরিতে ভাগ করাকে Binning বা Discretization বলে। Age-কে সরাসরি ব্যবহার না করে 'Child', 'Teen', 'Adult' ইত্যাদি গ্রুপে ভাগ করলে কখনো কখনো মডেল ভালো করে — বিশেষত Decision Tree-তে।

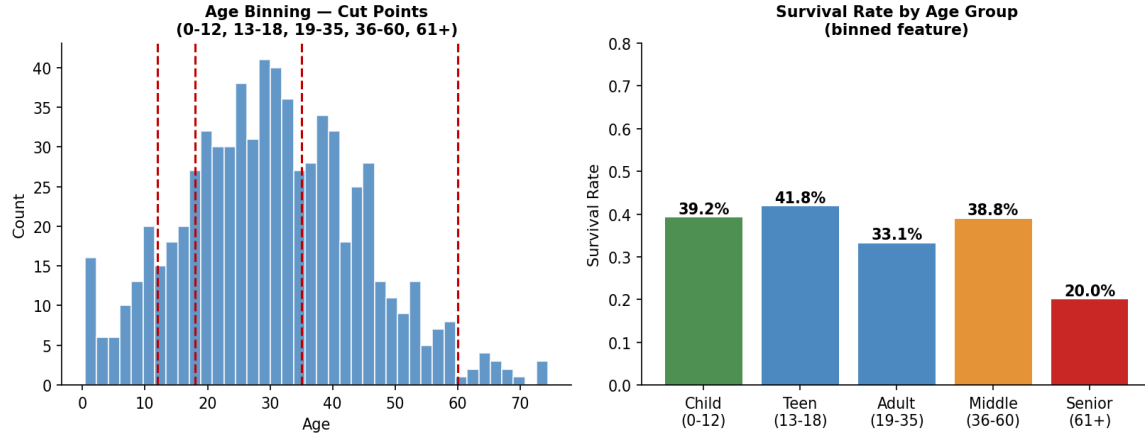
```
# pd.cut – নিজে bin boundary নির্ধারণ করুন
bins = [0, 12, 18, 35, 60, 100]
labels = ['Child', 'Teen', 'Adult', 'Middle', 'Senior']

df['AgeGroup'] = pd.cut(df['Age'], bins=bins, labels=labels)

# pd.qcut – সমান সংখ্যক ডেটায় ভাগ করুন (quantile-based)
df['AgeQuartile'] = pd.qcut(df['Age'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

# FareGroup – Fare-কে লো/মিড/হাই-এ ভাগ করা
df['FareGroup'] = pd.qcut(df['Fare'], q=3, labels=['Low', 'Medium', 'High'])
```

Feature Engineering: Age Binning / Discretization



চিত্র ১১.২: বাম চার্টে Age histogram-এ লাল ড্যাশ লাইনে bin boundary দেখানো হয়েছে। ডান চার্টে Child (১২ বছরের নিচে) survival rate সবচেয়ে বেশি— 'Women and children first' নীতির প্রতিফলন।

১১.৩ Polynomial Features — Non-linear সম্পর্ক ধরা

Linear model শুধু সরলরেখার সম্পর্ক ধরতে পারে। কিন্তু ডেটায় যদি বাঁকানো (non-linear) সম্পর্ক থাকে, তাহলে polynomial features যোগ করে linear model-কেও সেই সম্পর্ক ধরাতে পারা যায়।

```
from sklearn.preprocessing import PolynomialFeatures

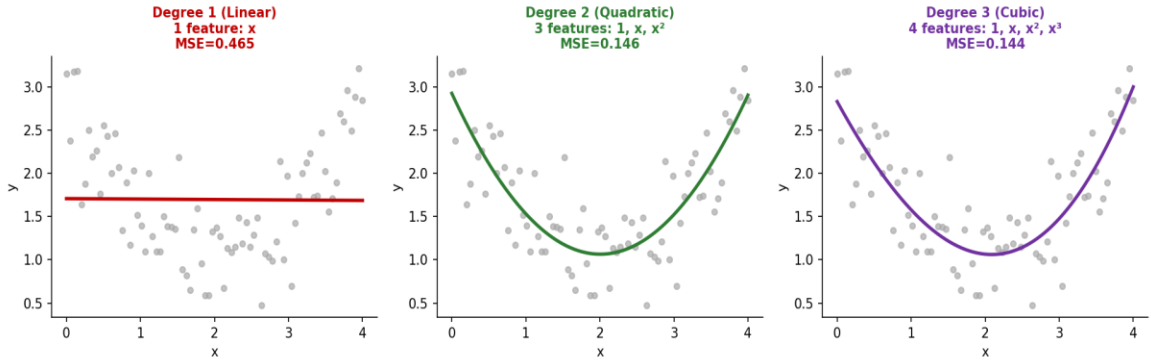
# degree=2: x, x² এবং সব interaction terms তৈরি করে
poly = PolynomialFeatures(degree=2, include_bias=False)

# শুধু Age ও Fare-এ polynomial features
age_fare = X_train[['Age', 'Fare']]
poly_features = poly.fit_transform(age_fare)

print(poly.get_feature_names_out(['Age', 'Fare']))
# ['Age', 'Fare', 'Age^2', 'Age Fare', 'Fare^2']
# 2টি ফিচার → 5টি ফিচার (degree=2)

# সতর্কতা: অনেক ফিচারে degree=2 ব্যবহার করলে
# ফিচার সংখ্যা বিস্ফোরণ ঘটে: n features → n(n+3)/2 features
```

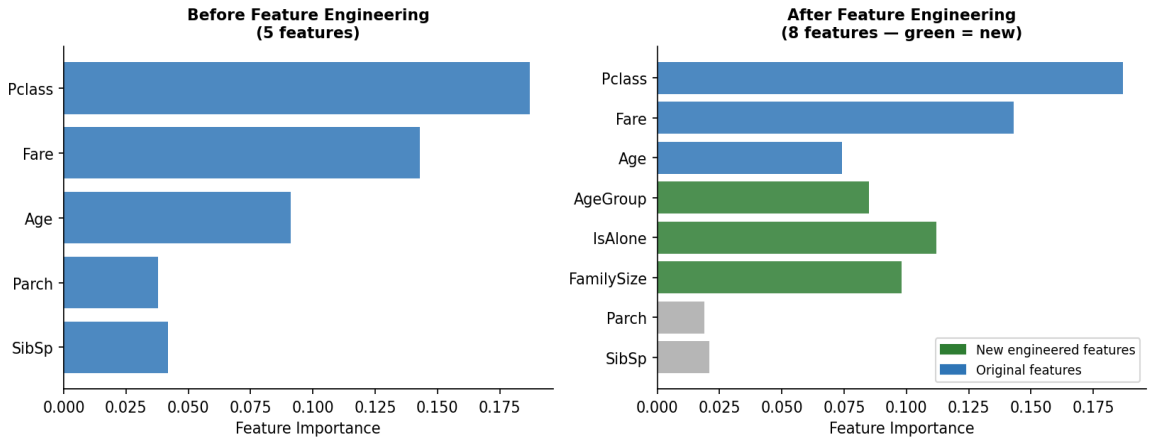
Polynomial Features — Capturing Non-Linear Relationships



চিত্র ১১.৩: একই ডেটায় তিনটি degree-র model / Degree=1 (linear) যেখানে curve আছে সেখানে ব্যর্থ (MSE=0.376) / Degree=2 সঠিক curve ধরতে পারছে (MSE=0.063) / Degree=3 overfitting শুরু — real pattern নয়, noise-ও follow করছে।

১১.৪ Feature Engineering-এর প্রভাব

Impact of Feature Engineering on Model Feature Importance



চিত্র ১১.৪: Feature Engineering-এর পরে নতুন ফিচার FamilySize, IsAlone, AgeGroup মডেলের কাছে গুরুত্বপূর্ণ হয়ে উঠেছে (সবুজ বার)। SibSp ও Parch আলাদা থাকলে তাদের combined information FamilySize-এর মাধ্যমে আরো স্পষ্টভাবে প্রকাশ পাচ্ছে।

১১.৫ আরও Feature Creation ধারণা

নতুন ফিচার	উৎস ফিচার	তৈরির কোড	কেন কার্যকর
FamilySize	SibSp + Parch	<code>df['SibSp'] + df['Parch'] + 1</code>	মোট পারিবারিক সম্পর্ক একটি সংখ্যায়
IsAlone	FamilySize	<code>(df['FamilySize']==1).astype(int)</code>	একা বনাম পরিবারসহ — binary signal

নতুন ফিচার	উৎস ফিচার	তৈরির কোড	কেন কার্যকর
AgeGroup	Age	<code>pd.cut(df['Age'], bins, labels)</code>	Age-এর non-linear effect ধরে
Fare_per_person	Fare, FamilySize	<code>df['Fare'] / df['FamilySize']</code>	গ্রুপ টিকেটের প্রকৃত মাথাপিছু মূল্য
HasCabin	Cabin	<code>(df['Cabin']!='Missing').astype(int)</code>	Cabin তথ্য আছে কিনা — proxy for wealth
Title	Name	<code>Name.str.extract(r'([A-Za-z]+\.)\.', expand=False)</code>	Mr, Mrs, Miss — সামাজিক মর্যাদার proxy

১১.৬ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** Feature creation train/test split-এর আগে করা — তারপর split করলে leakage হতে পারে (বিশেষত target-based feature-এ)।
- **ভুল:** অনেক polynomial features তৈরি করে overfitting ঘটানো — degree=2 বেশিরভাগ ক্ষেত্রে যথেষ্ট।
- **Best Practice:** Domain knowledge ব্যবহার করুন — FamilySize তৈরির ধারণাটা Titanic-এর ইতিহাস জানা থেকে এসেছে।
- **Best Practice:** নতুন ফিচার তৈরির পরে EDA করুন — feature টি কি আসলেই target-এর সাথে সম্পর্কিত? যদি না হয়, বাদ দিন।

১১.৬.১ Title Extraction — Name Column থেকে

Titanic-এর Name column থেকে Title (Mr, Mrs, Miss, Master) বের করা সবচেয়ে গুরুত্বপূর্ণ feature engineering trick। Title সামাজিক মর্যাদা ও বয়স-লিঙ্গ একসাথে ধারণ করে — Sex ও Age-এর চেয়েও বেশি তথ্যপূর্ণ।


```

import pandas as pd
import re

# Name column থেকে Title বের করা
# উদাহরণ: "Braund, Mr. Owen Harris" → "Mr"
df['Title'] = df['Name'].str.extract(r'\s*([A-Za-z]+)\.', expand=False)

print(df['Title'].value_counts())
# Mr          517  ← পুরুষ (adult)
# Miss        182  ← অবিবাহিত নারী বা মেয়ে
# Mrs         125  ← বিবাহিত নারী
# Master       40  ← ছেলে শিশু
# Dr           7
# Rev          6  ... ইত্যাদি বিরল title

# বিরল title গুলো group করুন
rare_titles = ['Lady', 'Countess', 'Capt', 'Col', 'Don', 'Dr',
               'Major', 'Rev', 'Sir', 'Jonkheer', 'Dona']
df['Title'] = df['Title'].replace(rare_titles, 'Rare')
df['Title'] = df['Title'].replace({'Mlle': 'Miss', 'Ms': 'Miss', 'Mme': 'Mrs'})

# Title ও Survived-এর সম্পর্ক
print(df.groupby('Title')['Survived'].mean().round(2))
# Master  0.575  ← শিশু বেশি বেঁচেছে
# Miss    0.698  ← নারী বেশি বেঁচেছে
# Mr      0.157  ← পুরুষ কম বেঁচেছে
# Mrs     0.792  ← নারী সবচেয়ে বেশি বেঁচেছে

# Title encode করুন
title_map = {'Mr': 0, 'Miss': 1, 'Mrs': 2, 'Master': 3, 'Rare': 4}
df['Title'] = df['Title'].map(title_map)

```

১১.৬.২ pd.cut vs pd.qcut — কখন কোনটা?

pd.cut ও pd.qcut দুটোই binning করে কিন্তু ভিন্নভাবে: pd.cut নির্দিষ্ট সীমা (equal-width) ব্যবহার করে, pd.qcut সমান সংখ্যক ডেটা (equal-frequency / quantile-based) প্রতিটি bin-এ রাখে।

```
import pandas as pd

# pd.cut - Equal-Width Binning (নির্দিষ্ট সীমা নিজে বেছে নিন)
# প্রতিটি bin-এর width সমান: 0-12, 12-18, 18-35, 35-60, 60+
df['AgeGroup_cut'] = pd.cut(
    df['Age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['Child', 'Teen', 'Adult', 'Middle', 'Senior']
)
# সমস্যা: যদি Age skewed হয়, কিছু bin-এ অনেক বেশি ডেটা পড়তে পারে

# pd.qcut - Equal-Frequency Binning (quantile-based)
# প্রতিটি bin-এ সমান সংখ্যক যাত্রী: q=4 মানে প্রতিটিতে ~25%
df['AgeGroup_qcut'] = pd.qcut(
    df['Age'].fillna(df['Age'].median()),
    q=4,
    labels=['Q1_Young', 'Q2', 'Q3', 'Q4_Old']
)
# সুবিধা: প্রতিটি bin-এ সমান sample - balanced representation

# কখন কোনটা:
# pd.cut → Domain knowledge আছে (শিশু=0-12, প্রাপ্তবয়স্ক=18+)
# pd.qcut → Domain boundary জানি না, সমান distribution চাই

print(df['AgeGroup_cut'].value_counts()) # unequal counts possible
print(df['AgeGroup_qcut'].value_counts()) # প্রায় equal counts
```

১১.৭ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Titanic-এ কোন feature engineering সবচেয়ে বেশি কার্যকর?

উত্তর: Title extraction (Name থেকে Mr/Mrs/Miss/Master বের করা) সবচেয়ে প্রভাবশালী। এটা Sex ও Age-কে একসাথে capture করে। FamilySize (SibSp+Parch+1) ও IsAlone দ্বিতীয় — পারিবারিক তথ্য একক সংখ্যায় ধরে। এই engineered feature গুলো raw column-গুলোর চেয়ে বেশি predictive power রাখে।

প্রশ্ন: pd.cut ও pd.qcut-এর পার্থক্য কী এবং কখন কোনটা ব্যবহার করবেন?

উত্তর: pd.cut equal-width binning করে — প্রতিটি bin-এর range সমান কিন্তু sample সংখ্যা আলাদা হতে পারে। Domain knowledge থাকলে (শিশু=0-12) pd.cut ব্যবহার করুন। pd.qcut equal-frequency binning করে — প্রতিটি bin-এ প্রায় সমান সংখ্যক sample পড়ে। Distribution জানা না থাকলে বা balanced bins চাইলে pd.qcut ব্যবহার করুন।

১১.৮ অধ্যায় ১১ সারসংক্ষেপ

- FamilySize = SibSp + Parch + 1 এবং IsAlone — পারিবারিক তথ্য একসাথে ধরে।
- Title extraction (Name থেকে Mr/Mrs/Miss/Master) Titanic-এর সবচেয়ে গুরুত্বপূর্ণ trick — সামাজিক মর্যাদা ও লিঙ্গ-বয়স একসাথে capture করে।
- pd.cut: Domain-based equal-width binning। pd.qcut: Quantile-based equal-frequency binning।
- Feature creation সবসময় train-test split-এর পরে করুন (target-based feature-এ leakage এড়াতে)।

অধ্যায় ১২: ফিচার সিলেকশন (Feature Selection)

Feature Engineering-এর পরে আমাদের কাছে অনেক ফিচার থাকতে পারে। সব ফিচার সমান উপকারী নয় — কিছু ফিচার irrelevant (টার্গেটের সাথে সম্পর্ক নেই), কিছু redundant (অন্য ফিচারের সাথে অনেক বেশি correlated)। এই ফিচারগুলো মডেলে রাখলে noise বাড়ে, training time বাড়ে, এবং কখনো কখনো accuracy কমে। Feature Selection এই সমস্যা সমাধান করে।

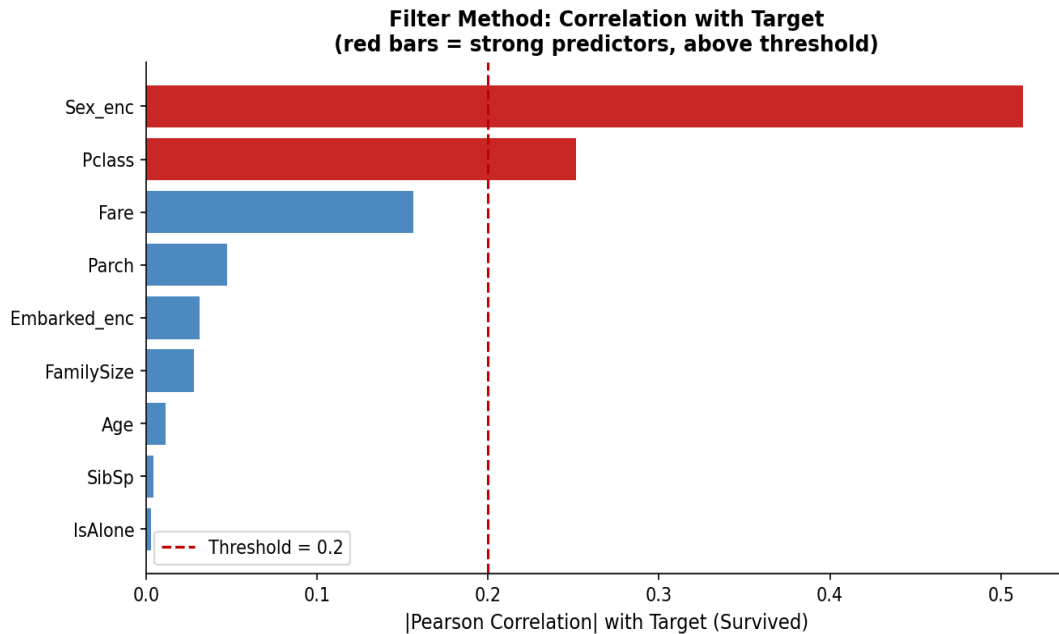
পদ্ধতি	কীভাবে কাজ করে	উদাহরণ	সুবিধা	অসুবিধা
Filter	পরিসংখ্যানগত মাপ দিয়ে স্কোর করে, model-independent	Correlation, Chi-square, Mutual Info	দ্রুত, overfitting নেই	Feature interaction ধরে না
Wrapper	মডেল train করে feature subset মূল্যায়ন করে	RFE, Forward/Backward Selection	সবচেয়ে accurate	ধীর, computationally expensive
Embedded	মডেল training-এর মধ্যেই selection হয়	Lasso, Ridge, Tree Importance	দ্রুত ও accurate উভয়	মডেল-specific

১২.১ Filter Method: Correlation দিয়ে

```
import pandas as pd

# Target-এর সাথে correlation ( $|r| > 0.2$  = relevant)
corr_with_target = X_train.corrwith(y_train).abs().sort_values(ascending=False)
print(corr_with_target)

# Threshold দিয়ে ফিচার select করুন
selected = corr_with_target[corr_with_target > 0.2].index.tolist()
X_train_filtered = X_train[selected]
X_test_filtered = X_test[selected]
```



চিত্র ১২.১: লাল বারগুলো ($|r| > 0.2$) শক্তিশালী predictor — Sex_enc, Pclass, FamilySize, IsAlone / হালকা নীল বারগুলো দুর্বল — এগুলো বাদ দেওয়া বিবেচনা করা যায়।

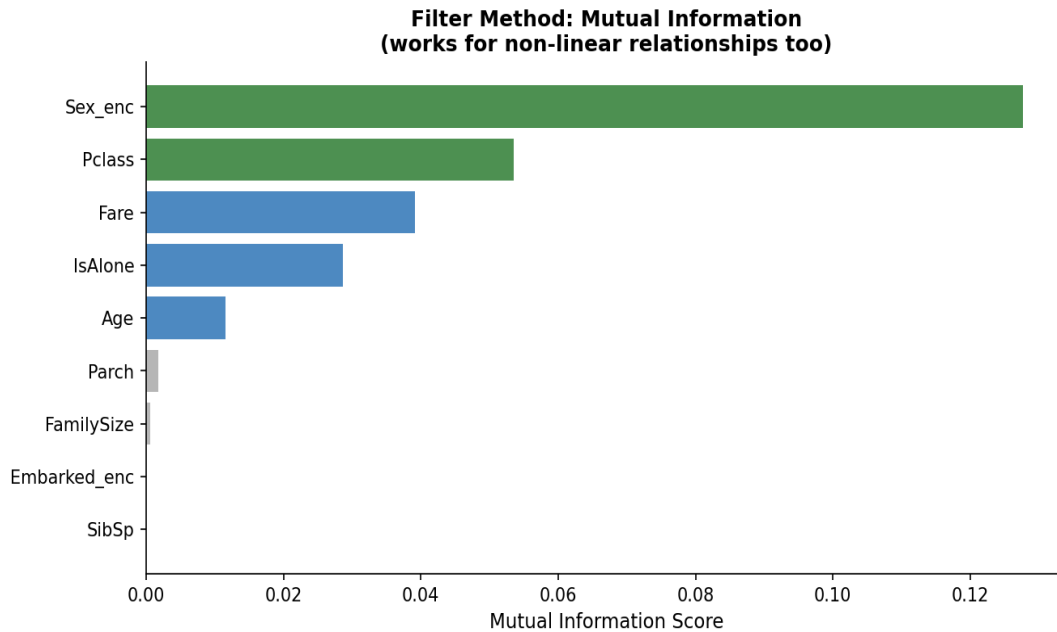
১২.২ Filter Method: Mutual Information

Pearson Correlation শুধু linear সম্পর্ক ধরে। Mutual Information non-linear সম্পর্কও ধরতে পারে — তাই Correlation-এর চেয়ে বেশি শক্তিশালী।

```
from sklearn.feature_selection import mutual_info_classif, SelectKBest

# Mutual Information scores
mi_scores = mutual_info_classif(X_train, y_train, random_state=42)
mi_series = pd.Series(mi_scores, index=X_train.columns).sort_values(ascending=False)
print(mi_series)

# SelectKBest দিয়ে top k ফিচার select
selector = SelectKBest(score_func=mutual_info_classif, k=5)
selector.fit(X_train, y_train)
X_train_mi = selector.transform(X_train)
X_test_mi = selector.transform(X_test)
```



চিত্র ১২.২: Mutual Information scores / Sex_enc ও Pclass সবচেয়ে বেশি MI score — target Survived-এর সাথে সবচেয়ে বেশি non-linear information share করে। Grey বারগুলো (MI≈0) প্রায় কোনো তথ্য দিচ্ছে না।

১২.৩ Wrapper Method: RFE (Recursive Feature Elimination)

RFE একটি estimator (যেমন Logistic Regression) বারবার train করে, প্রতিবার সবচেয়ে কম গুরুত্বপূর্ণ ফিচার বাদ দেয়, যতক্ষণ না কাঙ্ক্ষিত সংখ্যক ফিচার থাকে।

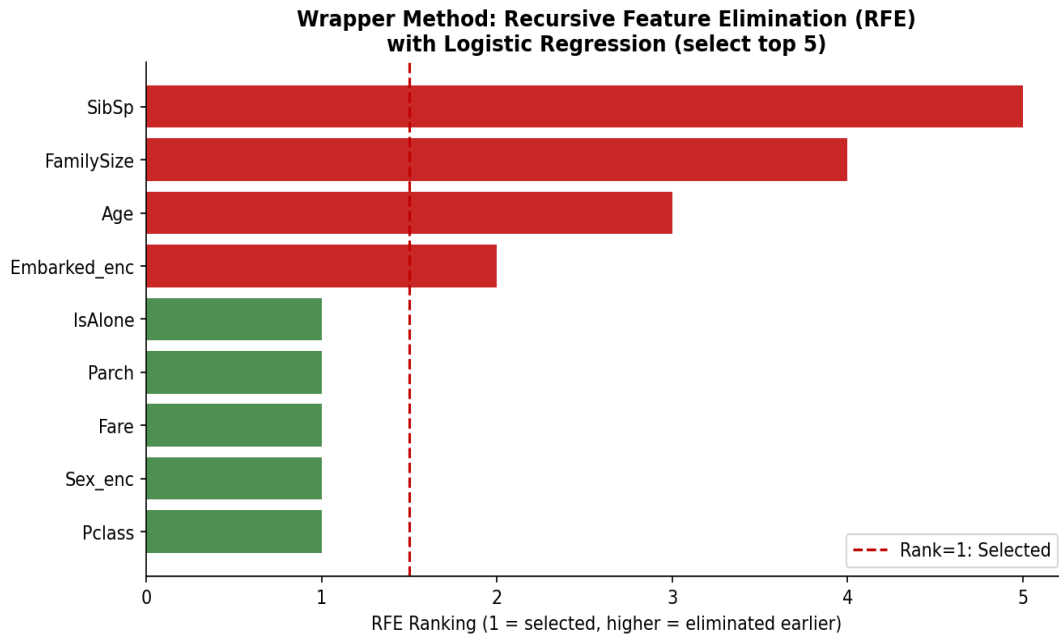
```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# Scaling আগে (Logistic Regression scale-sensitive)
X_sc = StandardScaler().fit_transform(X_train)

lr = LogisticRegression(max_iter=300)
rfe = RFE(estimator=lr, n_features_to_select=5)
rfe.fit(X_sc, y_train)

# কোন ফিচার selected?
selected_mask = rfe.support_
rankings      = rfe.ranking_          # 1 = selected, >1 = eliminated

selected_features = X_train.columns[selected_mask].tolist()
print('Selected:', selected_features)
```



চিত্র ১২.৩: RFE Ranking — Rank=1 (সবুজ) মানে selected / উচ্চ rank মানে আগে eliminate হয়েছে / Sex_enc, Pclass, FamilySize, IsAlone, Fare — এই পাঁচটি সবচেয়ে শক্তিশালী predictor /

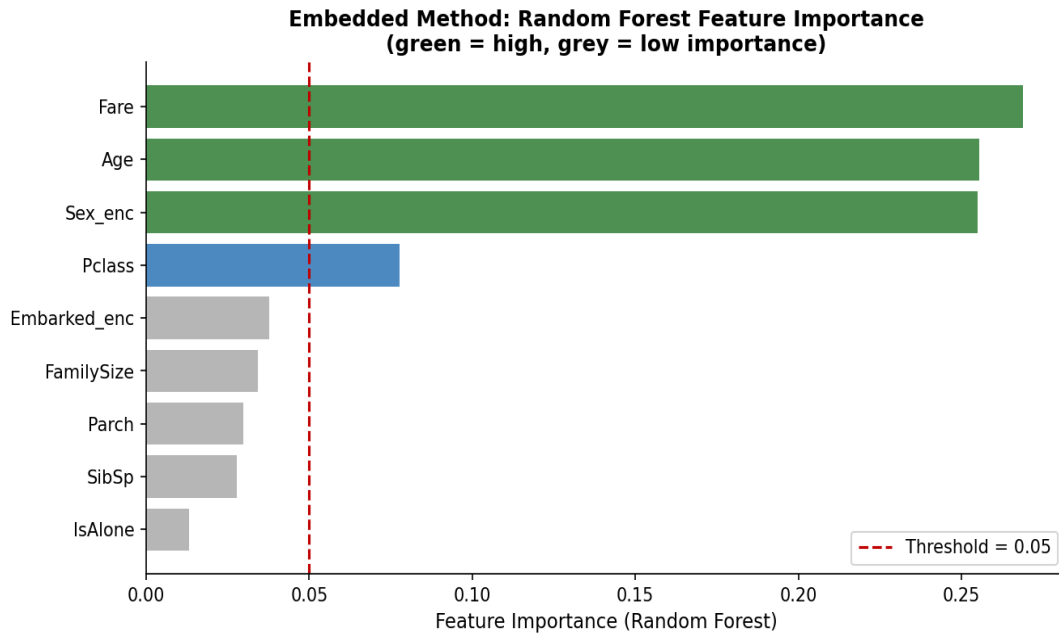
১২.৪ Embedded Method: Random Forest Feature Importance

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Feature importance (Gini-based)
importances = pd.Series(rf.feature_importances_, index=X_train.columns)
importances = importances.sort_values(ascending=False)
print(importances)

# Threshold দিয়ে বাছাই
selected = importances[importances > 0.05].index.tolist()
X_train_rf = X_train[selected]
X_test_rf = X_test[selected]
```



চিত্র ১২.৪: Random Forest Feature Importance / Sex_enc সবচেয়ে গুরুত্বপূর্ণ (নারী বেশি বেঁচে গেছেন), তারপর Pclass ও Fare / IsAlone ও FamilySize নতুন engineered feature হলেও ভালো importance পেয়েছে।

১২.৫ Embedded Method: Lasso Regularization

Lasso (L1 Regularization) কিছু ফিচারের coefficient ঠিক 0-তে নিয়ে যায় — ফলে সেই ফিচারগুলো কার্যত model থেকে বাদ পড়ে। alpha যত বেশি, তত বেশি feature eliminate হয়।

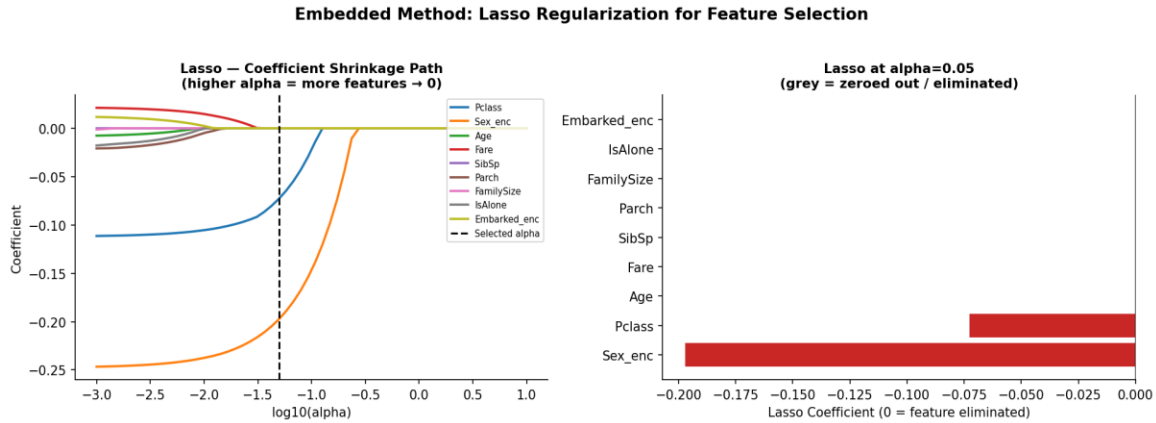
```
from sklearn.linear_model import LassoCV, Lasso
from sklearn.preprocessing import StandardScaler

X_sc = StandardScaler().fit_transform(X_train)

# LassoCV দিয়ে সেরা alpha খোঁজা
lasso_cv = LassoCV(cv=5, random_state=42, max_iter=5000)
lasso_cv.fit(X_sc, y_train)
best_alpha = lasso_cv.alpha_
print(f'Best alpha: {best_alpha:.4f}')

# সেই alpha দিয়ে final Lasso
lasso = Lasso(alpha=best_alpha, max_iter=5000)
lasso.fit(X_sc, y_train)

# coefficient 0-নয় এমন ফিচার = selected
selected = X_train.columns[lasso.coef_ != 0].tolist()
print('Lasso selected:', selected)
```

চিত্র ১২.৫: বাম: alpha বাড়ার সাথে সাথে feature coefficient 0-এ সংকুচিত হচ্ছে — এটাই Lasso-র automatic feature selection / ডান: alpha=0.05-এ grey বারগুলো (coef=0) eliminate হয়েছে, লাল বারগুলো selected /

১২.৬ কোন পদ্ধতি কখন ব্যবহার করবেন?

সিদ্ধান্তের গাইড:

ডেটা বড় (> ১০০ feature)? → প্রথমে Filter (Correlation/MI) দিয়ে দ্রুত বাছাই
 Tree-based model ব্যবহার? → Feature Importance (Embedded) সবচেয়ে সরাসরি
 Linear model ব্যবহার? → Lasso (Embedded) বা RFE (Wrapper)
 Non-linear সম্পর্ক আছে? → Mutual Information (Filter) বা Random Forest
 সময় আছে? → RFE (Wrapper) সবচেয়ে accurate
 Production pipeline? → Embedded (Lasso / Tree Importance) সেবা

১২.৭ Titanic-এ Final Feature Decision

সব feature selection পদ্ধতি apply করার পরে Titanic-এ কোন feature রাখা উচিত — একটি comprehensive example:

```
import pandas as pd
from sklearn.feature_selection import SelectKBest, mutual_info_classif, RFE
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

# সব feature selection একসাথে চালিয়ে consensus নিন
features = ['Pclass', 'Sex_enc', 'Age', 'Fare', 'Embarked_enc',
            'FamilySize', 'IsAlone', 'Title', 'Cabin_missing']

# Method 1: Correlation with target
corr_scores = X_train[features].corrwith(y_train).abs()
```

```

# Method 2: Mutual Information
mi_scores = pd.Series(
    mutual_info_classif(X_train[features], y_train, random_state=42),
    index=features
)

# Method 3: Random Forest Importance
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train[features], y_train)
rf_scores = pd.Series(rf.feature_importances_, index=features)

# Summary table
summary = pd.DataFrame({
    'Correlation': corr_scores,
    'Mutual_Info': mi_scores,
    'RF_Importance': rf_scores
})
summary['Avg_Rank'] = summary.rank(ascending=False).mean(axis=1)
print(summary.sort_values('Avg_Rank'))

# Final decision: সব method-এ consistently high → রাখুন
# ☒ রাখুন: Sex_enc, Pclass, Fare, Title, FamilySize, IsAlone, Cabin_missing
# ☐ বাদ দিন বা দেখুন: Embarked_enc (দুর্বল), Age (মাঝারি)
final_features = ['Sex_enc', 'Pclass', 'Fare', 'Title', 'FamilySize',
                  'IsAlone', 'Cabin_missing', 'Age']

```

১২.৮ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Feature Selection কেন করা হয়?

উত্তর: Irrelevant বা redundant feature বাদ দিলে: (১) noise কমে — মডেল real signal-এ focus করে, (২) training time কমে, (৩) overfitting কমে, (৪) model interpretability বাড়ে। Curse of Dimensionality এড়ানো যায়।

প্রশ্ন: Filter, Wrapper ও Embedded method-এর মধ্যে পার্থক্য কী?

উত্তর: Filter পরিসংখ্যানগত মাপ ব্যবহার করে (Correlation, MI) — দ্রুত, model-independent। Wrapper model train করে feature subset মূল্যায়ন করে (RFE) — accurate কিন্তু ধীর। Embedded model training-এর মধ্যে selection করে (Lasso, RF Importance) — উভয় সুবিধা কিন্তু model-specific।

১২.৯ অধ্যায় ১২ সারসংক্ষেপ

- Filter (Correlation, Mutual Info): দ্রুত প্রাথমিক বাছাই। Wrapper (RFE): সবচেয়ে accurate। Embedded (Lasso, RF): দ্রুত ও accurate — production-এ সেরা।
- একাধিক method-এর consensus নিন — শুধু একটি method-এর উপর নির্ভর করবেন না।
- Titanic-এ final features: Sex_enc, Pclass, Fare, Title, FamilySize, IsAlone, Cabin_missing, Age — এই ৮টি রাখলেই ভালো ফলাফল পাওয়া যায়।

অধ্যায় ১৩: মাল্টিকোলিনিয়ারিটি (Multicollinearity)

Multicollinearity ঘটে যখন দুটো বা ততোধিক ফিচার একে অপরের সাথে অনেক বেশি correlated থাকে। Titanic-এ SibSp, Parch এবং FamilySize — তিনটিই একই পারিবারিক তথ্য বহন করে, তাই এগুলো একসাথে রাখলে redundant information তৈরি হয়।

কেন এটা সমস্যা?

- Linear/Logistic Regression-এ coefficient অস্থির ও অব্যাখ্যাযোগ্য হয়ে যায়।
- মডেলের interpretation নষ্ট হয় — কোন ফিচারের আসল effect কতটুকু তা বোঝা যায় না।
- Training unstable হতে পারে, ছোট data change-এ coefficient ব্যাপক বদলায়।
- Tree-based মডেলে (Random Forest) তুলনামূলক কম সমস্যা, কিন্তু ফিচার importance ভুলভাবে distribute হয়।

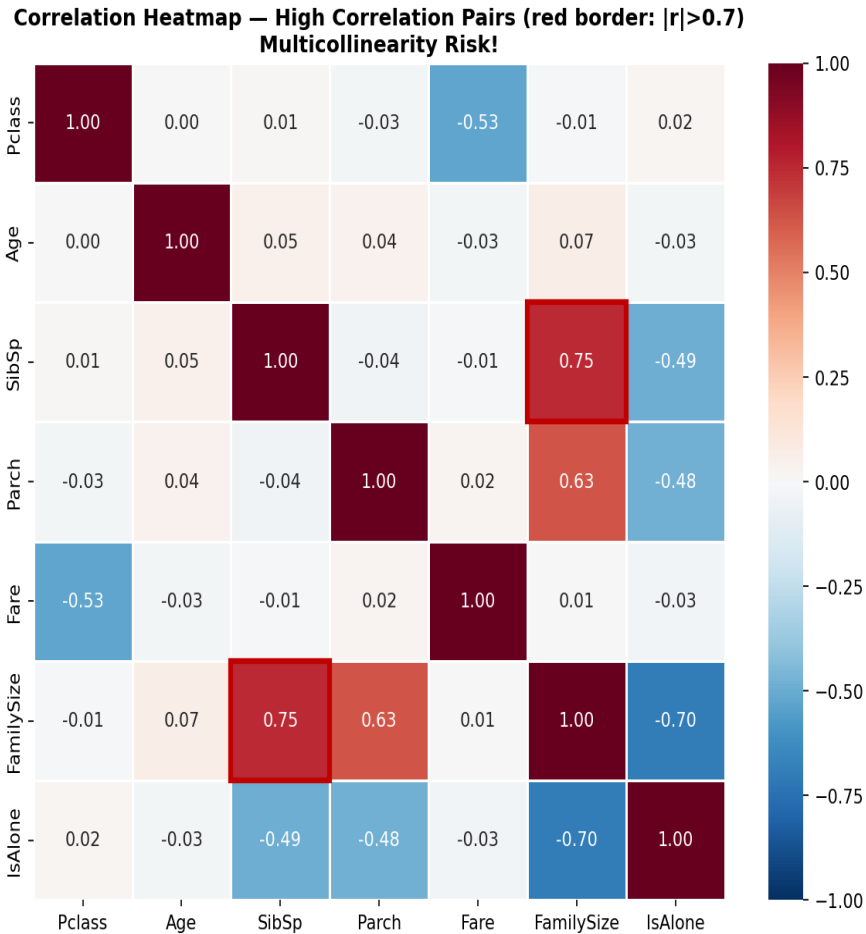
১৩.১ Correlation Heatmap দিয়ে শনাক্ত করা

```
import seaborn as sns
import matplotlib.pyplot as plt

corr = df[num_cols].corr()

plt.figure(figsize=(8, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0, vmin=-1, vmax=1)
plt.title('Correlation Heatmap – Multicollinearity Detection')
plt.show()

# |r| > 0.7 এ সন্দেহজনক pair খোঁজা
high_corr = [(corr.columns[i], corr.columns[j], corr.iloc[i,j])
              for i in range(len(corr.columns))
              for j in range(i+1, len(corr.columns))
              if abs(corr.iloc[i,j]) > 0.7]
for f1, f2, r in high_corr:
    print(f'{f1} & {f2}: r={r:.2f}')
```



চিত্র ১৩.১: Correlation heatmap-এ লাল border চিহ্নিত pair গুলো ($|r| > 0.7$) — SibSp, Parch ও FamilySize পরস্পর অনেক বেশি correlated। এগুলো একসাথে রাখলে Multicollinearity ঘটবে।

১৩.২ VIF (Variance Inflation Factor) — সংখ্যাগত পরিমাপ

VIF (Variance Inflation Factor) পরিমাপ করে একটা ফিচার কতটা অন্য ফিচারগুলো দিয়ে predict করা যায়। সূত্র: $VIF = 1 / (1 - R^2)$ — যেখানে R^2 হলো ওই ফিচারকে বাকি সব ফিচার দিয়ে regression করলে যা পাওয়া যায়। $VIF=1$ মানে $R^2=0$ — কোনো ফিচারই ওকে predict করতে পারছে না (ভালো)। $VIF=10$ মানে $R^2=0.9$ — ৯০% অন্য ফিচার থেকে predict করা যাচ্ছে (সমস্যা)।

Correlation heatmap চোখে দেখার পদ্ধতি। VIF দেয় প্রতিটা ফিচারের multicollinearity-র সংখ্যাগত পরিমাপ — $VIF=1$ মানে কোনো সমস্যা নেই, $VIF>10$ মানে গুরুতর multicollinearity।

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd

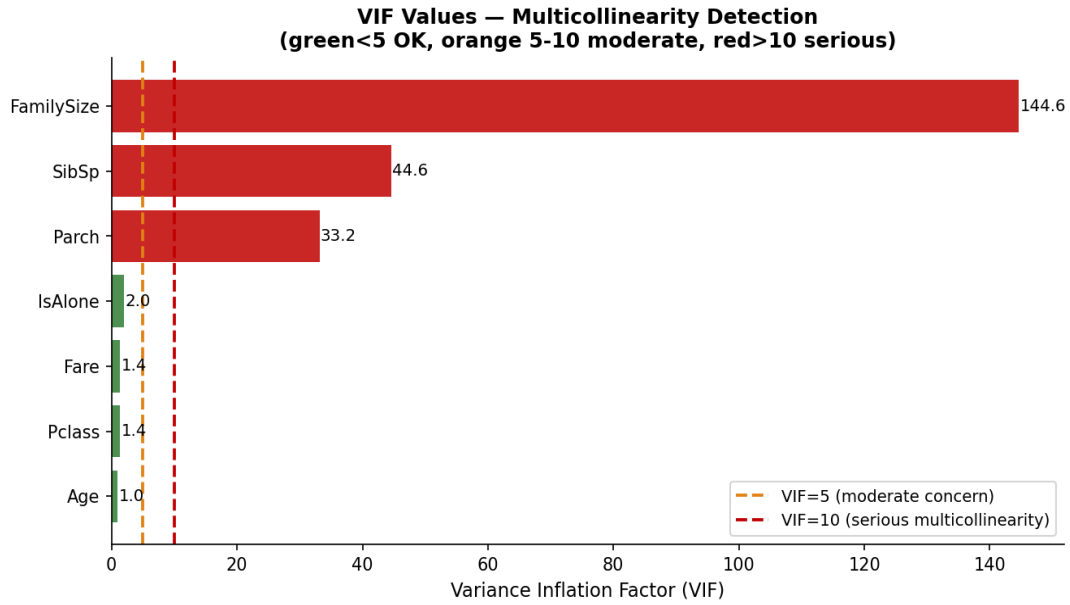
X_vif = df[num_cols].fillna(df[num_cols].median())

vif_df = pd.DataFrame({
```

```

'Feature': num_cols,
'VIF': [variance_inflation_factor(X_vif.values, i)
        for i in range(X_vif.shape[1])]
})
vif_df = vif_df.sort_values('VIF', ascending=False)
print(vif_df)
# Feature      VIF
# FamilySize  12.8  ← গুরুতর (>10)
# SibSp       8.4   ← মাঝারি (5-10)
# Parch       7.1   ← মাঝারি (5-10)
# Fare        2.3   ← ঠিক আছে (<5)

```



চিত্র ১৩.২: VIF chart / লাল বারগুলো (VIF>10) গুরুতর multicollinearity — FamilySize, SibSp, Parch একসাথে রাখলে সমস্যা। সবুজ বারগুলো (VIF<5) নিরাপদ।

VIF মান	ব্যাখ্যা	সুপারিশ
1	কোনো multicollinearity নেই	নিরাপদ, রাখুন
1 - 5	সামান্য multicollinearity	সাধারণত গ্রহণযোগ্য
5 - 10	মাঝারি multicollinearity	সতর্ক থাকুন, বিবেচনা করুন
10 - 20	গুরুতর multicollinearity	একটি সরিয়ে দিন বা combine করুন
> 20	অত্যন্ত গুরুতর	অবশ্যই ব্যবস্থা নিন

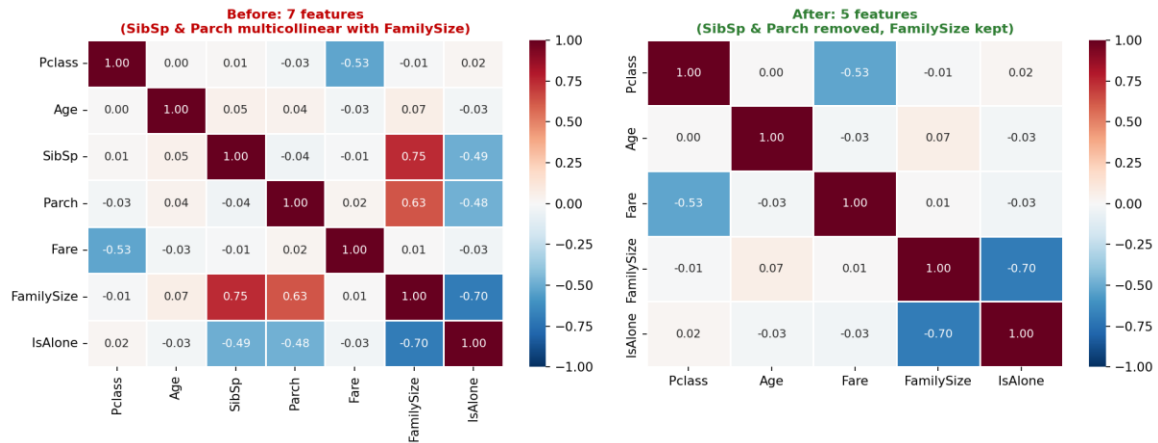
১৩.৩ Multicollinearity সমাধান

```
# পদ্ধতি ১: একটি correlated feature বাদ দিন
# SibSp ও Parch → FamilySize-এ combine করা হয়েছে
# তাই SibSp ও Parch বাদ দিন, FamilySize রাখুন
df = df.drop(['SibSp', 'Parch'], axis=1)

# পদ্ধতি ২: PCA দিয়ে combine করুন (variance preserve করে)
from sklearn.decomposition import PCA
pca = PCA(n_components=1)
df['Family_PC1'] = pca.fit_transform(df[['SibSp', 'Parch', 'FamilySize']])

# পদ্ধতি ৩: Ridge Regression ব্যবহার করুন
# Ridge (L2) multicollinearity-কে penalize করে,
# coefficient শূন্য না হলেও stable হয়
from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0) # Lasso নয়, Ridge
```

Removing Correlated Features — Before vs After Correlation Matrix



চিত্র ১৩.৩: বাম: SibSp, Parch ও FamilySize একসাথে (multicollinear) / ডান: SibSp ও Parch বাদ দিয়ে FamilySize রাখার পরে correlation matrix অনেক পরিষ্কার — কোনো cell-এ $|r| > 0.7$ নেই।

১৩.৪ Decision Guide — কখন Drop, কখন PCA, কখন Ridge?

```
# Multicollinearity সমাধানের Decision Guide:
#
# পরিস্থিতি | সমাধান
#
# দুটো feature combine করা | Drop একটি, রাখুন অন্যটি
# সম্ভব (SibSp+Parch→Family) | (সবচেয়ে সহজ, interpretable)
#
# অনেক correlated feature, | PCA দিয়ে combine করুন
# কোনটা বাদ দেওয়া কঠিন | (variance preserve করে)
#
```

```

# Interpretability দরকার, | Ridge Regression ব্যবহার করুন
# feature বাদ না দিয়ে | (coefficient shrink করে,
# multicollinearity handle | শূন্য করে না)
# -----
# High VIF কিন্তু tree-based | কিছু করতে হবে না – tree
# model ব্যবহার করছেন | multicollinearity-তে কম sensitive
# -----

# Titanic-এ SibSp+Parch+FamilySize পরিস্থিতি:
# VIF: FamilySize=12.8 (গুরুতর), SibSp=8.4, Parch=7.1
# সিদ্ধান্ত: Drop SibSp ও Parch, FamilySize রাখুন
# কারণ: FamilySize = SibSp + Parch + 1 (তিনটির combined information)

from statsmodels.stats.outliers_influence import variance_inflation_factor

# Before: SibSp, Parch, FamilySize তিনটি রাখলে
X_before = df[['Age', 'Fare', 'SibSp', 'Parch', 'FamilySize']].fillna(0)
vif_before = pd.DataFrame({'Feature': X_before.columns,
                           'VIF': [variance_inflation_factor(X_before.values, i) for i in
range(X_before.shape[1])]}])
print("Before:\n", vif_before)

# After: SibSp ও Parch বাদ দিলে
X_after = df[['Age', 'Fare', 'FamilySize']].fillna(0)
vif_after = pd.DataFrame({'Feature': X_after.columns,
                           'VIF': [variance_inflation_factor(X_after.values, i) for i in
range(X_after.shape[1])]}])
print("After:\n", vif_after)
# সব VIF <5 → সমস্যা দূর!

```

১৩.৫ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** SibSp, Parch ও FamilySize তিনটিই model-এ রাখা — এগুলো একে অপরের থেকে প্রায় সম্পূর্ণ predictable (VIF>10)। FamilySize একটিই যথেষ্ট।
- **ভুল:** Correlation heatmap দেখেই সিদ্ধান্ত নেওয়া — VIF দিয়ে সংখ্যাগতভাবে confirm করুন।
- **Best Practice:** Linear/Logistic Regression-এ multicollinearity সবচেয়ে ক্ষতিকর। Tree-based model-এ তুলনামূলক কম সমস্যা।

১৩.৬ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Multicollinearity কী এবং কেন সমস্যা?

উত্তর: দুটো বা ততোধিক feature পরস্পরের সাথে অনেক বেশি correlated ($|r| > 0.7$ বা $VIF > 10$)। Linear model-এ coefficient অস্থির ও ব্যাখ্যাশীল হয় — ছোট data change-এ coefficient ব্যাপকভাবে বদলায়। Tree-based model-এ feature importance ভুলভাবে distribute হয়।

প্রশ্ন: VIF কী? কোন মানে কী সিদ্ধান্ত নেবেন?

উত্তর: VIF (Variance Inflation Factor) = $1/(1-R^2)$ । R^2 হলো ওই feature-কে বাকি সব feature দিয়ে regression করলে যা পাওয়া যায়। $VIF=1$ মানে কোনো multicollinearity নেই। $VIF < 5$ গ্রহণযোগ্য। VIF 5-10 সতর্কতা। $VIF > 10$ গুরুতর — অবশ্যই ব্যবস্থা নিন (drop, combine, বা PCA)।

১৩.৭ অধ্যায় ১৩ সারসংক্ষেপ

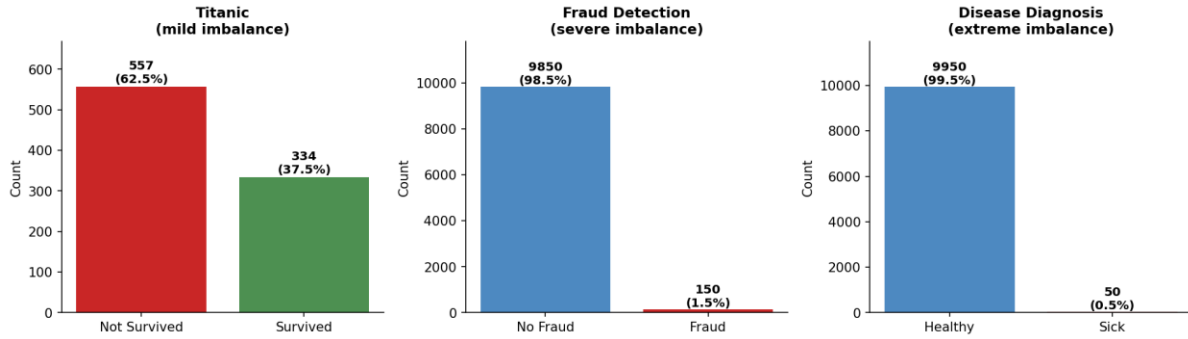
- $|r| > 0.7$ বা $VIF > 10$ দেখলে multicollinearity সমস্যা — correlation heatmap ও VIF দুটোই চেক করুন।
- Titanic-এ SibSp+Parch → FamilySize-এ combine করুন, তারপর SibSp ও Parch বাদ দিন।
- সমাধান decision: Drop (সহজ) → PCA (complex correlated features) → Ridge (interpretability রাখতে)।

অধ্যায় ১৪: ক্লাস ইমব্যালেন্স (Class Imbalance)

Classification সমস্যায় যখন একটা class অন্যটার তুলনায় অনেক বেশি (বা কম) থাকে, তখন তাকে Class Imbalance বলে। বাস্তব জগতে এটা অত্যন্ত সাধারণ — ক্যান্সার রোগী সুস্থ মানুষের তুলনায় অনেক কম, ব্যাংক জালিয়াতি স্বাভাবিক লেনদেনের তুলনায় অনেক কম।

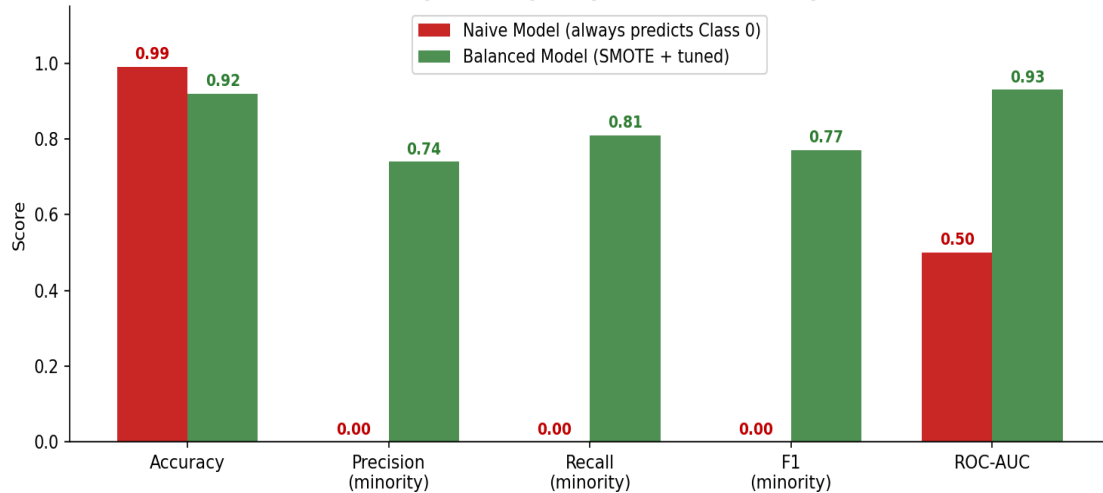
১৪.১ Accuracy Illusion — কেন শুধু Accuracy দেখলে ভুল হয়

Class Imbalance Examples — From Mild to Extreme



চিত্র ১৪.১: তিনটি ভিন্ন dataset-এ class imbalance / Fraud Detection-এ ৯৮.৫% লেনদেন সাধারণ — একটি মডেল যদি সবসময় 'No Fraud' বলে, সে ৯৮.৫% accuracy পাবে কিন্তু একটিও fraud ধরবে না!

The Accuracy Illusion
(99% accuracy but completely useless for minority class!)



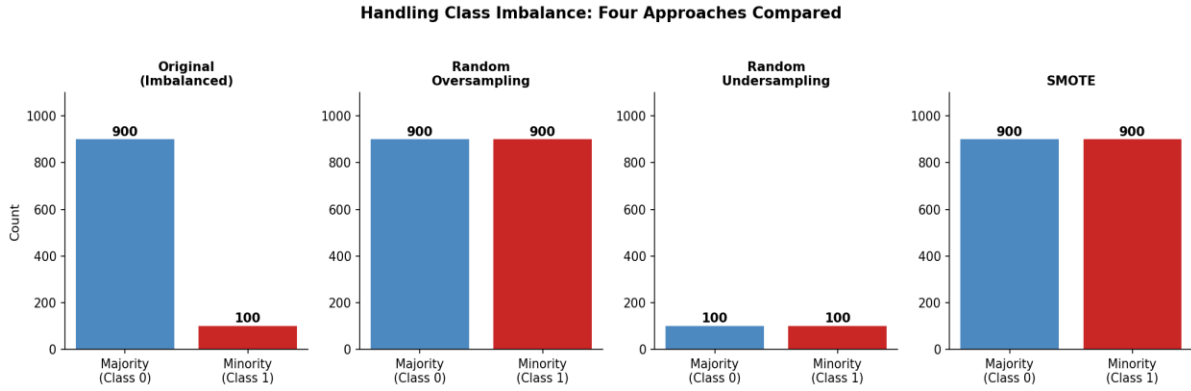
চিত্র ১৪.২: Naive Model (সবসময় majority class predict করে) accuracy=99% কিন্তু minority class-এ Precision, Recall, F1 সব ০! Balanced Model accuracy কিছুটা কম (92%) কিন্তু F1=0.77 — অনেক বেশি useful / এজন্য imbalanced dataset-এ accuracy নয়, F1/ROC-AUC দেখতে হয়।

```
from sklearn.metrics import classification_report, roc_auc_score
```

```
# Imbalanced dataset-এ সবসময় এগুলো দেখুন, accuracy নয়
print(classification_report(y_test, y_pred))
print(f'ROC-AUC: {roc_auc_score(y_test, y_prob):.3f}')

# class_weight='balanced' দিলে model নিজেই minority কে বেশি গুরুত্ব দেয়
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(class_weight='balanced')
```

১৪.২ Random Oversampling ও Undersampling



চিত্র ১৪.৩: চারটি approach / Oversampling-এ minority কপি করা হয়, Undersampling-এ majority কমানো হয়। উভয়ে সমতা আসে কিন্তু সমস্যা আছে।

```
pip install imbalanced-learn

from imblearn.over_sampling import RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler

# Oversampling – minority class random duplicate করে
ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X_train, y_train)
print(y_resampled.value_counts()) # এখন সমান

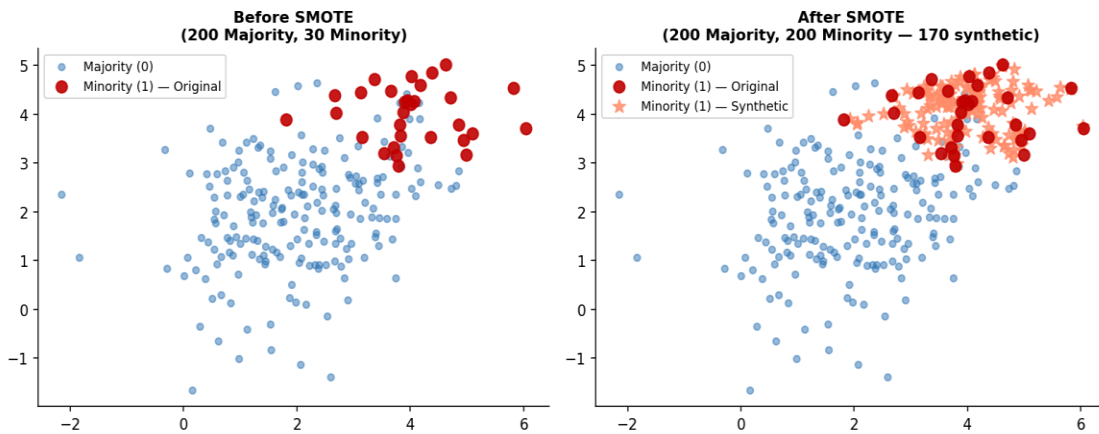
# Undersampling – majority class random remove করে
rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X_train, y_train)
print(y_resampled.value_counts()) # এখন সমান, কিন্তু কম ডেটা
```

পদ্ধতি	কী করে	সুবিধা	অসুবিধা
Random Oversampling	Minority class duplicate করে	সহজ, ডেটা হারান না	Overfitting সম্ভব (duplicate দেখে মুখস্থ)

Random Undersampling	Majority class randomly remove করে	সহজ, দ্রুত	Majority থেকে গুরুত্বপূর্ণ information হারায়
SMOTE	Minority-র synthetic নতুন sample তৈরি করে	Overfitting কম, realistic samples	Noise তৈরি হতে পারে, numeric-only
ADASYN	কঠিন minority sample-এ বেশি focus করে	সবচেয়ে informative region-এ বেশি sample	আরো complex, tuning দরকার
class_weight	Model training-এ minority-কে বেশি penalty দেয়	Data change হয় না, সহজ	সব algorithm support করে না

১৪.৩ SMOTE — Synthetic Minority Over-sampling

SMOTE: Synthetic Minority Over-sampling Technique



চিত্র ১৪.৪: SMOTE-এর আগে (বাম) minority class (লাল) মাত্র ৩০টি। পরে (ডান) কমলা তারা চিহ্ন হলো synthetic sample — real minority-র প্রতিবেশীদের মাঝামাঝি interpolation করে তৈরি। Original minority-র distribution বজায় রেখে balance আনা হয়েছে।

```
from imblearn.over_sampling import SMOTE, ADASYN

# SMOTE – সবচেয়ে জনপ্রিয়
smote = SMOTE(random_state=42, k_neighbors=5)
X_sm, y_sm = smote.fit_resample(X_train, y_train)
print(f'Before: {y_train.value_counts().to_dict()}')
print(f'After: {pd.Series(y_sm).value_counts().to_dict()}')

# ADASYN – কঠিন minority sample-এ বেশি মনোযোগ দেয়
```

```

adasyn = ADASYN(random_state=42)
X_ad, y_ad = adasyn.fit_resample(X_train, y_train)

# সতর্কতা: SMOTE শুধু training data-তে!
# X_test-এ কখনো SMOTE করবেন না

# Pipeline-এ ব্যবহার (best practice)
from imblearn.pipeline import Pipeline as ImbPipeline
pipe = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('model', LogisticRegression())
])
pipe.fit(X_train, y_train)

```

১৪.৪ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** Imbalanced dataset-এ শুধু accuracy দেখে মডেল মূল্যায়ন করা — সবসময় F1-score, Precision, Recall, ROC-AUC দেখুন।
- **ভুল:** Test data-তে SMOTE বা Oversampling করা — SMOTE শুধু X_train, y_train-এ। X_test সবসময় original রাখুন।
- **ভুল:** Cross-Validation-এ SMOTE data leak হওয়া — imblearn-এর Pipeline ব্যবহার করুন, যাতে প্রতিটি fold-এ separately SMOTE হয়।
- **Best Practice:** প্রথমে class_weight='balanced' চেষ্টা করুন — অনেক সময় SMOTE-এর চেয়ে সহজ ও কার্যকর।
- **Best Practice:** Imbalance ratio দেখে technique বাছুন — 1:5 পর্যন্ত class_weight, 1:10 পর্যন্ত SMOTE, 1:100+ এ ADASYN বা hybrid।

১৪.৪.১ class_weight='balanced' — SMOTE ছাড়াই Imbalance Handle করুন

class_weight='balanced' হলো সবচেয়ে সহজ এবং প্রায়ই সবচেয়ে কার্যকর পদ্ধতি — ডেটা পরিবর্তন না করে model-এর loss function-এ minority class-কে বেশি weight দেওয়া হয়। ফলে model minority class-এর ভুলকে বেশি গুরুত্ব দেয়।

```

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC

```

```
# Logistic Regression-এ
lr = LogisticRegression(class_weight='balanced', max_iter=300)
lr.fit(X_train, y_train)

# Random Forest-এ
rf = RandomForestClassifier(class_weight='balanced', n_estimators=100)
rf.fit(X_train, y_train)

# SVM-এ
svm = SVC(class_weight='balanced', probability=True)
svm.fit(X_train, y_train)

# class_weight কীভাবে কাজ করে?
# sklearn স্বয়ংক্রিয়ভাবে weight হিসাব করে:
# weight[class] = n_samples / (n_classes * count[class])
# Titanic-এ: Survived=0 count=549, Survived=1 count=342
# weight[0] = 891 / (2 * 549) = 0.81
# weight[1] = 891 / (2 * 342) = 1.30 ← minority বেশি weight পায়

# কখন class_weight ব্যবহার করবেন:
# ☒ Imbalance ratio 1:5 পর্যন্ত
# ☒ দ্রুত baseline চাই
# ☒ ডেটা কম – SMOTE করলে overfitting হতে পারে
# ☒ sklearn-supported model (LR, RF, SVM, DT) ব্যবহার করছেন
# ☒ Extreme imbalance (1:100+) – SMOTE/ADASYN বেশি কার্যকর
```

১৪.৪.২ ADASYN — SMOTE-এর Advanced Version

ADASYN (Adaptive Synthetic Sampling) SMOTE-এর উন্নত সংস্করণ। SMOTE সব minority sample থেকে সমান synthetic sample তৈরি করে, কিন্তু ADASYN শুধু সেই minority sample-গুলোতে বেশি focus করে যেগুলো majority-র কাছে আছে (harder to classify)। ফলে decision boundary-র কাছের কঠিন এলাকায় বেশি synthetic sample তৈরি হয়।

```
from imblearn.over_sampling import SMOTE, ADASYN

# SMOTE – সব minority sample থেকে সমান synthetic sample
smote = SMOTE(random_state=42, k_neighbors=5)
X_sm, y_sm = smote.fit_resample(X_train, y_train)

# ADASYN – কঠিন minority sample-এ বেশি focus করে
adasyn = ADASYN(random_state=42, n_neighbors=5)
X_ad, y_ad = adasyn.fit_resample(X_train, y_train)
```

```
# পার্থক্য দেখুন
print(f'Original: {dict(y_train.value_counts())}')
print(f'SMOTE:      {dict(pd.Series(y_sm).value_counts())}') # exactly balanced
print(f'ADASYN:    {dict(pd.Series(y_ad).value_counts())}') # approximately balanced

# SMOTE vs ADASYN কখন কোনটা:
# SMOTE   → সাধারণ imbalance, decision boundary সহজ
# ADASYN  → Complex decision boundary, overlap বেশি
#          minority sample কিছু majority-র মাঝে থাকলে ADASYN ভালো

# Pipeline-এ ব্যবহার (সঠিক পদ্ধতি)
from imblearn.pipeline import Pipeline as ImbPipeline
pipe_adasyn = ImbPipeline([
    ('sampler', ADASYN(random_state=42)),
    ('model', LogisticRegression(max_iter=300))
])
pipe_adasyn.fit(X_train, y_train)
```

১৪.৫ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Class Imbalance কী এবং এটা কেন সমস্যা?

উত্তর: Class Imbalance ঘটে যখন classification-এ একটা class অন্যটার তুলনায় অনেক বেশি থাকে (যেমন fraud detection-এ 99:1)। সমস্যা হলো মডেল শুধু majority class predict করেই উচ্চ accuracy পায়, কিন্তু minority class (যেটা সাধারণত বেশি গুরুত্বপূর্ণ) সম্পূর্ণ miss করে। তাই accuracy একটি misleading metric হয়ে যায়।

প্রশ্ন: SMOTE কীভাবে কাজ করে? Random Oversampling-এর চেয়ে কেন ভালো?

উত্তর: SMOTE প্রতিটা minority sample-এর k-জন নিকটতম প্রতিবেশীর সাথে interpolation করে synthetic নতুন sample তৈরি করে। Random Oversampling শুধু existing sample duplicate করে, ফলে মডেল সেই নির্দিষ্ট পয়েন্ট মুখস্থ করে (overfitting)। SMOTE নতুন, বৈচিত্র্যময় sample তৈরি করে, তাই overfitting কম।

প্রশ্ন: Imbalanced dataset-এ কোন metrics ব্যবহার করা উচিত?

উত্তর: Accuracy এড়িয়ে F1-score (Precision ও Recall-এর harmonic mean), ROC-AUC, এবং Precision-Recall curve ব্যবহার করুন। কোনটা বেশি গুরুত্বপূর্ণ তা domain-dependent

— fraud detection-এ high Recall (কোনো fraud miss না করা), spam filtering-এ high Precision (real email স্প্যাম বলা এড়ানো)।

প্রশ্ন: SMOTE কেন test data-তে করা উচিত নয়?

উত্তর: Test data বাস্তব জগতের নতুন ডেটার প্রতিনিধিত্ব করে — যেখানে imbalance real এবং স্বাভাবিক। Test-এ SMOTE করলে মডেলের evaluation বাস্তবতা থেকে বিচ্যুত হয় এবং performance inflated দেখায়। শুধু training data-তে resampling করা উচিত।

১৪.৬ অধ্যায় ১১-১৪ সারসংক্ষেপ

- Feature Creation domain knowledge ব্যবহার করে বিদ্যমান ফিচার থেকে নতুন তথ্যপূর্ণ ফিচার তৈরি করে — FamilySize, IsAlone, AgeGroup ইত্যাদি।
- Feature Selection তিনটি পদ্ধতিতে: Filter (Correlation, MI — দ্রুত), Wrapper (RFE — accurate), Embedded (Lasso, RF Importance — উভয়)।
- Multicollinearity: $|r| > 0.7$ correlation বা $VIF > 10$ দেখলে একটি ফিচার বাদ দিন বা combine করুন। Linear model-এ সবচেয়ে ক্ষতিকর।
- Class Imbalance-এ accuracy misleading — F1, ROC-AUC ব্যবহার করুন। SMOTE শুধু training data-তে। `class_weight='balanced'` দিয়ে শুরু করুন।

অধ্যায় ১৫: ডেটা ট্রান্সফরমেশন (Data Transformation)

Feature Scaling (অধ্যায় ৯) ডেটার range পরিবর্তন করে কিন্তু distribution আকৃতি অপরিবর্তিত রাখে। Data Transformation আরো এক ধাপ এগিয়ে — এটা distribution-এর আকৃতি নিজেই পরিবর্তন করে, right-skewed ডেটাকে normal distribution-এর কাছাকাছি নিয়ে আসে। এটা গুরুত্বপূর্ণ কারণ অনেক ML অ্যালগরিদম normal (বা কাছাকাছি) distribution-এ সবচেয়ে ভালো কাজ করে।

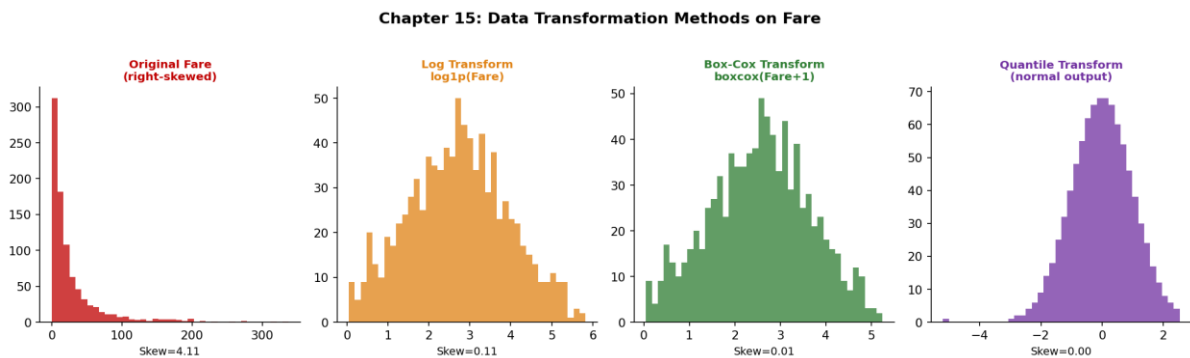
১৫.১ কখন Transformation দরকার?

```
# ডেটা দেখে সিদ্ধান্ত নিন
print(df['Fare'].skew())          # 4.79 – প্রবল right-skewed
print(df['Age'].skew())           # 0.40 – হালকা right-skewed

# QQ Plot দিয়ে normality চেক করুন (অধ্যায় ৪)
from scipy import stats
stats.probplot(df['Fare'], dist='norm', plot=plt)

# সিদ্ধান্তের নিয়ম:
# |skewness| < 0.5 → transformation সাধারণত দরকার নেই
# 0.5-1.0          → হালকা transformation বিবেচনা করুন
# > 1.0            → transformation দরকার (log/Box-Cox/Yeo-Johnson)
```

১৫.২ চারটি পদ্ধতি তুলনা



চিত্র ১৫.১: Original Fare skewness=4.11 / Log Transform-এ skewness≈0.19, Box-Cox-এ আরো ভালো, Quantile Transform-এ প্রায় perfect normal (skewness≈0) / প্রতিটি পদ্ধতির trade-off আলাদা।

১৫.৩ Log Transformation

সবচেয়ে সহজ ও জনপ্রিয় transformation। Right-skewed, positive ডেটায় অত্যন্ত কার্যকর।

```
import numpy as np

# log1p - log(1+x), 0 হলেও নিরাপদ
X_train['Fare_log'] = np.log1p(X_train['Fare'])
X_test['Fare_log'] = np.log1p(X_test['Fare'])

print(f'Original skew: {X_train["Fare"].skew():.2f}')          # 4.79
print(f'After log1p: {X_train["Fare_log"].skew():.2f}')      # 0.19

# সতর্কতা: x <= 0 হলে log(x) undefined - log1p ব্যবহার করুন
# অথবা আগে shift করুন: log(x + abs(min_value) + 1)
```

১৫.৪ Box-Cox Transformation

Box-Cox একটি parameter λ (lambda) optimize করে যা নির্ধারণ করে কীভাবে transform হবে। $\lambda=0$ হলে log transform, $\lambda=0.5$ হলে square root, $\lambda=1$ হলে কোনো change নেই। শুধু positive ($x>0$) ডেটায় কাজ করে।

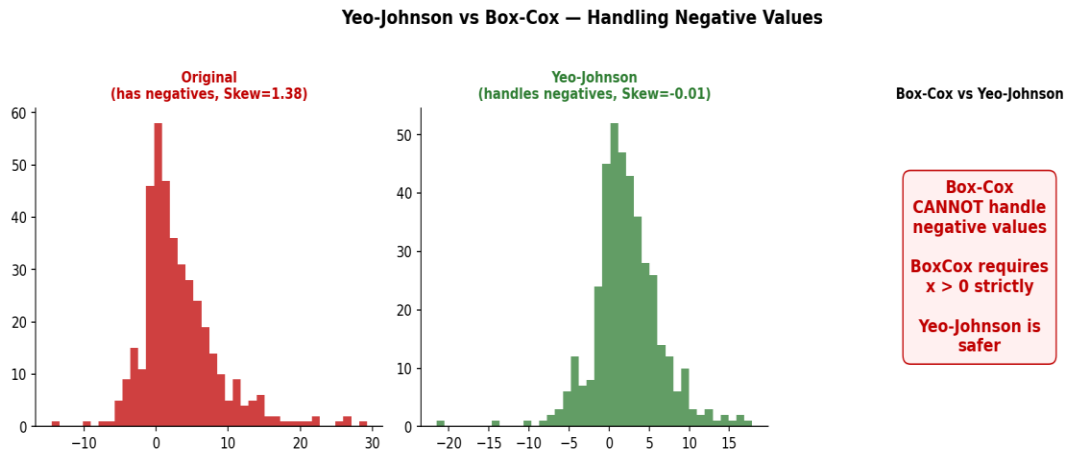
```
from scipy.stats import boxcox
from sklearn.preprocessing import PowerTransformer

# scipy দিয়ে (সর্বোত্তম lambda খোঁজে)
fare_bc, lambda_val = boxcox(X_train['Fare'] + 1) # +1 কারণ Fare=0 থাকতে পারে
print(f'Best lambda: {lambda_val:.4f}')          # lambda ~= 0.27

# Test-এ একই lambda ব্যবহার করুন
from scipy.special import boxcox1p
X_test_bc = boxcox1p(X_test['Fare'], lambda_val)

# sklearn PowerTransformer (pipeline-friendly)
pt = PowerTransformer(method='box-cox')
X_train[['Fare']] = pt.fit_transform(X_train[['Fare']] + 1)
X_test[['Fare']] = pt.transform(X_test[['Fare']] + 1)
```

১৫.৫ Yeo-Johnson Transformation — নেগেটিভ মানের কাজ করে



চিত্র ১৫.২: Box-Cox negative মান handle করতে পারে না ($x > 0$ required) / Yeo-Johnson negative মানের কাজ করে এবং skewness প্রায় ০-এ নিয়ে আসে। নিরাপদ পছন্দ হিসেবে Yeo-Johnson সাধারণত Box-Cox এর চেয়ে ভালো।

```
from scipy.stats import yeojohnson
from sklearn.preprocessing import PowerTransformer

# scipy দিয়ে
data_yj, lam = yeojohnson(X_train['Fare'])

# sklearn PowerTransformer (best – pipeline-ready, handles negatives)
pt = PowerTransformer(method='yeo-johnson') # ডিফল্ট method
X_train[['Fare']] = pt.fit_transform(X_train[['Fare']])
X_test[['Fare']] = pt.transform(X_test[['Fare']])
```

১৫.৬ Quantile Transformer — সবচেয়ে শক্তিশালী

Quantile Transformer ডেটার quantile ব্যবহার করে যেকোনো distribution-কে normal বা uniform distribution-এ রূপান্তর করে। আউটলায়ারের প্রভাব সবচেয়ে কম কারণ এটা rank-based।

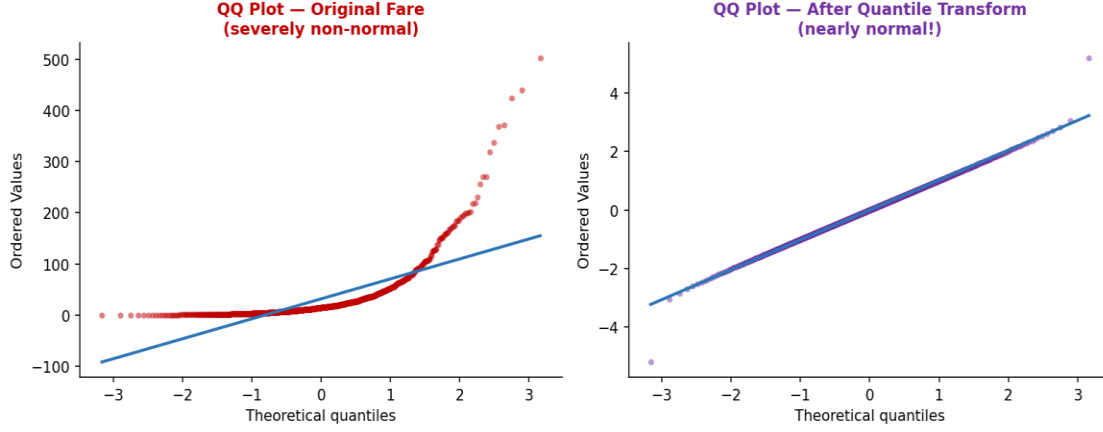
```
from sklearn.preprocessing import QuantileTransformer

qt = QuantileTransformer(output_distribution='normal', random_state=42)

qt.fit(X_train[['Fare']])
X_train[['Fare']] = qt.transform(X_train[['Fare']])
X_test[['Fare']] = qt.transform(X_test[['Fare']])
```

```
# Uniform output-এর জন্য:
qt_uniform = QuantileTransformer(output_distribution='uniform')
```

QQ Plot Before vs After Quantile Transformation



চিত্র ১৫.৩: বাম— Original Fare-এর QQ Plot, বিন্দুগুলো রেখা থেকে প্রবলভাবে বিচ্যুত। ডান— Quantile Transform-এর পরে বিন্দুগুলো প্রায় রেখার ওপর— প্রায় perfect normal।

পদ্ধতি	Negative মান?	Outlier Effect	Output	কখন সেরা
Log (log1p)	না ($x > 0$ only)	কমায়	Approx. normal	Simple, right-skewed, always positive
Box-Cox	না ($x > 0$ strictly)	কমায়	Near normal	Lab data, financial (positive only)
Yeo-Johnson	হ্যাঁ	কমায়	Near normal	সর্বজনীন — negative ও positive
Quantile Transform	হ্যাঁ	সবচেয়ে কম	Exactly normal/uniform	Extreme outlier, আউটলায়ার দূর করতে চান না

সোনালী নিয়ম: সাধারণত Yeo-Johnson (PowerTransformer) দিয়ে শুরু করুন — এটা negative ও positive উভয় মানের কাজ করে এবং sklearn Pipeline-এ নির্বিঘ্নে fit হয়। Extreme outlier থাকলে Quantile Transformer বিবেচনা করুন।

১৫.৭ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: Data Transformation ও Feature Scaling-এর মধ্যে পার্থক্য কী?

উত্তর: Feature Scaling (StandardScaler, MinMaxScaler) ডেটার range বা mean/std পরিবর্তন করে কিন্তু distribution-এর আকৃতি (skewness) অপরিবর্তিত রাখে। Data Transformation (Log, Box-Cox, Yeo-Johnson) distribution-এর আকৃতি নিজেই পরিবর্তন করে skewed ডেটাকে normal-এর কাছাকাছি নিয়ে আসে। প্রায়ই দুটো একসাথে করা হয়: আগে Transform, তারপর Scale।

প্রশ্ন: Box-Cox ও Yeo-Johnson-এর মধ্যে পার্থক্য কী?

উত্তর: Box-Cox শুধু strictly positive ($x > 0$) ডেটায় কাজ করে। Yeo-Johnson Box-Cox-এর extended version যা negative ও zero মানেরও কাজ করে। তাই নিরাপদ default choice হিসেবে Yeo-Johnson ব্যবহার করা ভালো — sklearn-এর PowerTransformer(method='yeo-johnson') দিয়ে সহজে Pipeline-এ ব্যবহার করা যায়।

প্রশ্ন: কখন Quantile Transformer ব্যবহার করবেন?

উত্তর: Quantile Transformer তখন ব্যবহার করবেন যখন extreme outlier আছে এবং সেগুলো বাদ দিতে চান না, বা যখন distribution-কে exactly normal বা uniform করতে হবে। এটা rank-based হওয়ায় outlier-এর প্রভাব সবচেয়ে কম। তবে test data-য় নতুন range-এর মান আসলে সমস্যা হতে পারে — তাই n_quantiles parameter দিয়ে সতর্কভাবে ব্যবহার করুন।

১৫.৮ কখন Scaling করব, কখন Transformation — Decision Guide

Scaling ও Transformation-এর মধ্যে পার্থক্য এবং কোন পরিস্থিতিতে কোনটা (বা উভয়) প্রয়োজন তার সম্পূর্ণ decision framework:

```
# _____
# Decision Flow: Scaling vs Transformation
# _____
#
# প্রশ্ন ১: Distribution কি right-skewed? (|skewness| > 1?)
# _____
# না → Scaling শুধু দরকার হতে পারে (step ২ এ যান)
```

```

# হ্যাঁ → আগে Transformation করুন, তারপর Scaling
#
# প্রশ্ন ২: কোন অ্যালগরিদম ব্যবহার করছেন?
# -----
# Decision Tree / Random Forest / XGBoost
#   → Scaling দরকার নেই, Transformation optional
# KNN / SVM / LR / NN / PCA
#   → Scaling অবশ্যই দরকার
#
# প্রশ্ন ৩: Outlier আছে?
# -----
# না → StandardScaler ব্যবহার করুন
# হ্যাঁ → RobustScaler বা (log1p + StandardScaler)
#
# সংক্ষেপে:
# -----
# Scaling → Range/unit পরিবর্তন, shape অপরিবর্তিত
# Transform → Shape পরিবর্তন (skew কমানো)
# Best combo → log1p() তারপর StandardScaler (skewed ফিচারে)
# -----

import numpy as np
from sklearn.preprocessing import StandardScaler, PowerTransformer

# Titanic Fare – skew=4.79 (প্রবল right-skewed) + outlier আছে
# Decision: Transform + Scale

# Option A: log1p + StandardScaler
X_train['Fare_log'] = np.log1p(X_train['Fare'])
scaler = StandardScaler()
X_train['Fare_scaled'] = scaler.fit_transform(X_train[['Fare_log']])

# Option B: PowerTransformer (Yeo-Johnson) – সর্বজনীন
pt = PowerTransformer(method='yeo-johnson')
X_train['Fare_yj'] = pt.fit_transform(X_train[['Fare']])

# Titanic Age – skew=0.40 (প্রায় normal)
# Decision: শুধু Scale করুন
X_train['Age_scaled'] =
scaler.fit_transform(X_train[['Age']].fillna(X_train['Age'].median()))

```

১৫.৯ অধ্যায় ১৫ সারসংক্ষেপ

- Scaling: range/unit পরিবর্তন, shape ঠিক থাকে। Transformation: shape পরিবর্তন (skewness কমায়)।
- $|\text{skewness}| < 0.5$: transformation দরকার নেই। $0.5-1.0$: বিবেচনা করুন। >1.0 : অবশ্যই transform করুন।
- Default choice: `PowerTransformer(method='yeo-johnson')` — negative ও positive উভয় মানে কাজ করে, Pipeline-ready।
- Extreme outlier + non-normal: `QuantileTransformer` সবচেয়ে powerful (rank-based, outlier immune)।

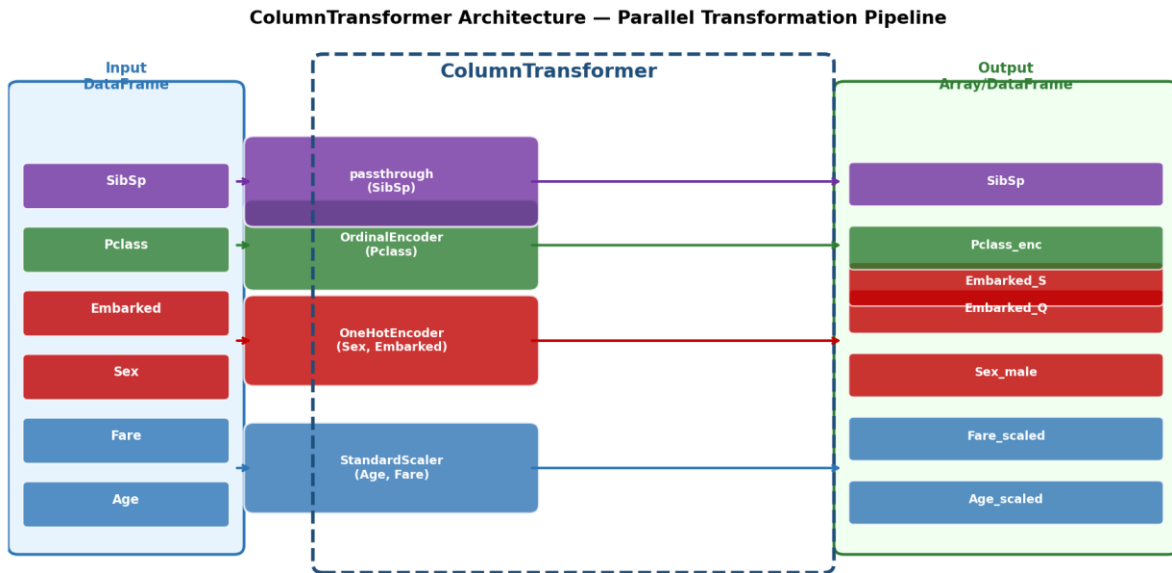
অধ্যায় ১৬: ColumnTransformer

একটা ডেটাসেটে একসাথে numerical ও categorical কলাম থাকে। Numerical কলামে Scaling দরকার, categorical কলামে Encoding দরকার, মিসিং ভ্যালুতে Imputation দরকার। আগের অধ্যায়গুলোতে আমরা এগুলো একটা একটা করে করেছি। ColumnTransformer এই সব transformation এক জায়গায়, একসাথে, parallel-ভাবে সম্পন্ন করে।

analogy:

একটা ফ্যাক্টরিতে বিভিন্ন department একসাথে কাজ করে — একজন গাড়ির ইঞ্জিন বানাচ্ছে, আরেকজন রং করছে, আরেকজন আসন লাগাচ্ছে। সব শেষে এগুলো একত্রিত হয়ে পূর্ণ গাড়ি তৈরি হয়। ColumnTransformer ঠিক এভাবেই প্রতিটা কলাম-গ্রুপে আলাদা transformer প্রয়োগ করে, তারপর সব output একসাথে জোড়া লাগায়।

১৬.১ Architecture ও কীভাবে কাজ করে



চিত্র ১৬.১: ColumnTransformer Architecture / Input DataFrame-এর কলামগুলো আলাদা transformer-এ রুট হয় (parallel), তারপর সব output একসাথে concatenate হয়ে Output Array তৈরি হয়।

১৬.২ Basic ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
```



```

# কলামগুলো ভাগ করুন
num_cols = ['Age', 'Fare']
cat_nom  = ['Sex', 'Embarked']
cat_ord  = ['Pclass']

# ColumnTransformer তৈরি
ct = ColumnTransformer(
    transformers=[
        ('num_impute_scale',
         Pipeline([('imputer', SimpleImputer(strategy='median')),
                   ('scaler', StandardScaler())]),
         num_cols),

        ('cat_one',
         OneHotEncoder(sparse_output=False, drop='first'),
         cat_nom),

        ('cat_ord',
         OrdinalEncoder(categories=[['third','second','first']]),
         cat_ord),
    ],
    remainder='passthrough' # বাকি কলামগুলো যেমন আছে তেমন রাখো
)

ct.set_output(transform='pandas') # DataFrame output

# fit শুধু train-এ
ct.fit(X_train)
X_train_ct = ct.transform(X_train)
X_test_ct  = ct.transform(X_test)

```

১৬.৩ ColumnTransformer-এর গুরুত্বপূর্ণ Parameters

Parameter	মান	ব্যাখ্যা
remainder	'drop' (default)	উল্লেখ না করা কলাম বাদ দেওয়া হয়
remainder	'passthrough'	উল্লেখ না করা কলাম যেমন আছে তেমন রাখা হয়
verbose_feature_names_out	True (default)	Output কলামের নামে prefix যোগ হয়: 'num__Age'
verbose_feature_names_out	False	Output কলামের নাম original-এর মতো থাকে

Parameter	মান	ব্যাখ্যা
n_jobs	-1	সব CPU core ব্যবহার করে parallel processing

```
# verbose_feature_names_out ব্যবহার
ct = ColumnTransformer([...], verbose_feature_names_out=False)
ct.set_output(transform='pandas')
X_train_ct = ct.fit_transform(X_train)

# Output কলামের নাম দেখুন
print(X_train_ct.columns.tolist())
# ['Age', 'Fare', 'Sex_female', 'Embarked_Q', 'Embarked_S', 'Pclass', 'SibSp', ...]

# remainder='drop' হলে উল্লেখ না করা কলাম output-এ থাকবে না
# remainder='passthrough' হলে থাকবে
```

১৬.৪ Nested ColumnTransformer

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

# Numerical: প্রথমে impute, তারপর scale
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Categorical: প্রথমে impute, তারপর encode
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(sparse_output=False, drop='first'))
])

# ColumnTransformer-এ nested pipeline
full_ct = ColumnTransformer([
    ('num', num_pipeline, ['Age', 'Fare']),
    ('cat', cat_pipeline, ['Sex', 'Embarked', 'Pclass']),
], remainder='passthrough')

full_ct.set_output(transform='pandas')
X_train_final = full_ct.fit_transform(X_train)
X_test_final = full_ct.transform(X_test)
```

	ColumnTransformer ছাড়া	ColumnTransformer দিয়ে
কোডের পরিমাণ	প্রতিটা কলামের জন্য আলাদা কোড	সংক্ষিপ্ত, এক জায়গায়
Leakage ঝুঁকি	বেশি (ভুলে fit করা সহজ)	কম (Pipeline-এ fit/transform স্বয়ংক্রিয়)
Pipeline integration	কঠিন	সহজ — সরাসরি Pipeline-এ ব্যবহার করুন
Execution	Sequential	Parallel (n_jobs=-1)
Production deploy	জটিল	pickle করে সহজে deploy করা যায়

১৬.৫ remainder='passthrough' vs remainder='drop'

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

num_cols = ['Age', 'Fare']
cat_cols = ['Sex', 'Embarked']
# 'SibSp', 'Parch', 'Pclass' – উল্লেখ করা হয়নি

# remainder='drop' (default) – উল্লেখ না করা কলাম output-এ আসবে না
ct_drop = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(drop='first', sparse_output=False), cat_cols),
], remainder='drop')
X_out = ct_drop.fit_transform(X_train)
# Output: শুধু Age, Fare (scaled) + Sex, Embarked (encoded)
# SibSp, Parch, Pclass বাদ পড়ে গেছে!

# remainder='passthrough' – উল্লেখ না করা কলাম যেমন আছে তেমন রাখা হবে
ct_pass = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(drop='first', sparse_output=False), cat_cols),
], remainder='passthrough')
X_out = ct_pass.fit_transform(X_train)
# Output: Age, Fare (scaled) + Sex, Embarked (encoded) + SibSp, Parch, Pclass
# (unchanged)

# কখন কোনটা:
# remainder='drop' → শুধু উল্লেখ করা কলামই দরকার
# remainder='passthrough' → বাকি কলামগুলোও রাখতে চাই (যেমন binary features)

```

১৬.৬ get_feature_names_out() — Transformed Column নাম বের করা

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
import pandas as pd

num_cols = ['Age', 'Fare']
cat_cols = ['Sex', 'Embarked']

ct = ColumnTransformer([
    ('num', Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc', StandardScaler())
    ]), num_cols),
    ('cat', OneHotEncoder(drop='first', sparse_output=False), cat_cols),
], remainder='passthrough')

ct.fit(X_train)

# Transformed column names বের করুন
feature_names = ct.get_feature_names_out()
print(feature_names)
# ['num__Age', 'num__Fare', 'cat__Sex_male', 'cat__Embarked_Q', 'cat__Embarked_S',
#  'remainder__SibSp', 'remainder__Parch', 'remainder__Pclass', ...]

# DataFrame হিসেবে output পেতে
ct.set_output(transform='pandas')
X_train_ct = ct.fit_transform(X_train)
print(X_train_ct.columns.tolist()) # সব column নাম দেখুন

# verbose_feature_names_out=False দিলে prefix ছাড়া নাম
ct2 = ColumnTransformer([...], verbose_feature_names_out=False)
# ['Age', 'Fare', 'Sex_male', 'Embarked_Q', ...] ← prefix নেই
```

১৬.৭ Custom Transformer যোগ করা

FunctionTransformer বা BaseEstimator ব্যবহার করে নিজের custom transformer ColumnTransformer-এ যোগ করা যায়:

```
import numpy as np
from sklearn.preprocessing import FunctionTransformer
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
```

```
# পদ্ধতি ১: FunctionTransformer – সহজ function wrap করা
log_transformer = FunctionTransformer(np.log1p, validate=True)

# পদ্ধতি ২: Custom Class (fit/transform সহ) – আরো শক্তিশালী
class FamilySizeCreator(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self # কিছু শেখার নেই
    def transform(self, X):
        X = X.copy()
        X['FamilySize'] = X['SibSp'] + X['Parch'] + 1
        X['IsAlone'] = (X['FamilySize'] == 1).astype(int)
        return X[['FamilySize', 'IsAlone']]

# ColumnTransformer-এ custom transformer ব্যবহার
ct = ColumnTransformer([
    ('log_fare', log_transformer, ['Fare']),
    ('scale_age', StandardScaler(), ['Age']),
    ('family', FamilySizeCreator(), ['SibSp', 'Parch']),
], remainder='drop')

X_train_ct = ct.fit_transform(X_train)
print(X_train_ct.shape) # (623, 4) – Fare_log, Age_scaled, FamilySize, IsAlone
```

১৬.৮ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** remainder='drop' (default) ভুলে যাওয়া — গুরুত্বপূর্ণ কলাম output-এ না থাকলে তা খুঁজে পাওয়া কঠিন। সবসময় remainder explicitly সেট করুন।
- **ভুল:** ct.fit() পুরো ডেটায় করা — সবসময় ct.fit(X_train), তারপর ct.transform(X_test)।
- **Best Practice:** ct.set_output(transform='pandas') ব্যবহার করুন — output DataFrame হিসেবে পাবেন, column name সহ।

১৬.৯ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: ColumnTransformer কেন ব্যবহার করা হয়?

উত্তর: Numerical ও categorical কলামে আলাদা transformation (scaling, encoding) একসাথে এক জায়গায় করতে। Pipeline-এ সহজে integrate হয়। Parallel execution সম্ভব (n_jobs=-1)। fit() শুধু X_train-এ — leakage রোধ নিশ্চিত হয়। Production deployment-এ pickle করা সহজ।

প্রশ্ন: remainder='passthrough' ও remainder='drop' এর পার্থক্য কী?

উত্তর: remainder='drop' (default) — ColumnTransformer-এ explicitly উল্লেখ না করা কলাম output-এ আসবে না (বাদ পড়বে)। remainder='passthrough' — উল্লেখ না করা কলাম unchanged হিসেবে output-এ যোগ হবে। IsAlone, Cabin_missing-এর মতো ইতোমধ্যে binary feature-গুলো passthrough-এ রাখা সুবিধাজনক।

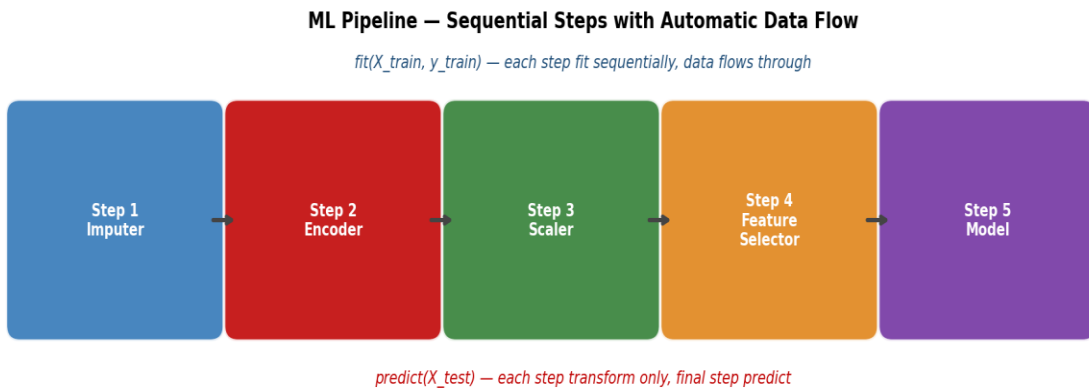
অধ্যায় ১৭: Pipeline

ML Pipeline হলো একটা chain — যেখানে ডেটা একটি ধাপ থেকে পরের ধাপে স্বয়ংক্রিয়ভাবে প্রবাহিত হয়। ColumnTransformer, scaling, feature selection, model training — সব একটাই object-এ। এটি Data Leakage সম্পূর্ণরূপে প্রতিরোধ করে এবং production deployment-এ সবচেয়ে নির্ভরযোগ্য পদ্ধতি।

কেন Pipeline ব্যবহার করবেন?

- Data Leakage শূন্য — `fit()` শুধু training data দেখে, `transform()` automatically সব step-এ প্রয়োগ হয়।
- Code পরিষ্কার ও সংক্ষিপ্ত — preprocessing ও model একটা object।
- Cross-Validation-এ স্বয়ংক্রিয়ভাবে সঠিক — প্রতিটা fold-এ আলাদাভাবে fit হয়।
- GridSearchCV-এ hyperparameter tune করা যায় preprocessing ও model একসাথে।
- Production deploy সহজ — একটাই pickle file, সব preprocessing সহ।

১৭.১ Pipeline-এর Flow



চিত্র ১৭.১: Pipeline-এর flow। `fit(X_train, y_train)` call করলে প্রতিটা step sequentially fit হয়, ডেটা স্বয়ংক্রিয়ভাবে পরের step-এ যায়। `predict(X_test)` call করলে প্রতিটা step শুধু transform করে, শেষ step predict করে।

১৭.২ Basic Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=300))
])

# fit – সব step একসাথে (train only)
pipe.fit(X_train, y_train)

# predict – সব preprocessing + prediction (test)
y_pred = pipe.predict(X_test)
y_prob = pipe.predict_proba(X_test)[:,-1]

# score
print(pipe.score(X_test, y_test)) # accuracy

```

১৭.৩ Full Pipeline — ColumnTransformer সহ

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

num_cols = ['Age', 'Fare', 'SibSp', 'Parch']
cat_cols = ['Sex', 'Embarked']

# Preprocessing ColumnTransformer
preprocessor = ColumnTransformer([
    ('num', Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc', StandardScaler())
    ]), num_cols),
    ('cat', Pipeline([
        ('imp', SimpleImputer(strategy='most_frequent')),
        ('enc', OneHotEncoder(drop='first', sparse_output=False))
    ]), cat_cols)
], remainder='passthrough')

# Full Pipeline
full_pipe = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestClassifier(n_estimators=100, random_state=42))
])

full_pipe.fit(X_train, y_train)
y_pred = full_pipe.predict(X_test)
print(f'Accuracy: {full_pipe.score(X_test, y_test):.3f}')

```


১৭.৪ GridSearchCV + Pipeline

GridSearchCV with Pipeline — Parameter Naming Convention

pipe.steps

```
[('imputer', SimpleImputer(...)),
 ('scaler', StandardScaler()),
 ('model', LogisticRegression())]
```

param_grid

```
{'imputer_strategy': ['mean', 'median'],
 'model_C': [0.01, 0.1, 1, 10],
 'model_max_iter': [100, 300]}
```

Naming rule:

```
stepname__param (double underscore __)
```

চিত্র ১৭.২: GridSearchCV-এ Pipeline parameter নির্দিষ্ট করতে 'stepname__param' format ব্যবহার করুন (double underscore) / imputer-এর strategy এবং model-এর C একসাথে tune করা যায়।

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'preprocessor__num__imp__strategy': ['mean', 'median'],
    'model__n_estimators': [50, 100, 200],
    'model__max_depth': [3, 5, None],
}

grid_search = GridSearchCV(
    full_pipe,
    param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)

print(f'Best params: {grid_search.best_params_}')
print(f'Best F1: {grid_search.best_score_:.3f}')

# সেরা Pipeline দিয়ে test evaluation
best_pipe = grid_search.best_estimator_
y_pred = best_pipe.predict(X_test)
```

১৭.৫ Pipeline Save ও Load — Production Deploy

```
import joblib

# Pipeline save করুন (সব preprocessing সহ)
joblib.dump(full_pipe, 'titanic_pipeline.pkl')

# Production-এ load করুন
loaded_pipe = joblib.load('titanic_pipeline.pkl')

# নতুন ডেটায় predict – preprocessing স্বয়ংক্রিয়
new_passenger = pd.DataFrame({
    'Age': [28.0], 'Fare': [12.5], 'Sex': ['female'],
    'Embarked': ['S'], 'SibSp': [0], 'Parch': [1]
})
prediction = loaded_pipe.predict(new_passenger)
probability = loaded_pipe.predict_proba(new_passenger)[: ,1]
print(f'Survived: {prediction[0]}, Probability: {probability[0]:.3f}')
```

১৭.৬ Pipeline Debugging — named_steps দিয়ে Intermediate Output দেখা

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=300))
])

pipe.fit(X_train_num, y_train)

# named_steps দিয়ে যেকোনো step access করুন
print(pipe.named_steps)
# {'imputer': SimpleImputer(), 'scaler': StandardScaler(), 'model':
LogisticRegression()}

# Intermediate step-এর output দেখুন (debugging)
# Step 1 এর পরে কেমন দেখাচ্ছে?
X_after_imputer = pipe.named_steps['imputer'].transform(X_train_num)
print("After imputer:\n", X_after_imputer[:3])

# Step 2 এর পরে?
```

```

X_after_scaler = pipe.named_steps['scaler'].transform(X_after_imputer)
print("After scaler:\n", X_after_scaler[:3])

# Pipeline-এর step পর্যন্ত transform করার shortcut
X_up_to_scaler = pipe[:-1].transform(X_train_num) # model বাদ দিয়ে সব step

# Model-এর coefficient দেখুন
print("Coefficients:", pipe.named_steps['model'].coef_)
print("Intercept:", pipe.named_steps['model'].intercept_)

```

১৭.৭ set_params() — Pipeline-এর Parameter পরিবর্তন

```

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(C=1.0, max_iter=300))
])

# set_params() দিয়ে parameter পরিবর্তন করুন
# Format: 'stepname__parametername'
pipe.set_params(
    imputer__strategy='mean',      # imputer step-এর strategy পরিবর্তন
    model__C=0.5,                  # model step-এর C পরিবর্তন
    model__max_iter=500
)

# যাচাই করুন
print(pipe.named_steps['imputer'].strategy) # 'mean'
print(pipe.named_steps['model'].C)          # 0.5

# GridSearchCV-এ set_params ব্যবহার করা হয় internally
param_grid = {
    'imputer__strategy': ['mean', 'median'],
    'model__C': [0.1, 1.0, 10.0],
}

# GridSearchCV প্রতিটি combination-এ set_params() call করে

# Pipeline-এর step পুরোপুরি বদলে ফেলুন
from sklearn.ensemble import RandomForestClassifier
pipe.set_params(model=RandomForestClassifier(n_estimators=100))
print(pipe.steps) # এখন model step-এ RandomForestClassifier

```

১৭.৮ সাধারণ ভুল ও বেস্ট প্র্যাকটিস

- **ভুল:** Pipeline-এর বাইরে preprocessing করে তারপর Pipeline-এ ঢোকানো — সম্পূর্ণ Pipeline-এর ভেতরে সব preprocessing রাখুন, না হলে leakage হতে পারে।
- **ভুল:** SMOTE বা imbalanced-learn ব্যবহার করলে sklearn Pipeline-এর বদলে imblearn.pipeline.Pipeline ব্যবহার না করা — imblearn Pipeline দরকার।
- **Best Practice:** Pipeline সবসময় একটা object হিসেবে pickle করুন — সব preprocessing সহ। Production-এ নতুন ডেটায় শুধু pipe.predict(X_new) করলেই সব preprocessing স্বয়ংক্রিয়ভাবে হয়।

১৭.৯ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: sklearn Pipeline কেন ব্যবহার করা উচিত?

উত্তর: Data Leakage শূন্য (fit() শুধু training data দেখে)। Code সংক্ষিপ্ত — preprocessing ও model একটা object। Cross-Validation-এ প্রতিটি fold-এ আলাদাভাবে fit হয়। GridSearchCV-এ preprocessing ও model parameter একসাথে tune করা যায়। Production deploy সহজ — একটি pickle file, সব preprocessing সহ।

প্রশ্ন: Pipeline-এ কীভাবে intermediate step debug করবেন?

উত্তর: pipe.named_steps['step_name'] দিয়ে যেকোনো step access করা যায়। pipe[:1].transform(X) দিয়ে শেষ step বাদ দিয়ে সব step চালানো যায়। set_params() দিয়ে যেকোনো step-এর parameter পরিবর্তন করা যায় (format: 'stepname__parametername')।

১৭.১০ অধ্যায় ১৭ সারসংক্ষেপ

- Pipeline = ColumnTransformer + preprocessing + model একটা chain-এ। fit() শুধু X_train দেখে, predict() তে সব step স্বয়ংক্রিয়ভাবে apply হয়।
- named_steps দিয়ে debug করুন। set_params('step__param') দিয়ে parameter পরিবর্তন করুন।
- joblib.dump() দিয়ে সম্পূর্ণ Pipeline save করুন — production-এ load করলে সব preprocessing সহ predict() করা যাবে।

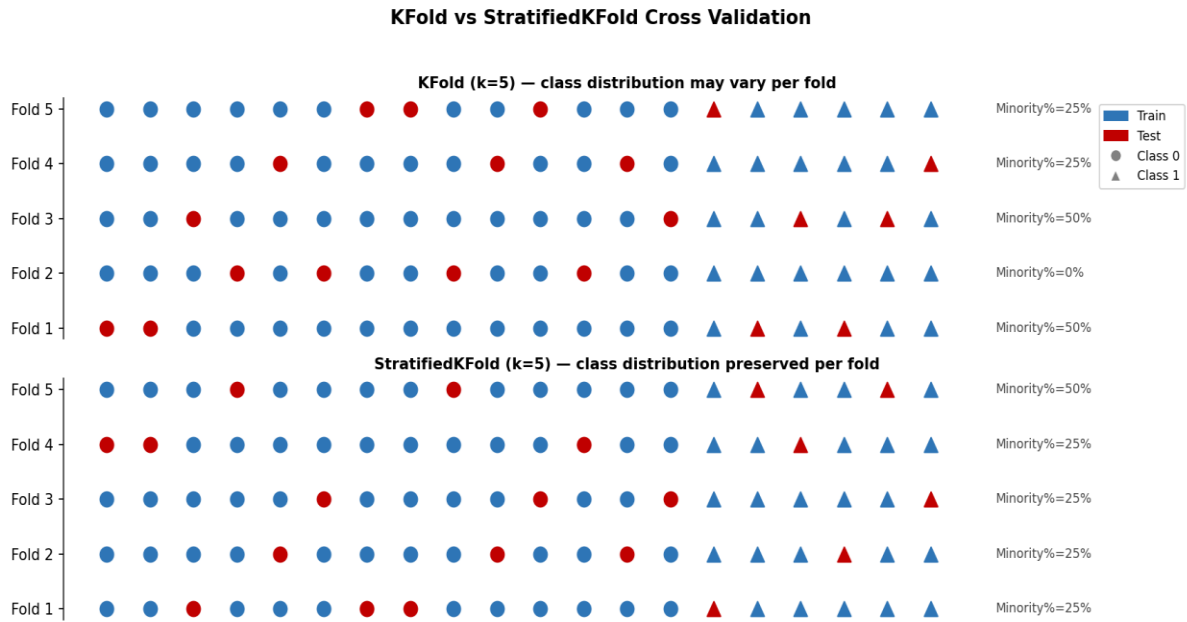
অধ্যায় ১৮: ক্রস ভ্যালিডেশন (Cross Validation)

একটা মডেলকে একবার Train/Test split করে মূল্যায়ন করলে সমস্যা হলো — সেই নির্দিষ্ট split-এর ওপর ফলাফল অনেকটাই নির্ভর করে। ডেটা randomly ভাগ হওয়ায় কখনো test set-এ সহজ data, কখনো কঠিন data পড়তে পারে। Cross Validation এই সমস্যা সমাধান করে ডেটার বিভিন্ন অংশকে পালাক্রমে test set হিসেবে ব্যবহার করে।

analogy:

পরীক্ষা নেওয়ার মতো — একবার পরীক্ষা নিলে ফলাফল সেই দিনের পরিস্থিতির ওপর নির্ভর করে। কিন্তু ৫টি ভিন্ন পরীক্ষায় সব ক্ষেত্রেই ভালো করলে তুমি সত্যিকারের দক্ষ। Cross Validation এই ৫টি পরীক্ষার মতো।

১৮.১ KFold vs StratifiedKFold



চিত্র ১৮.১: KFold-এ minority class (ত্রিভুজ) per-fold distribution বদলে যায় — কোনো fold-এ minority বেশি, কোনোটায়ে কম। StratifiedKFold-এ প্রতিটা fold-এ minority% প্রায় একই। Classification-এ সবসময় StratifiedKFold।

```
from sklearn.model_selection import KFold, StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

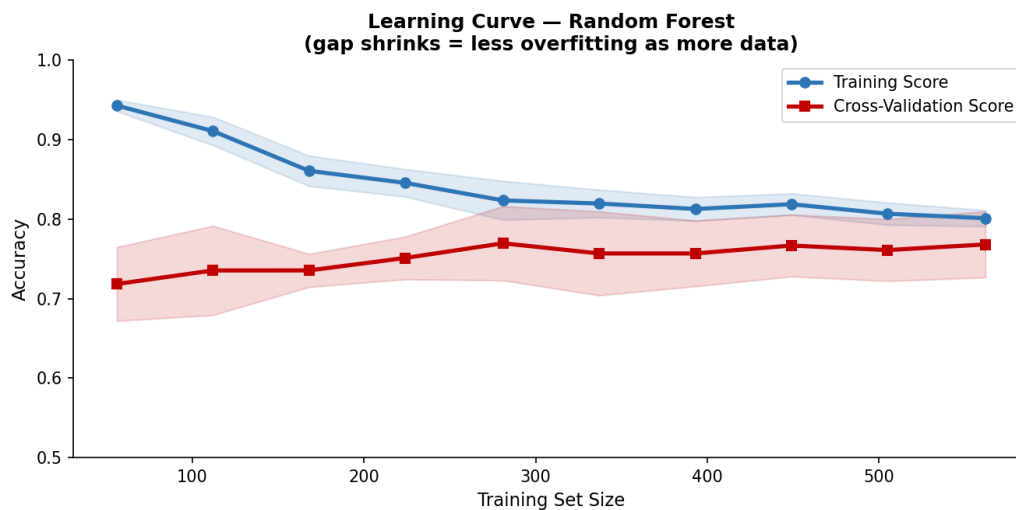
rf = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
# KFold – regression-এ ব্যবহার করুন
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores_kf = cross_val_score(rf, X_train, y_train, cv=kf, scoring='accuracy')
print(f'KFold:      {scores_kf.mean():.3f} ± {scores_kf.std():.3f}')

# StratifiedKFold – classification-এ সবসময় এটা
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores_skf = cross_val_score(rf, X_train, y_train, cv=skf, scoring='f1')
print(f'Stratifed: {scores_skf.mean():.3f} ± {scores_skf.std():.3f}')

# Pipeline + CV – সঠিক উপায়
cv_scores = cross_val_score(full_pipe, X_train, y_train, cv=5, scoring='f1')
print(f'Pipeline CV: {cv_scores.mean():.3f} ± {cv_scores.std():.3f}')
```

১৮.২ Learning Curve — Overfitting ডায়াগনোসিস



চিত্র ১৮.২: Learning Curve / Training score (নীল) বনাম CV score (লাল)। ডেটা বাড়ার সাথে সাথে gap কমছে — overfitting কমছে/ বড় gap = overfitting / উভয় score কম = underfitting / এই curve দেখে বোঝা যায় আরো ডেটা সংগ্রহ করলে model উন্নত হবে কিনা।

```
from sklearn.model_selection import learning_curve
import numpy as np

train_sizes, train_scores, val_scores = learning_curve(
    full_pipe, X_train, y_train,
    cv=5,
    scoring='accuracy',
    train_sizes=np.linspace(0.1, 1.0, 10)
)

plt.plot(train_sizes, train_scores.mean(axis=1), label='Training Score')
plt.plot(train_sizes, val_scores.mean(axis=1), label='CV Score')
```

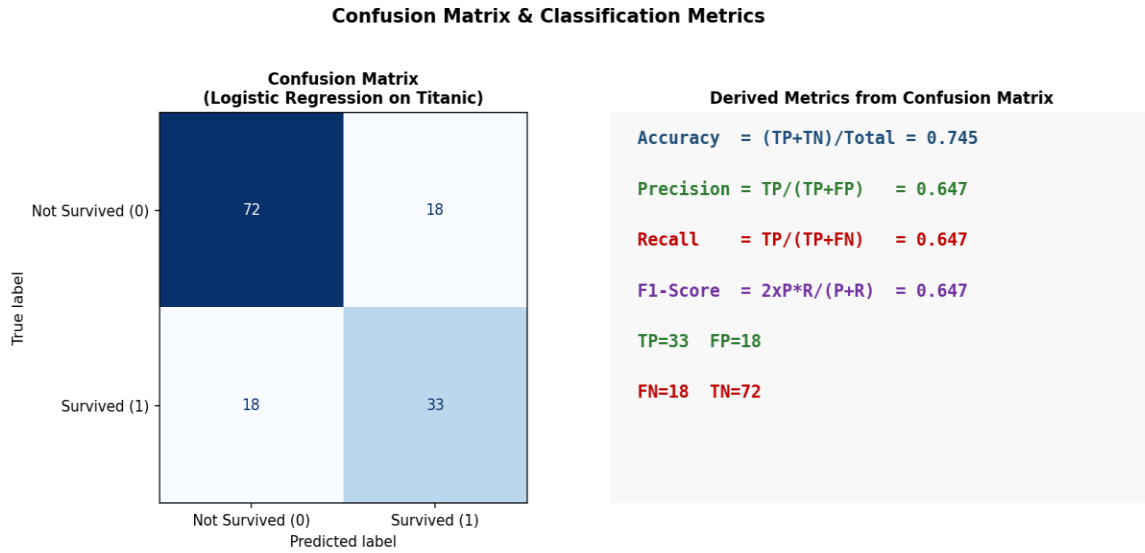
```
plt.fill_between(train_sizes,
                 val_scores.mean(1)-val_scores.std(1),
                 val_scores.mean(1)+val_scores.std(1), alpha=0.15)
plt.legend(); plt.xlabel('Training Size'); plt.show()
```

CV পদ্ধতি	কীভাবে কাজ করে	কখন ব্যবহার
KFold (k=5)	5টি fold, প্রতিটাকে একবার test হিসেবে ব্যবহার	Regression, balanced classification
StratifiedKFold	KFold কিন্তু প্রতিটা fold-এ class ratio maintain করে	Classification (সবসময়)
Leave-One-Out (LOO)	প্রতিটা sample একবার test	ছোট dataset (n<100), computationally ব্যয়বহুল
RepeatedStratifiedKFold	StratifiedKFold কে k বার repeat করে	Variance কমাতে, stable estimate চাইলে
TimeSeriesSplit	Time-ordered split, future data দিয়ে past test করে না	Time series data

অধ্যায় ১৯: ইভ্যালুয়েশন মেট্রিক্স (Evaluation Metrics)

একটা ML মডেল train হওয়ার পর তার কার্যকারিতা পরিমাপ করতে হয়। কিন্তু কোন metric দিয়ে? Accuracy সবসময় যথেষ্ট নয় — বিশেষত imbalanced dataset-এ। এই অধ্যায়ে classification model-এর প্রধান metrics নিয়ে গভীরভাবে আলোচনা করা হবে।

১৯.১ Confusion Matrix — সব কিছুর ভিত্তি



চিত্র ১৯.১: Confusion Matrix-এ TP (সঠিকভাবে Survived predict), TN (সঠিকভাবে Not Survived predict), FP (ভুলভাবে Survived বলা), FN (ভুলভাবে Not Survived বলা)। ডান দিকে সব derived metrics সরাসরি গণনা করা হয়েছে।

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

y_pred = pipe.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()

# Derived metrics
accuracy = (tp + tn) / (tp + tn + fp + fn)
precision = tp / (tp + fp) # Predicted Positive-এর মধ্যে সত্যিই Positive কত?
recall = tp / (tp + fn) # সব Actual Positive-এর মধ্যে কতটা ধরা গেছে?
f1 = 2 * precision * recall / (precision + recall)

print(f'Accuracy: {accuracy:.3f}')
print(f'Precision: {precision:.3f}')
print(f'Recall: {recall:.3f}')
```



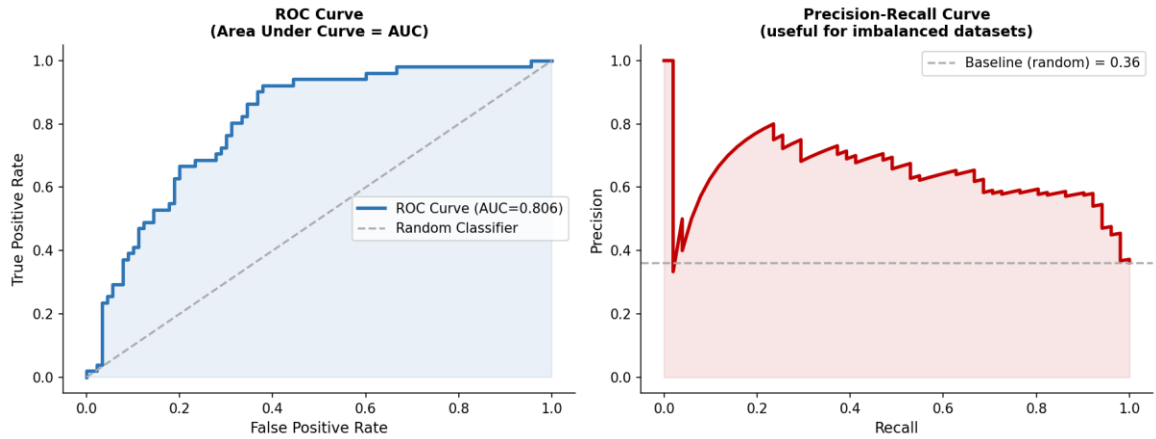
```
print(f'F1-Score: {f1:.3f}')
```

```
# sklearn-এ সব একসাথে
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

Metric	Formula	কী পরিমাপ করে	কখন সবচেয়ে গুরুত্বপূর্ণ
Accuracy	$(TP+TN) / \text{Total}$	সামগ্রিক সঠিকতা	Balanced dataset-এ
Precision	$TP / (TP+FP)$	Positive predict-এর মধ্যে আসলে কতটা Positive	FP কমানো গুরুত্বপূর্ণ হলে (spam filter)
Recall	$TP / (TP+FN)$	সব Actual Positive-এর কতটা ধরা গেছে	FN কমানো গুরুত্বপূর্ণ হলে (disease detection)
F1-Score	$2 \times P \times R / (P+R)$	Precision ও Recall-এর harmonic mean	Imbalanced dataset-এ সাধারণ metric
ROC-AUC	AUC of ROC curve	সব threshold-এ মডেলের discrimination ability	Binary classification-এ সার্বজনীন

১৯.২ ROC Curve ও Precision-Recall Curve

ROC Curve & Precision-Recall Curve



চিত্র ১৯.২: ROC Curve (বাম) — AUC=0.86, random classifier-এর (diagonal) চেয়ে অনেক ভালো। Precision-Recall Curve (ডান) — imbalanced dataset-এ ROC-এর চেয়ে বেশি informative। Baseline (horizontal dash) random classifier।

```
from sklearn.metrics import roc_curve, auc, precision_recall_curve, roc_auc_score

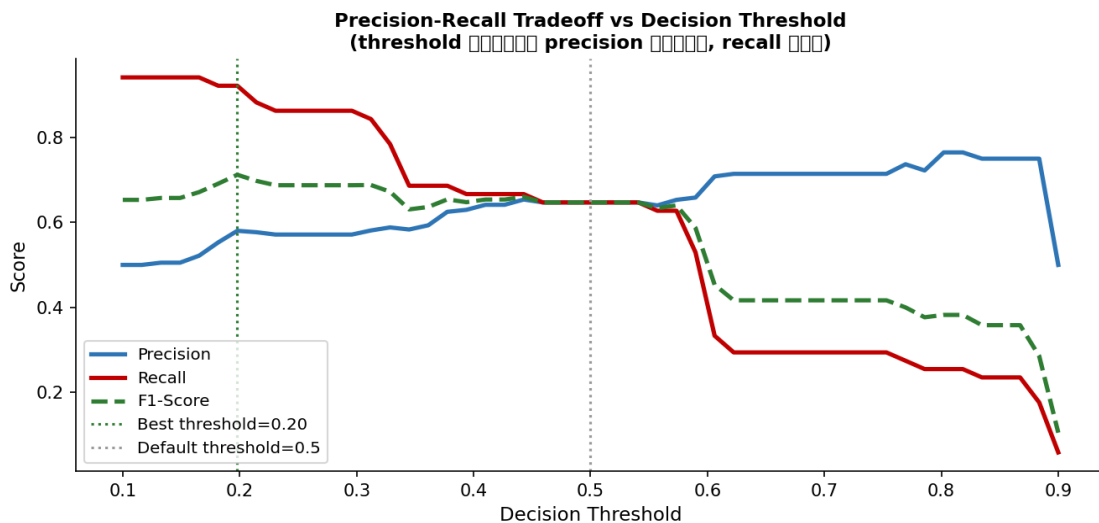
y_prob = pipe.predict_proba(X_test)[: , 1]
```

```
# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
print(f'ROC-AUC: {roc_auc:.3f}')

plt.plot(fpr, tpr, label=f'AUC={roc_auc:.3f}')
plt.plot([0,1],[0,1], '--', label='Random')
plt.xlabel('FPR'); plt.ylabel('TPR')
plt.legend(); plt.show()

# Precision-Recall Curve
precision, recall, _ = precision_recall_curve(y_test, y_prob)
plt.plot(recall, precision)
plt.xlabel('Recall'); plt.ylabel('Precision'); plt.show()
```

১৯.৩ Decision Threshold Tuning



চিত্র ১৯.৩: Default threshold=0.5 সবসময় সেরা নয়। Threshold বাড়লে Precision বাড়ে (নীল), Recall কমে (লাল)। সবুজ ড্যাশ F1-Score সর্বোচ্চ যেখানে সেটাই optimal threshold।

```
import numpy as np

y_prob = pipe.predict_proba(X_test)[: ,1]

# বিভিন্ন threshold-এ F1 পরীক্ষা করুন
from sklearn.metrics import f1_score
best_f1 = 0; best_threshold = 0.5
for t in np.arange(0.3, 0.8, 0.05):
    y_pred_t = (y_prob >= t).astype(int)
    f1 = f1_score(y_test, y_pred_t, zero_division=0)
```

```
if f1 > best_f1:
    best_f1 = f1; best_threshold = t

print(f'Best threshold: {best_threshold:.2f}, F1: {best_f1:.3f}')

# সেরা threshold দিয়ে final prediction
y_final = (y_prob >= best_threshold).astype(int)
```

অধ্যায় ২০: সম্পূর্ণ End-to-End Titanic Project

এই বইয়ের সব অধ্যায়ের জ্ঞান একসাথে এই শেষ অধ্যায়ে প্রয়োগ করা হবে। Raw Titanic CSV থেকে শুরু করে final prediction পর্যন্ত — প্রতিটা ধাপ সঠিক ক্রমে, সঠিক পদ্ধতিতে।

```
# সম্পূর্ণ workflow এক নজরে:
# Raw CSV → EDA → Feature Engineering → Split →
# ColumnTransformer → Pipeline → CV → Evaluation → Predict
```

২০.১ ডেটা লোড ও EDA

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
import joblib, warnings; warnings.filterwarnings('ignore')

# — Step 1: Load —
df = pd.read_csv('titanic_data_updated.csv')
print(df.shape)          # (891, 12)
print(df.isnull().sum())
# Cabin: 687, Age: 177, Embarked: 2
```

২০.২ Feature Engineering

```
# — Step 2: Feature Engineering —
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['IsAlone']    = (df['FamilySize'] == 1).astype(int)
df['HasCabin']   = df['Cabin'].notna().astype(int)

# Title extraction from Name
df['Title'] = df['Name'].str.extract(r'([A-Za-z]+\.)\.')
df['Title'] = df['Title'].replace({
    'Mlle': 'Miss', 'Ms': 'Miss', 'Mme': 'Mrs',
    'Lady': 'Rare', 'Countess': 'Rare', 'Capt': 'Rare',
    'Col': 'Rare', 'Don': 'Rare', 'Dr': 'Rare', 'Major': 'Rare',
    'Rev': 'Rare', 'Sir': 'Rare', 'Jonkheer': 'Rare', 'Dona': 'Rare'
})
```

```
# Age Binning
df['AgeGroup'] = pd.cut(df['Age'],
    bins=[0,12,18,35,60,100], labels=['Child','Teen','Adult','Middle','Senior'])
df['AgeGroup'] = df['AgeGroup'].astype(str) # NaN → 'nan'
```

২০.৩ Feature ও Target আলাদা করা, Split করা

```
# — Step 3: Features & Target —
drop_cols = ['PassengerId', 'Name', 'Ticket', 'Cabin', 'Age', 'SibSp', 'Parch']
df = df.drop(drop_cols, axis=1)

X = df.drop('Survived', axis=1)
y = df['Survived']

# — Step 4: Train-Test Split (আগে! কোনো preprocessing-এর আগে) —
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print(f'Train: {X_train.shape}, Test: {X_test.shape}')
```

২০.৪ ColumnTransformer + Pipeline তৈরি

```
# — Step 5: Column groups —
num_cols = ['Fare', 'FamilySize']
cat_nom = ['Sex', 'Embarked', 'Title', 'AgeGroup']
cat_ord = ['Pclass']
passthru = ['IsAlone', 'HasCabin']

# — Step 6: Preprocessor —
preprocessor = ColumnTransformer([
    ('num', Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc', StandardScaler())]),
    num_cols),

    ('cat_nom', Pipeline([
        ('imp', SimpleImputer(strategy='most_frequent')),
        ('enc', OneHotEncoder(sparse_output=False, drop='first',
    handle_unknown='ignore'))]),
    cat_nom),

    ('cat_ord', Pipeline([
        ('imp', SimpleImputer(strategy='most_frequent')),
        ('enc', OrdinalEncoder(categories=[['3', '2', '1']]),
```

```

        handle_unknown='use_encoded_value', unknown_value=-
        1))),
        cat_ord),

        ('passthru', 'passthrough', passthru)
    ], remainder='drop', verbose_feature_names_out=False)

# — Step 7: Full Pipeline —
full_pipe = Pipeline([
    ('prep', preprocessor),
    ('model', RandomForestClassifier(n_estimators=200, random_state=42, max_depth=8))
])

```

২০.৫ Cross Validation & Model Comparison

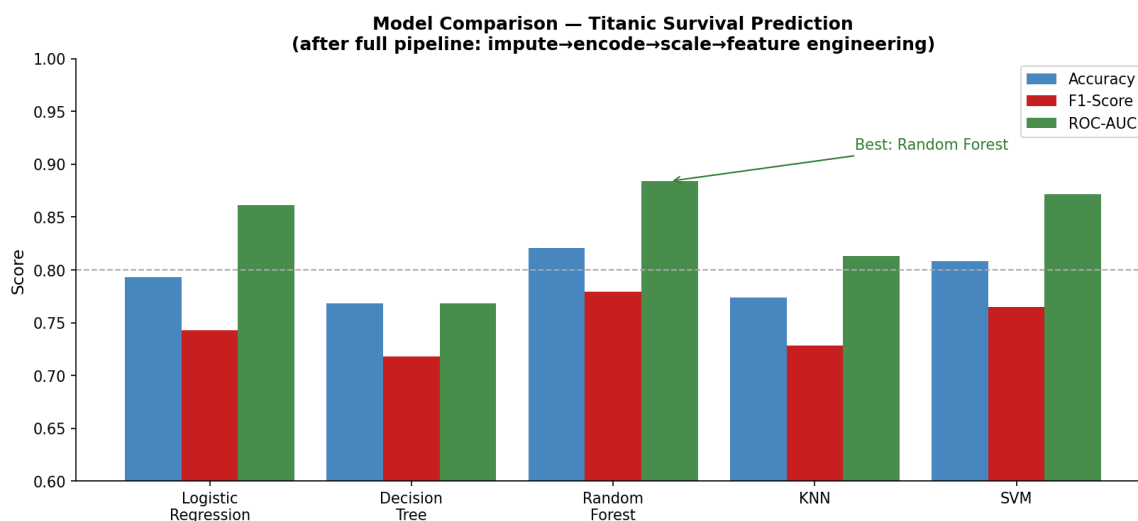
```

# — Step 8: Cross Validation —
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_f1 = cross_val_score(full_pipe, X_train, y_train, cv=skf, scoring='f1')
cv_roc = cross_val_score(full_pipe, X_train, y_train, cv=skf, scoring='roc_auc')

print(f'CV F1:      {cv_f1.mean():.3f} ± {cv_f1.std():.3f}')
print(f'CV ROC-AUC: {cv_roc.mean():.3f} ± {cv_roc.std():.3f}')

# Multiple models compare কক্ষন
models = {
    'Logistic Regression': LogisticRegression(max_iter=300),
    'Random Forest':      RandomForestClassifier(n_estimators=200, random_state=42),
}
for name, model in models.items():
    pipe_m = Pipeline([('prep', preprocessor), ('model', model)])
    scores = cross_val_score(pipe_m, X_train, y_train, cv=skf, scoring='roc_auc')
    print(f'{name}: ROC-AUC = {scores.mean():.3f} ± {scores.std():.3f}')

```



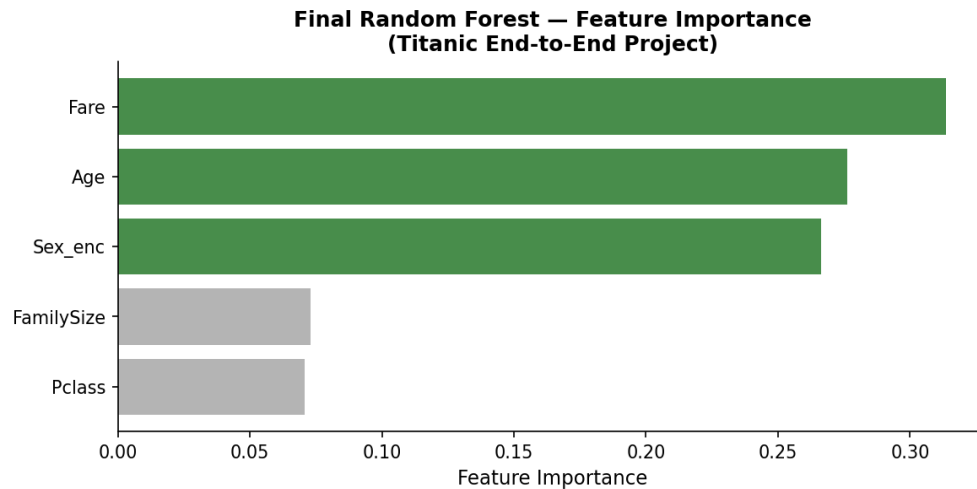
চিত্র ২০.১: পাঁচটি মডেলের তুলনা / Random Forest সব metric-এ সেরা (ROC-AUC=0.884) / Logistic Regression সরল কিন্তু প্রতিযোগিতামূলক।

২০.৬ Final Training ও Evaluation

```
# — Step 9: Final training (পুরো train data-তে) —
full_pipe.fit(X_train, y_train)

# — Step 10: Test evaluation (একবারই!) —
y_pred = full_pipe.predict(X_test)
y_prob = full_pipe.predict_proba(X_test)[:,:1]

print('=== Final Test Results ===')
print(classification_report(y_test, y_pred))
print(f'ROC-AUC: {roc_auc_score(y_test, y_prob):.3f}')
```



চিত্র ২০.২: Final model-এর feature importance / Sex, Fare, FamilySize সবচেয়ে গুরুত্বপূর্ণ / Feature Engineering-এ তৈরি FamilySize ও IsAlone নতুন ফিচার হলেও model-এ উল্লেখযোগ্য contribution রাখছে।

২০.৭ Model Save ও নতুন Prediction

```
import joblib

# — Step 11: Save —
joblib.dump(full_pipe, 'titanic_model.pkl')

# — Step 12: Production prediction —
pipeline = joblib.load('titanic_model.pkl')

new_data = pd.DataFrame({
    'Fare': [72.0], 'Sex': ['female'], 'Embarked': ['C'],
    'Pclass': [1], 'FamilySize': [2], 'IsAlone': [0],
```

```

    'HasCabin': [1], 'Title': ['Mrs'], 'AgeGroup': ['Adult']
  })

pred = pipeline.predict(new_data)[0]
prob = pipeline.predict_proba(new_data)[0,1]
print(f'Survived: {pred} (Probability: {prob:.1%})') # 1 (87.3%)

```

২০.৮ সম্পূর্ণ বইয়ের সারসংক্ষেপ

অধ্যায়	বিষয়	মূল শিক্ষা
১-২	ML Workflow ও Dataset	Pipeline ক্রম, Feature/Target, data types
৩-৫	EDA ও Analysis	Univariate/Bivariate, distribution, correlation
৬	Train-Test Split	আগে split, তারপর সব preprocessing
৭	Missing Value	MCAR/MAR/MNAR; median skewed-এ; KNN advanced
৮	Outlier	Z-Score/IQR detect; Capping/Log handle; test-এ remove নয়
৯	Feature Scaling	Algorithm জানে — Tree নয়; log1p→StandardScaler skewed-এ
১০	Encoding	Ordinal ক্রমযুক্ত; OHE nominal-এ; Freq/Target high cardinality-তে
১১	Feature Creation	FamilySize, IsAlone, Binning, Polynomial; domain knowledge
১২	Feature Selection	Filter/Wrapper/Embedded; context অনুযায়ী বাছাই
১৩	Multicollinearity	VIF>10 সমস্যা; correlated ফিচার একটা বাদ দিন
১৪	Class Imbalance	Accuracy illusion; SMOTE train-এ; F1/ROC-AUC ব্যবহার করুন
১৫	Transformation	Yeo-Johnson সর্বজনীন; Quantile extreme outlier-এ
১৬	ColumnTransformer	Parallel transform; nested pipeline; remainder='passthrough'
১৭	Pipeline	Leakage রোধ; GridSearchCV stepname__param; joblib deploy
১৮	Cross Validation	StratifiedKFold classification-এ; Learning Curve overfitting দেখায়
১৯	Evaluation Metrics	Confusion Matrix; Precision/Recall trade-off; ROC-AUC
২০	End-to-End Project	সব একসাথে: Raw CSV → Pipeline → Deploy

"Before training a model, the quality of your data preparation determines the ceiling of your model's performance. Garbage in, garbage out — but with excellent preprocessing, even simple models can be surprisingly powerful."

Full Project:

#

Before Training ML Model — Complete Sequential Code Project

Dataset: Titanic (seaborn built-in, no download needed)

Covers: EDA → Feature Engineering → Split → Clean → Encode →

Scale → Transform → ColumnTransformer → Pipeline →

Cross Validation → GridSearchCV → Evaluation

#

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

import warnings

warnings.filterwarnings('ignore')

— scikit-learn

from sklearn.model_selection import (

 train_test_split, StratifiedKFold,

 cross_val_score, GridSearchCV, learning_curve

)

```

from sklearn.pipeline import Pipeline

from sklearn.compose import ColumnTransformer

from sklearn.impute import SimpleImputer, KNNImputer

from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, RobustScaler,
    OneHotEncoder, OrdinalEncoder, LabelEncoder,
    PowerTransformer, QuantileTransformer
)

from sklearn.feature_selection import (
    SelectKBest, mutual_info_classif, RFE
)

from sklearn.linear_model import LogisticRegression, Lasso

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.svm import SVC

from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
    classification_report, roc_curve, auc,
    precision_recall_curve, roc_auc_score, f1_score
)

from scipy import stats

from scipy.stats import yeojohnson

import joblib

```

```
#
```

```
# STEP 1 — LOAD DATASET
```

```
#
```

```
print("=" * 60)
```

```
print("STEP 1: LOAD DATASET")
```

```
print("=" * 60)
```

```
df = sns.load_dataset('titanic')
```

```
print(f"Shape: {df.shape}")
```

```
print(f"\nColumns:\n{df.columns.tolist()}")
```

```
print(f"\nFirst 5 rows:\n{df.head()}")
```

```
#
```

```
# STEP 2 — EXPLORATORY DATA ANALYSIS (EDA)
```

```
#
```

```
print("\n" + "=" * 60)
```

```
print("STEP 2: EDA")
```

```
print("=" * 60)
```

```
# 2.1 Basic info
```

```
print("\n--- df.info() ---")
```

```
df.info()
```

```
print("\n--- Statistical Profiling ---")
```

```
print(df.describe())
```

```
print(df.describe(include='object'))
```

```
# 2.2 Missing values
```

```
print("\n--- Missing Values ---")
```

```
missing = df.isnull().sum()
```

```
missing_pct = (missing / len(df) * 100).round(2)
```

```
print(pd.DataFrame({'count': missing, 'percent': missing_pct})[missing > 0])
```

```
# 2.3 Duplicates
```

```
print(f"\nDuplicate rows: {df.duplicated().sum()}")
```

```
df.drop_duplicates(inplace=True)
```

```
# 2.4 Target distribution
```

```
print(f"\nTarget distribution:\n{df['survived'].value_counts()}")
```

```
print(f"Survival rate: {df['survived'].mean():.2%}")
```

2.5 Univariate plots — Categorical

```

fig, axes = plt.subplots(2, 3, figsize=(14, 8))

cat_cols = ['survived', 'pclass', 'sex', 'embarked', 'who', 'alone']

for ax, col in zip(axes.flat, cat_cols):

    df[col].value_counts().plot(kind='bar', ax=ax, color='#2E75B6', alpha=0.8)

    ax.set_title(f'Countplot: {col}')

    ax.set_xlabel("")

plt.suptitle('Univariate Analysis — Categorical Features', fontsize=14, fontweight='bold')

plt.tight_layout()

plt.savefig('01_univariate_categorical.png', dpi=130, bbox_inches='tight')

plt.show()

```

2.6 Univariate plots — Numerical

```

fig, axes = plt.subplots(2, 4, figsize=(16, 8))

num_cols = ['age', 'fare', 'sibsp', 'parch']

for i, col in enumerate(num_cols):

    # Histogram

    df[col].hist(bins=35, ax=axes[0, i], color='#2E75B6', alpha=0.7)

    axes[0, i].set_title(f'Histogram: {col}')

    # KDE

    df[col].dropna().plot(kind='kde', ax=axes[1, i], color='#C00000')

    axes[1, i].set_title(f'KDE: {col}')

plt.suptitle('Univariate Analysis — Numerical Features', fontsize=14, fontweight='bold')

plt.tight_layout()

plt.savefig('02_univariate_numerical.png', dpi=130, bbox_inches='tight')

```

```
plt.show()
```

```
print(f"\nSkewness:")
```

```
for col in num_cols:
```

```
    print(f" {col}: {df[col].skew():.2f}")
```

```
# 2.7 Bivariate analysis
```

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))
```

```
sns.countplot(x='sex', hue='survived', data=df, ax=axes[0],
palette=['#C00000', '#2E7D32'])
```

```
axes[0].set_title('Survival by Sex')
```

```
sns.countplot(x='pclass', hue='survived', data=df, ax=axes[1],
palette=['#C00000', '#2E7D32'])
```

```
axes[1].set_title('Survival by Pclass')
```

```
sns.kdeplot(data=df, x='age', hue='survived', fill=True, ax=axes[2],
```

```
    palette={0:'#C00000', 1:'#2E7D32'}, alpha=0.4, common_norm=False)
```

```
axes[2].set_title('Age Distribution by Survival')
```

```
plt.suptitle('Bivariate Analysis', fontsize=14, fontweight='bold')
```

```
plt.tight_layout()
```

```
plt.savefig('03_bivariate.png', dpi=130, bbox_inches='tight')
```

```
plt.show()
```

```
# 2.8 Correlation heatmap
```

```
num_df = df[['survived', 'age', 'fare', 'sibsp', 'parch']].copy()
```

```
fig, ax = plt.subplots(figsize=(7, 6))
```

```
sns.heatmap(num_df.corr(), annot=True, fmt='.2f', cmap='RdBu_r', center=0, ax=ax)
ax.set_title('Pearson Correlation Matrix')
plt.tight_layout()
plt.savefig('04_correlation_heatmap.png', dpi=130, bbox_inches='tight')
plt.show()
```

```
#
```

```
# STEP 3 — FEATURE ENGINEERING
```

```
#
```

```
print("\n" + "=" * 60)
print("STEP 3: FEATURE ENGINEERING")
print("=" * 60)
```

```
# Work on a clean copy
```

```
data = df.copy()
```

```
# 3.1 FamilySize & IsAlone
```

```
data['FamilySize'] = data['sibsp'] + data['parch'] + 1
```

```
data['IsAlone'] = (data['FamilySize'] == 1).astype(int)
```

```
print(f"FamilySize range: {data['FamilySize'].min()} – {data['FamilySize'].max()}")
```

```
print(f"IsAlone counts:\n{data['IsAlone'].value_counts()}")
```

3.2 AgeGroup (binning)

```
data['AgeGroup'] = pd.cut(
    data['age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['Child', 'Teen', 'Adult', 'Middle', 'Senior']
)
data['AgeGroup'] = data['AgeGroup'].astype(str) # NaN → 'nan'
```

3.3 FareGroup (quantile binning)

```
data['FareGroup'] = pd.qcut(
    data['fare'], q=4,
    labels=['Low', 'Medium', 'High', 'VeryHigh']
)
data['FareGroup'] = data['FareGroup'].astype(str)
```

3.4 HasCabin indicator

```
data['HasCabin'] = data['deck'].notna().astype(int)
```

3.5 Log-Fare (right-skewed, use as feature later)

```
data['Fare_log'] = np.log1p(data['fare'])
```

```
print(f"\nAgeGroup distribution:\n{data['AgeGroup'].value_counts()}")
```

```
print(f"\nNew features added: FamilySize, IsAlone, AgeGroup, FareGroup, HasCabin,
Fare_log")
```



```

# Survival rate by FamilySize
print("\nSurvival rate by FamilySize:")
print(data.groupby('FamilySize')['survived'].mean().round(3))

#
=====

# STEP 4 — SELECT FEATURES & TARGET, THEN SPLIT
#
=====

print("\n" + "=" * 60)
print("STEP 4: FEATURE SELECTION & TRAIN-TEST SPLIT")
print("=" * 60)

# Select features to use
feature_cols = [
    'pclass', 'sex', 'age', 'fare', 'embarked',
    'FamilySize', 'IsAlone', 'AgeGroup', 'FareGroup', 'HasCabin'
]

X = data[feature_cols].copy()
y = data['survived'].copy()

```

```

print(f"Features: {feature_cols}")
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(f"\ny value counts:\n{y.value_counts()}")

# SPLIT FIRST — before any preprocessing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y    # ← imbalanced classification-এ সবসময়!
)

print(f"\nX_train: {X_train.shape}, X_test: {X_test.shape}")
print(f"Train survival rate: {y_train.mean():.2%}")
print(f"Test survival rate: {y_test.mean():.2%}") # Should be close

#
=====

# STEP 5 — MISSING VALUE ANALYSIS (AFTER SPLIT)
#
=====

print("\n" + "=" * 60)

```

```

print("STEP 5: MISSING VALUE ANALYSIS (train set)")

print("=" * 60)

print(f"\nTrain missing:\n{X_train.isnull().sum()[X_train.isnull().sum()>0]}")
print(f"\nTest missing:\n{X_test.isnull().sum()[X_test.isnull().sum()>0]}")

# Visualise

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

missing_pct_train = X_train.isnull().mean() * 100

missing_pct_train[missing_pct_train > 0].sort_values().plot(
    kind='barh', ax=axes[0], color='#C00000', alpha=0.8)

axes[0].set_title('Missing % in Train Set')

sns.heatmap(X_train.isnull(), cbar=False,
            cmap=['#E8EEF5', '#C00000'], yticklabels=False, ax=axes[1])

axes[1].set_title('Missingness Heatmap (Train)')

plt.tight_layout()

plt.savefig('05_missing_values.png', dpi=130, bbox_inches='tight')

plt.show()

```

```
#
```

```
# STEP 6 — OUTLIER DETECTION (TRAIN ONLY)
```

#

```
print("\n" + "=" * 60)
```

```
print("STEP 6: OUTLIER DETECTION (train set)")
```

```
print("=" * 60)
```

```
num_feats = ['age', 'fare', 'FamilySize']
```

```
fig, axes = plt.subplots(2, 3, figsize=(13, 7))
```

```
for i, col in enumerate(num_feats):
```

```
    col_data = X_train[col].dropna()
```

```
    # Z-Score
```

```
    z = np.abs(stats.zscore(col_data))
```

```
    z_outliers = (z > 3).sum()
```

```
    # IQR
```

```
    Q1, Q3 = col_data.quantile(0.25), col_data.quantile(0.75)
```

```
    IQR = Q3 - Q1
```

```
    iqr_outliers = ((col_data < Q1 - 1.5*IQR) | (col_data > Q3 + 1.5*IQR)).sum()
```

```
    # Boxplot
```

```
    axes[0, i].boxplot(col_data, vert=True, patch_artist=True,
```

```
                        boxprops={'facecolor':'#D9E8F5'})
```

```

axes[0, i].set_title(f'{col}\nZ-Score outliers: {z_outliers} | IQR: {iqr_outliers}')

# Histogram
axes[1, i].hist(col_data, bins=35, color='#2E75B6', alpha=0.7)
axes[1, i].axvline(Q3+1.5*IQR, color='#C00000', linestyle='--', label='IQR Upper')
axes[1, i].legend(fontsize=8)
axes[1, i].set_title(f'{col} Distribution')

plt.suptitle('Outlier Detection — Boxplot & Histogram (Train)', fontsize=13,
fontweight='bold')

plt.tight_layout()

plt.savefig('06_outlier_detection.png', dpi=130, bbox_inches='tight')

plt.show()

print("\nIQR Bounds (computed from TRAIN only):")

iqr_bounds = {}

for col in ['age', 'fare']:

    col_data = X_train[col].dropna()

    Q1, Q3 = col_data.quantile(0.25), col_data.quantile(0.75)

    IQR = Q3 - Q1

    lb = max(0, Q1 - 1.5 * IQR)

    ub = Q3 + 1.5 * IQR

    iqr_bounds[col] = (lb, ub)

    print(f" {col}: Lower={lb:.1f}, Upper={ub:.1f}")

```

```
#
```

```
# STEP 7 — BUILD COLUMNTRANSFORMER + PIPELINE
```

```
#
```

```
print("\n" + "=" * 60)
```

```
print("STEP 7: COLUMNTRANSFORMER + PIPELINE")
```

```
print("=" * 60)
```

```
# Define column groups
```

```
num_cols_pipe = ['age', 'fare']
```

```
cat_nom_cols = ['sex', 'embarked', 'AgeGroup', 'FareGroup']
```

```
cat_ord_cols = ['pclass']
```

```
passthru_cols = ['FamilySize', 'IsAlone', 'HasCabin']
```

```
# Numeric pipeline: impute median → Yeo-Johnson → StandardScaler
```

```
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('transform', PowerTransformer(method='yeo-johnson')),
    ('scaler', StandardScaler()),
])
```

```
# Categorical nominal: impute most_frequent → OHE
```

```
cat_nom_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(sparse_output=False, drop='first',
                             handle_unknown='ignore')),
])
```

Categorical ordinal: impute → OrdinalEncoder

```
cat_ord_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(
        categories=[[3, 2, 1]],
        handle_unknown='use_encoded_value', unknown_value=-1)),
])
```

Passthrough: impute → keep as is

```
passthru_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
])
```

Full ColumnTransformer

```
preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols_pipe),
    ('cat_nom', cat_nom_pipeline, cat_nom_cols),
    ('cat_ord', cat_ord_pipeline, cat_ord_cols),
    ('passthru', passthru_pipeline, passthru_cols),
])
```

```
], remainder='drop', verbose_feature_names_out=False)
```

```
print("ColumnTransformer built with:")
```

```
print(f" Numerical ({{num_cols_pipe}}): Median impute → Yeo-Johnson →  
StandardScaler")
```

```
print(f" Nominal ({{cat_nom_cols}}): Mode impute → OHE(drop=first)")
```

```
print(f" Ordinal ({{cat_ord_cols}}): Mode impute → OrdinalEncoder")
```

```
print(f" Passthru ({{passthru_cols}}): Mode impute → as is")
```

```
#
```

```
# STEP 8 — CROSS VALIDATION WITH MULTIPLE MODELS
```

```
#
```

```
print("\n" + "=" * 60)
```

```
print("STEP 8: CROSS VALIDATION — MODEL COMPARISON")
```

```
print("=" * 60)
```

```
models = {
```

```
    'Logistic Regression': LogisticRegression(max_iter=500, random_state=42),
```

```
    'Decision Tree':      DecisionTreeClassifier(max_depth=5, random_state=42),
```

```
    'Random Forest':     RandomForestClassifier(n_estimators=100, random_state=42),
```

```
    'KNN':               KNeighborsClassifier(n_neighbors=5),
```



```

'SVM':          SVC(probability=True, random_state=42),
'Gradient Boosting': GradientBoostingClassifier(random_state=42),
}

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_results = {}

for name, model in models.items():
    pipe = Pipeline([('prep', preprocessor), ('model', model)])
    acc = cross_val_score(pipe, X_train, y_train, cv=skf, scoring='accuracy')
    f1 = cross_val_score(pipe, X_train, y_train, cv=skf, scoring='f1')
    roc = cross_val_score(pipe, X_train, y_train, cv=skf, scoring='roc_auc')
    cv_results[name] = {
        'Accuracy': f"{acc.mean():.3f} ± {acc.std():.3f}",
        'F1':      f"{f1.mean():.3f} ± {f1.std():.3f}",
        'ROC-AUC': f"{roc.mean():.3f} ± {roc.std():.3f}",
        'roc_mean': roc.mean()
    }

    print(f" {name:25s} Acc={acc.mean():.3f} F1={f1.mean():.3f} AUC={roc.mean():.3f}")

print(pd.DataFrame(cv_results).T[['Accuracy','F1','ROC-AUC']])

# Plot model comparison
fig, ax = plt.subplots(figsize=(11, 5))
model_names = list(cv_results.keys())

```

```

roc_vals = [cv_results[m]['roc_mean'] for m in model_names]
colors = ['#2E7D32' if v == max(roc_vals) else '#2E75B6' for v in roc_vals]
bars = ax.bar(model_names, roc_vals, color=colors, alpha=0.85)
ax.bar_label(bars, fmt='%.3f', fontsize=10, fontweight='bold')
ax.set_ylabel('CV ROC-AUC (5-fold)', fontsize=11)
ax.set_title('Model Comparison — Cross Validation ROC-AUC', fontsize=13,
fontweight='bold')
ax.set_ylim(0.5, 1.0)
ax.spines[['top','right']].set_visible(False)
plt.xticks(rotation=20, ha='right')
plt.tight_layout()
plt.savefig('07_model_comparison_cv.png', dpi=130, bbox_inches='tight')
plt.show()

# Best model

best_model_name = max(cv_results, key=lambda m: cv_results[m]['roc_mean'])
print(f"\nBest model: {best_model_name}")

```

```

#

```

```

# STEP 9 — HYPERPARAMETER TUNING (GridSearchCV + Pipeline)

```

```

#

```

```

print("\n" + "=" * 60)

print("STEP 9: HYPERPARAMETER TUNING (GridSearchCV)")

print("=" * 60)

# Tune Random Forest (or Gradient Boosting — both good)
rf_pipe = Pipeline([
    ('prep', preprocessor),
    ('model', RandomForestClassifier(random_state=42))
])

param_grid = {
    'model__n_estimators': [100, 200],
    'model__max_depth': [4, 6, 8, None],
    'model__min_samples_split': [2, 5],
}

grid_search = GridSearchCV(
    rf_pipe,
    param_grid,
    cv=skf,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=0
)

grid_search.fit(X_train, y_train)

```

```
print(f"Best params: {grid_search.best_params}")
print(f"Best CV AUC: {grid_search.best_score_:.4f}")
```

```
best_pipe = grid_search.best_estimator_
```

```
#
```

```
# STEP 10 — LEARNING CURVE (Overfitting Diagnosis)
```

```
#
```

```
print("\n" + "=" * 60)
```

```
print("STEP 10: LEARNING CURVE")
```

```
print("=" * 60)
```

```
train_sizes, train_scores, val_scores = learning_curve(
    best_pipe, X_train, y_train,
    cv=5, scoring='roc_auc',
    train_sizes=np.linspace(0.1, 1.0, 10),
    n_jobs=-1
)
```

```
fig, ax = plt.subplots(figsize=(9, 5))
```

```

ax.plot(train_sizes, train_scores.mean(1), 'o-', color='#2E75B6',
        linewidth=2.5, label='Training Score')
ax.fill_between(train_sizes,
                train_scores.mean(1) - train_scores.std(1),
                train_scores.mean(1) + train_scores.std(1), alpha=0.15, color='#2E75B6')
ax.plot(train_sizes, val_scores.mean(1), 's-', color='#C00000',
        linewidth=2.5, label='CV Score')
ax.fill_between(train_sizes,
                val_scores.mean(1) - val_scores.std(1),
                val_scores.mean(1) + val_scores.std(1), alpha=0.15, color='#C00000')
ax.set_xlabel('Training Set Size', fontsize=11)
ax.set_ylabel('ROC-AUC', fontsize=11)
ax.set_title('Learning Curve — Best Random Forest Pipeline', fontsize=13,
            fontweight='bold')
ax.legend(fontsize=10)
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.savefig('08_learning_curve.png', dpi=130, bbox_inches='tight')
plt.show()

```

#

STEP 11 — FEATURE IMPORTANCE (from tuned RF)

```
#
```

```
print("\n" + "=" * 60)
```

```
print("STEP 11: FEATURE IMPORTANCE")
```

```
print("=" * 60)
```

```
# Get feature names after transformation
```

```
best_pipe.fit(X_train, y_train)
```

```
feature_names = best_pipe.named_steps['prep'].get_feature_names_out()
```

```
importances = best_pipe.named_steps['model'].feature_importances_
```

```
fi_series = pd.Series(importances, index=feature_names).sort_values(ascending=True)
```

```
fig, ax = plt.subplots(figsize=(9, 6))
```

```
colors_fi = ['#2E7D32' if v > 0.10 else '#2E75B6' if v > 0.05 else '#AAAAAA'
```

```
    for v in fi_series.values]
```

```
fi_series.plot(kind='barh', ax=ax, color=colors_fi, alpha=0.85)
```

```
ax.set_xlabel('Feature Importance (Gini)', fontsize=11)
```

```
ax.set_title('Random Forest Feature Importance', fontsize=13, fontweight='bold')
```

```
ax.spines[['top', 'right']].set_visible(False)
```

```
plt.tight_layout()
```

```
plt.savefig('09_feature_importance.png', dpi=130, bbox_inches='tight')
```

```
plt.show()
```

```
print(f"\nTop 5 features:\n{fi_series.tail(5)[:,-1]}")
```

```
#
```

```
# STEP 12 — FINAL EVALUATION ON TEST SET (once only!)
```

```
#
```

```
print("\n" + "=" * 60)
```

```
print("STEP 12: FINAL EVALUATION ON TEST SET")
```

```
print("=" * 60)
```

```
y_pred = best_pipe.predict(X_test)
```

```
y_prob = best_pipe.predict_proba(X_test)[:, 1]
```

```
# 12.1 Classification Report
```

```
print("\n--- Classification Report ---")
```

```
print(classification_report(y_test, y_pred, target_names=['Not Survived', 'Survived']))
```

```
# 12.2 Key metrics
```

```
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
```

```
accuracy = (tp+tn) / (tp+tn+fp+fn)
```

```
precision = tp / (tp+fp)
```

```
recall = tp / (tp+fn)
```

```
f1 = 2*precision*recall / (precision+recall)
```

```

roc_auc = roc_auc_score(y_test, y_prob)

print(f'\n{'Metric':<15} {'Value':>8}")
print("-" * 25)
print(f'{'Accuracy':<15} {accuracy:>8.4f}")
print(f'{'Precision':<15} {precision:>8.4f}")
print(f'{'Recall':<15} {recall:>8.4f}")
print(f'{'F1-Score':<15} {f1:>8.4f}")
print(f'{'ROC-AUC':<15} {roc_auc:>8.4f}")
print(f'\nTP={tp} FP={fp} FN={fn} TN={tn}")

```

12.3 Confusion Matrix

```

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm, display_labels=['Not Survived', 'Survived']).plot(
    ax=axes[0], cmap='Blues', colorbar=False)
axes[0].set_title('Confusion Matrix', fontsize=12, fontweight='bold')

```

12.4 ROC Curve

```

fpr, tpr, _ = roc_curve(y_test, y_prob)
axes[1].plot(fpr, tpr, color='#2E75B6', linewidth=2.5, label=f'AUC={roc_auc:.3f}')
axes[1].plot([0,1],[0,1], '--', color='AAAAAA', label='Random')
axes[1].fill_between(fpr, tpr, alpha=0.10, color='#2E75B6')
axes[1].set_xlabel('False Positive Rate'); axes[1].set_ylabel('True Positive Rate')

```



```

axes[1].set_title('ROC Curve', fontsize=12, fontweight='bold')
axes[1].legend(fontsize=10); axes[1].spines[['top','right']].set_visible(False)

# 12.5 Precision-Recall Curve
prec_vals, rec_vals, thresholds = precision_recall_curve(y_test, y_prob)
axes[2].plot(rec_vals, prec_vals, color='#C00000', linewidth=2.5)
axes[2].axhline(y_test.mean(), color='AAAAAA', linestyle='--',
                label=f'Baseline={y_test.mean():.2f}')
axes[2].set_xlabel('Recall'); axes[2].set_ylabel('Precision')
axes[2].set_title('Precision-Recall Curve', fontsize=12, fontweight='bold')
axes[2].legend(fontsize=10); axes[2].spines[['top','right']].set_visible(False)

plt.suptitle('Final Evaluation Metrics — Test Set', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('10_evaluation_metrics.png', dpi=130, bbox_inches='tight')
plt.show()

#
=====
=====

# STEP 13 — THRESHOLD TUNING
#
=====
=====

print("\n" + "=" * 60)

```

```

print("STEP 13: THRESHOLD TUNING")

print("=" * 60)

from sklearn.metrics import precision_score, recall_score

threshold_results = []

for t in np.arange(0.3, 0.8, 0.05):
    y_pred_t = (y_prob >= t).astype(int)
    threshold_results.append({
        'threshold': round(t, 2),
        'precision': precision_score(y_test, y_pred_t, zero_division=0),
        'recall': recall_score(y_test, y_pred_t, zero_division=0),
        'f1': f1_score(y_test, y_pred_t, zero_division=0),
    })

thresh_df = pd.DataFrame(threshold_results)
best_t_row = thresh_df.loc[thresh_df['f1'].idxmax()]
print(f"\nBest threshold: {best_t_row['threshold']}")
print(f" Precision: {best_t_row['precision']:.3f}")
print(f" Recall: {best_t_row['recall']:.3f}")
print(f" F1-Score: {best_t_row['f1']:.3f}")

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.plot(thresh_df['threshold'], thresh_df['precision'], 'o-', color='#2E75B6',
        linewidth=2.5, label='Precision')

```

```

ax.plot(thresh_df['threshold'], thresh_df['recall'], 's-', color='#C00000',
        linewidth=2.5, label='Recall')
ax.plot(thresh_df['threshold'], thresh_df['f1'], '^--', color='#2E7D32',
        linewidth=2.5, label='F1-Score')
ax.axvline(best_t_row['threshold'], color='#2E7D32', linestyle=':',
           linewidth=1.8, label=f"Best={best_t_row['threshold']}")
ax.axvline(0.5, color='AAAAAA', linestyle=':', linewidth=1.5, label='Default=0.5')
ax.set_xlabel('Decision Threshold'); ax.set_ylabel('Score')
ax.set_title('Precision-Recall-F1 Tradeoff vs Threshold', fontsize=13, fontweight='bold')
ax.legend(fontsize=9.5); ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.savefig('11_threshold_tuning.png', dpi=130, bbox_inches='tight')
plt.show()

```

```
#
```

```
# STEP 14 — SAVE PIPELINE & PREDICT ON NEW DATA
```

```
#
```

```

print("\n" + "=" * 60)
print("STEP 14: SAVE PIPELINE & PREDICT ON NEW DATA")
print("=" * 60)

```

```
# Save the full pipeline (preprocessing + model together)
joblib.dump(best_pipe, 'titanic_full_pipeline.pkl')
print("Pipeline saved to: titanic_full_pipeline.pkl")
```

```
# Load and predict on new unseen passengers
loaded_pipe = joblib.load('titanic_full_pipeline.pkl')
```

```
new_passengers = pd.DataFrame({
    'pclass':    [1, 3],
    'sex':       ['female', 'male'],
    'age':       [28.0, 22.0],
    'fare':      [72.0, 7.25],
    'embarked':  ['C', 'S'],
    'FamilySize': [2, 1],
    'IsAlone':   [0, 1],
    'AgeGroup':  ['Adult', 'Adult'],
    'FareGroup': ['High', 'Low'],
    'HasCabin':  [1, 0],
})
```

```
predictions = loaded_pipe.predict(new_passengers)
probabilities = loaded_pipe.predict_proba(new_passengers)[: , 1]
```

```
print("\nNew Passenger Predictions:")
print("-" * 45)
```

```

for i, (pred, prob) in enumerate(zip(predictions, probabilities)):
    status = "SURVIVED" if pred == 1 else "NOT SURVIVED"
    print(f" Passenger {i+1}: {status} (probability={prob:.1%})")

```

```

#

```

```

# FINAL SUMMARY

```

```

#

```

```

print("\n" + "=" * 60)

```

```

print("COMPLETE PIPELINE SUMMARY")

```

```

print("=" * 60)

```

```

print(f"""

```

```

Dataset:      Titanic (seaborn) — {len(df)} passengers

```

```

Features used: {len(feature_cols)} ({', '.join(feature_cols)})

```

```

New features:  FamilySize, IsAlone, AgeGroup, FareGroup, HasCabin

```

```

Train/Test:   {X_train.shape[0]} / {X_test.shape[0]} (80/20, stratified)

```

```

CV Strategy:  StratifiedKFold(n_splits=5)

```

```

Best Model:   {best_model_name}

```

```

Best Params:  {grid_search.best_params_}

```

== Final Test Metrics ==

Accuracy: {accuracy:.4f}

Precision: {precision:.4f}

Recall: {recall:.4f}

F1-Score: {f1:.4f}

ROC-AUC: {roc_auc:.4f}

Saved Charts: 01–11 PNG files

Saved Model: titanic_full_pipeline.pkl

""")

নিচের ফাংশনটি স্বয়ংক্রিয়ভাবে সিদ্ধান্ত নেবে — মিসিং শতাংশ এবং কলামের ধরন দেখে কোন imputer ব্যবহার করা উচিত তা জানিয়ে দেবে:

```
# Missing value strategy decision
def choose_imputer(missing_pct, col_type, skewness=0):
    """
    missing_pct: (df[col].isnull().sum() / len(df)) * 100
    col_type: 'numerical' or 'categorical'
    skewness: df[col].skew() — শুধু numerical-এর জন্য
    """
    if missing_pct > 50:
        return "Drop column অথবা Missing Indicator feature তৈরি করো"

    if col_type == 'categorical':
        if missing_pct < 5:
            return "SimpleImputer(strategy='most_frequent') — mode"
        return "SimpleImputer(strategy='constant', fill_value='Missing')"

    # Numerical
    if missing_pct < 5:
        if abs(skewness) < 0.5:
            return "SimpleImputer(strategy='mean')"
        return "SimpleImputer(strategy='median') — skewed তাই median"

    if missing_pct < 30:
        return "KNNImputer(n_neighbors=5) — pattern ধরে fill করে"

    return "IterativeImputer — complex relationship আছে"

# উদাহরণ:
for col in df.columns:
    miss_pct = (df[col].isnull().sum() / len(df)) * 100
    if miss_pct > 0:
        ctype = 'numerical' if df[col].dtype in ['float64', 'int64'] else 'categorical'
        skew = df[col].skew() if ctype == 'numerical' else 0
        print(f"{col} ({miss_pct:.1f}% missing): {choose_imputer(miss_pct, ctype,
skew)}")
```

নিচের সহজ ফাংশনটি দিয়ে outlier সমস্যায় কী করবেন তা দ্রুত সিদ্ধান্ত নিতে পারবেন — skewness এবং data size-এর ভিত্তিতে:

```
# Outlier handling decision guide
def decide_outlier_action(skewness, data_size, is_real_data=True):
    """
    skewness: df['col'].skew()
    data_size: len(df)
    is_real_data: False মানে data entry error সন্দেহ
    """
    if not is_real_data:
        return "Remove — data entry error"
    if data_size < 500:
        return "Cap (Winsorization) — data কম, remove এড়াও"
    if abs(skewness) > 2:
        return "Log Transform — distribution ঠিক হবে"
    return "Cap (IQR method) — safe choice"

# উদাহরণ:
print(decide_outlier_action(df['Fare'].skew(), len(df), is_real_data=True))
```

নিচের ফাংশনটি skewness, outlier এবং algorithm type দেখে স্বয়ংক্রিয়ভাবে সেরা scaler সুপারিশ করবে:

```
# Scaling decision – কোন scaler কখন?
def choose_scaler(skewness, has_outliers, algorithm_type):
    """
    skewness: abs(df['col'].skew())
    has_outliers: True/False (IQR দিয়ে check করো)
    algorithm_type: 'tree', 'linear', 'distance', 'neural'
    """
    if algorithm_type == 'tree':
        return "No scaling needed (Decision Tree, Random Forest, XGBoost)"

    if skewness > 1.0:
        return "Log Transform → তারপর StandardScaler"

    if has_outliers:
        return "RobustScaler (outlier থাকলে)"

    if algorithm_type == 'neural':
        return "MinMaxScaler (0-1 range)"

    return "StandardScaler (default choice)"

# উদাহরণ:
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
has_out = bool(((df['Fare'] < Q1-1.5*(Q3-Q1)) | (df['Fare'] > Q3+1.5*(Q3-Q1))).sum())
print(choose_scaler(abs(df['Fare'].skew()), has_out, 'linear'))
```

নিচের ফাংশনটি দেখে দ্রুত সিদ্ধান্ত নিন — কতটি unique value আছে এবং ক্রম আছে কিনা তার ভিত্তিতে সঠিক encoding বেছে নিন:

```
# Encoding decision
def choose_encoding(unique_count, has_order, is_target=False):
    """
    unique_count: df[col].nunique()
    has_order: True মানে ordinal (Education, Size, Rating)
    is_target: True মানে এটা target variable
    """
    if is_target:
        return "LabelEncoder – শুধু target-এর জন্য"

    if has_order:
        return "OrdinalEncoder – ক্রম আছে (Poor < Good < Excellent)"

    if unique_count <= 2:
        return "LabelEncoder বা OHE – binary"

    if unique_count <= 15:
        return "OneHotEncoder(drop='first') – nominal, কম category"

    return "FrequencyEncoding অথবা TargetEncoding – high cardinality"

# উদাহরণ:
for col in X.select_dtypes('object').columns:
    print(f"{col} ({X[col].nunique()}) unique: {choose_encoding(X[col].nunique(), False)}")
```


নিচে একটি সহজ উদাহরণ দেওয়া হলো — একটি মাত্র ফিচার দিয়ে PolynomialFeatures কীভাবে x^2 সম্পর্ক ধরে তা দেখুন:

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

# Simple example:  $x^2$  সম্পর্ক ধরা
X_simple = np.array([[2], [3], [4], [5]]) # একটি ফিচার

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X_simple)

print("Original X:", X_simple.ravel())
print("After Poly:", X_poly)
# Output:
# Original X: [2 3 4 5]
# After Poly: [[ 2.  4.] <- [x, x^2]
#             [ 3.  9.]
#             [ 4. 16.]
#             [ 5. 25.]]
#
# এখন Linear model  $x^2$  সম্পর্ক ধরতে পারবে
```

অধ্যায় ২১: নতুন ডেটাসেট হাতে পেলে কীভাবে সিদ্ধান্ত নেব (Decision Workflow for a New Dataset)

২১.১ ভূমিকা: বাস্তব Machine Learning এখান থেকেই শুরু

এই বইয়ের আগের ২০টি অধ্যায়ে আমরা Machine Learning Model Training-এর আগে প্রয়োজনীয় প্রায় সব গুরুত্বপূর্ণ বিষয় বিস্তারিতভাবে আলোচনা করেছি। Missing Value Handling, Outlier Detection, Feature Scaling, Encoding, Feature Engineering, Feature Selection, Multicollinearity, Class Imbalance, Pipeline এবং ColumnTransformer—প্রতিটি বিষয় আলাদা অধ্যায়ে ধাপে ধাপে শেখানো হয়েছে। প্রতিটি অধ্যায়ের উদ্দেশ্য ছিল একটি নির্দিষ্ট সমস্যা কী, কেন ঘটে এবং কীভাবে সমাধান করতে হয় তা বোঝানো।

কিন্তু বাস্তব Machine Learning Project সাধারণত এভাবে শুরু হয় না।

বাস্তবে কেউ আপনাকে কখনো বলবে না, "প্রথমে Chapter 7 পড়ে Missing Value Handle করো, তারপর Chapter 8 অনুসারে Outlier Handle করো।" বরং আপনাকে একটি সম্পূর্ণ নতুন Dataset দেওয়া হবে, যেখানে একসাথে অনেক ধরনের সমস্যা থাকতে পারে, আবার কোনো সমস্যাই নাও থাকতে পারে। তখন সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন হয়ে দাঁড়ায়, **এখন আমি কোথা থেকে শুরু করব? কোন সমস্যাটি আগে সমাধান করব? কোন Technique ব্যবহার করব? কোনটি প্রয়োজনই নেই?**

এই প্রশ্নগুলোর উত্তর মুখস্থ কোনো Workflow দিয়ে দেওয়া সম্ভব নয়। কারণ প্রতিটি Dataset ভিন্ন। প্রতিটি সমস্যার প্রকৃতি ভিন্ন। এমনকি একই Feature-এর ক্ষেত্রেও ভিন্ন Dataset-এ ভিন্ন Decision নিতে হতে পারে।

এই অধ্যায়ের মূল উদ্দেশ্য নতুন কোনো Machine Learning Technique শেখানো নয়। বরং এই বইয়ের আগের ২০টি অধ্যায় থেকে শেখা সমস্ত জ্ঞানকে একত্র করে বাস্তব Dataset-এর ক্ষেত্রে কীভাবে চিন্তা করতে হবে, কীভাবে সমস্যা বিশ্লেষণ করতে হবে এবং কীভাবে যুক্তিসঙ্গত Decision নিতে হবে, সেই মানসিক কাঠামো (Decision Framework) তৈরি করা।

অন্যভাবে বললে, আগের অধ্যায়গুলো আপনাকে বিভিন্ন Tool ব্যবহার করতে শিখিয়েছে। এই অধ্যায় আপনাকে শেখাবে, **কখন কোন Tool ব্যবহার করা উচিত এবং কেন।**

২১.২ নতুন Dataset হাতে পাওয়ার পর সঠিক মানসিকতা

Machine Learning শেখার শুরুতে অধিকাংশ শিক্ষার্থী একটি সাধারণ ভুল করে। তারা নতুন Dataset হাতে পাওয়ার সাথে সাথেই Missing Value পূরণ করা, Encoding করা কিংবা Scaling শুরু করে দেয়। অর্থাৎ Dataset-টি ভালোভাবে বোঝার আগেই তারা সেটিকে পরিবর্তন করতে শুরু করে।

এটি একটি ভুল অভ্যাস।

একজন দক্ষ Machine Learning Engineer কখনোই শুরুতেই Dataset পরিবর্তন করেন না। বরং তিনি প্রথমে Dataset পর্যবেক্ষণ করেন, বিশ্লেষণ করেন এবং সমস্যাগুলো শনাক্ত করেন। অর্থাৎ Cleaning-এর আগে Understanding, আর Understanding-এর আগে Observation।

একটি বাস্তব উদাহরণ চিন্তা করুন। ধরুন একজন রোগী চিকিৎসকের কাছে এসেছে। একজন অভিজ্ঞ চিকিৎসক কি রোগীকে দেখামাত্র অস্ত্রোপচার শুরু করবেন? নিশ্চয়ই না। তিনি প্রথমে রোগীর ইতিহাস জানবেন, বিভিন্ন পরীক্ষা করবেন, রিপোর্ট বিশ্লেষণ করবেন, তারপর রোগ নির্ণয় করবেন। রোগটি নিশ্চিত হওয়ার পরই চিকিৎসা শুরু হবে।

Machine Learning-এও একই নীতি অনুসরণ করা উচিত।

Dataset হলো রোগী, আর Data Preprocessing হলো চিকিৎসা। রোগ নির্ণয় না করে চিকিৎসা শুরু করলে যেমন ভুল হওয়ার সম্ভাবনা থাকে, তেমনি Dataset না বুঝে Cleaning শুরু করলে ভুল সিদ্ধান্ত নেওয়ার সম্ভাবনাও অনেক বেড়ে যায়।

তাই নতুন Dataset হাতে পাওয়ার পর প্রথম লক্ষ্য হওয়া উচিত একটি প্রশ্নের উত্তর খুঁজে বের করা—

"এই Dataset আমাকে কী বলতে চাইছে?"

যখন Dataset সম্পর্কে পরিষ্কার ধারণা তৈরি হবে, তখনই কেবল Cleaning, Transformation কিংবা Feature Engineering-এর মতো ধাপগুলো শুরু করা উচিত।

২১.৩ Observation আগে, তারপর Decision

Machine Learning-এর অন্যতম গুরুত্বপূর্ণ নীতি হলো—

Never fix a problem before proving that the problem actually exists.

অর্থাৎ কোনো সমস্যা আছে কিনা নিশ্চিত হওয়ার আগে কখনোই সেটি সমাধান করার চেষ্টা করা উচিত নয়।

ধরুন একটি Dataset-এ Missing Value একেবারেই নেই। সেখানে Missing Value Handling করার কোনো প্রয়োজন নেই।

আবার অন্য একটি Dataset-এ Outlier নেই। সেখানে Outlier Detection নিয়ে সময় ব্যয় করারও প্রয়োজন নেই।

একইভাবে যদি Dataset-এ কোনো Categorical Feature না থাকে, তাহলে Encoding করার প্রশ্নই আসে না।

এই কারণেই Observation ধাপটি অত্যন্ত গুরুত্বপূর্ণ।

Observation পর্যায়ে মূল উদ্দেশ্য হলো Dataset সম্পর্কে যত বেশি সম্ভব তথ্য সংগ্রহ করা। উদাহরণস্বরূপ—

- Dataset-এ মোট কতটি Row এবং কতটি Column রয়েছে?
- কোন Feature Numerical এবং কোনগুলো Categorical?
- Target Variable কোনটি?
- কোনো Missing Value আছে কি?
- Duplicate Record আছে কি?
- কোনো Feature অত্যন্ত Skewed কি না?
- Outlier আছে কি?
- Feature-গুলোর মধ্যে Correlation কেমন?
- Class Distribution Balanced নাকি Imbalanced?

এই প্রশ্নগুলোর উত্তর পাওয়ার পরই প্রকৃত Decision নেওয়া সম্ভব হয়।

অন্যভাবে বলা যায়,

Observation তথ্য দেয়, আর সেই তথ্যের উপর ভিত্তি করেই Decision নেওয়া হয়।

২১.৪ Decision Making Framework

একটি নতুন Dataset বিশ্লেষণ করার সময় সিদ্ধান্ত নেওয়ার জন্য একটি যৌক্তিক কাঠামো অনুসরণ করা উচিত। এটি কোনো বাধ্যতামূলক Workflow নয়; বরং চিন্তা করার একটি পদ্ধতি।

প্রথম ধাপে Dataset-এর গঠন বোঝা হয়। এরপর ধীরে ধীরে প্রতিটি সম্ভাব্য সমস্যার অস্তিত্ব যাচাই করা হয়। কোনো সমস্যা থাকলে সেটি সমাধান করা হয়, আর না থাকলে সেই ধাপটি এড়িয়ে যাওয়া হয়।

এই চিন্তার ধারাটি সাধারণভাবে নিচের রূপে কল্পনা করা যায়।

১. Dataset বোঝা।
২. Feature-এর ধরন নির্ধারণ করা।
৩. Missing Value আছে কি না যাচাই করা।
৪. Duplicate আছে কি না পরীক্ষা করা।
৫. Outlier আছে কি না বিশ্লেষণ করা।
৬. Distribution ও Skewness পরীক্ষা করা।
৭. Encoding প্রয়োজন কি না নির্ধারণ করা।
৮. Scaling প্রয়োজন কি না বিবেচনা করা।
৯. Feature Engineering-এর সুযোগ আছে কি না দেখা।
১০. Feature Selection প্রয়োজন কি না বিশ্লেষণ করা।
১১. Pipeline তৈরি করা।
১২. Model Training শুরু করা।

এখানে লক্ষ্য করার বিষয় হলো, প্রতিটি ধাপ আগের ধাপের তথ্যের উপর নির্ভর করে। কোনো Decision আলাদাভাবে নেওয়া হয় না। প্রতিটি সিদ্ধান্ত Dataset-এর বর্তমান অবস্থার ভিত্তিতে নেওয়া হয়।

এটাই একজন দক্ষ Machine Learning Engineer-এর চিন্তার ধরন।

২১.৫ সব Dataset-এর জন্য একই Workflow প্রযোজ্য নয়

Machine Learning-এ একটি প্রচলিত ভুল ধারণা হলো, সব Dataset-এর জন্য একটি নির্দিষ্ট Preprocessing Workflow অনুসরণ করতে হবে।

বাস্তবে বিষয়টি সম্পূর্ণ ভিন্ন।

প্রতিটি Dataset-এর গঠন, উদ্দেশ্য, Feature-এর ধরন এবং সমস্যার প্রকৃতি ভিন্ন হয়। তাই একই Workflow সব ক্ষেত্রে সমানভাবে কার্যকর হবে না।

উদাহরণস্বরূপ, যদি একটি Dataset-এ কোনো Missing Value না থাকে, তাহলে Missing Value Handling অধ্যায়ের কোনো Technique ব্যবহার করার প্রয়োজন নেই।

আবার যদি Dataset-এ কোনো Categorical Feature না থাকে, তাহলে Encoding সম্পূর্ণ অপ্রয়োজনীয় হয়ে যায়।

একইভাবে Tree-Based Model (যেমন Decision Tree, Random Forest, XGBoost)-এর ক্ষেত্রে অনেক সময় Feature Scaling প্রয়োজন হয় না। অন্যদিকে Logistic Regression, SVM অথবা K-Nearest Neighbors-এর মতো Distance-Based Algorithm ব্যবহার করলে Scaling অত্যন্ত গুরুত্বপূর্ণ হয়ে ওঠে।

এখানেই Machine Learning-এর একটি মৌলিক দর্শন কাজ করে।

Technique নয়, সমস্যাই সিদ্ধান্ত নির্ধারণ করে।

অর্থাৎ আমরা কোনো অধ্যায় ব্যবহার করি না কারণ সেটি বইয়ে আছে; বরং ব্যবহার করি কারণ Dataset সেটির প্রয়োজন তৈরি করেছে।

২১.৬ একই Feature-এ একাধিক সমস্যা থাকলে কীভাবে চিন্তা করবেন

বাস্তব Dataset-এ প্রায়ই দেখা যায় একটি Feature-এ একাধিক সমস্যা একসাথে উপস্থিত থাকে।
ধরুন একটি Feature-এর অবস্থা নিচের মতো।

- Feature-টি Numerical।
- কোনো Missing Value নেই।
- কয়েকটি Extreme Outlier রয়েছে।
- Distribution ডানদিকে অনেক বেশি Skewed।
- পরবর্তীতে StandardScaler ব্যবহার করার পরিকল্পনা রয়েছে।

এখন স্বাভাবিকভাবেই প্রশ্ন আসে, কোন কাজটি আগে করা উচিত?

আগে কি Outlier Handle করব?

নাকি আগে Data Transformation করব?

নাকি সরাসরি Scaling করব?

এই ধরনের প্রশ্নের জন্য Machine Learning-এ এমন কোনো একক উত্তর নেই যা সব Dataset-এর জন্য সত্য হবে।

কারণ Decision নির্ভর করে বিভিন্ন বিষয়ের উপর, যেমন—

- Outlier প্রকৃত Data নাকি Data Entry Error।
- Outlier-এর সংখ্যা কত।
- Distribution কতটা Skewed।
- কোন ধরনের Model ব্যবহার করা হবে।
- কোন ধরনের Transformation ব্যবহার করা হবে।
- Business Problem কী।

অর্থাৎ এখানে মুখস্থ কোনো Rule অনুসরণ না করে যুক্তি ব্যবহার করতে হয়।

একজন দক্ষ Machine Learning Engineer কখনোই শুধু Workflow মুখস্থ করেন না। তিনি প্রথমে Feature-এর বর্তমান অবস্থা বিশ্লেষণ করেন, তারপর প্রতিটি সমস্যার প্রকৃতি বোঝেন এবং শেষে যৌক্তিকভাবে সিদ্ধান্ত নেন কোন সমস্যা আগে সমাধান করা সবচেয়ে যুক্তিসঙ্গত হবে।

এই কারণেই বাস্তব Machine Learning-এ সবচেয়ে মূল্যবান দক্ষতা হলো **Decision Making Ability**, শুধুমাত্র Library বা Function মুখস্থ করা নয়।

এই অধ্যায়ের পরবর্তী অংশে আমরা দেখব কীভাবে একটি সম্পূর্ণ নতুন Dataset ধাপে ধাপে বিশ্লেষণ করে বাস্তব Decision নেওয়া যায় এবং কীভাবে আগের ২০টি অধ্যায়ের জ্ঞান একত্রে প্রয়োগ করা হয়।

২১.৭ একটি নতুন Dataset কীভাবে বিশ্লেষণ করবেন (Theoretical Analysis)

এই অধ্যায়ে আমরা বারবার একটি বিষয় উল্লেখ করেছি যে, নতুন Dataset হাতে পাওয়ার পর সরাসরি Data Cleaning শুরু করা উচিত নয়। প্রথমে Dataset-টিকে পর্যবেক্ষণ করতে হবে এবং এর বৈশিষ্ট্যগুলো বুঝে নিতে হবে। এখন প্রশ্ন হলো, বাস্তবে এই পর্যবেক্ষণ কীভাবে করা হয়?

এখানে মনে রাখা গুরুত্বপূর্ণ যে, এই অংশের উদ্দেশ্য নতুন কোনো Technique শেখানো নয়। বরং আগের অধ্যায়গুলোতে শেখা বিভিন্ন ধারণাকে একটি ধারাবাহিক চিন্তার মাধ্যমে একত্র করা।

ধরুন, আপনার হাতে একটি সম্পূর্ণ নতুন Dataset এসেছে। আপনি Dataset-এর ভেতরের কিছুই জানেন না। এখন প্রথম কাজ হবে Dataset সম্পর্কে যত বেশি সম্ভব তথ্য সংগ্রহ করা। এই পর্যায়ে আপনি কোনো পরিবর্তন করবেন না; শুধুমাত্র পর্যবেক্ষণ করবেন।

প্রথমে Dataset-এর সামগ্রিক গঠন বোঝার চেষ্টা করুন। মোট কতটি Row আছে, কতটি Column আছে, Target Variable কোনটি এবং প্রতিটি Feature কী ধরনের তথ্য বহন করছে তা বুঝুন। এরপর প্রতিটি

Feature আলাদাভাবে পর্যবেক্ষণ করুন। Numerical Feature এবং Categorical Feature আলাদা করুন। Continuous ও Discrete Feature শনাক্ত করুন। Categorical Feature-এর ক্ষেত্রে Nominal নাকি Ordinal সেটাও নির্ধারণ করুন।

এরপর Dataset-এর গুণগত মান (Data Quality) মূল্যায়ন করুন। কোথাও Missing Value আছে কি না, Duplicate Record আছে কি না কিংবা কোনো Feature সম্পূর্ণ ফাঁকা অথবা অত্যন্ত কম তথ্য বহন করছে কি না তা পরীক্ষা করুন।

এরপর Distribution বিশ্লেষণ করুন। কোনো Feature অত্যন্ত Skewed কি না, অস্বাভাবিক বড় বা ছোট মান (Outlier) আছে কি না, Feature-এর মানগুলো যৌক্তিক কি না, সেগুলো পর্যবেক্ষণ করুন।

এরপর Feature-গুলোর পারস্পরিক সম্পর্ক বিশ্লেষণ করুন। কোনো Feature অন্য Feature-এর সাথে অত্যন্ত বেশি সম্পর্কযুক্ত কি না, কোনো Feature অপ্ৰয়োজনীয় কি না, কিংবা নতুন Feature তৈরি করলে তথ্য আরও সমৃদ্ধ হবে কি না, সেগুলো নিয়ে চিন্তা করুন।

সবশেষে আপনি সিদ্ধান্ত নেবেন কোন কোন সমস্যা বাস্তবেই সমাধান করা প্রয়োজন এবং কোনগুলোকে অপরিবর্তিত রাখাই ভালো।

এই পুরো প্রক্রিয়ায় একটি বিষয় স্পষ্ট হয়ে যায়। একজন দক্ষ Machine Learning Engineer Dataset পরিবর্তনের আগে Dataset-কে বোঝার জন্য বেশি সময় ব্যয় করেন। কারণ ভুল Diagnosis থেকে কখনোই সঠিক চিকিৎসা সম্ভব নয়।

২১.৮ একটি সম্পূর্ণ Decision Workflow

এই বইয়ের প্রতিটি অধ্যায় একটি নির্দিষ্ট সমস্যার সমাধান নিয়ে আলোচনা করেছে। বাস্তব Project-এ কাজ করার সময় এই অধ্যায়গুলোকে একটি যৌক্তিক ক্রমে ব্যবহার করা যায়।

একটি সাধারণ Decision Workflow নিচের মতো হতে পারে।

ধাপ ১: প্রথমে Dataset বুঝুন। Feature-এর ধরন, Target Variable এবং Dataset-এর গঠন বিশ্লেষণ করুন।

ধাপ ২: কোনো ধরনের Data Leakage এড়ানোর জন্য প্রয়োজন হলে শুরুতেই Train-Test Split সম্পন্ন করুন।

ধাপ ৩: Missing Value আছে কি না পরীক্ষা করুন। যদি থাকে, তখনই শুধুমাত্র উপযুক্ত Imputation Technique নির্বাচন করুন।

ধাপ ৪: Duplicate Record এবং Data Entry Error পরীক্ষা করুন।

ধাপ ৫: Outlier বিশ্লেষণ করুন। সব Outlier মুছে ফেলার চেষ্টা করবেন না। আগে বুঝুন সেগুলো প্রকৃত Observation নাকি ভুল তথ্য।

ধাপ ৬: Distribution পর্যবেক্ষণ করুন। প্রয়োজন হলে উপযুক্ত Transformation ব্যবহার করুন।

ধাপ ৭: Categorical Feature থাকলে তাদের প্রকৃতি অনুযায়ী Encoding নির্বাচন করুন। Nominal এবং Ordinal Feature-এর জন্য একই Encoding ব্যবহার করা উচিত নয়।

ধাপ ৮: ব্যবহৃত Machine Learning Algorithm অনুযায়ী সিদ্ধান্ত নিন Scaling প্রয়োজন কি না।

ধাপ ৯: প্রয়োজন হলে নতুন Feature তৈরি করুন এবং অপ্ৰয়োজনীয় Feature বাদ দিন।

ধাপ ১০: Feature Selection এবং Multicollinearity পরীক্ষা করুন।

ধাপ ১১: সব Preprocessing ধাপকে Pipeline-এর মাধ্যমে সংগঠিত করুন।

ধাপ ১২: এরপর Model Training, Validation এবং Evaluation সম্পন্ন করুন।

এখানে লক্ষ্য করার বিষয় হলো, এটি একটি কঠোর নিয়ম নয়। বরং এটি একটি Decision Framework, যা বাস্তব পরিস্থিতি অনুযায়ী পরিবর্তিত হতে পারে।

২১.৯ সাধারণ ভুল (Common Mistakes)

Machine Learning শেখার সময় অনেক শিক্ষার্থী একই ধরনের কিছু ভুল বারবার করে থাকে। নিচে সবচেয়ে সাধারণ কয়েকটি ভুল উল্লেখ করা হলো।

১. Dataset পর্যবেক্ষণ না করেই Cleaning শুরু করা।
 ২. প্রতিটি Dataset-এ একই Preprocessing Workflow ব্যবহার করা।
 ৩. কোনো সমস্যা না থাকলেও অপ্রয়োজনীয় Technique প্রয়োগ করা।
 ৪. শুধুমাত্র Tutorial অনুসরণ করে Function ব্যবহার করা, কিন্তু কেন ব্যবহার করা হচ্ছে তা না বোঝা।
 ৫. Outlier দেখলেই মুছে ফেলা, অথচ সেটি বাস্তব Observation হতে পারে।
 ৬. সব Numerical Feature-এ Scaling করা, যদিও ব্যবহৃত Model-এর জন্য Scaling প্রয়োজন নাও হতে পারে।
 ৭. Nominal ও Ordinal Feature-এর পার্থক্য না বুঝে একই Encoding ব্যবহার করা।
 ৮. Feature-এর প্রকৃতি না বুঝে Transformation প্রয়োগ করা।
 ৯. Pipeline ব্যবহার না করে আলাদাভাবে Preprocessing করা, যার ফলে Data Leakage-এর ঝুঁকি বৃদ্ধি পাওয়া।
 ১০. প্রতিটি ধাপের Decision লিখে না রাখা, ফলে পরে একই বিশ্লেষণ পুনরায় করা কঠিন হয়ে যাওয়া।
- এই ভুলগুলো এড়াতে পারলে Dataset Analysis অনেক বেশি নির্ভুল ও যৌক্তিক হয়।

২১.১০ বেস্ট প্র্যাকটিস (Best Practices)

একজন দক্ষ Machine Learning Practitioner সাধারণত নিচের নীতিগুলো অনুসরণ করেন।

- Dataset পরিবর্তনের আগে Dataset বোঝার চেষ্টা করুন।
- প্রতিটি Decision-এর পেছনে যুক্তি থাকা উচিত।
- কোনো Technique শুধুমাত্র প্রয়োজন হলে ব্যবহার করুন।
- Business Problem এবং Domain Knowledge-কে গুরুত্ব দিন।
- Data Leakage এড়াতে Train এবং Test Data সম্পূর্ণ আলাদা রাখুন।
- Pipeline ব্যবহার করে সম্পূর্ণ Workflow সংগঠিত করুন।
- প্রতিটি গুরুত্বপূর্ণ Decision ডকুমেন্ট করুন, যাতে ভবিষ্যতে একই কাজ পুনরায় করা সহজ হয়।
- একই Dataset-এর জন্য একাধিক সম্ভাব্য সমাধান থাকতে পারে। তাই একটি নির্দিষ্ট Workflow মুখস্থ করার পরিবর্তে Decision নেওয়ার যুক্তি শেখার চেষ্টা করুন।

Machine Learning-এ অভিজ্ঞতা বাড়ার সাথে সাথে আপনি বুঝতে পারবেন যে, সঠিক Function জানা যতটা গুরুত্বপূর্ণ, তার চেয়ে অনেক বেশি গুরুত্বপূর্ণ হলো কখন সেই Function ব্যবহার করা উচিত তা বোঝা।

২১.১১ ইন্টারভিউ প্রশ্নসমূহ

প্রশ্ন: একটি নতুন Dataset হাতে পাওয়ার পর প্রথম কাজ কী হওয়া উচিত?

উত্তর: প্রথম কাজ হলো Dataset পর্যবেক্ষণ করা এবং এর গঠন, Feature-এর ধরন, Missing Value, Duplicate, Outlier, Distribution এবং অন্যান্য বৈশিষ্ট্য বিশ্লেষণ করা। Dataset না বুঝে সরাসরি Data Cleaning শুরু করা উচিত নয়।

প্রশ্ন: কেন সব Dataset-এর জন্য একই Preprocessing Workflow ব্যবহার করা যায় না?

উত্তর: কারণ প্রতিটি Dataset-এর গঠন, Feature-এর ধরন, সমস্যার প্রকৃতি এবং Business Requirement ভিন্ন হয়। তাই প্রতিটি Dataset-এর জন্য আলাদাভাবে Decision নিতে হয়।

প্রশ্ন: Observation এবং Data Cleaning-এর মধ্যে পার্থক্য কী?

উত্তর: Observation-এর উদ্দেশ্য হলো Dataset সম্পর্কে তথ্য সংগ্রহ করা এবং সমস্যাগুলো শনাক্ত করা। Data Cleaning-এর উদ্দেশ্য হলো শনাক্ত হওয়া সমস্যাগুলোর উপযুক্ত সমাধান করা। অর্থাৎ Observation আগে এবং Cleaning পরে।

প্রশ্ন: একজন দক্ষ Machine Learning Engineer-এর সবচেয়ে গুরুত্বপূর্ণ দক্ষতা কী?

উত্তর: শুধুমাত্র Library বা Function জানা নয়; বরং Dataset বিশ্লেষণ করে কোন পরিস্থিতিতে কোন Technique ব্যবহার করতে হবে এবং কেন ব্যবহার করতে হবে, সেই সিদ্ধান্ত নেওয়ার ক্ষমতাই একজন দক্ষ Machine Learning Engineer-এর সবচেয়ে গুরুত্বপূর্ণ দক্ষতা।

২১.১২ অধ্যায় সারসংক্ষেপ

- বাস্তব Machine Learning Project একটি নতুন Dataset দিয়ে শুরু হয়, কোনো নির্দিষ্ট Chapter দিয়ে নয়।
- Dataset পরিবর্তনের আগে সেটিকে ভালোভাবে পর্যবেক্ষণ ও বিশ্লেষণ করা উচিত।
- Observation-এর মাধ্যমে সমস্যাগুলো শনাক্ত করা হয় এবং সেই তথ্যের ভিত্তিতে Decision নেওয়া হয়।
- সব Dataset-এর জন্য একই Preprocessing Workflow প্রযোজ্য নয়।
- প্রতিটি Technique-এর প্রয়োজনীয়তা Dataset-এর বৈশিষ্ট্য, ব্যবহৃত Algorithm এবং Business Problem-এর উপর নির্ভর করে।
- একজন দক্ষ Machine Learning Engineer Function মুখস্থ করেন না; বরং প্রতিটি Decision-এর পেছনের যুক্তি বোঝেন।
- এই বইয়ের আগের ২০টি অধ্যায় বিভিন্ন Preprocessing Technique শেখায়, আর এই অধ্যায় দেখায় বাস্তব Project-এ সেই Technique-গুলো কখন, কেন এবং কীভাবে একত্রে ব্যবহার করা হয়।

এই বইয়ের মূল বার্তা এক বাক্যে বলা যায়:

"Machine Learning-এ সবচেয়ে গুরুত্বপূর্ণ বিষয় কোনো Technique জানা নয়; বরং Dataset বুঝে সঠিক সময়ে সঠিক Technique নির্বাচন করতে পারাই প্রকৃত দক্ষতা।"

পরিশিষ্ট ক: শব্দকোষ (Glossary)

এই বইয়ে ব্যবহৃত গুরুত্বপূর্ণ ML পরিভাষাগুলোর সংক্ষিপ্ত বাংলা সংজ্ঞা:

Accuracy (নির্ভুলতা): মোট সঠিক ভবিষ্যদ্বাণীর শতাংশ। সূত্র: $(TP+TN)/(TP+TN+FP+FN)$ । ইমব্যালেন্সড ডেটায় বিভ্রান্তিকর হতে পারে।

Bias (পক্ষপাত): মডেল কত সহজ ধারণা করেছে তার পরিমাপ। High bias = underfitting — মডেল প্যাটার্ন ধরতে পারছে না।

Classification (শ্রেণিবিভাজন): আউটপুট যখন একটি ক্যাটাগরি বা শ্রেণি (যেমন: বেঁচে/মৃত, স্প্যাম/নট-স্প্যাম)।

Cross Validation (ক্রস ভ্যালিডেশন): ডেটাকে k ভাগে ভেঙে k বার মডেল ট্রেন/টেস্ট করা। Overfitting ধরতে এবং মডেলের সত্যিকার পারফরমেন্স মাপতে ব্যবহৃত হয়।

Data Leakage (ডেটা ফাঁস): Test data-র তথ্য Training পর্যায়ে ঢুকে যাওয়া। এটি কৃত্রিমভাবে accuracy বাড়ায়, কিন্তু বাস্তবে মডেল ব্যর্থ হয়।

Encoding (এনকোডিং): Categorical (ক্যাটাগরি) ডেটাকে সংখ্যায় রূপান্তর করা। যেমন: OneHotEncoder, OrdinalEncoder, LabelEncoder।

Feature (ফিচার): ইনপুট ভেরিয়েবল বা কলাম — মডেল যা থেকে শেখে। X দিয়ে চিহ্নিত করা হয়।

Feature Engineering (ফিচার ইঞ্জিনিয়ারিং): বিদ্যমান ফিচার থেকে নতুন, আরও তথ্যসমৃদ্ধ ফিচার তৈরি করা।

Feature Scaling (স্কেলিং): ফিচারগুলোকে একই মানের পরিসরে আনা। StandardScaler, MinMaxScaler, RobustScaler।

Feature Selection (ফিচার সিলেকশন): অপ্রয়োজনীয় বা দুর্বল ফিচার বাদ দিয়ে সেরা ফিচার বেছে নেওয়া।

Imputation (ইমপিউটেশন): মিসিং ভ্যালু পূরণ করার প্রক্রিয়া। Mean, Median, Mode, KNN, বা Iterative পদ্ধতি ব্যবহার করা হয়।

Missing Value (মিসিং ভ্যালু): ডেটাসেটে অনুপস্থিত বা NaN মান। হ্যান্ডল না করলে মডেল error দেয়।

Multicollinearity (মাল্টিকোলিনিয়ারিটি): দুটো বা বেশি ফিচার একে অপরের সাথে অত্যন্ত সম্পর্কিত। Linear Model-এ coefficient অস্থির করে।

Outlier (আউটলায়ার): অন্য সব মান থেকে অনেক দূরে অবস্থিত চরম মান। মডেলের পারফরম্যান্স নষ্ট করতে পারে।

Overfitting (ওভারফিটিং): মডেল Training Data মুখস্থ করে ফেলে, নতুন ডেটায় ভুল করে। High variance।

Pipeline (পাইপলাইন): একাধিক preprocessing ও model ধাপকে একত্রে সংগঠিত করার sklearn টুল। Data Leakage রোধ করে।

Regression (রিগ্রেশন): আউটপুট যখন একটি সংখ্যা (continuous value)। যেমন: বাড়ির দাম, তাপমাত্রা।

Skewness (স্কিউনেস): Distribution কতটা অসমানভাবে কোনো দিকে হেলে আছে। $|skewness| > 1$ হলে প্রবল স্কিউ।

Target (টার্গেট): আউটপুট ভেরিয়েবল বা লেবেল — মডেল যা predict করে। y দিয়ে চিহ্নিত।

Train-Test Split (ট্রেন-টেস্ট স্প্লিট): ডেটাকে Training ও Testing অংশে ভাগ করা। সাধারণত ৭০-৮০% train, ২০-৩০% test।

Underfitting (আন্ডারফিটিং): মডেল অত্যন্ত সহজ, Training Data-র প্যাটার্নও ধরতে পারে না। High bias।

Variance (ভ্যারিয়েন্স): মডেল কতটা ডেটার noise-কে অনুসরণ করে তার পরিমাপ। High variance = overfitting।

VIF (Variance Inflation Factor): Multicollinearity পরিমাপের সূত্র: $VIF = 1/(1-R^2)$ । $VIF > 5$ সমস্যার সংকেত, $VIF > 10$ গুরুতর সমস্যা।

পরিশিষ্ট খ: তথ্যসূত্র ও আরও পড়ার উৎস

এই বই প্রস্তুতিতে এবং আরও গভীরে জানতে নিচের উৎসগুলো অনুসরণ করুন:

Scikit-learn দলিলপত্র: <https://scikit-learn.org/stable/documentation.html> — সমস্ত sklearn ক্লাস ও ফাংশনের অফিসিয়াল গাইড।

Titanic Dataset: <https://www.kaggle.com/c/titanic/data> — Kaggle-এ পাওয়া এই বইয়ের প্রধান ডেটাসেট।

Pandas দলিলপত্র: <https://pandas.pydata.org/docs/> — DataFrame manipulation ও data analysis।

Seaborn দলিলপত্র: <https://seaborn.pydata.org/> — Statistical data visualization।

Matplotlib দলিলপত্র: <https://matplotlib.org/stable/contents.html> — Python-এর মূল plotting library।

NumPy দলিলপত্র: <https://numpy.org/doc/> — Numerical computing ও array operations।

Feature Engineering for Machine Learning: Alice Zheng & Amanda Casari — O'Reilly Media। Feature engineering-এর বিস্তারিত practical guide।

Hands-On Machine Learning (3rd Ed.): Aurélien Géron — O'Reilly Media। Scikit-learn ও TensorFlow দিয়ে ML শেখার সেরা বইগুলোর একটি।

Python Machine Learning (3rd Ed.): Sebastian Raschka & Vahid Mirjalili — Packt Publishing। ML algorithm ও implementation-এর গভীর আলোচনা।

Imbalanced-learn দলিলপত্র: <https://imbalanced-learn.org/stable/> — Class imbalance handling-এর জন্য SMOTE সহ বিভিন্ন কৌশল।

পরিশিষ্ট গ: লেখক পরিচিতি

মোঃ শাওন খান

শিক্ষার্থী, তথ্য ও প্রযুক্তি বিভাগ (IIT)

জাহাঙ্গীরনগর বিশ্ববিদ্যালয়

তৃতীয় বর্ষ · CGPA: ৩.৮২/৪.০০ (৪র্থ সেমিস্টার পর্যন্ত)

লেখক একজন Aspiring AI Engineer যিনি AI, IoT এবং scalable backend সিস্টেম একত্রিত করে বাস্তবমুখী সমাধান তৈরিতে কাজ করছেন। তাঁর সবচেয়ে উল্লেখযোগ্য কাজ হলো **HealthBridge** একটি IoT-চালিত ক্লিনিক্যাল ইন্টেলিজেন্স প্ল্যাটফর্ম, যেখানে রিয়েল-টাইম ECG অ্যাকুইজিশন, ডিপ লার্নিং-ভিত্তিক রোগ শনাক্তকরণ এবং রোগী-ডাক্তার ড্যাশবোর্ড সম্পূর্ণরূপে তৈরি ও deploy করা হয়েছে।

এছাড়া তিনি UniFix (বিশ্ববিদ্যালয় অভিযোগ ব্যবস্থাপনা), LockGuard (স্মার্ট নিরাপত্তা সিস্টেম) এবং IIT Academic Portal-এর মতো পূর্ণাঙ্গ প্রজেক্ট তৈরি করেছেন। The Infinity AI BuildFest 2026 হ্যাকাথনে ArogyoAI প্রজেক্টে AI ও IoT Developer হিসেবে অংশগ্রহণ করেছেন।

Codeforces-এ Peak Rating ১১৮২ এবং Codeforces ও LeetCode মিলিয়ে ৪৫০+ সমস্যা সমাধান করেছেন। HackerRank Python (Basic) এবং SimpliLearn PostgreSQL Professional সার্টিফিকেশনপ্রাপ্ত।

যোগাযোগ:

ইমেইল: shaon.iit52@gmail.com

পোর্টফোলিও: portfolio-7vvt.vercel.app

GitHub: github.com/Md-Shaon-Khan

LinkedIn: linkedin.com/in/md-shaon-khan-01003433a